Verification Continuum™

Synopsys Synplify Pro® for Pango

Reference

August 2022

SYNOPSYS®

Copyright Notice and Proprietary Information

© 2022 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at
http://www.synopsys.com/Company/Pages/Trademarks.aspx.
All other product or company names may be trademarks of their respective owners.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
690 East Middlefield Road
Mountain View, CA 94043
www.synopsys.com

August 2022

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Contents

Chapter 1: Product Overview

Overview of the Synthesis Tool 12
    Synplify Pro Features 12

Graphic User Interface 14

Getting Help 16

Chapter 2: User Interface Overview

The Project View 20
    Project Management View 21

The Project Results View 23
    Project Status Tab 23
    Report Tab 27
    Implementation Directory 28
    Process View 30

Other Windows and Views 33
    Dockable GUI Entities 34
    Watch Window 34
    Tcl Script and Messages Windows 37
    Tcl Script Window 38
    Message Viewer 38
    Output Windows (Tcl Script and Watch Windows) 42
    Text Editor View 42
    Context Help Editor Window 45
    Interactive Attribute Examples 47

Using the Mouse 49
    Mouse Operation Terminology 49
    Using Mouse Strokes 50
    Using the Mouse Buttons 51
    Using the Mouse Wheel 53

© 2022 Synopsys, Inc.

Toolbars 54
Project Toolbar 55
Analyst Toolbar 56
Text Editor Toolbar 58
FSM Viewer Toolbar 59
Tools Toolbar 60

Keyboard Shortcuts 61

Buttons and Options 69

Chapter 3: HDL Analyst Tool

HDL Analyst Views and Commands 74
RTL View 75
Technology View 77
Hierarchy Browser 79
FSM Viewer Window 81
Filtered and Unfiltered Schematic Views 84
Accessing HDL Analyst Commands 85

Schematic Objects and Their Display 86
Object Information 86
Sheet Connectors 87
Primitive and Hierarchical Instances 88
Transparent and Opaque Display of Hierarchical Instances 89
Hidden Hierarchical Instances 90
Schematic Display 91

Basic Operations on Schematic Objects 95
Finding Schematic Objects 95
Selecting and Unselecting Schematic Objects 96
Crossprobing Objects 97
Dragging and Dropping Objects 99

Multiple-sheet Schematics 100
Controlling the Amount of Logic on a Sheet 100
Navigating Among Schematic Sheets 100
Multiple Sheets for Transparent Instance Details 102

Exploring Design Hierarchy 103
Pushing and Popping Hierarchical Levels 103
Navigating With a Hierarchy Browser 106
Looking Inside Hierarchical Instances 108

Filtering and Flattening Schematics 111
Commands That Result in Filtered Schematics 111

Combined Filtering Operations 112
Returning to The Unfiltered Schematic 113
Commands That Flatten Schematics 113
Selective Flattening 114
Filtering Compared to Flattening 115

Timing Information and Critical Paths 117
Timing Reports 117
Critical Paths and the Slack Margin Parameter 118
Examining Critical Path Schematics 119

Chapter 4: Constraint Guidelines

Constraint Types 122

Constraint Files 123

FDC Constraints 125

Methods for Creating Constraints 125

Constraint Checking 127

Database Object Search 129

Forward Annotation 129

Auto Constraints 130

Chapter 5: Input and Result Files

Input Files 132
HDL Source Files 133

Libraries 136
Open Verification Library (Verilog) 136
The Generic Technology Library 137
ASIC Library Files 138

Output Files 140

Log File 145

Timing Reports 150
Timing Report Header 151
Performance Summary 151
Clock Relationships 153
Interface Information 154
Asynchronous Clock Report 155

© 2022 Synopsys, Inc.

Constraint Checking Report 157
Gated Clock Conversion Report 164

Chapter 6: RAM and ROM Inference

Guidelines and Support for RAM Inference 166
Block RAM Examples 167
Block RAM Mode Examples 167
Single-Port Block RAM Examples 171
Single-Port RAM with Read Address Registered Example 171
Single-Port RAM with RAM Output Registered Examples 172
Dual-Port Block RAM Examples 173
True Dual-Port RAM Examples 176
Initial Values for RAMs 179
Example 1: RAM Initialization 180
Example 2: Cross-Module Referencing for RAM Initialization 181
Initialization Data File 183
Forward Annotation of Initial Values 186
RAM Instantiation with SYNCORE 186
ROM Inference 187

Chapter 7: SynCore IP Tool

SYNCore FIFO Compiler 192
Synchronous FIFO Overview 192
Specifying FIFOs with SYNCore 193
SYNCore FIFO Wizard 197
FIFO Read and Write Operations 207
FIFO Ports 208
FIFO Parameters 211
FIFO Status Flags 213
FIFO Programmable Flags 216
SYNCore RAM Compiler 223
Specifying RAMs with SYNCore 223
SYNCore RAM Wizard 231
Single-Port Memories 235
Dual-Port Memories 237
Read/Write Timing Sequences 242
SYNCore Byte-Enable RAM Compiler 245
Functional Overview 245

© 2022 Synopsys, Inc.

Specifying Byte-Enable RAMs with SYNCore 246
SYNCore Byte-Enable RAM Wizard 252
Read/Write Timing Sequences 255
Parameter List 258

SYNCore ROM Compiler 260
Functional Overview 260
Specifying ROMs with SYNCore 262
SYNCore ROM Wizard 266
Single-Port Read Operation 270
Dual-Port Read Operation 271
Parameter List 271

SYNCore Adder/Subtractor Compiler 274
Functional Description 274
Specifying Adder/Subtractors with SYNCore 275
SYNCore Adder/Subtractor Wizard 282
Adder 286
Subtractor 289
Dynamic Adder/Subtractor 292

SYNCore Counter Compiler 297
Functional Overview 297
Specifying Counters with SYNCore 298
SYNCore Counter Wizard 304
UP Counter Operation 308
Down Counter Operation 308
Dynamic Counter Operation 309

Appendix A: Designing with Pango

Pango Technology Support 314
Supported Device Families 314
Netlist Format 314

Pango Features 315
Register and Latch Support 315
Combinational Logic Support 315
RAM Support 316
Block RAM Support 316
Distributed RAM Support 317
Distributed ROM Support 317
DSP Support 318
Multiplier Support 318
I/O Primitives 318

© 2022 Synopsys, Inc.

Pango Design Options 319
FSM Compiler 319
Resource Sharing 319
Pipelining 320
Retiming 320
Gated and Generated Clocks 320
Pango Device Support 322
Pango Device Mapping Options 322
set_option Command for Pango Architectures 324
Pango Technologies 324
Pango Attribute and Directive Summary 325

SYNOPSYS®
Silicon to Software™

CHAPTER 1

Product Overview

This document is part of a set that includes reference and procedural information for the Synopsys® FPGA synthesis tool. The reference manual provides additional details about the synthesis tool user interface, commands, and features. Use this information to supplement the user guide tasks, procedures, design flows, and result analysis.

The following provide an introduction to the synthesis tool.

Overview of the Synthesis Tool, on page 12

- Graphic User Interface, on page 14
- Getting Help, on page 16

© 2022 Synopsys, Inc.

Overview of the Synthesis Tool

This section introduces the technology, main features, and user interface of the Synplify Pro FPGA synthesis tool. See the following for details:

- Synplify Pro Features, on page 12
- Graphic User Interface, on page 14
- Getting Help, on page 16

Synplify Pro Features

The Synopsys FPGA synthesis tool has the following built-in features:

- The HDL Analyst® analysis and debugging environment, a graphical tool for analysis and crossprobing. See Analyzing With the HDL Analyst Tool, on page 237 and Analyzing With the Standard HDL Analyst Tool, on page 304 in the *User Guide*.
- The Text Editor window, with a language-sensitive editor for writing and editing HDL code. See Text Editor View, on page 42.
- The SCOPE® (Synthesis Constraint OPtimization Environment®) tool, which provides a spreadsheet-like interface for managing timing constraints and design attributes. See SCOPE Constraints Editor, on page 197.
- FSM Compiler, a symbolic compiler that performs advanced finite state machine (FSM) optimizations. See Running the FSM Compiler, on page 399.
- The FSM Viewer, for viewing state transitions in detail.See Using the FSM Viewer, on page 257.
- The Tcl window, a command line interface for running TCL scripts. See Tcl Script Window, on page 38.
- The Timing Analyst window, which allows you to generate timing schematics and reports for specified paths for point-to-point timing analysis.
- Other special windows, or *views*, for analyzing your design, including the Watch Window and Message Viewer (see The Project View, on page 20).

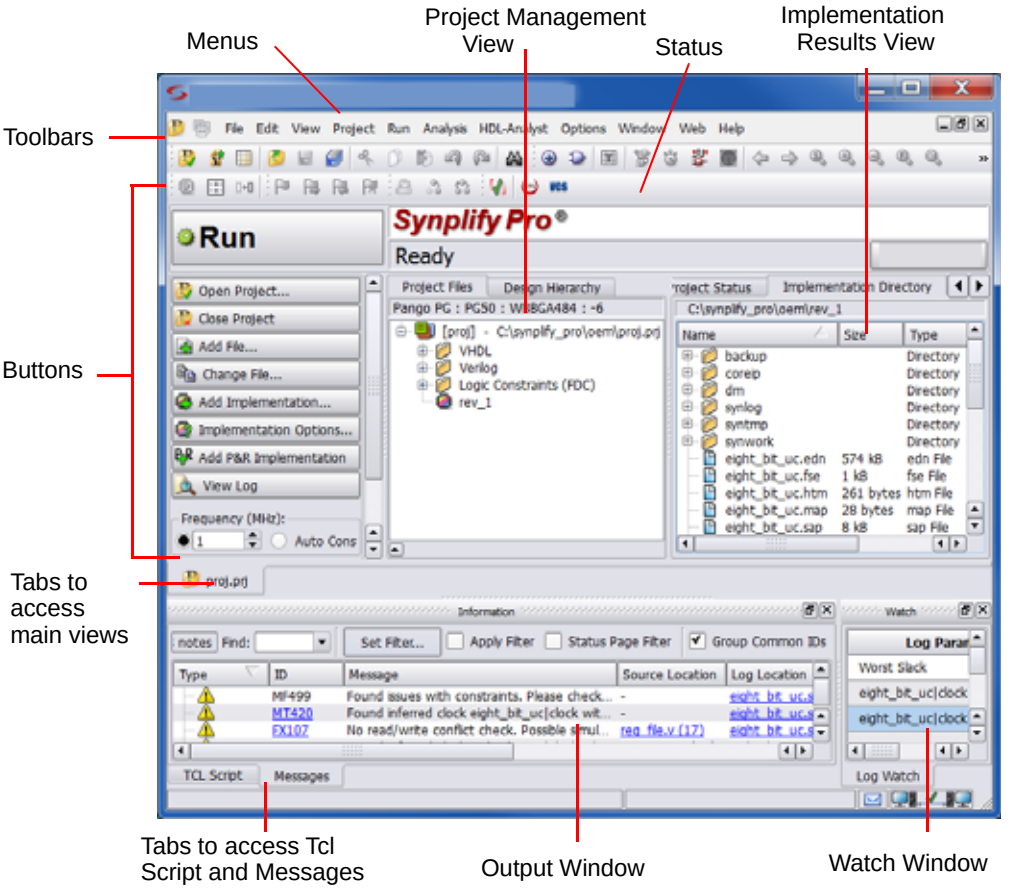
© 2022 Synopsys, Inc.

- Certain optimizations, like pipelining and retiming are only available with these tools.
- Advanced analysis features like crossprobing.

© 2022 Synopsys, Inc.

Graphic User Interface

The following figure shows the graphical user interface for the Synplify Pro tool.



The following table shows where you can find information about different parts of the GUI, some of which are not shown in the above figure. For more information, see the *User Guide*.

For information about ...	See ...
Project window	The Project View, on page 20
HDL Analyst view	Chapter 7, *Analyzing with HDL Analyst*
Text Editor view	Text Editor View, on page 42
Tcl window	Tcl Script Window, on page 38
Watch Window	Watch Window, on page 34
SCOPE spreadsheet	SCOPE Constraints Editor, on page 197
Other views and windows	The Project View, on page 20
Menu commands and their dialog boxes	Chapter 5, *User Interface Commands*
Toolbars	Toolbars, on page 54
Buttons	Buttons and Options, on page 69
Context-sensitive popup menus and their dialog boxes	Chapter 6, *GUI Popup Menu Commands*
Online help	Use the F1 keyboard shortcut or click the Help button in a dialog box. See Help Menu, on page 420, for more information.

© 2022 Synopsys, Inc.

Getting Help

You can access the information online from the Help menu, or refer to the corresponding manual. The following table shows you how the information is organized.

Finding Information

For help with ...	Refer to the ...
How to...	*User Guide*
Flow information	*User Guide*
FPGA Implementation Tools	Synopsys Web Page (Web->FPGA Implementation Tools menu command from within the software)
Synthesis features	*User Guide* and *Reference Manual*
Language and syntax	*Language Support Reference Manual*
Attributes and directives	*Attribute Reference Manual*
Tcl language	Online help (Help->Tcl Help)
Synthesis Tcl commands	*Command Reference Manual* or type help followed by the command name in the Tcl window
Using tool-specific features and attributes	*User Guide*
Error and warning messages	Click the message ID code

© 2022 Synopsys, Inc.

Document Set

This document is part of a series of books included with the Synopsys FPGA synthesis software tool. The set consists of the following books that are packaged with the tool:

- *FPGA Synthesis User Guide*
- *FPGA Synthesis Reference*
- *FPGA Synthesis Command Reference*
- *FPGA Synthesis Attributes and Directives Reference*
- *FPGA Synthesis Language Support Reference*

© 2022 Synopsys, Inc.

© 2022 Synopsys, Inc.

SYNOPSYS®
Silicon to Software™

CHAPTER 2

User Interface Overview

This chapter presents tools and technologies that are built into the Synopsys FPGA synthesis software to enhance your productivity.

This chapter describes the following aspects of the graphical user interface (GUI):

- The Project View, on page 20
- The Project Results View, on page 23
- Other Windows and Views, on page 33
- Using the Mouse, on page 49
- Toolbars, on page 54
- Keyboard Shortcuts, on page 61
- Buttons and Options, on page 69

© 2022 Synopsys, Inc.

The Project View

The Project View is the main interface to the tool. The Project View consists of a Project Management View on the left and a Project Results View on the right. See the following for an overview:

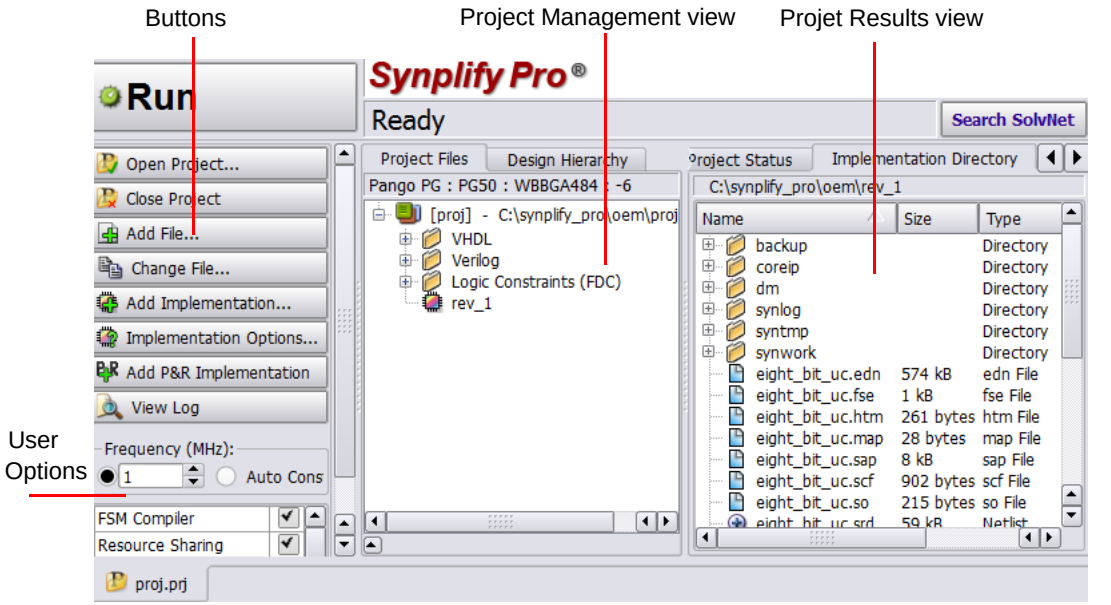
- Multiple Pane Project View, on page 20
- Project Management View, on page 21

Multiple Pane Project View

The Project Management view is on the left side of the window, and is used to create or open projects, create new implementations, set device options, and initiate design synthesis. You can use it to manage and synthesize hierarchical designs. The Project Results view is on the right.

The following figure shows the main parts of the interface. Additional details about the project view are described here:

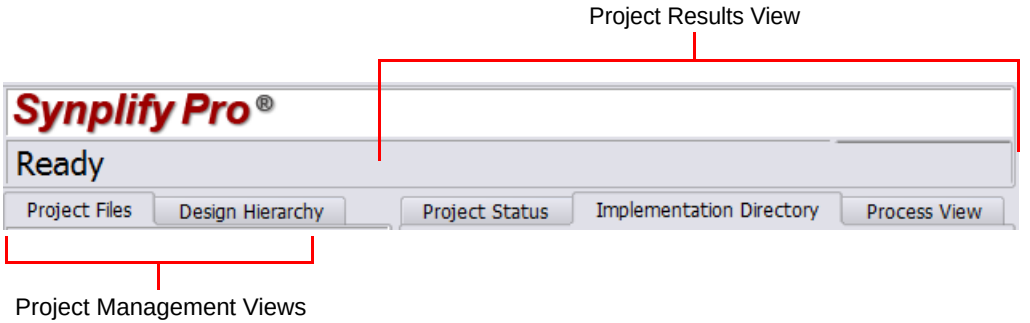
- Project Management View, on page 21
- The Project Results View, on page 23



The Project view has the following main parts:

Project View Interface	Description
Status	Displays the tool name or the current status of the synthesis job that is running. Clicking in this area displays additional information about the current job.
Buttons and options	Allows immediate access to some of the more common commands. See Buttons and Options, on page 69 for details.
Project Management view	Lists the projects and implementations, and their associated HDL source files and constraint files.
Implementation Results view	Lists the result of the synthesis runs for the implementations of your design. You can only view one set of implementation results at a time. Click an implementation in the Project view to make it active and view its result files. The Project Results view includes the following: • Project Status Tab—provides an overview of the project settings and at-a-glance summary of synthesis messages and reports. • Implementation Directory—lists the names and types of the result files, and the dates they were last modified. • Process View—gives you instant visibility to the synthesis flow. See The Project Results View, on page 23 for more information.

Project Management View



© 2022 Synopsys, Inc.

The Project Management view is on the left side of the window, and is used to create or open projects, create new implementations, set device options, and initiate design synthesis. The graphical user interface (GUI) lets you manage hierarchical designs that can be synthesized independently and imported back to the top-level project in a team design flow. The following figure shows the Project view as it appears in the interface.

The Project view contains two tabs with different views of the design that help you manage hierarchical projects, which is not currently supported.

Both tabs in this view have right-click popup menu commands for managing design files and hierarchy. For descriptions of these commands, see Project Management View Popup Folder Commands, on page 432.

© 2022 Synopsys, Inc.

The Project Results View

The Project Results view appears on the right side of the Project view and contains the results of the synthesis runs for the implementations of your design. The Project Results view includes the following:

- Project Status Tab
- Implementation Directory
- Process View
- Report Tab

Project Status Tab

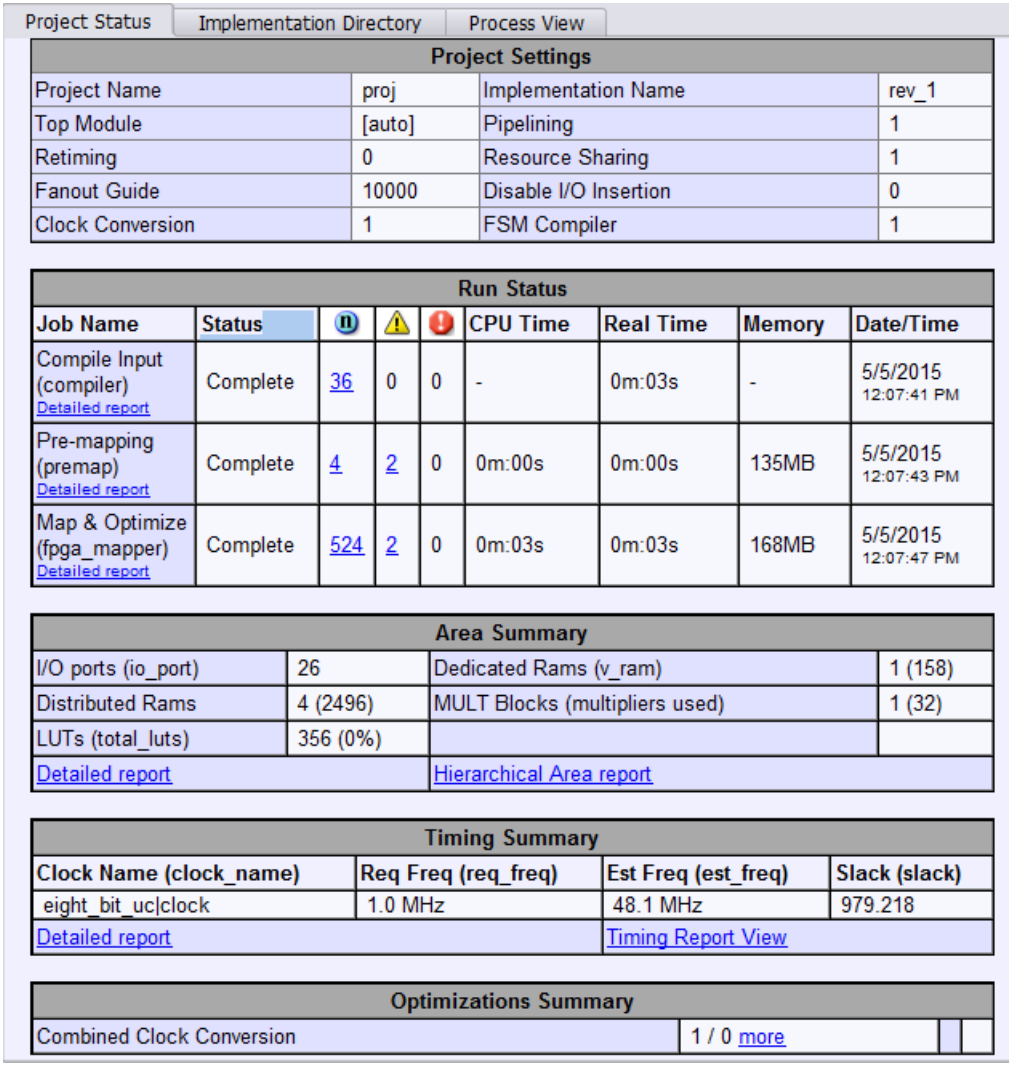
The Project Status view provides an overview of the project settings and at-a-glance summary of synthesis messages and reports such as an area or optimization summary for the active implementation. You can track the status and settings for your design and easily navigate to reports and messages in the Project view.

To display this window, click the Project Status tab in the Project view. An overview for the project is displayed in a spreadsheet format for each of the following sections:

- Project Settings
- Run Status
- Reports

For details about how to access synthesis results, see Accessing Specific Reports Quickly, on page 163.

© 2022 Synopsys, Inc.



Project Status | Implementation Directory | Process View

Project Settings

Project Name	proj	Implementation Name	rev_1
Top Module	[auto]	Pipelining	1
Retiming	0	Resource Sharing	1
Fanout Guide	10000	Disable I/O Insertion	0
Clock Conversion	1	FSM Compiler	1

Run Status

Job Name	Status				CPU Time	Real Time	Memory	Date/Time
Compile Input (compiler) Detailed report	Complete	36	0	0	-	0m:03s	-	5/5/2015 12:07:41 PM
Pre-mapping (premap) Detailed report	Complete	4	2	0	0m:00s	0m:00s	135MB	5/5/2015 12:07:43 PM
Map & Optimize (fpga_mapper) Detailed report	Complete	524	2	0	0m:03s	0m:03s	168MB	5/5/2015 12:07:47 PM

Area Summary

I/O ports (io_port)	26	Dedicated Rams (v_ram)	1 (158)
Distributed Rams	4 (2496)	MULT Blocks (multipliers used)	1 (32)
LUTs (total_luts)	356 (0%)		
Detailed report		Hierarchical Area report	

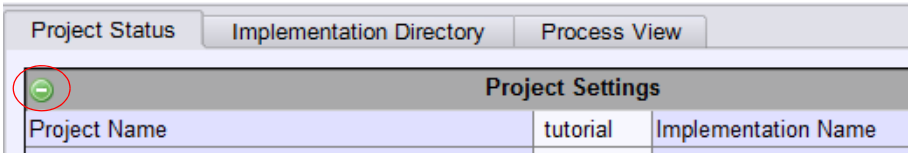
Timing Summary

Clock Name (clock_name)	Req Freq (req_freq)	Est Freq (est_freq)	Slack (slack)
eight_bit_uc|clock	1.0 MHz	48.1 MHz	979.218
Detailed report		Timing Report View	

Optimizations Summary

Combined Clock Conversion	1 / 0 more

You can expand or collapse each section of the Project Status view by clicking on the + or - icon in the upper left-corner of each section.



Project Settings

Project Settings is populated with the project settings from the run_options.txt file after a synthesis run. This section displays information, like the following:

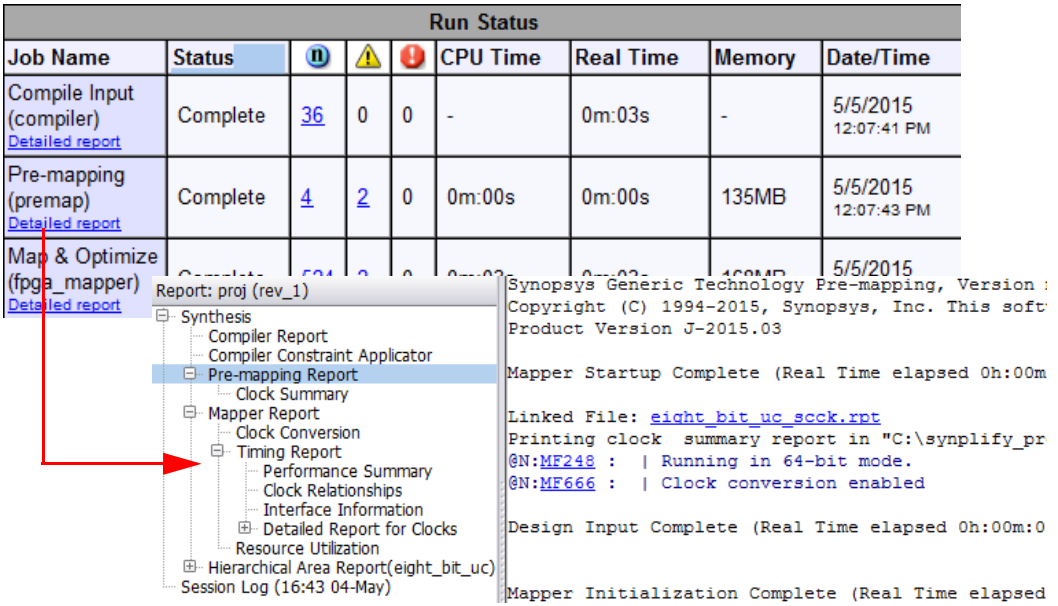
- Project name, top-level module, and implementation name
- Project options currently specified, such as Pipelining, Retiming, Resource Sharing, Fanout Guide, and Disable I/O Insertion.

Run Status

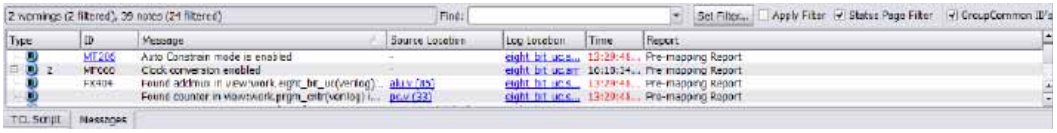
The Run Status table gets updated during and after a synthesis run. This section displays job status information for the compiler, premap job, mapper, and place-and-route runs, as needed. This section displays information about the synthesis run:

- Job name - Jobs include Compiler Input, Premap, and Map & Optimize. The job might have a Detailed Report link. When you click this link, it takes you to the corresponding report in the log file.

© 2022 Synopsys, Inc.



- Status - Reports whether the job is running or completed.
- Notes, Warnings, and Errors – These columns are headed by the respective icons and display the number of messages. The messages themselves are displayed in the Messages tab, beside the TCL Script tab. Links are available to the error message and the log location.



The message numbers may not match for designs with compile points. The numbers reflect the top-level design.

- Real and CPU times, peak memory, and a timestamp.

Reports

The mapper summary table generates various reports such as

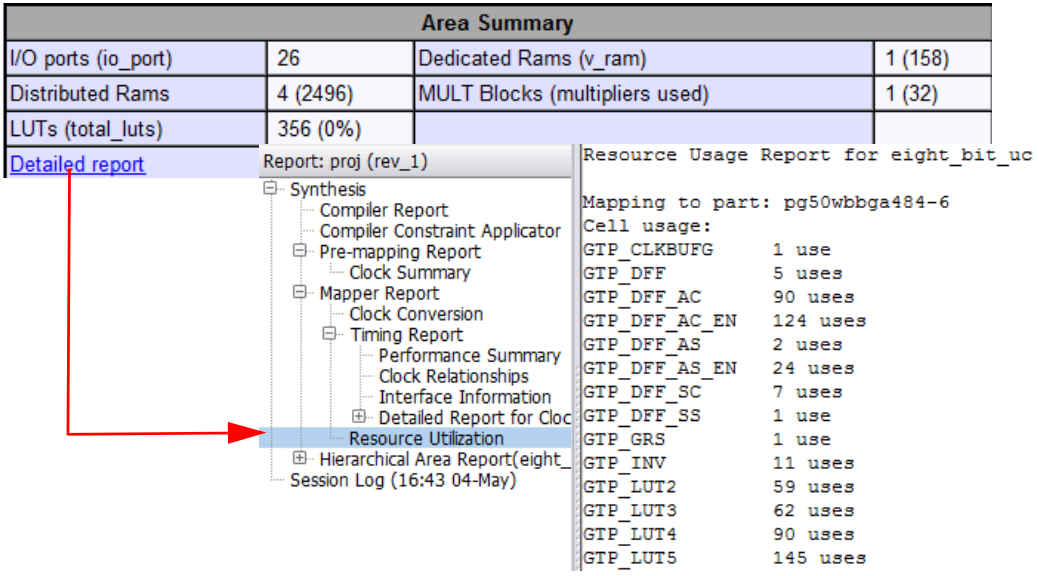
- Area Summary
- Compile Point Summary

- Optimization Summary

Click the Detailed Report link when applicable, to go to the log file and information about the selected report. These reports are written to the synlog folder for the active implementation.

Area Summary

For example, the Area Summary contains a resource usage count for components such as registers, LUTs, and I/O ports in the design. Click the Detailed report link to display the usage count information in the design for this report.



Report Tab

Some reporting such as the Hierarchical Area Report are written to the Report tab of the Project Results view. These reports are typically not included in the log file; therefore, they are displayed separately.

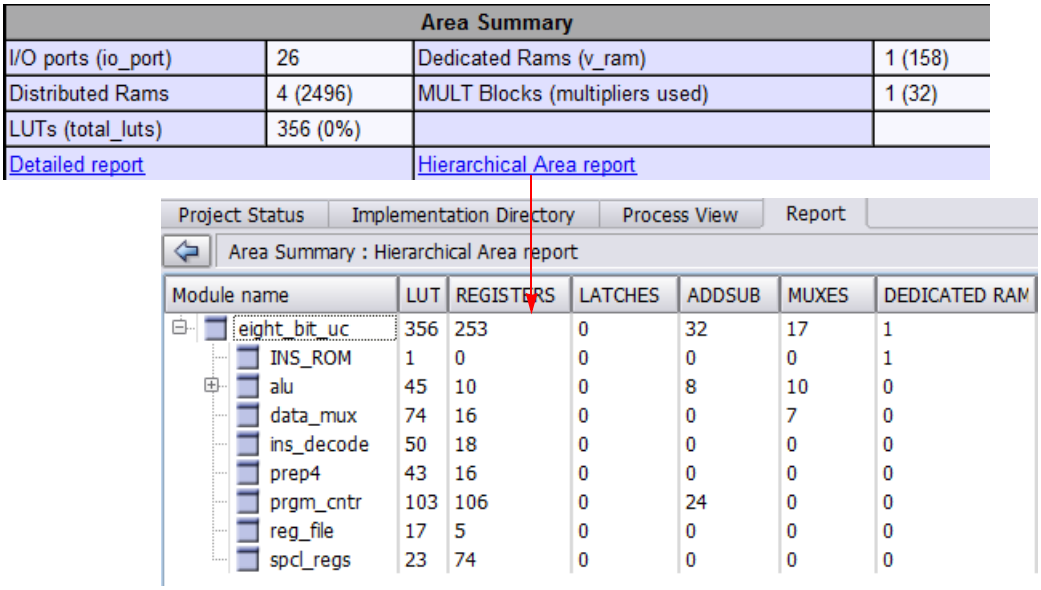
Hierarchical Area Report

A Hierarchical Area report is generated in a Report tab that you can access from the Project Status view. This report generates area usage for components such as sequential and combinational logic, RAM, and DSP blocks.

© 2022 Synopsys, Inc.

You can locate the Hierarchical Area report file in the following Implementation Directory: `/synlog/report`.

Use the arrow icon ( ) to return to the main Project Status view.



Area Summary			
I/O ports (io_port)	26	Dedicated Rams (v_ram)	1 (158)
Distributed Rams	4 (2496)	MULT Blocks (multipliers used)	1 (32)
LUTs (total_luts)	356 (0%)		
Detailed report		Hierarchical Area report	

Project Status | Implementation Directory | Process View | Report

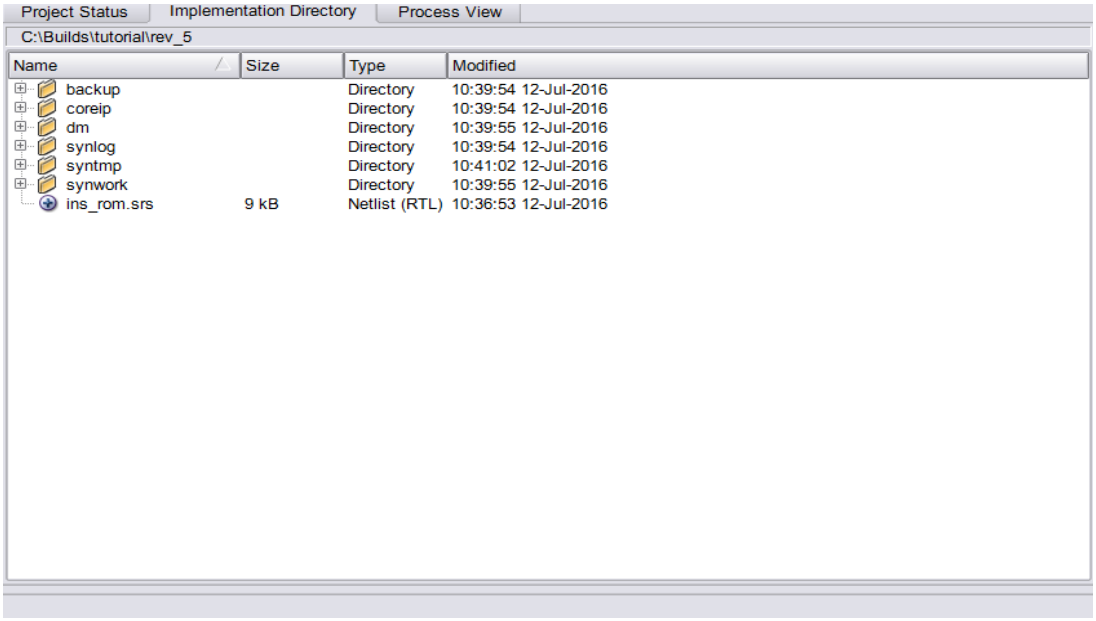
Area Summary : Hierarchical Area report

Module name	LUT	REGISTERS	LATCHES	ADDSUB	MUXES	DEDICATED RAM
eight_bit_uc	356	253	0	32	17	1
INS_ROM	1	0	0	0	0	1
alu	45	10	0	8	10	0
data_mux	74	16	0	0	7	0
ins_decode	50	18	0	0	0	0
prep4	43	16	0	0	0	0
prgm_cntr	103	106	0	24	0	0
reg_file	17	5	0	0	0	0
spcl_regs	23	74	0	0	0	0

Implementation Directory

An implementation is one version of a project, run with certain parameter or option settings. You can synthesize again, with a different set of options, to get a different implementation. In the Project view, an implementation is shown in the folder of its project; the active implementation is highlighted. You can display multiple implementations in the same Project view. The output files generated for the active implementation are displayed in the Implementation Directory.

© 2022 Synopsys, Inc.

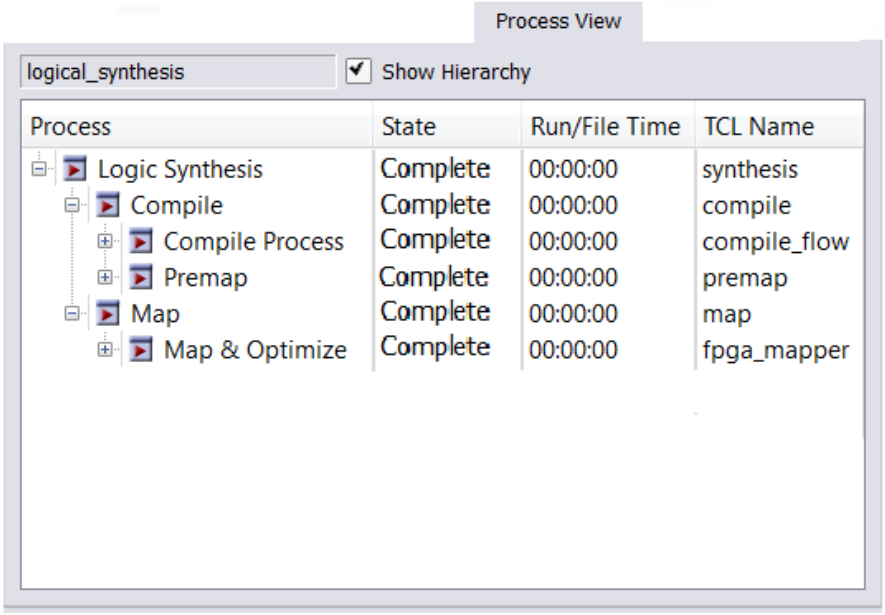


© 2022 Synopsys, Inc.

Process View

As process flow jobs become more complex, the benefits of exposing the underlying job flow is extremely valuable. The Process View gives you this visibility to track the design progress for the synthesis and place-and-route job flows.

Click the Process View tab on the right side of the Project Results view. This displays the job flow hierarchy run on the active implementation and is a function of this current implementation and its project settings.



Process View Displays and Controls

The Process View shows the current state of a job and allows you to control the run. You can see various aspects of the synthesis process flow, such as logical synthesis, premap, map, and placement. If you run place and route, you can see its job processes as well.

Appropriate jobs of the process flow contains the following information:

- Job Input and Output Files
- Completion State

  Displays if the job generated an error, warning, or was canceled.
- Job State
  - Out-of-date – Job needs to be run.
  - Running – Job is active.
  - Complete – Job has completed and is up-to-date.
  - Complete * – Job is up-to-date, so the job is skipped.
- Run/File Time – Job process flow runtime in real time or file creation date timestamp.
- Job TCL Command – Job process name.

Each job has the following control commands that allows you to run jobs at any stage of the design process, for example map. Right-click any job icon and select one of the following commands from the popup menu:

- Cancel *jobProcess* that is running
- Disable *jobProcess* that you do not want to run
- Run this *jobProcess* only
- Run to this *jobProcess* from the beginning of run
- Run from this *jobProcess* to the end of run

© 2022 Synopsys, Inc.

Hierarchical Job Flows

A hierarchical job flow runs two or more subordinate jobs. Primitive jobs launch an executable, but have no subordinate jobs. The Logical Synthesis flow is a hierarchical job that runs the Compile and Map flows.

The state of a hierarchical job depends on the state of its subordinate jobs.

- If a subordinate job is out-of-date, then its parent job is out-of-date.
- If a subordinate job has an error, then its parent job terminates with this error.
- If a subordinate job has been canceled, then its parent job is canceled as well.
- If a subordinate job is running, then its parent job is also running.

The Process View is a hierarchical tree view. To collapse or expand the main hierarchical tree, enable or disable the Show Hierarchy option. Use the plus or minus icon to expand or collapse each process flow to show the details of the jobs. The icons below are used to show the information for the state of each process:

- Red arrow ( ) – Job is out-of-date and needs to be rerun.
- Green arrow () – Job is up-to-date.
- Red Circle with! ( ) - Job encountered an error.

© 2022 Synopsys, Inc.

Other Windows and Views

Besides the Project view, the tool provides other windows and views that help you manage input and output files, direct the synthesis process, and analyze your design and its results. The following windows and views are described here:

- Dockable GUI Entities, on page 34
- Watch Window, on page 34
- Tcl Script and Messages Windows, on page 37
- Tcl Script Window, on page 38
- Message Viewer, on page 38
- Output Windows (Tcl Script and Watch Windows), on page 42
- Text Editor View, on page 42
- Context Help Editor Window, on page 45
- Interactive Attribute Examples, on page 47

See the following for descriptions of other views and windows that are not covered here:

Project view	The Project View, on page 20
SCOPE	SCOPE Tabs, on page 198

© 2022 Synopsys, Inc.

Dockable GUI Entities

Some of the main GUI entities can appear as either independent windows or docked elements of the main application window. These entities include the menu bar, Watch window, Tcl window, and various toolbars (see the description of each entity for details). Docked elements function effectively as *panes* of the application window; you can drag the border between two such panes to adjust their relative areas.

Watch Window

The Watch window displays selected information from the log file (see Log File, on page 145) as a spreadsheet of parameters that you select to monitor. The values are updated when synthesis finishes.

Watch Window Display

Display of the Watch window is controlled by the View ->Watch Window command. By default, the Watch window is below the Project view in the lower right corner of the main application window.

To access the Watch window configuration menu, right-click in any cell. Select Configure Watch to display the Log Watch Configuration dialog box.



In the Watch window, indicate which implementations to watch under Watch Selection. The selected implementation(s) will display in the Watch window.

You can move the Watch window anywhere on the screen; you can make it float in its own window (named Watch Window) or dock it at a docking area (an edge) of the application window. Double-click in the banner to toggle between docked and floating.

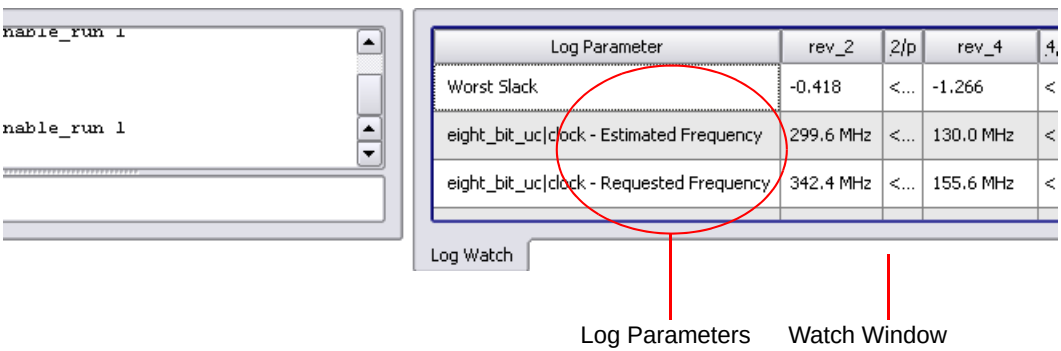
The Watch window has a special positioning popup menu that you access by right-clicking the window border. The following commands are in the menu:

Command	Description
Allow Docking	A toggle: when enabled, the window can be docked.
Hide	Hides the window; use View ->Watch Window to show it again.
Float in Main Window	A toggle: when enabled, the window is floated (undocked).

Right-clicking the window *title bar* when the Watch window is floating displays an alternative popup menu with commands Hide and Move; Move lets you position the window using either the arrow keys or the mouse.

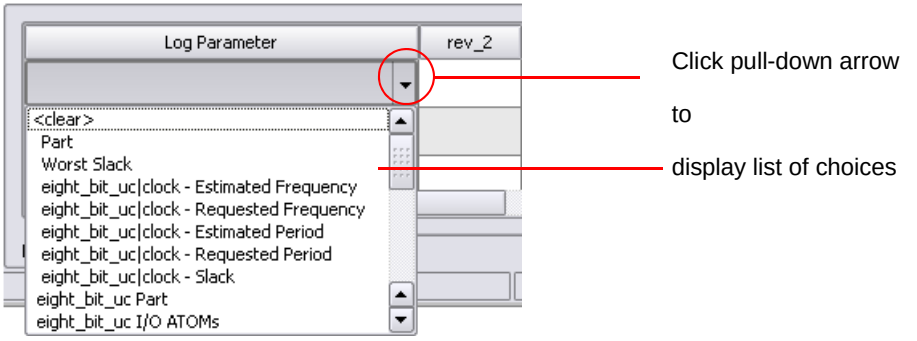
Using the Watch Window

You can view and compare the results of multiple implementations in the Watch window.



© 2022 Synopsys, Inc.

To choose log parameters from a pull-down menu, click in the Log Parameter section of the window. Click the pull-down arrow that appears to display the parameter list choices:

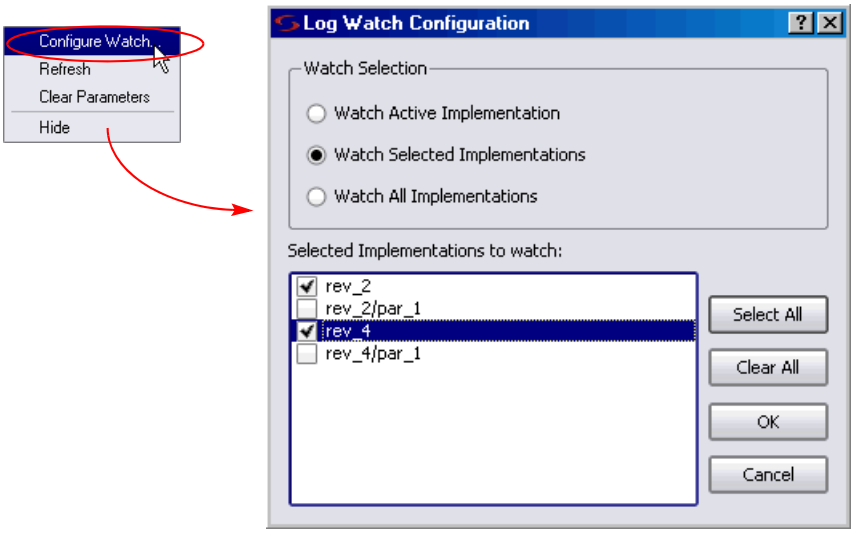


The Watch window creates an entry for each implementation of a project:

Log Parameter	rev_2	rev_4
Worst Slack	-0.418	-1.266
eight_bit_uc|clock - Estimated Frequency	299.6 MHz	130.0 MHz
eight_bit_uc|clock - Requested Frequency	342.4 MHz	155.6 MHz

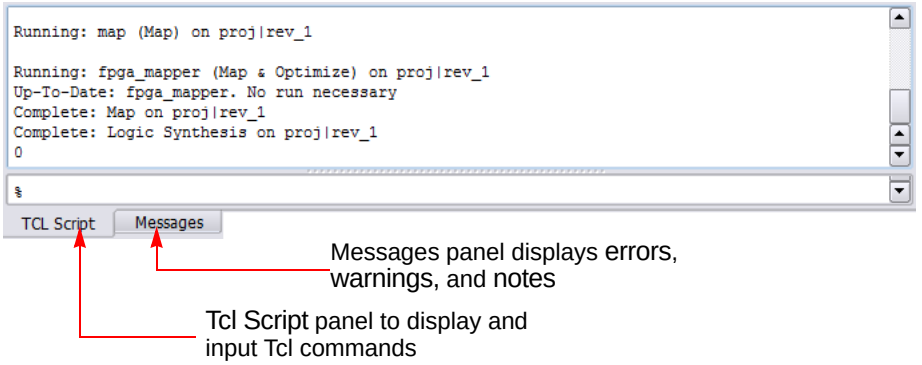
To choose the implementations to watch, use the Log Watch Configuration dialog box. To display this box, right-click in the Watch window, then choose Configure Watch in the popup menu. Enable Watch Selected Implementations, then choose the implementations you want to watch in the list Selected Implementations to watch. The other buttons let you watch only the active implementation or all implementations.

© 2022 Synopsys, Inc.



Tcl Script and Messages Windows

The Tcl window has tabs for the Tcl Script and Messages windows. By default, the Tcl windows are located below the Project Tree view in the lower left corner of the main application window.



You can float the Tcl windows by clicking on a window edge while holding the Ctrl or Shift key. You can then drag the window to float it anywhere on the screen or dock it at an edge of the application window. Double-click in the banner to toggle between docked and floating.

© 2022 Synopsys, Inc.

Right-clicking the Tcl windows *title bar* when the window is floating displays a popup menu with commands Hide and Move. Hide removes the window (use View ->Tcl Window to redisplay the window). Move lets you position the window using either the arrow keys or the mouse.

For more information about the Tcl windows, see Tcl Script Window, on page 38 and Message Viewer, on page 38.

Tcl Script Window

The Tcl Script window is an interactive command shell that implements the Tcl command-line interface. You can type or paste Tcl commands at the prompt ("% "). For a list of the available commands, type "`help *`" (without the quotes) at the prompt. For general information about Tcl syntax, choose Help ->TCL.

The Tcl script window also displays each command executed in the course of running the synthesis tool, regardless of whether it was initiated from a menu, button, or keyboard shortcut. Right-clicking inside the Tcl window displays a popup menu with the Copy, Paste, Hide, and Help commands.

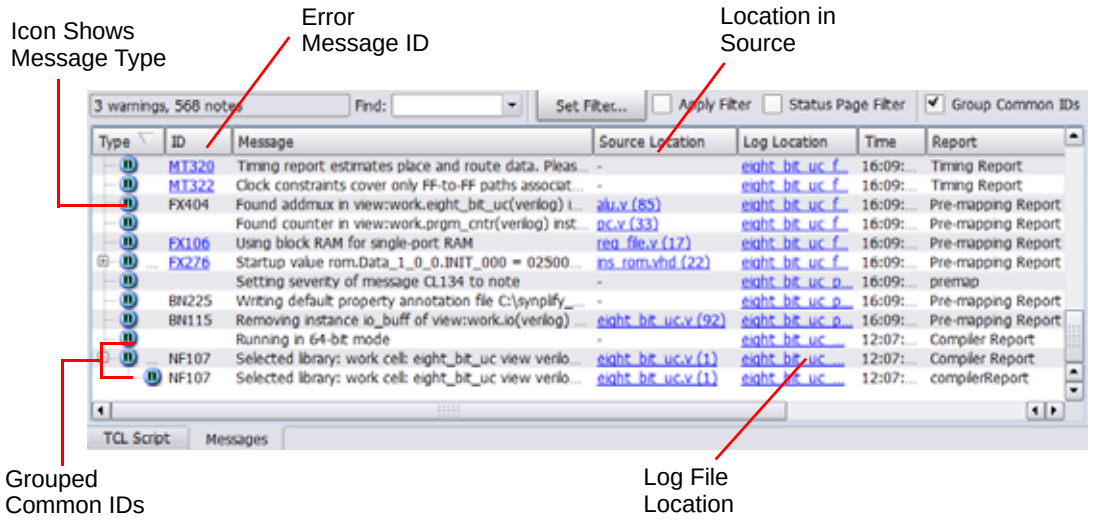
See also

- Tcl Synthesis Commands, on page 17, for information about the Tcl synthesis commands.
- Generating a Job Script, on page 455 in the *User Guide*.

Message Viewer

To display errors, warnings, and notes after running the synthesis tool, click the Messages tab in the Tcl Window. A spreadsheet-style interactive interface appears.

© 2022 Synopsys, Inc.



In the Messages panel, you can:

- Drag the pane divider with the mouse to change the relative column size.
- Click the ID entry to open online help for the error, warning, or note.
- Click a Source Location entry to go to the section of code in the source HDL file that is causing the message.
- Click a Log Location entry to go to its location in the log file.

The following table describes the contents of the Messages panel. You can sort the messages by clicking the column headers. For further sorting, use Find and Filter. For details about using this window, see Checking Results in the Message Viewer, on page 175 in the *User Guide*.

Item	Description
Find	Type into this field to find errors, warnings, or notes.
Filter	Opens the Warning Filter dialog box. See Messages Filter, on page 41.
Apply Filter	Enable/disable the last saved filter.

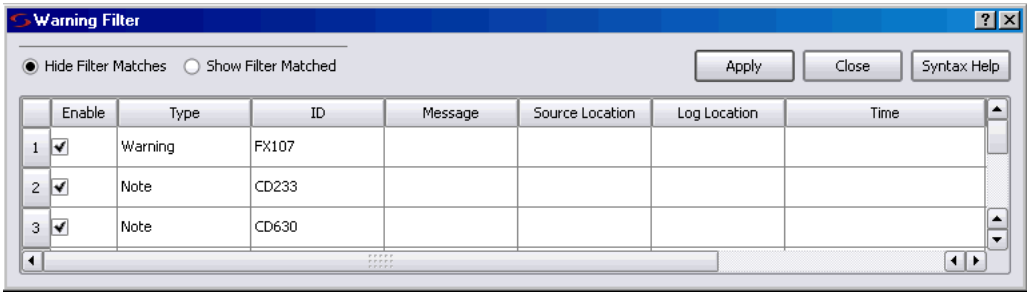
© 2022 Synopsys, Inc.

Item	Description
Group Common ID's	Enable/disable grouping of repeated messages. Groups are indicated by a number next to the type icon. There are two types of groups: • The same warning or note ID appears in multiple source files indicated by a dash in the source files column. • Multiple warnings or notes in the same line of source code indicated by a bracketed number.
Type	The icons indicate the type of message: Error Warning Note Advisory A plus sign next to an icon indicates that repeated messages are grouped together. Click the plus sign to expand and view the various occurrences of the message.
ID	This is the message ID. You can select an underlined ID to launch help on the message.
Message	The error, warning, or note message text.
Source Location	The HDL source file that generated the error, warning, or note message.
Log Location	The location of the error, warning, or note message in the log file.
Time	The time that the error, warning, or note message was recorded in the log file for the various stages of synthesis (for example: compiler, premap, and map). If you rerun synthesis, only new messages generate a new timestamp for this session. **Note:** Once synthesis has run to completion, all the `srr` files for the different stages of synthesis are merged into one unified `srr` file. If you exit the GUI, these timestamps remain the same when you re-open the same project in the GUI again.
Report	Indicates which section of the Log File report the error appears, for example Compiler or Mapper.

© 2022 Synopsys, Inc.

Messages Filter

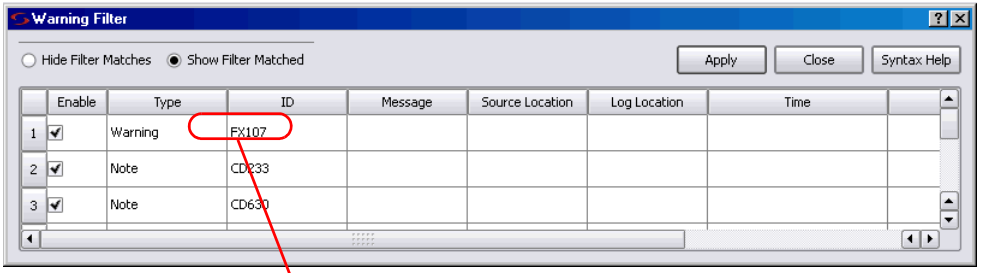
You filter which errors, warnings, and notes appear in the Messages panel of the Tcl Window using match criteria for each field. The selections are combined to produce the result. You can elect to hide or show the warnings that match the criteria you set. See Checking Results in the Message Viewer, on page 175 in the *User Guide*.



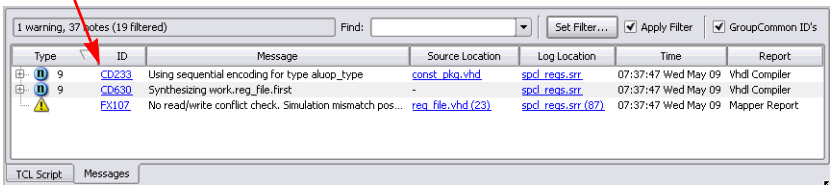
Item	Description
Hide Filter Matches	Hides matched criteria in the Messages Panel.
Show Filter Matches	Shows matched criteria in the Messages Panel.
Syntax Help	Gives quick syntax descriptions.
Apply	Applies the filter criteria to the Messages Panel report, without closing the window.
Type, ID, Message, Source Location, Log Location, Time, Report	Log file report criteria to use when filtering.

The following is a filtering example.

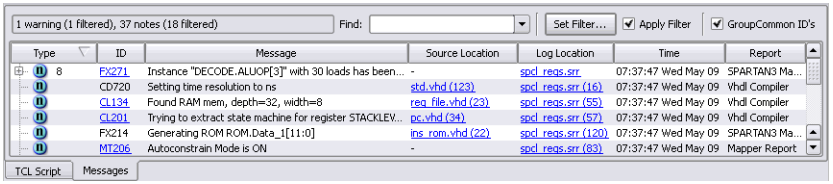
© 2022 Synopsys, Inc.



Show Filter Matches



Hide Filter Matches



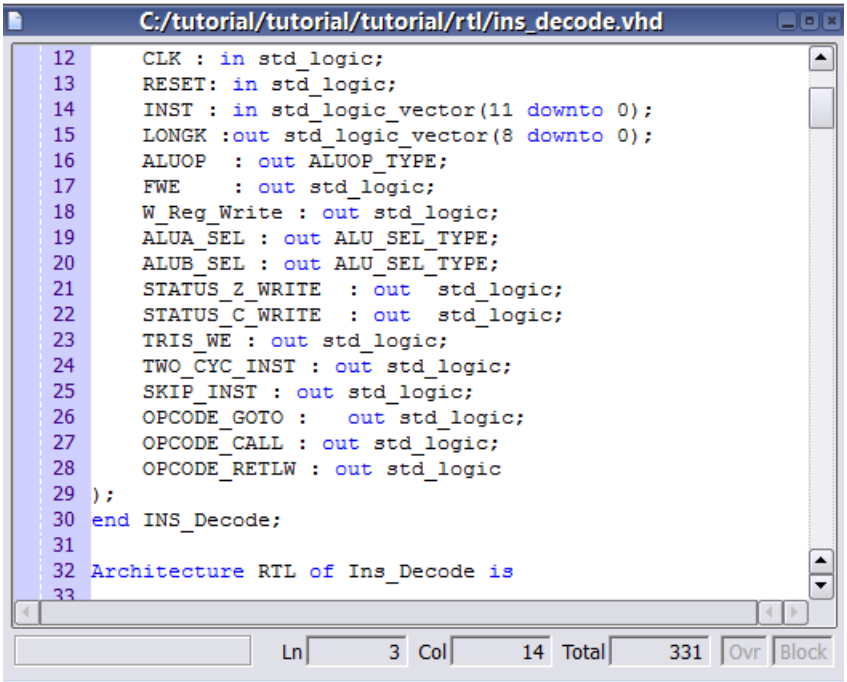
Output Windows (Tcl Script and Watch Windows)

The Output windows are the Tcl Script and Log Watch windows. To display or hide them, use View->Output Windows from the main menu. Refer to Watch Window, on page 34 and Tcl Script and Messages Windows, on page 37 for more information.

Text Editor View

The Text Editor view displays text files. These can be constraint files, source code files, or other informational or report files. You can enter and edit text in the window. You use this window to update source code and fix syntax or synthesis errors. You can also use it to crossprobe the design. For information about using the Text Editor, see Editing HDL Source Files with the Built-in Text Editor, on page 37 in the *User Guide*.

© 2022 Synopsys, Inc.



Opening the Text Editor

To open the Text Editor to edit an existing file, do one of the following:

- Double-click a source code file (v or vhd) in the Project view.
- Choose File ->Open. In the dialog box displayed, double-click a file to open it.

  With the Microsoft® Windows® operating system, you can instead drag and drop a source file from a Windows folder into the gray background area of the GUI (*not* into any particular view).

To open the Text Editor on a new file, do one of the following:

- Choose File ->New, then specify the kind of text file you want to create.
- Click the HDL icon ( ) to create and edit an HDL source file.

The Text Editor colors HDL source code keywords such as module and output blue and comments green.

© 2022 Synopsys, Inc.

Text Editor Features

The Text Editor has the features listed in the following table.

Feature	Description
Color coding	Keywords are blue, comments green, and strings red. All other text is black.
Editing text	You can use the Edit menu or keyboard shortcuts for basic editing operations like Cut, Copy, Paste, Find, Replace, and Goto.
Completing keywords	To complete a keyword, type enough characters to make the string unique and then press the Esc key.
Indenting a block of text	The Tab key indents a selected block of text to the right. Shift-Tab indents text to the left.
Inserting a bookmark	Click the line you want to bookmark. Choose Edit ->Toggle Bookmark, type Ctrl-F2, or click the Toggle Bookmark icon () on the Edit toolbar. The line number is highlighted to indicate that there is a bookmark at the beginning of the line.
Deleting a bookmark	Click the line with the bookmark. Choose Edit ->Toggle Bookmark, type Ctrl-F2, or click the Toggle Bookmark icon () on the Edit toolbar.
Deleting all bookmarks	Choose Edit ->Delete all Bookmarks, type Ctrl-Shift-F2, or click the Clear All Bookmarks icon () on the Edit toolbar.
Editing columns	Press and hold Alt, then drag the mouse down a column of text to select it.
Commenting out code	Choose Edit ->Advanced ->Comment Code. The rest of the current line is commented out: the appropriate comment prefix is inserted at the current text cursor position.
Checking syntax	Use Run ->Syntax Check to highlight syntax errors, such as incorrect keywords and punctuation, in source code. If the active window shows an HDL file, then only that file is checked. Otherwise, the entire project is checked.
Checking synthesis	Use Run ->Synthesis Check to highlight hardware-related errors in source code, like incorrectly coded flip-flops. If the active window shows an HDL file, then only that file is checked. Otherwise, the entire project is checked.

© 2022 Synopsys, Inc.

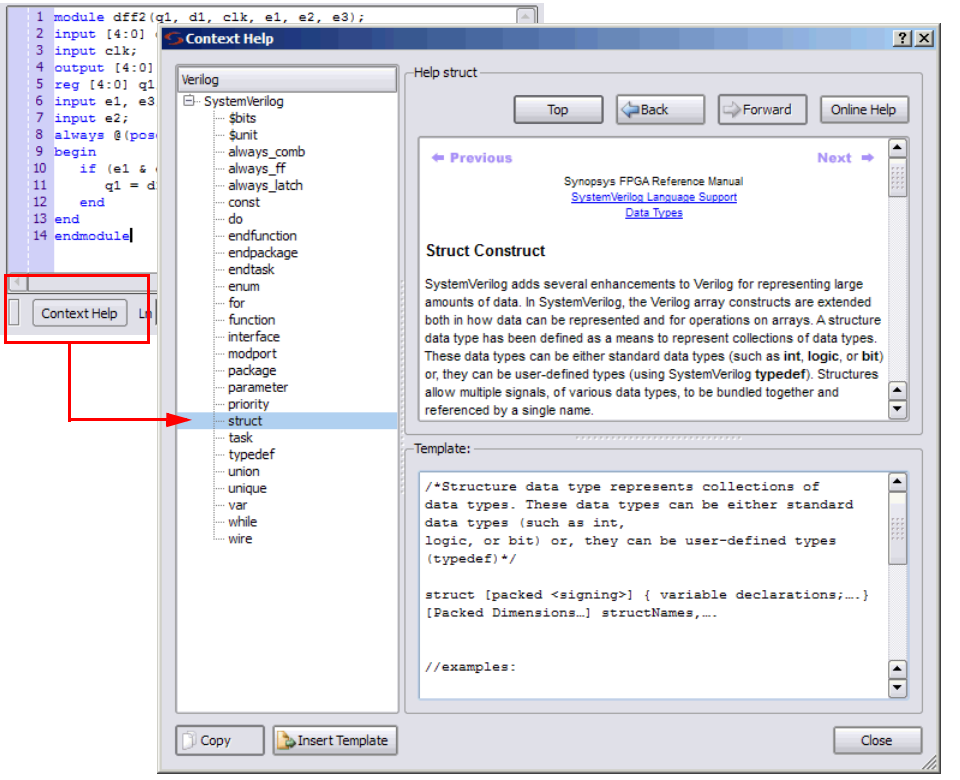
See also:

- Editor Options Command, on page 403, for information on setting Text Editor preferences.
- File Menu, on page 286, for information on printing setup operations.
- Edit Menu Commands for the Text Editor, on page 292, for information on Text Editor editing commands.
- Text Editor Popup Menu, on page 423, for information on the Text Editor popup menu.
- Text Editor Toolbar, on page 58, for information on bookmark icons of the Edit toolbar.
- Keyboard Shortcuts, on page 61, for information on keyboard shortcuts that can be used in the Text Editor.

Context Help Editor Window

Use the Context Help button to copy Verilog, SystemVerilog, or VHDL constructs into your source file or Tcl constraint commands into your Tcl file. When you load a Verilog/SystemVerilog/VHDL file or Tcl file into the UI, the Context Help button displays at the bottom of the window. Click this button to display the Context Help Editor.

© 2022 Synopsys, Inc.



When you select a construct in the left-side of the window, the online help description for the construct is displayed. If the selected construct has this feature enabled, the online help topic is displayed on the top of the window and a generic code or command template for that construct is displayed at the bottom. The Insert Template button is also enabled. When you click the Insert Template button, the code or command shown in the template window is inserted into your file at the location of the cursor. This allows you to easily insert the code or constraint command and modify it for the design that you are going to synthesize. If you want to copy only parts of the template, select the code or constraint command you want to insert and click Copy. You can then paste it into your file.

© 2022 Synopsys, Inc.

Field/Option	Description
Top	Takes you to the top of the context help page for the selected construct.
Back	Takes you back to the last context help page previously viewed.
Forward	Once you have gone back to a context help page, use Forward to return to the original context help page from where you started.
Online Help	Brings up the interactive online help for the synthesis tool.
Copy	Allows you to copy selected code from the Template file and paste it into the editor file.
Insert Template	Automatically copies the code description in its entirety from the Template file to the editor file.

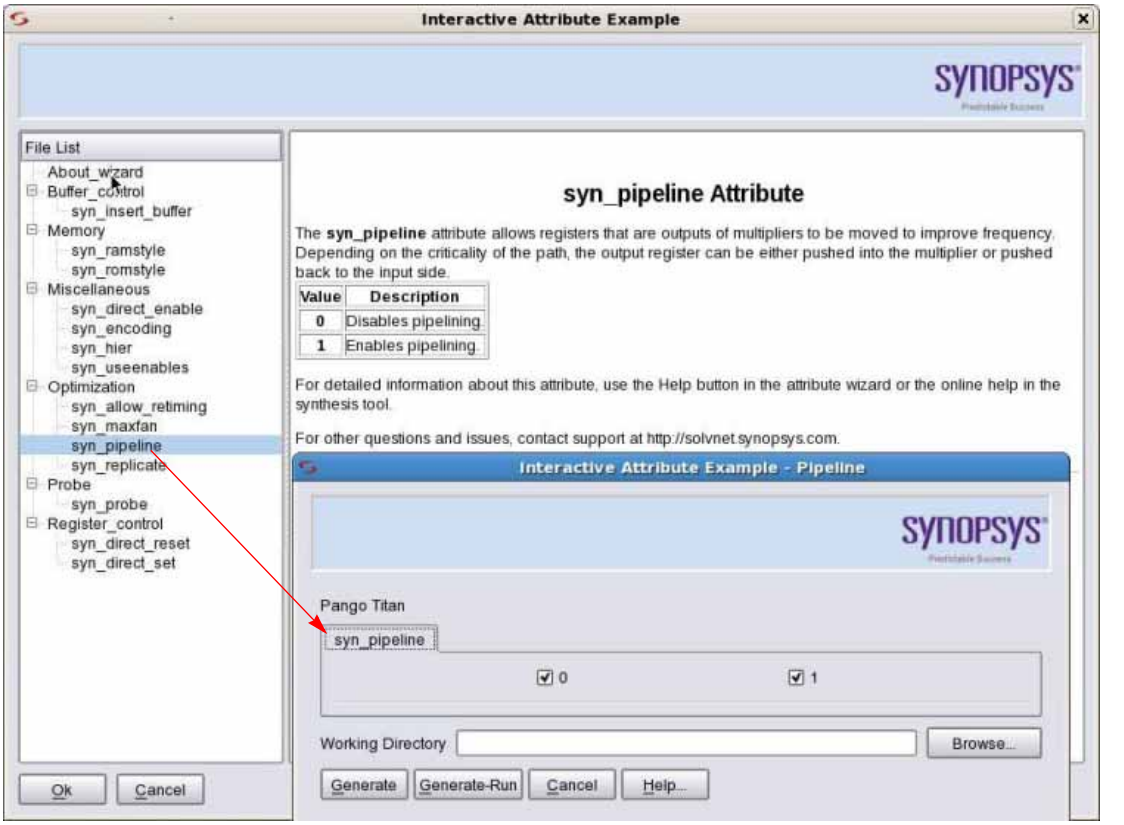
Interactive Attribute Examples

The Interactive Attribute Examples wizard lets you select pre-defined attributes to run in a project. To use this tool:

1. Launch the wizard from Help->Interactive Attribute Examples in the Project view.
2. Double-click an attribute to start the wizard.
3. Specify the Working Directory location to write your project.
4. Click Generate to generate a project for your attribute.

   A project will be created with an implementation for each attribute value selected.

© 2022 Synopsys, Inc.



Double-click on attribute to bring up the Interactive Attribute wizard

5. Click Generate Run to run synthesis for all the implementations. When synthesis completes:
   - The Technology view opens to show how the selected attribute impacts synthesis.
   - You can compare resource utilization and timing information between implementations in the Log Watch window.

Using the Mouse

The mouse button operations in Synopsys FPGA products are standard; refer to Mouse Operation Terminology for a summary of supported functions. The Synopsys FPGA tools also provide support for:

- Using Mouse Strokes, on page 50
- Using the Mouse Buttons, on page 51
- Using the Mouse Wheel, on page 53

Mouse Operation Terminology

The following terminology is used to refer to mouse operations:

Term	Meaning
Click	Click with the *left* mouse button: press then release it without moving the mouse.
Double-click	Click the left mouse button twice rapidly, without moving the mouse.
Right-click	Click with the right mouse button.
Drag	Press the left mouse button, hold it down while moving the mouse, then release it. Dragging an object moves the object to where the mouse is released; then, releasing is sometimes called *dropping*. Dragging initiated when the mouse is not over an object often traces a selection rectangle, whose diagonal corners are at the press and release positions.
Press	Depress a mouse button; unless otherwise indicated, the left button is implied. It is sometimes used as an abbreviation for *press and hold*.
Hold	Keep a mouse button depressed. It is sometimes used as an abbreviation for *press and hold*.
Release	Stop holding a mouse button depressed.

© 2022 Synopsys, Inc.

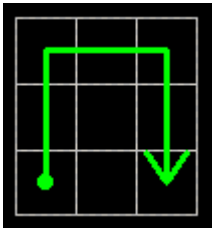
Using Mouse Strokes

Mouse strokes are used to quickly perform simple repetitive commands. Mouse strokes are drawn by pressing and holding the right mouse button as you draw the pattern. The stroke must be at least 16 pixels in width or height to be recognized. You will see a green mouse trail as you draw the stroke (the actual color depends on the window background color).

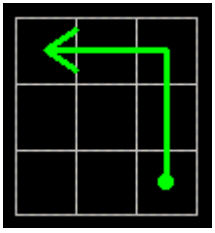
Some strokes are context sensitive. That is, the interpretation of the stroke depends upon the window in which the stroke is started. For example, in an Analyst view, the right stroke means *Next Sheet*. In a dialog box, the right stroke means *OK*.

For information on each of the available mouse strokes, consult the Mouse Stroke Tutor.

The strokes you draw are interpreted on a grid of one to three rows. Some strokes are similar, differing only in the number of columns or rows, so it may take a little practice to draw them correctly. For example, the strokes for Redo and Back differ in that the Redo stroke is back and forth horizontally, within a single-row grid, while the Back stroke involves vertical movement as well.



Redo Last Operation

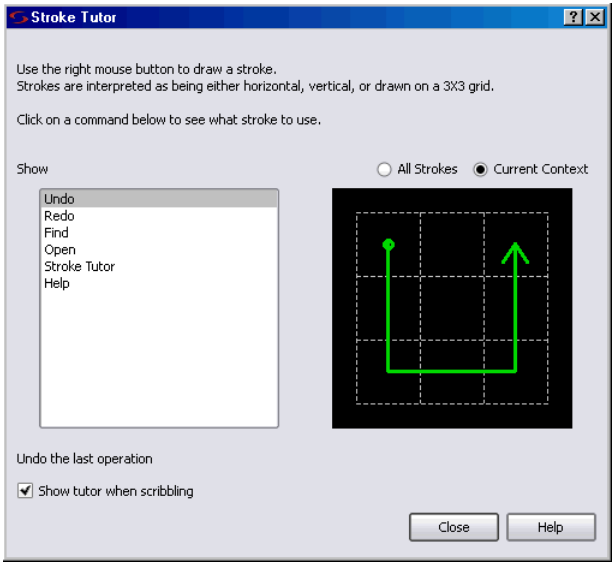


Back to Previous View

The Mouse Stroke Tutor

Do one of the following to access the Mouse Stroke Tutor:

- Help->Stroke Tutor
- Draw a question mark stroke ("?")
- Scribble (Show tutor when scribbling must be enabled on the Stroke Help dialog box)



The tutor displays the available strokes along with a description and a diagram of the stroke. You can draw strokes while the tutor is displayed.

Mouse strokes are context sensitive. When viewing the Stroke Tutor, you can choose All Strokes or Current Context to view just the strokes that apply to the context of where you invoked the tutor. For example, if you draw the "?" stroke in an Analyst window, the Current Context option in the tutor shows only those strokes recognized in the Analyst window.

You can display the tutor while working in a window such as the Analyst RTL view. However you cannot display the tutor while a modal dialog is displayed, as input is restricted to the modal dialog.

Using the Mouse Buttons

The operations you can perform using mouse buttons include the following:

- You select an object by clicking it. You deselect a selected object by clicking it. Selecting an object by clicking it deselects all previously selected objects.
- You can select and deselect multiple objects by pressing and holding the Control key (Ctrl) while clicking each of the objects.

© 2022 Synopsys, Inc.

- You can select a range of objects in a Hierarchy Browser, as follows:
  - select the first object in the range
  - scroll the tree of objects, if necessary, to display the last object in the range
  - press and hold the Shift key while clicking the last object in the range

  Selecting a range of objects in a Hierarchy Browser crossprobes to the corresponding schematic, where the same objects are automatically selected.
- You can select all of the objects in a region by tracing a selection rectangle around them (lassoing).
- You can select text by dragging the mouse over it. You can alternatively select text containing no white space (such as spaces) by double-clicking it.
- Double-clicking sometimes selects an object and immediately initiates a default action associated with it. For example, double-clicking a source file in the Project view opens the file in a Text Editor window.
- You can access a contextual popup menu by clicking the right mouse button. The menu displayed is specific to the current context, including the object or window under the mouse.

  For example, right-clicking a project name in the Project view displays a popup menu with operations appropriate to the project file. Right-clicking a source (HDL) file in the Project view displays a popup menu with operations applicable to source files.

  Right-clicking a selectable object in an HDL Analyst schematic also *selects* it, and deselects anything that was selected. The resulting popup menu applies only to the selected object. See Text Editor View, on page 42, and Technology View, on page 77, for information on HDL Analyst views.

Most of the mouse button operations involve selecting and deselecting objects. To use the mouse in this way in an HDL Analyst schematic, the mouse pointer must be the cross-hairs symbol: ┼. If the cross-hairs pointer is not displayed, right-click the schematic background to display it.

© 2022 Synopsys, Inc.

Using the Mouse Wheel

If your mouse has a wheel and you are using a Microsoft Windows platform, you can use the wheel to scroll and zoom, as follows:

- Whenever only a horizontal scroll bar is visible, rotating the wheel scrolls the window horizontally.
- Whenever a vertical scroll bar is visible, rotating the wheel scrolls the window vertically.
- Whenever both horizontal and vertical scroll bars are visible, rotating the wheel while pressing and holding the Shift key scrolls the window horizontally.
- In a window that can be zoomed, such as a graphics window, rotating the wheel while pressing and holding the Ctrl key zooms the window.

© 2022 Synopsys, Inc.

Toolbars

Toolbars provide a quick way to access common menu commands by clicking their icons. The following standard toolbars are available:

- Project Toolbar — Project control and file manipulation.
- Analyst Toolbar — Manipulation of compiled and mapped schematic views.
- Text Editor Toolbar — Text editor bookmark commands.
- FSM Viewer Toolbar — Display of finite state machine (FSM) information.
- Tools Toolbar — Opens supporting tools.

You can enable or disable the display of individual toolbars – see Toolbar Command, on page 306.

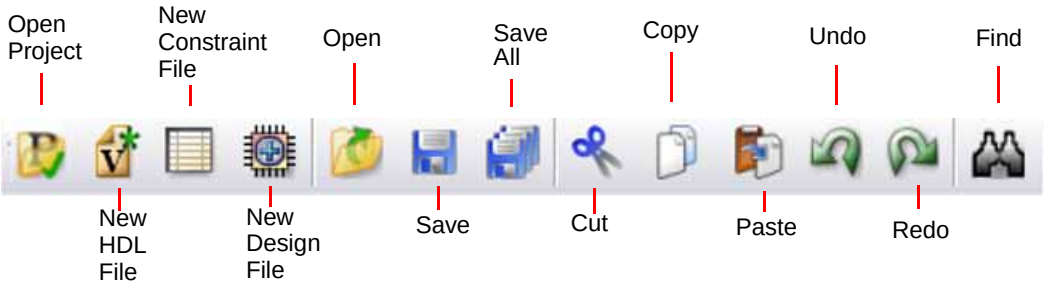
By dragging a toolbar, you can move it anywhere on the screen: you can make it float in its own window or dock it at a docking area (an edge) of the application window. To move the menu bar to a docking area without docking it there (that is, to leave it floating), press and hold the Ctrl or Shift key while dragging it.

Right-clicking the window *title bar* when a toolbar is floating displays a popup menu with commands Hide and Move. Hide removes the window. Move lets you position the window using either the arrow keys or the mouse.

© 2022 Synopsys, Inc.

Project Toolbar

The Project toolbar provides the following icons, by default:



The following table describes the default Project icons. Each is equivalent to a File or Edit menu command; for more information, see the following:

- File Menu, on page 286
- Edit Menu, on page 291

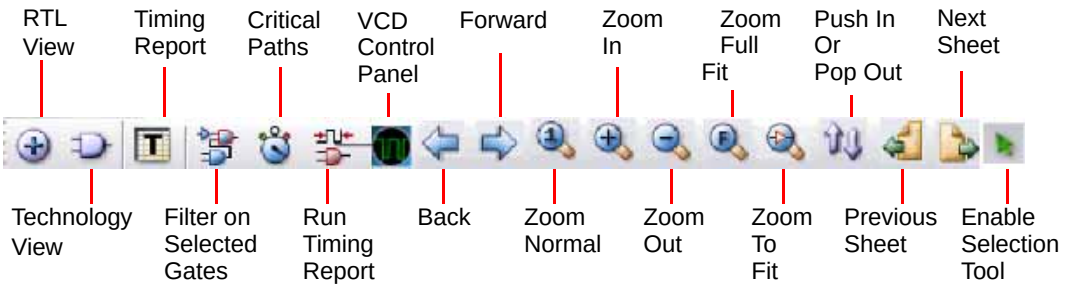
Icon	Description
Open Project	Displays the Open Project dialog box to create a new project or to open an existing project. Same as File ->Open Project.
New HDL file	Opens the Text Editor window with a new, empty source file. Same as File ->New, Verilog File or VHDL File.
New Constraint File (SCOPE)	Opens the SCOPE spreadsheet with a new, empty constraint file. Same as File ->New, Constraint File (SCOPE).
New Design Plan	Not applicable for Pango technologies.
Open	Displays the Open dialog box, to open a file. Same as File ->Open.
Save	Saves the current file. If the file has not yet been saved, this displays the Save As dialog box, where you specify the filename. The kind of file depends on the active view. Same as File ->Save.

© 2022 Synopsys, Inc.

Icon	Description
Save All	Saves all files associated with the current design. Same as File ->Save All.
Cut	Cuts text or graphics from the active view, making it available to Paste. Same as Edit ->Cut.
Paste	Pastes previously cut or copied text or graphics to the active view. Same as Edit ->Paste.
Undo	Undoes the last action taken. Same as Edit ->Undo.
Redo	Performs the action undone by Undo. Same as Edit ->Redo.
Find	Finds text in the Text Editor or objects in an RTL view or Technology view. Same as Edit ->Find.

Analyst Toolbar

The Analyst toolbar becomes active after a design has been compiled. The toolbar provides the following icons, by default:



The following table describes the default Analyst icons. Each is equivalent to an HDL Analyst menu command – see HDL Analyst Menu, on page 382, for more information.

Icon	Description
 RTL View	Opens a new, hierarchical RTL view: a register transfer-level schematic of the compiled design, together with the associated Hierarchy Browser. Same as HDL Analyst ->RTL ->Hierarchical View.
 Technology View	Opens a new, hierarchical Technology view: a technology-level schematic of the mapped (synthesized) design, together with the associated Hierarchy Browser. Same as HDL Analyst ->Technology ->Hierarchical View.
 Timing Report View	Not applicable for Pango technologies.
 Filter Schematic	Filters your entire design to show only the selected objects. The result is a *filtered* schematic. Same as HDL Analyst ->Filter Schematic.
 Show Critical Path	Filters your design to show only the instances (and their paths) whose slack times are within the slack margin of the worst slack time of the design (see HDL Analyst ->Set Slack Margin). The result is flat if the entire design was already flat. The icon Show Critical Path also enables HDL Analyst ->Show Timing Information. Available only in a Technology view. Not available in a Timing view. Same as HDL Analyst ->Show Critical Path.
 Timing Analyst	Generates and displays a custom timing report and view. The timing report provides more information than the default report (specific paths or more than five paths) or one that provides timing based on additional analysis constraint files. See Analysis Menu, on page 370. Only available for certain device technologies. Same as Analysis ->Timing Analyst.
 VCD Panel	Not applicable for Pango technologies.
 Back	Goes backward in the history of displayed sheets of the current HDL Analyst view. Same as View ->Back.

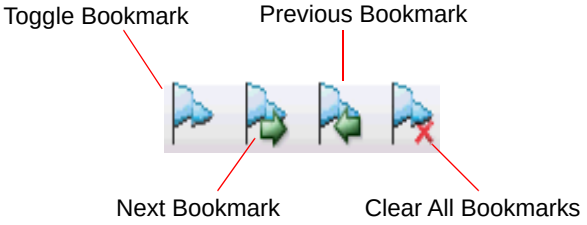
© 2022 Synopsys, Inc.

Icon	Description
 Forward	Goes forward in the history of displayed sheets of the current HDL Analyst view. Same as View ->Forward.
 Zoom 100%	Zooms in at a 1:1 ratio and centers the active view where you click. If the view is already normal size, it re-centers the view at the new click location. Same as View ->Normal View.[a]
 Zoom In  Zoom Out	Zooms the view in or out. Buttons stay active until deselected. Same as View ->Zoom In or View ->Zoom Out.[a]
 Zoom Full	Zoom that reduces the active view to display the entire design. Same as View ->Full View.[b]
 Zoom Selected	When selected, zooms in on only the selected objects to the full window size.
 Push/Pop Hierarchy	Toggles traversing the hierarchy using the push/pop mode. Same as View ->Push/Pop Hierarchy.
 Previous Sheet	Displays the previous sheet of a multiple-sheet schematic. Same as View ->Previous Sheet.
 Next Sheet	Displays the next sheet of a multiple-sheet schematic. Same as View ->Previous Sheet.
 Select Tool	Switches from zoom to the selection tool.

a. Available only in the SCOPE spreadsheet, FSM Viewer, RTL views, and Technology views.
b. Available only in the FSM Viewer, RTL views, and Technology views.

Text Editor Toolbar

The Edit toolbar is active whenever the Text Editor is active. You use it to edit *bookmarks* in the file. (Other editing operations are located on the Project toolbar – see Project Toolbar, on page 55.) The Edit toolbar provides the following icons, by default:

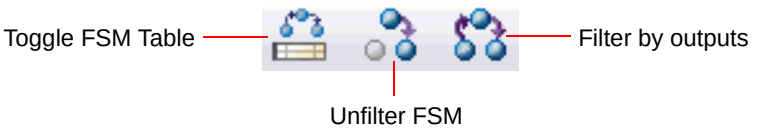


The following table describes the default Edit icons. Each is available in the Text Editor, and each is equivalent to an Edit menu command there – see Edit Menu Commands for the Text Editor, on page 292, for more information.

Icon	Description
Toggle Bookmark	Alternately inserts and removes a bookmark at the line that contains the text cursor. Same as Edit ->Toggle bookmark.
Next Bookmark	Takes you to the next bookmark. Same as Edit ->Next bookmark.
Previous Bookmark	Takes you to the previous bookmark. Same as Edit ->Previous bookmark.
Clear All Bookmarks	Removes all bookmarks from the Text Editor window. Same as Edit ->Delete all bookmarks.

FSM Viewer Toolbar

When you push down into a state machine primitive in an RTL view, the FSM Viewer displays and enables the FSM toolbar. The FSM Viewer graphically displays the states and transitions. It also lists them in table form. By default, the FSM toolbar provides the following icons, providing access to common FSM Viewer commands.



© 2022 Synopsys, Inc.

The following table describes the default FSM icons. Each is available in the FSM viewer, and each is equivalent to a View menu command available there – see View Menu, on page 303, for more information.

Icon	Description
 Toggle FSM Table	Toggles the display of state-and-transition tables. Same as View->FSM Table.
 Unfilter FSM	Restores a filtered FSM diagram so that all the states and transitions are showing. Same as View->Unfilter.
 Filter by outputs	Hides all but the selected state(s), their output transitions, and the destination states of those transitions. Same as View->Filter->By output transitions.

Tools Toolbar

The Tools Toolbar opens supporting tools.

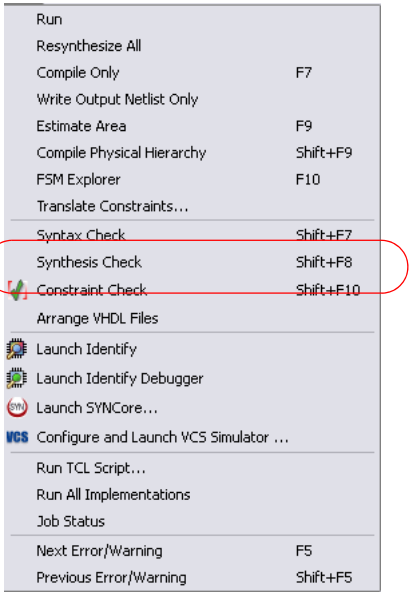
Icon	Description
 Constraint Check	Checks the syntax and applicability of the timing constraints in the constraint file for your project and generates a report (*project_name*_cck.rpt). Same as Run->Constraint Check.
 Launch Identify Instrumentor	Not available for Pango technologies.
 Launch Identify Debugger	Not available for Pango technologies.
 Launch SYNCore	Launches the SYNCore IP wizard. This tool helps you build IP blocks such as memory models for your design. For more information, see Launch SYNCore Command, on page 358.
 Launch SystemDesigner	Not available for Pango technologies.
 VCS Simulator	Configures and launches the VCS simulator.

© 2022 Synopsys, Inc.

Keyboard Shortcuts

Keyboard shortcuts are key sequences that you type in order to run a command. Menus list keyboard shortcuts next to the corresponding commands.

For example, to check syntax, you can press and hold the Shift key while you type the F7 key, instead of using the menu command Run ->Syntax Check.



The following table describes the keyboard shortcuts.

© 2022 Synopsys, Inc.

Keyboard Shortcut	Description
b	In an RTL or Technology view, shows all logic between two or more selected objects (instances, pins, ports). The result is a *filtered* schematic. Limited to the current schematic. Same as HDL Analyst ->Current Level ->Expand Paths (see HDL Analyst Menu: Filtering and Flattening Commands, on page 385).
Ctrl-++ (number pad)	In the FSM Viewer, hides all but the selected state(s), their output transitions, and the destination states of those transitions. Same as View ->Filter ->By output transitions.
Ctrl-+- (number pad)	In the FSM Viewer, hides all but the selected state(s), their input transitions, and the origin states of those transitions. Same as View ->Filter ->By input transitions.
Ctrl-+* (number pad)	In the FSM Viewer, hides all but the selected state(s), their input and output transitions, and their predecessor and successor states. Same as View ->Filter ->By any transition.
Ctrl-1	In an RTL or Technology view, zooms the active view, when you click, to full (normal) size. Same as View ->Normal View.
Ctrl-a	Centers the window on the design. Same as View ->Pan Center.
Ctrl-b	In an RTL or Technology view, shows all logic between two or more selected objects (instances, pins, ports). The result is a *filtered* schematic. Operates hierarchically, on lower levels as well as the current schematic. Same as HDL Analyst ->Hierarchical ->Expand Paths (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).
Ctrl-c	Copies the selected object. Same as Edit ->Copy. This shortcut is sometimes available even when Edit ->Copy is not. See, for instance, Find Command (HDL Analyst), on page 296.)
Ctrl-d	In an RTL or Technology view, selects the driver for the selected net. Operates hierarchically, on lower levels as well as the current schematic. Same as HDL Analyst->Hierarchical ->Select Net Driver (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).

© 2022 Synopsys, Inc.

Keyboard Shortcut	Description
Ctrl-e	In an RTL or Technology view, expands along the paths from selected pins or ports, according to their directions, to the nearest objects (no farther). The result is a *filtered* schematic. Operates hierarchically, on lower levels as well as the current schematic. Same as HDL Analyst->Hierarchical ->Expand (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).
Ctrl-Enter (Return)	In the FSM Viewer, hides all but the selected state(s). Same as View->Filter->Selected (see View Menu, on page 303).
Ctrl-f	Finds the selected object. Same as Edit->Find.
Ctrl-F2	Alternately inserts and removes a bookmark to the line that contains the text cursor. Same as Edit->Toggle bookmark (see Edit Menu Commands for the Text Editor, on page 292).
Ctrl-F4	Closes the current window. Same as File ->Close.
Ctrl-F6	Toggles between active windows.
Ctrl-g	In the Text Editor, jumps to the specified line. Same as Edit->Goto (see Edit Menu Commands for the Text Editor, on page 292). In an RTL or Technology view, selects the sheet number in a multiple-page schematic. Same as View->View Sheets (see View Menu: RTL and Technology Views Commands, on page 305).
Ctrl-h	In the Text Editor, replaces text. Same as Edit->Replace (see Edit Menu Commands for the Text Editor, on page 292).
Ctrl-i	In an RTL or Technology view, selects instances connected to the selected net. Operates hierarchically, on lower levels as well as the current schematic. Same as HDL Analyst->Hierarchical->Select Net Instances (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).
Ctrl-j	In an RTL or Technology view, displays the unfiltered schematic sheet that contains the net driver for the selected net. Operates hierarchically, on lower levels as well as the current schematic. Same as HDL Analyst->Hierarchical->Goto Net Driver (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).

© 2022 Synopsys, Inc.

Keyboard Shortcut	Description
Ctrl-l	In the FSM Viewer, or an RTL or Technology view, toggles zoom locking. When locking is enabled, if you resize the window the displayed schematic is resized proportionately, so that it occupies the same portion of the window. Same as View->Zoom Lock (see View Menu Commands: All Views, on page 303).
Ctrl-m	In an RTL or Technology view, expands inside the subdesign, from the lower-level port that corresponds to the selected pin, to the nearest objects (no farther). Same as HDL Analyst->Hierarchical->Expand Inwards (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).
Ctrl-n	Creates a new file or project. Same as File->New.
Ctrl-o	Opens an existing file or project. Same as File->Open.
Ctrl-p	Prints the current view. Same as File->Print.
Ctrl-q	In an RTL or Technology view, toggles the display of visual properties of instances, pins, nets, and ports in a design.
Ctrl-r	In an RTL or Technology view, expands along the paths from selected pins or ports, according to their directions, until registers, ports, or black boxes are reached. The result is a *filtered* schematic. Operates hierarchically, on lower levels as well as the current schematic. Same as HDL Analyst->Hierarchical->Expand to Register/Port (see HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383).
Ctrl-s	In the Project View, saves the file. Same as File ->Save.
Ctrl-t	Toggles display of the Tcl window. Same as View ->Tcl Window (see View Menu, on page 303).
Ctrl-u	In the Text Editor, changes the selected text to lower case. Same as Edit->Advanced->Lowercase (see Edit Menu Commands for the Text Editor, on page 292). In the FSM Viewer, restores a filtered FSM diagram so that all the states and transitions are showing. Same as View->Unfilter (see View Menu: FSM Viewer Commands, on page 305).
Ctrl-v	Pastes the last object copied or cut. Same as Edit ->Paste.

© 2022 Synopsys, Inc.

Keyboard Shortcut	Description
Ctrl-x	Cuts the selected object(s), making it available to Paste. Same as Edit ->Cut.
Ctrl-y	In an RTL or Technology view, goes forward in the history of displayed sheets for the current HDL Analyst view. Same as View->Forward (see View Menu: RTL and Technology Views Commands, on page 305). In other contexts, performs the action undone by Undo. Same as Edit->Redo.
Ctrl-z	In an RTL or Technology view, goes backward in the history of displayed sheets for the current HDL Analyst view. Same as View->Back (see View Menu: RTL and Technology Views Commands, on page 305). In other contexts, undoes the last action. Same as Edit ->Undo.
Ctrl-Shift-F2	Removes all bookmarks from the Text Editor window. Same as Edit ->Delete all bookmarks (see Edit Menu Commands for the Text Editor, on page 292).
Ctrl-Shift-h	In an RTL or Technology view, shows all pins on selected *transparent* hierarchical (non-primitive) instances. Pins on primitives are always shown. Available only in a filtered schematic. Same as HDL Analyst ->Show All Hier Pins (see HDL Analyst Menu: Analysis Commands, on page 389).
Ctrl-Shift-i	In an RTL or Technology view, selects all instances on the current schematic level (all sheets). This does *not* select instances on other levels. Same as HDL Analyst->Select All Schematic->Instances (see HDL Analyst Menu, on page 382).
Ctrl-Shift-p	In an RTL or Technology view, selects all ports on the current schematic level (all sheets). This does *not* select ports on other levels. Same as HDL Analyst->Select All Schematic->Ports (see HDL Analyst Menu, on page 382).
Ctrl-Shift-u	In the Text Editor, changes the selected text to lower case. Same as Edit->Advanced->Uppercase (see Edit Menu Commands for the Text Editor, on page 292).

© 2022 Synopsys, Inc.

Keyboard Shortcut	Description
d	In an RTL or Technology view, selects the driver for the selected net. Limited to the current schematic. Same as HDL Analyst ->Current Level ->Select Net Driver (see HDL Analyst Menu, on page 382).
Delete (DEL)	Removes the selected files from the project. Same as Project->Remove Files From Project.
e	In an RTL or Technology view, expands along the paths from selected pins or ports, according to their directions, to the nearest objects (no farther). Limited to the current schematic. Same as HDL Analyst->Current Level->Expand (see HDL Analyst Menu, on page 382).
F1	Provides context-sensitive help. Same as Help->Help.
F2	In an RTL or Technology view, toggles traversing the hierarchy using the push/pop mode. Same as View->Push/Pop Hierarchy (see View Menu: RTL and Technology Views Commands, on page 305). In the Text Editor, takes you to the next bookmark. Same as Edit->Next bookmark (see Edit Menu Commands for the Text Editor, on page 292).
F4	In the Project view, adds a file to the project. Same as Project->Add Source File (see Build Project Command, on page 290). In an RTL or Technology view, zooms the view so that it shows the entire design. Same as View->Full View (see View Menu: RTL and Technology Views Commands, on page 305).
F5	Displays the next source file error. Same as Run->Next Error/Warning (see Run Menu, on page 353).
F7	Compiles your design, without mapping it. Same as Run->Compile Only (see Run Menu, on page 353).
F8	Synthesizes (compiles and maps) your design. Same as Run->Synthesize (see Run Menu, on page 353).
F11	Toggles zooming in. Same as View->Zoom In (see View Menu: RTL and Technology Views Commands, on page 305).

© 2022 Synopsys, Inc.

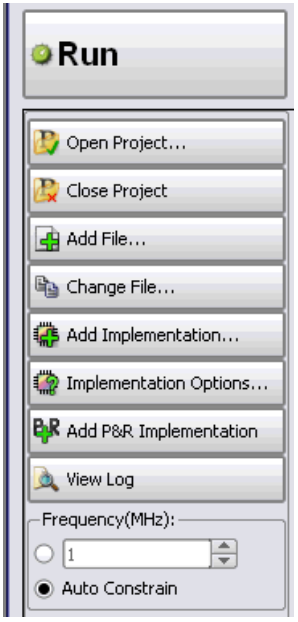
Keyboard Shortcut	Description
F12	In an RTL or Technology view, filters your entire design to show only the selected objects. Same as HDL Analyst->Filter Schematic – see HDL Analyst Menu: Filtering and Flattening Commands, on page 385.
i	In an RTL or Technology view, selects instances connected to the selected net. Limited to the current schematic. Same as HDL Analyst->Current Level->Select Net Instances (see HDL Analyst Menu, on page 382).
j	In an RTL or Technology view, displays the unfiltered schematic sheet that contains the net driver for the selected net. Same as HDL Analyst->Current Level->Goto Net Driver (see HDL Analyst Menu, on page 382).
r	In an RTL or Technology view, expands along the paths from selected pins or ports, according to their directions, until registers, ports, or black boxes are reached. The result is a *filtered* schematic. Limited to the current schematic. Same as HDL Analyst ->Current Level->Expand to Register/Port (see HDL Analyst Menu, on page 382).
Shift-F2	In the Text Editor, takes you to the previous bookmark.
Shift-F4	Allows you to add source files to your project (Project->Add Source Files).
Shift-F5	Displays the previous source file error. Same as Run->Previous Error/Warning (see Run Menu, on page 353).
Shift-F7	Checks source file syntax. Same as Run->Syntax Check (see Run Menu, on page 353).
Shift-F8	Checks synthesis. Same as Run->Synthesis Check (see Run Menu, on page 353).
Shift-F10	Checks the timing constraints in the constraint files in your project and generates a report (*project_name*_cck.rpt). Same as Run->Constraint Check (see Run Menu, on page 353). In an RTL or Technology view, lets you pan (scroll) the schematic by dragging it with the mouse. Same as View ->Pan (see View Menu: RTL and Technology Views Commands, on page 305).

© 2022 Synopsys, Inc.

Keyboard Shortcut	Description
Shift-F11	Toggles zooming out. Same as View->Zoom Out (see View Menu, on page 303).
Shift-Left Arrow	Displays the previous sheet of a multiple-sheet schematic.
Shift-Right Arrow	Displays the next sheet of a multiple-sheet schematic.
Shift-s	Dissolves the selected instances, showing their lower-level details. Dissolving an instance one level replaces it, in the current sheet, by what you would see if you pushed into it using the push/pop mode. The rest of the sheet (not selected) remains unchanged. The number of levels dissolved is the Dissolve Levels value in the Schematic Options dialog box. The type (filtered or unfiltered) of the resulting schematic is unchanged from that of the current schematic. However, the effect of the command is different in filtered and unfiltered schematics. Same as HDL Analyst ->Dissolve Instances – see HDL Analyst Command, on page 385.

Buttons and Options

The Project view contains several buttons and a few additional features that give you immediate access to some of the more common commands and user options.



The following table describes the Project View buttons and options.

Button/Option	Action
Open Project...	Opens a new or existing project. Same as File->Open Project (see Open Project Command, on page 291).
Close Project	Closes the current project. Same as File->Close Project (see Run Menu, on page 353).
Add File...	Adds a source file to the project. Same as Project->Add Source File (see Build Project Command, on page 290).

© 2022 Synopsys, Inc.

Button/Option	Action
Change File...	Replaces one source file with another. Same as Project ->Change File (see Change File Command, on page 315).
Add Implementation	Creates a new implementation.
Implementation Options	Displays the Implementation Options dialog box, where you can set various options for synthesis. Same as Project ->Implementation Options (see Implementation Options Command, on page 324).
Add P&R Implementation	Not applicable for Pango technologies.
View Log	Displays the log file. Same as View ->View Log File (see View Menu, on page 303).
Frequency (MHz)	Sets the global frequency, which you can override locally with attributes. Same as enabling the Frequency (MHz) option on the Constraints panel of the Implementation Options dialog box.
Auto Constrain	When Auto Constrain is enabled and no clocks are defined, the software automatically constrains the design to achieve best possible timing by reducing periods of individual clock and the timing of any timed I/O paths in successive steps. See Using Auto Constraints, on page 345 in the *User Guide* for detailed information about using this option. You can also set this option on the Constraints panel of the Implementation Options dialog box.
Automatic Compile Point	Enables the automatic compile point flow, which can analyze a design and identify modules that can automatically be defined as compile points and mapped in parallel using multiprocessing. See The Automatic Compile Point Flow, on page 428 in the *User Guide*.
FSM Compiler	Turning on this option enables special FSM optimizations. Same as enabling the FSM Compiler option on the Options panel of the Implementation Options dialog box (see Using the Mouse, on page 49 and Optimizing State Machines, on page 398 in the *User Guide*).

Button/Option	Action
Resource Sharing	When enabled, makes the compiler use resource sharing techniques. This option does not affect resource sharing by the mapper. The option is the same as the Resource Sharing option on the Options panel of the Implementation Options dialog box. See Sharing Resources, on page 396 in the *User Guide* for usage details.
Pipelining	When enabled, uses register balancing and pipeline registers on multipliers and ROMs. Same as enabling the Pipelining option on the Options panel of the Implementation Options dialog box. See Pipelining, on page 376in the *User Guide*.
Retiming	When enabled, improves the timing performance of sequential circuits. The retiming process moves storage devices (flip-flops) across computational elements with no memory (gates/LUTs) to improve the performance of the circuit. This option also adds a retiming report to the log file. Same as enabling the Retiming option on the Options panel of the Implementation Options dialog box. Use the syn_allow_retiming attribute to enable or disable retiming for individual flip-flops. See syn_allow_retiming, on page 21 for syntax details. Note: Pipelining is automatically enabled when retiming is enabled.
Run	Runs synthesis (compilation and mapping). Same as the Run->Synthesize command (see Run Menu, on page 353).

© 2022 Synopsys, Inc.

© 2022 Synopsys, Inc.

SYNOPSYS®
Silicon to Software™

CHAPTER 3

HDL Analyst Tool

The HDL Analyst tool helps you examine your design and synthesis results, and analyze how you can improve design performance and area.

The following describe the HDL Analyst tool and the operations you can perform with it.

- HDL Analyst Views and Commands, on page 74
- Schematic Objects and Their Display, on page 86
- Basic Operations on Schematic Objects, on page 95
- Multiple-sheet Schematics, on page 100
- Exploring Design Hierarchy, on page 103
- Filtering and Flattening Schematics, on page 111
- Timing Information and Critical Paths, on page 117

© 2022 Synopsys, Inc.

HDL Analyst Views and Commands

The HDL Analyst tool graphically displays information in two schematic views, the RTL and Technology views (see Text Editor View, on page 42 and Technology View, on page 77 for information). The graphic representation is useful for analyzing and debugging your design, because you can visualize where coding changes or timing constraints might reduce area or increase performance.

This section gives you information about the following:

- Hierarchy Browser, on page 79
- FSM Viewer Window, on page 81
- Filtered and Unfiltered Schematic Views, on page 84
- Accessing HDL Analyst Commands, on page 85

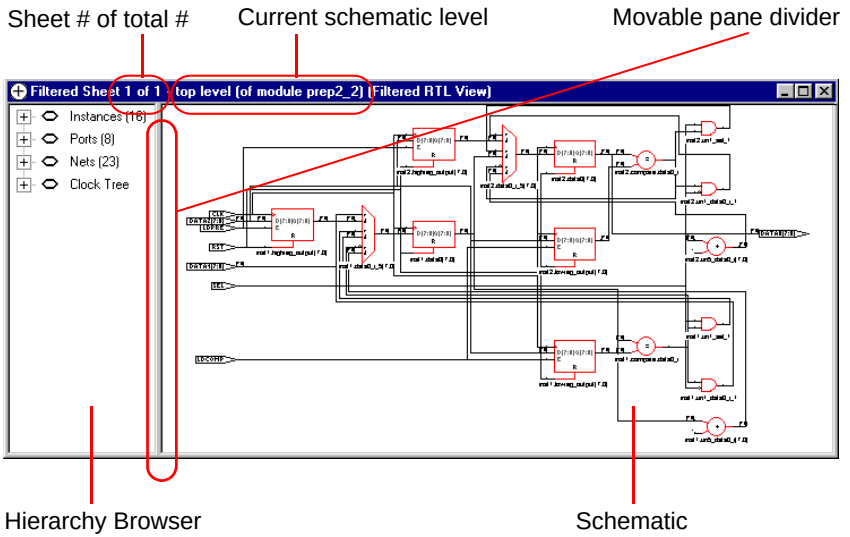
© 2022 Synopsys, Inc.

RTL View

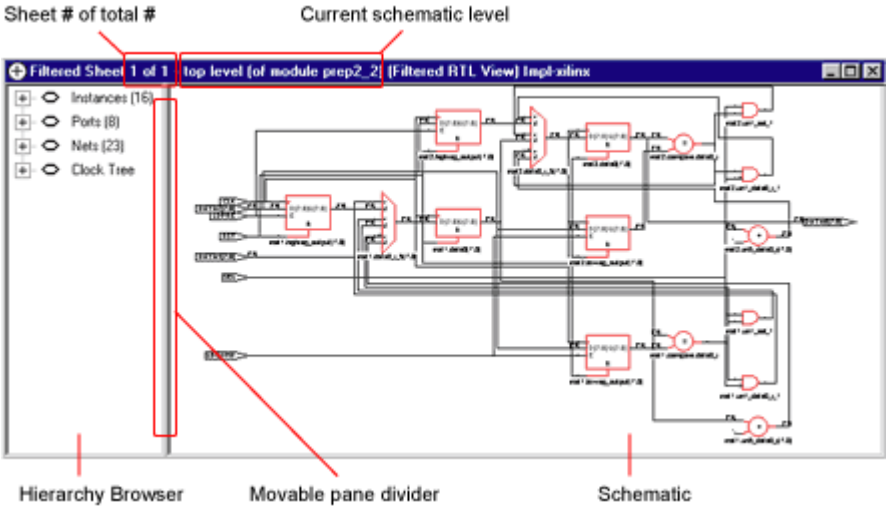
The RTL view provides a high-level, technology-independent, graphic representation of your design after compilation, using technology-independent components like variable-width adders, registers, large multiplexers, and state machines. RTL views correspond to the srs netlist files generated during compilation. RTL views are only available after your design has been successfully compiled. For information about the other HDL Analyst view (the Technology view generated after mapping), see Technology View, on page 77.

To display an RTL view, first compile or synthesize your design, then select HDL Analyst->RTL and choose Hierarchical View or Flattened View, or click the RTL icon ( ).

An RTL view has two panes, a Hierarchy Browser on the left and an RTL schematic on the right. You can drag the pane divider with the mouse to change the relative pane sizes. For more information about the Hierarchy Browser, see Hierarchy Browser, on page 79. Your design is drawn as a set of schematics. The schematic for a design module (or the top-level) consists of one or more sheets, only one of which is visible in a given view at any time. The title bar of the window indicates the current hierarchical schematic level, the current sheet, and the total number of sheets for that level.



© 2022 Synopsys, Inc.



The design in the RTL schematic can be hierarchical or flattened. Further, the view can consist of the entire design or part of it. Different commands apply, depending on the kind of RTL view.

The following table lists where to find further information about the RTL view:

For information about ...	See ...
Hierarchy Browser	Hierarchy Browser, on page 79
Procedures for RTL view operations like crossprobing, searching, pushing/popping, filtering, flattening, etc.	Working in the Standard Schematic, on page 261 of the *User Guide*.
Explanations or descriptions of features like object display, filtering, flattening, etc.	HDL Analyst Tool, on page 73
Commands for RTL view operations like filtering, flattening, etc.	Accessing HDL Analyst Commands, on page 85 HDL Analyst Command, on page 383

© 2022 Synopsys, Inc.

For information about ...	See ...
Viewing commands like zooming, panning, etc.	View Menu: RTL and Technology Views Commands, on page 305
History commands: Back and Forward	View Menu: RTL and Technology Views Commands, on page 305
Search command	Find Command (HDL Analyst), on page 296

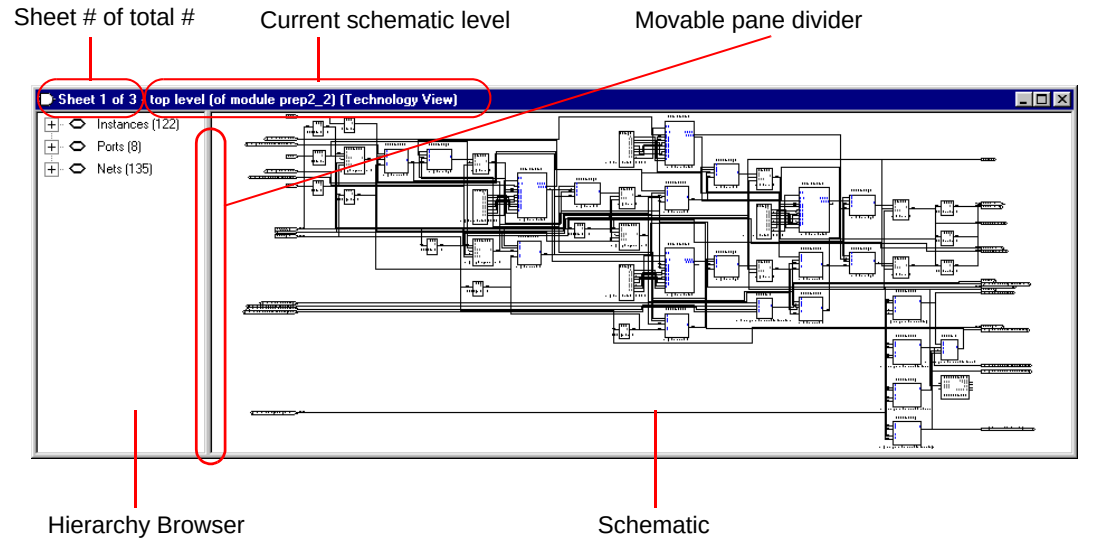
Technology View

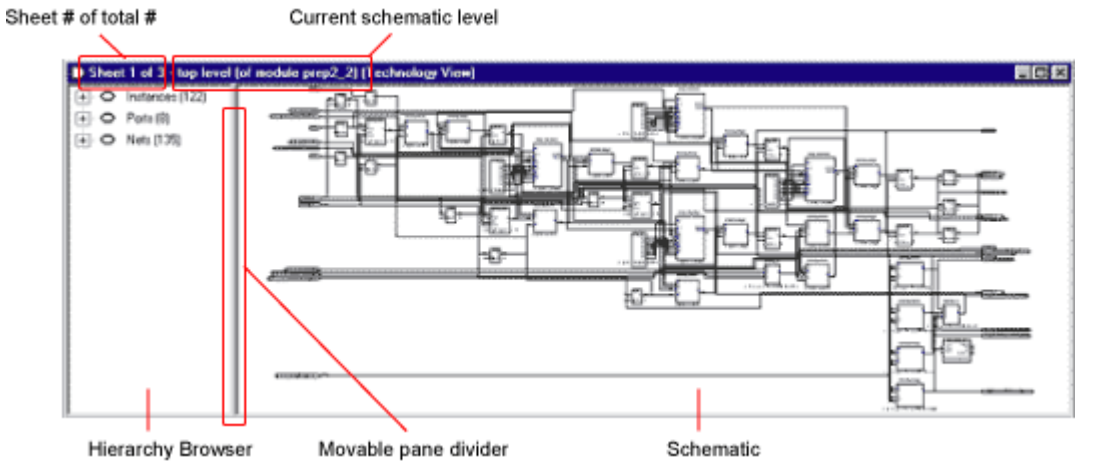
A Technology view provides a low-level, technology-specific view of your design after mapping, using components such as look-up tables, cascade and carry chains, multiplexers, and flip-flops. Technology views are only available after your design has been synthesized (compiled and mapped). For information about the other HDL Analyst view (the RTL view generated after compilation), see RTL View, on page 75.

To display a Technology view, first synthesize your design, and then either select a view from the HDL Analyst->Technology menu (Hierarchical View, Flattened View, Flattened to Gates View, Hierarchical Critical Path, or Flattened Critical Path) or select the Technology view icon ( ).

A Technology view has two panes: a Hierarchy Browser on the left and an RTL schematic on the right. You can drag the pane divider with the mouse to change the relative pane sizes. For more information about the Hierarchy Browser, see Hierarchy Browser, on page 79. Your design is drawn as a set of schematics at different design levels. The schematic for a design module (or the top level) consists of one or more sheets, only one of which is visible in a given view at any time. The title bar of the window indicates the current schematic level, the current sheet, and the total number of sheets for that level.

© 2022 Synopsys, Inc.





The schematic design can be hierarchical or flattened. Further, the view can consist of the entire design or a part of it. Different commands apply, depending on the kind of view. In addition to all the features available in RTL views, Technology views have two additional features, critical path filtering and flattening to gates.

The following table lists where to find further information about the Technology view:

For information about ...	See ...
Hierarchy Browser	Hierarchy Browser, on page 79
Procedures for Technology view operations like crossprobing, searching, pushing/popping, filtering, flattening, etc.	Working in the Standard Schematic, on page 261 of the *User Guide*
Explanations or descriptions of features like object display, filtering, flattening, etc.	HDL Analyst Tool, on page 73
Commands for Technology view operations like filtering, flattening, etc.	Accessing HDL Analyst Commands, on page 85 HDL Analyst Command, on page 383
Viewing commands like zooming, panning, etc.	View Menu: RTL and Technology Views Commands, on page 305
History commands: Back and Forward	View Menu: RTL and Technology Views Commands, on page 305
Search command	Find Command (HDL Analyst), on page 296

Hierarchy Browser

The Hierarchy Browser is the left pane in the RTL and Technology views. (See RTL View, on page 75 and Technology View, on page 77.) The Hierarchy Browser categorizes the design objects in a series of trees, and lets you browse the design hierarchy or select objects. Selecting an object in the Browser selects that object in the schematic. The objects are organized as shown in the following table, with a symbol that indicates the object type. See Hierarchy Browser Symbols, on page 80 for common symbols.

Instances	Lists all the instances and primitives in the design. In a Technology view, it includes all technology-specific primitives.

© 2022 Synopsys, Inc.

Ports	Lists all the ports in the design.
Nets	Lists all the nets in the design.
Clock Tree	Lists all the instances and ports that drive clock pins in an RTL view. If you select everything listed under Clock Tree and then use the Filter Schematic command, you see a filtered view of all clock pin drivers in your design. Registers are not shown in the resulting schematic, unless they drive clocks. This view can help you determine what to define as clocks.

A tree node can be expanded or collapsed by clicking the associated icons: the square plus (⊞) or minus (⊟) icons, respectively. You can also expand or collapse all trees at the same time by right-clicking in the Hierarchy Browser and choosing Expand All or Collapse All.

You can use the keyboard arrow keys (left, right, up, down) to move between objects in the Hierarchy Browser, or you can use the scroll bar. Use the Shift or Ctrl keys to select multiple objects. See Navigating With a Hierarchy Browser, on page 106 for more information about using the Hierarchy Browser for navigation and crossprobing.

Hierarchy Browser Symbols

Common symbols used in Hierarchy Browsers are listed in the following table.

Symbol	Description	Symbol	Description
	Folder		Buffer
	Input port		AND gate
	Output port		NAND gate
	Bidirectional port		OR gate
	Net		NOR gate
	Other primitive instance		XOR gate
	Hierarchical instance		XNOR gate
	Technology-specific primitive or inferred ROM		Adder

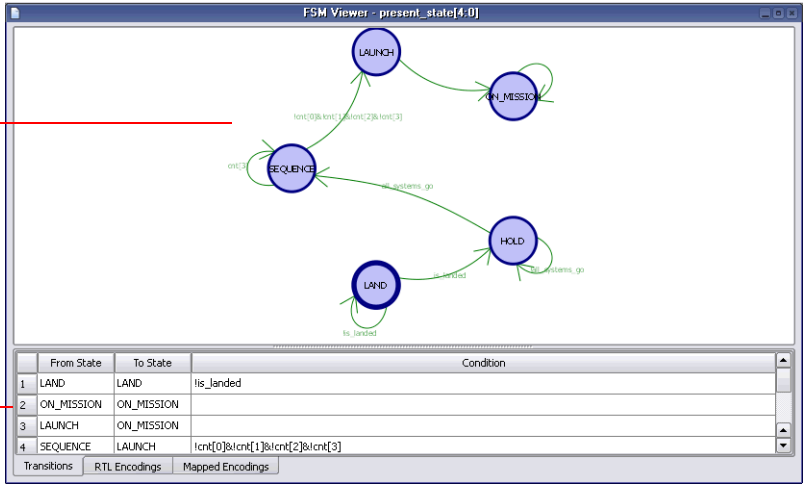
Symbol	Description	Symbol	Description
	Register or inferred state machine		Multiplier
	Multiplexer		Equal comparator
	Tristate		Less-than comparator
	Inverter		Less-than-or-equal comparator

FSM Viewer Window

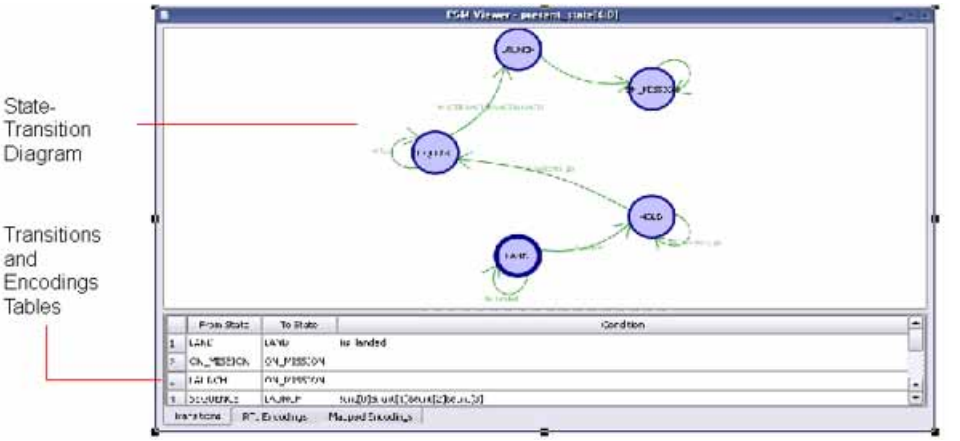
Pushing down into a state machine primitive in the RTL view displays the FSM Viewer and enables the FSM toolbar. The FSM Viewer contains graphical information about the finite state machines (FSMs) in your design. The window has a state-transition diagram and tables of transitions and state encodings.

State-Transition Diagram

Transitions and Encodings Tables



© 2022 Synopsys, Inc.



For the FSM Viewer to display state machine names for a Verilog design, you must use the Verilog parameter keyword. If you specify state machine names using the define keyword, the FSM Viewer displays the binary values for the state machines, rather than their names.

You can toggle display of the FSM tables on and off with the Toggle FSM Table icon ( ) on the FSM toolbar. The FSM tables are in the following panels:

- The Transitions panel describes, for each transition, the From State, To State, and Condition of transition.
- The RTL Encodings panel describes the correlation, in the RTL view, between the states (State) and the outputs (Register) of the FSM cell.
- The Mapped Encodings panel describes the correlation, in the Technology view, between the states (State) and their encodings into technology-specific registers. The information in this panel is available only after the design has been synthesized.

The following table describes FSM Viewer operations.

To accomplish this ...	Do this ...
Open the FSM Viewer	Run the FSM Compiler. Use the push/pop mode in the RTL view to push down into the FSM and open the FSM Viewer window.
Hide/display the table	Use the FSM icons.
Filter selected states and their transitions	Select the states. Right-click and choose the filter criteria from the popup, or use the FSM icons.
Display the encoding properties of a state	Select a state. Right-click to display its encoding properties (RTL or Mapped).
Display properties for the state machine	Right-click the window, outside the state-transition diagram. The property sheet shows the selected encoding method, the number of states, and the total number of transitions among states.
Crossprobe	Double-click a register in an RTL or Technology view to see the corresponding code. Select a state in the FSM view to highlight the corresponding code or register in other open views.

© 2022 Synopsys, Inc.

See also:

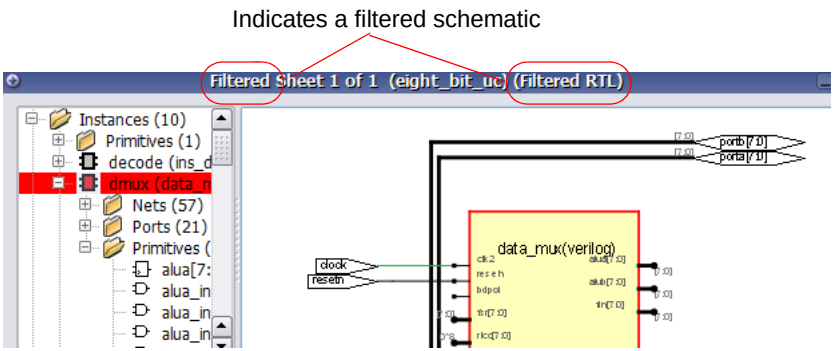
- Pushing and Popping Hierarchical Levels, on page 103, for information on the operation of pushing into a state machine.
- FSM Viewer Toolbar, on page 59, for information on the FSM icons.

See Using the FSM Viewer, on page 257 of the *User Guide* for more information on using the FSM viewer.

Filtered and Unfiltered Schematic Views

HDL Analyst views (Text Editor View, on page 42 and Technology View, on page 77) consist of schematics that let you analyze your design graphically. The schematics can be filtered or unfiltered. The distinction is important because the kind of view determines how objects are displayed for certain commands.

- Unfiltered schematics display all the objects in your design, at appropriate hierarchical levels.
- Filtered schematics show only a subset of the objects in your design, because the other objects have been filtered out by some operation. The Hierarchy Browser in the filtered view always list all the objects in the design, not just the filtered objects. Some commands, such as HDL Analyst -> Show Context, are only available in filtered schematics. Views with a filtered schematic have the word Filtered in the title bar.



Filtering commands affect only the displayed schematic, not the underlying design. See the following topics:

- For a detailed description of filtering, see Filtering and Flattening Schematics, on page 111.
- For procedures on using filtering, see Filtering Schematics, on page 308 in the *User Guide*.

Accessing HDL Analyst Commands

You can access HDL Analyst commands in many ways, depending on the active view, the currently selected objects, and other design context factors. The software offers these alternatives to access the commands:

- HDL Analyst and View menus
- HDL Analyst popup menus appear when you right-click in an HDL Analyst view. The popup menu is context-sensitive, and includes commonly used commands from the HDL Analyst and View menus, as well as some additional commands.
- HDL Analyst toolbar icons provide shortcuts to commonly used commands

For brevity, this document primarily refers to the menu method of accessing the commands and does not list alternative access methods.

See also:

- HDL Analyst Menu, on page 382
- View Menu, on page 303
- RTL and Technology Views Popup Menus, on page 438
- Analyst Toolbar, on page 56

© 2022 Synopsys, Inc.

Schematic Objects and Their Display

Schematic objects are the objects that you manipulate in an HDL Analyst schematic: instances, ports, and nets. Instances can be categorized in different ways, depending on the operation: hidden/unhidden, transparent/opaque, or primitive/hierarchical. The following topics describe schematic objects and the display of associated information in more detail:

- Object Information, on page 86
- Sheet Connectors, on page 87
- Primitive and Hierarchical Instances, on page 88
- Hidden Hierarchical Instances, on page 90
- Transparent and Opaque Display of Hierarchical Instances, on page 89
- Schematic Display, on page 91

For most objects, you select them to perform an operation. For some objects like sheet connectors, you do not select them but right-click them and select from the popup menu commands.

Object Information

To obtain information about specific objects, you can view object properties with the Properties command from the right-click popup menu, or place the pointer over the object and view the object information displayed. With the latter method, information about the object displays in these two places until you move the pointer away:

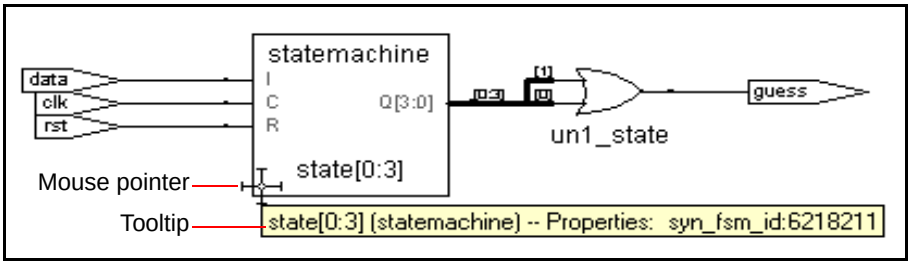
- The status bar at the bottom of the synthesis window displays the name of the instance, net, port, or sheet connector and other relevant information. If HDL Analyst->Show Timing Information is enabled, the status bar also displays timing information for the object. Here is an example of the status bar information for a net:

  ```
  Net clock (local net clock) Fanout=4
  ```

  You can enable and disable the display of status bar information by toggling the command View -> Status Bar.

© 2022 Synopsys, Inc.

- In a tooltip at the mouse pointer
  Displays the name of the object and any attached attributes. The following figure shows tooltip information for a state machine:



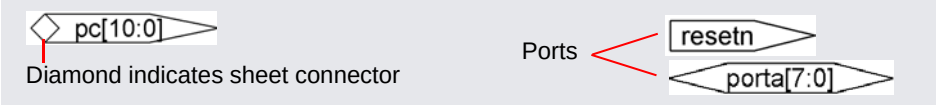
  To disable tooltip display, select View -> Toolbars and disable the Show Tooltips option. Do this if you want to reduce clutter.

See also

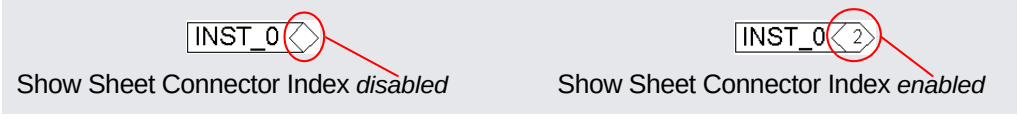
- Pin and Pin Name Display for Opaque Objects, on page 92
- HDL Analyst Options Command, on page 408

Sheet Connectors

When the HDL Analyst tool divides a schematic into multiple sheets, sheet connector symbols indicate how sheets are related. A sheet connector symbol is like a port symbol, but it has an empty diamond with sheet numbers at one end. Use the Options->HDL Analyst Options command (see Sheet Size Panel, on page 415) to control how the schematic is divided into multiple sheets.



If you enable the Show Sheet Connector Index option in the (Options->HDL Analyst Options), the empty diamond becomes a hexagon with a list of the connected sheets. You go to a connecting sheet by right-clicking a sheet connector and choosing the sheet number from the popup menu. The menu has as many sheet numbers as there are sheets connected to the net at that point.



© 2022 Synopsys, Inc.

See also

- Multiple-sheet Schematics, on page 100
- HDL Analyst Options Command, on page 408
- RTL and Technology Views Popup Menus, on page 438

Primitive and Hierarchical Instances

HDL Analyst instances are either primitive or hierarchical, and sorted into these categories in the Hierarchy Browser. Under Instances, the browser first lists hierarchical instances, and then lists primitive instances under Instances->Primitives.

Primitive Instances

Although some primitive objects have hierarchy, the term is used here to distinguish these objects from *user-defined* hierarchies. Primitive instances include the following:

RTL View	Technology View
High-level logic primitives, like XOR gates or priority-encoded multiplexers	Black boxes
Inferred ROMs, RAMs, and state machines	Technology-specific primitives, like LUTs or FPGA block RAMs
Black boxes	
Technology-specific primitives, like LUTs or FPGA block RAMs	

In a schematic, logic gate primitives are represented with standard schematic symbols, and technology-specific primitives with various symbols (see Hierarchy Browser Symbols, on page 80). You can push into primitives like technology-specific primitives, inferred ROMs, and inferred state machines to view internal details. You cannot push into logic primitives.

© 2022 Synopsys, Inc.

Hierarchical Instances

Hierarchical instances are user-defined hierarchies; all other instances are considered to be primitives. Hierarchical instances correspond to Verilog modules and VHDL entities.

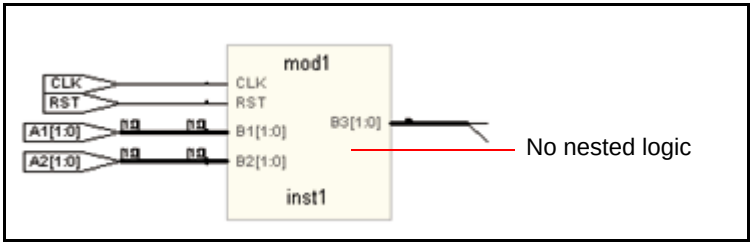
The Hierarchy Browser lists hierarchical instances under Instances, and uses this symbol: . In a schematic, the display of hierarchical instances depends on the combination of the following:

- Whether the instance is transparent or opaque. Transparent instances show their internal details nested inside them; opaque instances do not. You cannot directly control whether an object is transparent or opaque; the views are automatically generated by certain commands. See Transparent and Opaque Display of Hierarchical Instances, on page 89 for details.
- Whether the instance is hidden or not. This is user-controlled, and you can hide instances so that they are ignored by certain commands. See Hidden Hierarchical Instances, on page 90 for more information.

Transparent and Opaque Display of Hierarchical Instances

A hierarchical instance can be displayed transparently or opaquely. You cannot directly control the display; certain commands cause instances to be transparent. The distinction between transparent and opaque is important because some commands operate differently on transparent and opaque instances. For example, in a filtered schematic Flatten Current Schematic flattens only transparent hierarchical instances.

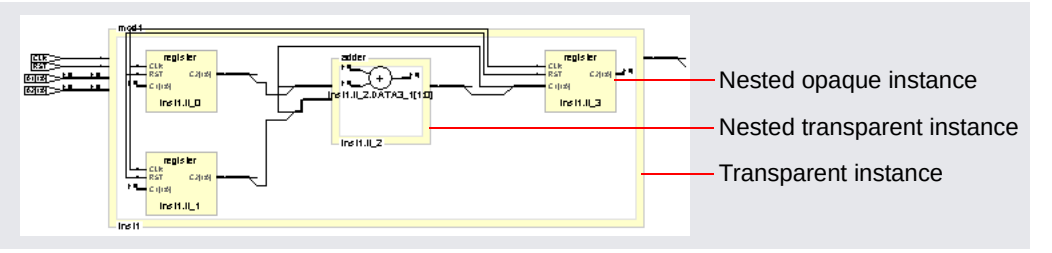
- Opaque instances are pale yellow boxes, and do not display their internal hierarchy. This is the default display.



© 2022 Synopsys, Inc.

- Transparent instances display some or all their lower-level hierarchy nested inside a hollow box with a pale yellow border. Transparent instances are only displayed in filtered schematics, and are a result of certain commands. See Looking Inside Hierarchical Instances, on page 108 for information about commands that generate transparent instances.

  A transparent instance can contain other opaque or transparent instances nested inside. The details inside a transparent instance are independent schematic objects and you can operate on them independently: select, push into, hide, and so on. Performing an operation on a transparent object does not automatically perform it on any of the objects nested inside it, and conversely.



See also

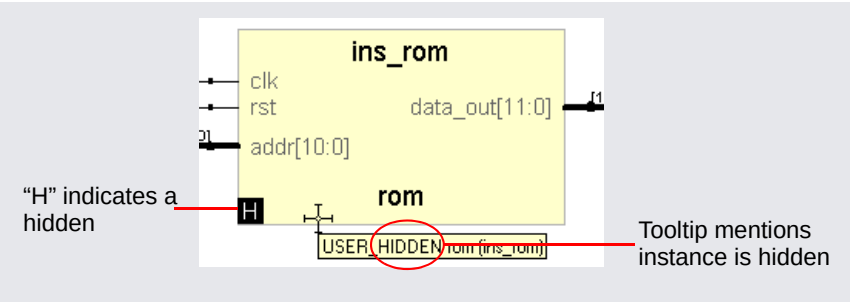
- Looking Inside Hierarchical Instances, on page 108
- Multiple Sheets for Transparent Instance Details, on page 102
- Filtered and Unfiltered Schematic Views, on page 84

Hidden Hierarchical Instances

Certain commands do not operate on the lower-level hierarchy of hidden instances, so you can hide instances to focus the operation of a command and improve performance. You hide opaque or transparent hierarchical instances with the Hide Instances command (described in RTL and Technology Views Popup Menus, on page 438). Hiding and unhiding only affects the current HDL Analyst view, and does not affect the Hierarchy Browser. You can hide and unhide instances as needed. The hierarchical logic of a hidden instance is not removed from the design; it is only excluded from certain operations.

© 2022 Synopsys, Inc.

The schematics indicate hidden hierarchical instances with a small H in the lower left corner. When the mouse pointer is over a hidden instance, the status bar and the tooltip indicate that the instance is hidden.



Schematic Display

The HDL Analyst Options dialog box controls general properties for all HDL Analyst views, and can determine the display of schematic object information. Setting a display option affects all objects of the given type in all views. Some schematic options only take effect in schematic windows opened after the setting change; others affect existing schematic windows as well.

The following are some commonly used settings that affect the display of schematic objects. See HDL Analyst Options Command, on page 408 for a complete list of display options.

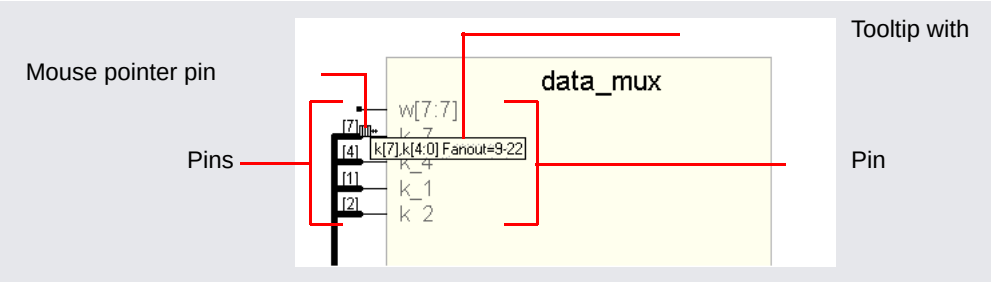
Option	Controls the display of ...
Show Cell Interior	Internal logic of technology-specific primitives
Compress Buses	Buses as bundles
Dissolve Levels	Hierarchical levels in a view flattened with HDL Analyst -> Dissolve Instances or Dissolve to Gates, by setting the number of levels to dissolve.

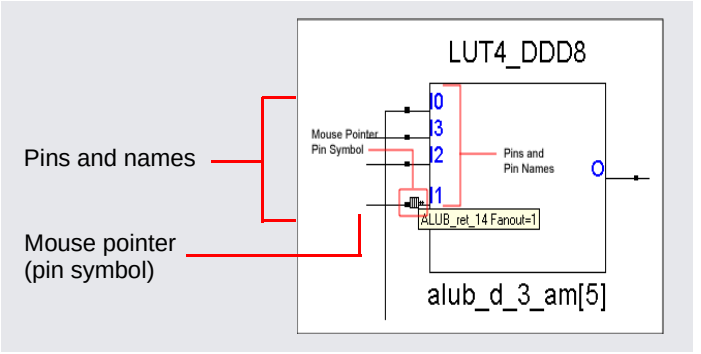
© 2022 Synopsys, Inc.

Option	Controls the display of ...
Instances Filtered Instances Instances added for expansion	Instances on a schematic by setting limits to the number of instances displayed
Instance Name Show Conn Name Show Symbol Name Show Port Name	Object labels
Show Pin Name HDL Analyst->Show All Hier Pins	Pin names. See Pin and Pin Name Display for Opaque Objects, on page 92 and Pin and Pin Name Display for Transparent Objects, on page 93 for details.

Pin and Pin Name Display for Opaque Objects

Although it always displays the pins, the software does not automatically display pin names for opaque hierarchical instances, technology-specific primitives, RAMS, ROMs, and state machines. To display pin names for these objects, enable Options-> HDL Analyst Options->Text->Show Pin Name. The following figures illustrate this display. The first figure shows pins and pin names of an opaque hierarchical instance, and the second figure shows the pins of a technology-specific primitive with its cell contents not displayed.





Pin and Pin Name Display for Transparent Objects

This section discusses pin name display for transparent hierarchical instances in filtered views and technology-specific primitives.

Transparent Hierarchical Instances

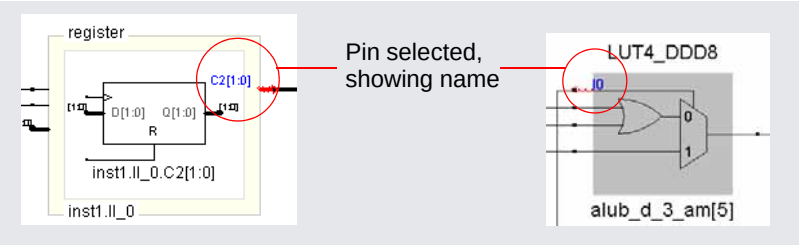
In a filtered schematic, some of the pins on a transparent hierarchical instance might not be displayed because of filtering. To display all the pins, select the instance and select HDL Analyst -> Show All Hier Pins.

To display pin names for the instance, enable Options->HDL Analyst Options->Text ->Show Pin Name. The software temporarily displays the pin name when you move the cursor over a pin. To keep the pin name displayed even after you move the cursor away, select the pin. The name remains until you select something else.

Primitives

To display pin names for technology primitives in the Technology view, enable Options-> HDL Analyst Options->Text->Show Pin Name. The software displays the pin names until the option is disabled. If Show Pin Name is enabled when Options-> HDL Analyst Options->General->Show Cell Interior is also enabled, the primitive is treated like a transparent hierarchical instance, and primitive pin names are only displayed when the cursor moves over the pins. To keep a pin name displayed even after you move the cursor away, select the pin. The name remains until you select something else.

© 2022 Synopsys, Inc.



See also:

- HDL Analyst Options Command, on page 408
- Controlling the Amount of Logic on a Sheet, on page 100
- Analyzing Timing in Schematic Views, on page 328 in the *User Guide*

Basic Operations on Schematic Objects

Basic operations on schematic objects include the following:

- Finding Schematic Objects, on page 95
- Selecting and Unselecting Schematic Objects, on page 96
- Crossprobing Objects, on page 97
- Dragging and Dropping Objects, on page 99

For information about other operations on schematics and schematic objects, see the following:

- Filtering and Flattening Schematics, on page 111
- Timing Information and Critical Paths, on page 117
- Multiple-sheet Schematics, on page 100
- Exploring Design Hierarchy, on page 103

Finding Schematic Objects

You can use the following techniques to find objects in the schematic. For step-by-step procedures using these techniques, see Finding Objects (Standard), on page 283 in the *User Guide*.

- Zooming and panning
- HDL Analyst Hierarchy Browser

  You can use the Hierarchy Browser to browse and find schematic objects. This can be a quick way to locate an object by name if you are familiar with the design hierarchy. See Browsing With the Hierarchy Browser, on page 283 in the *User Guide* for details.

- Edit -> Find command

  The Edit -> Find command is described in Find Command (HDL Analyst), on page 296. It displays the Object Query dialog box, which lists schematic objects by type (Instances, Symbols, Nets, or Ports) and lets you use wildcards to find objects by name. You can also fine-tune your search by setting a range for the search.

© 2022 Synopsys, Inc.

This command selects all found objects, whether or not they are displayed in the current schematic. Although you can search for hidden instances, you cannot find objects that are inside hidden instances at a lower level. Temporarily hiding an instance thus further refines the search range by excluding the internals of a a given instance. This can be very useful when working with transparent instances, because the lower-level details appear at the current level, and cannot be excluded by choosing Current Level Only. See Using Find for Hierarchical and Restricted Searches, on page 285 in the *User Guide*.

- Edit -> Find command combined with filtering

  Edit->Find enhances filtering. Use Find to select by name and hierarchical level, and then filter the design to limit the display to the current selection. Unselected objects are removed. Because Find only adds to the current selection (it never deselects anything already selected), you can use successive searches to build up exactly the selection you need, before filtering.

- Filtering before searching with Edit->Find

  Filtering helps you to fine-tune the range of a search. You can search for objects just within a filtered schematic by limiting the search range to the Current Level Only.

  Filtering adds to the expressive power of displaying search results. You can find objects on different sheets and filter them to see them all together at once. Filtering collapses the hierarchy visually, showing lower-level details nested inside transparent higher-level instances. The resulting display combines the advantage of a high-level, abstract view with detail-rich information from lower levels.

  See Filtering and Flattening Schematics, on page 111 for further information.

Selecting and Unselecting Schematic Objects

Whenever an object is selected in one place it is selected and highlighted everywhere else in the synthesis tool, including all Hierarchy Browsers, all schematics, and the Text Editor. Many commands operate on the currently selected objects, whether or not those objects are visible.

© 2022 Synopsys, Inc.

The following briefly list selection methods; for a concise table of selection procedures, see Selecting Objects in the RTL/Technology Views, on page 268 in the *User Guide*.

Using the Mouse to Select a Range of Schematic Objects

In a Hierarchy Browser, you can select a *range* of schematic objects by clicking the name of an object at one end of the range, then holding the Shift key while clicking the name of an object at the other end of the range.To use the mouse for selecting and unselecting objects in a schematic, the cross-hairs symbol () must appear as the mouse pointer. If this is not currently the case, right-click the schematic background.

Using Commands to Select Schematic Objects

You can select and deselect schematic objects using the commands in the HDL Analyst menu, or use Edit->Find to find and select objects by name.

The HDL Analyst menu commands that affect selection include the following:

- Expansion commands like Expand, Expand to Register/Port, Expand Paths, and Expand Inwards select the objects that result from the expansion. This means that (except for Expand to Register/Port) you can perform successive expansions and expand the set of objects selected.
- The Select All Schematic and Select All Sheet commands select all instances or ports on the current schematic or sheet, respectively.
- The Select Net Driver and Select Net Instances commands select the appropriate objects according to the hierarchical level you have chosen.
- Deselect All deselects all objects in *all* HDL Analyst views.

See also

- Finding Schematic Objects, on page 95
- HDL Analyst Menu, on page 382

Crossprobing Objects

Crossprobing helps you diagnose where coding changes or timing constraints might reduce area or increase performance. When you crossprobe, you select an object in one place and it or its equivalent is automatically selected and

© 2022 Synopsys, Inc.

highlighted in other places. For example, selecting text in the Text Editor automatically selects the corresponding logic in all HDL Analyst views. Whenever a net is selected, it is highlighted through all the hierarchical instances it traverses, at all schematic levels.

Crossprobing Between Different Views

You can crossprobe objects (including logic inside hidden instances) between RTL views, Technology views, the FSM Viewer, HDL source code files, and other text files. Some RTL and source code objects are optimized away during synthesis, so they cannot be crossprobed to certain views.

The following table summarizes crossprobing to and from HDL Analyst (RTL and Technology) views. For information about crossprobing procedures, see Crossprobing, on page 230 in the *User Guide*.

From ...	To ...	Do this ...
Text Editor: log file	Text Editor: HDL source file	Double-click a log file note, error, or warning. The corresponding HDL source code appears in the Text Editor.
Text Editor: HDL code	Analyst view FSM Viewer	The RTL view or Technology view must be open. Select the code in the Text Editor that corresponds to the object(s) you want to crossprobe. The object corresponding to the selected code is automatically selected in the target view, if an HDL source file is in the Text Editor. Otherwise, right-click and choose the Select in Analyst command. To cross-probe from text other than source code, first select Options->HDL Analyst Options and then enable Enhanced Text Crossprobing.
FSM Viewer	Analyst view	The target view must be open. The state machine must be encoded with the onehot style to crossprobe from the transition table. Select a state anywhere in the FSM Viewer (bubble diagram or transition table). The corresponding object is automatically selected in the HDL Analyst view.

© 2022 Synopsys, Inc.

From ...	To ...	Do this ...
Analyst view FSM Viewer	Text Editor	Double-click an object. The source code corresponding to the object is automatically selected in the Text Editor, which is opened to show the selection. If you just select an object, without double-clicking it, the corresponding source code is still selected and displayed in the editor (provided it is open), but the editor window is not raised to the front.
Analyst view	Another open view	Select an object in an HDL Analyst view. The object is automatically selected in all open views. If the target view is the FSM Viewer, then the state machine must be encoded as onehot.
Tcl window	Text Editor	Double-click an error or warning message (available in the Tcl window errors or warnings panel, respectively). The corresponding source code is automatically selected in the Text Editor, which is opened to show the selection.
Text Editor: any text containing instance names, like a timing report	Corresponding instance	Highlight the text, then right-click & choose Select or Filter. Use this to filter critical paths reported in a text file by the FPGA timing analysis tool.

Dragging and Dropping Objects

You can drag and drop objects like instances, nets and pins from the HDL Analyst schematic views to other windows to help you analyze your design or set constraints. You can drag and drop objects from an RTL or Technology views to the following other windows:

- SCOPE editor
- Text editor window
- Tcl window

© 2022 Synopsys, Inc.

Multiple-sheet Schematics

When there is too much logic to display on a single sheet, the HDL Analyst tool uses additional schematic sheets. Large designs can take several sheets. In a hierarchical schematic, each module consists of one or more sheets. Sheet connector symbols (Sheet Connectors, on page 87) mark logic connections from one sheet to the next.

For more information, see

- Controlling the Amount of Logic on a Sheet, on page 100
- Navigating Among Schematic Sheets, on page 100
- Multiple Sheets for Transparent Instance Details, on page 102

Controlling the Amount of Logic on a Sheet

You can control the amount of logic on a schematic sheet using the options in Options->HDL Analyst Options->Sheet Size. The Maximum Instances option sets the maximum number of instances on an unfiltered schematic sheet. The Maximum Filtered Instances option sets the maximum number of instances displayed at any given hierarchical level on a filtered schematic sheet.

See also:

- HDL Analyst Options Command, on page 408
- Setting Schematic Preferences, on page 272 of the *User Guide*.

Navigating Among Schematic Sheets

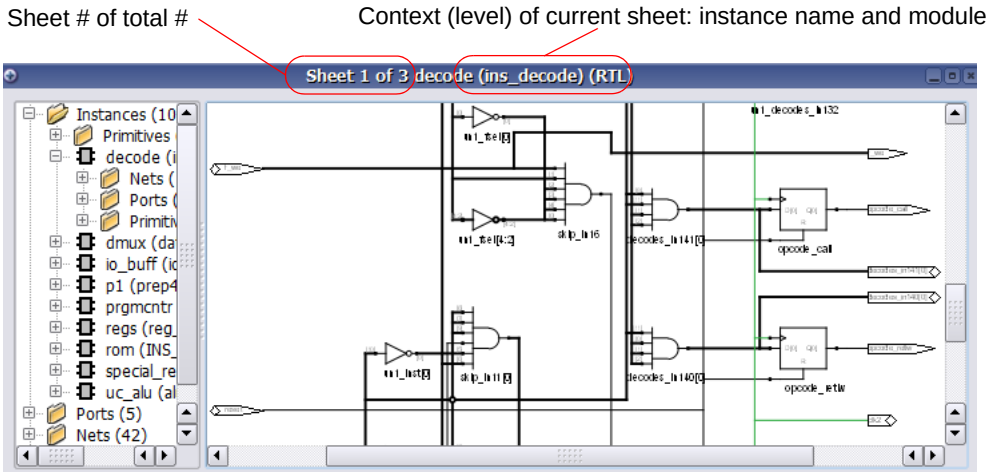
This section describes how to navigate among the sheets in a given schematic. The window title bar lets you know where you are at any time.

Multisheet Orientation in the Title Bar

The window title bar of an RTL view or Technology view indicates the current context. For example, ins_decode (of module decode) in the title indicates that the current schematic level displays the instance ins_decode (which is of module decode). The objects shown are those comprising that instance.

© 2022 Synopsys, Inc.

The title bar also indicates, for the current schematic, the number of the displayed sheet, and the total number of sheets — for example, sheet 2 of 4. A schematic is initially opened to its first sheet.



Navigating Among Sheets

You can navigate among different sheets of a schematic in these ways:

- Follow a sheet connector, by right-clicking it and choosing a connecting sheet from the popup menu
- Use the sheet navigation commands of the View menu: Next Sheet, Previous Sheet, and View Sheets, or their keyboard shortcut or icon equivalents
- Use the history navigation commands of the View menu (Back and Forward), or their keyboard shortcuts or icon equivalents to navigate to sheets stored in the display history

For details, see Working with Multisheet Schematics, on page 270 in the *User Guide*.

You can navigate among different design levels by pushing and popping the design hierarchy. Doing so adds to the display history of the View menu, so you can retrace your push/pop steps using View -> Back and View->Forward. After pushing down, you can either pop back up or use View->Back.

© 2022 Synopsys, Inc.

See also:

- Filtering and Flattening Schematics, on page 111
- View Menu: RTL and Technology Views Commands, on page 305
- Pushing and Popping Hierarchical Levels, on page 103

Multiple Sheets for Transparent Instance Details

The details of a transparent instance in a filtered view are drawn in two ways:

- Generally, these interior details are spread out over multiple sheets at the same schematic level (module) as the instance that contains them. You navigate these sheets as usual, using the methods described in Navigating Among Schematic Sheets, on page 100.
- If the number of nested contents exceeds the limit set with the Filtered Instances option (Options->HDL Analyst Options), the nested contents are drawn on separate sheets. The parent hierarchical instance is empty, with a notation (for example, Go to sheets 4-16) inside it, indicating which sheets contain its lower-level details. You access the sheets containing the lower-level details using the sheet navigation commands of the View menu, such as Next Sheet.

See also:

- Controlling the Amount of Logic on a Sheet, on page 100
- View Menu: RTL and Technology Views Commands, on page 305

© 2022 Synopsys, Inc.

Exploring Design Hierarchy

The hierarchy in your design can be explored in different ways. The following sections explain how to move between hierarchical levels:

- Pushing and Popping Hierarchical Levels, on page 103
- Navigating With a Hierarchy Browser, on page 106
- Looking Inside Hierarchical Instances, on page 108

Pushing and Popping Hierarchical Levels

You can navigate your design hierarchy by pushing down into a high-level schematic object or popping back up. Pushing down into an object takes you to a lower-level schematic that shows the internal logic of the object. Popping up from a lower level brings you back to the parent higher-level object.

Pushing and popping is best suited for traversing the hierarchy of a specific object. If you want a more general view of your design hierarchy, use the Hierarchy Browser instead. See Navigating With a Hierarchy Browser, on page 106 and Looking Inside Hierarchical Instances, on page 108 for other ways of viewing design hierarchy.

Pushable Schematic Objects

To push into an instance, it must have hierarchy. You can push into the object regardless of its position in the design hierarchy; for example, you can push into the object if it is shown nested inside a transparent instance. You can push down into the following kinds of schematic objects:

- Non-hidden hierarchical instances. To push into a hidden instance, unhide it first.
- Technology-specific primitives (not logic primitives)
- Inferred ROMs and state machines in RTL views. Inferred ROMs, RAMs, and state machines do not appear in Technology views, because they are resolved into technology-specific primitives.

© 2022 Synopsys, Inc.

When you push/pop, the HDL Analyst window displays the appropriate level of design hierarchy, except in the following cases:

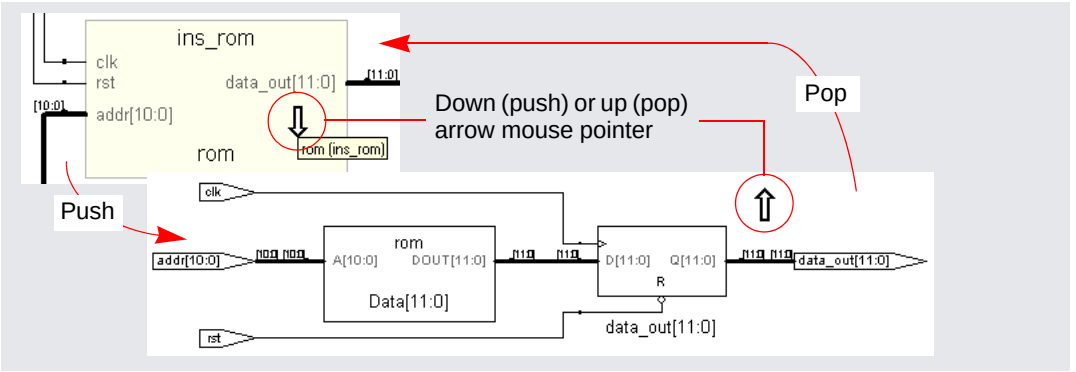
- When you push into an inferred state machine in an RTL view, the `statemachine.info` text file opens. The FSM Viewer opens, with graphical information about the FSM. See the FSM Viewer Window, on page 81, for more information.
- When you push into an inferred ROM in an RTL view, the Text Editor window opens and displays the ROM data table (`rom.info` file).

You can use the following indicators to determine whether you can push into an object:

- The mouse pointer shape when Push/Pop mode is enabled. See How to Push and Pop Hierarchical Levels, on page 104 for details.
- A small H symbol () in the lower left corner indicates a hidden instance, and you cannot push into it.
- The Hierarchy Browser symbols indicates the type of instance and you can use that to determine whether you can push into an object. For example, hierarchical instance (), technology-specific primitive (), logic primitive such as XOR (), or other primitive instance (). The browser symbol does not indicate whether or not an instance is hidden.
- The *status bar* at the bottom of the main synthesis tool window reports information about the object under the pointer, including whether or not it is a hidden instance or a primitive.

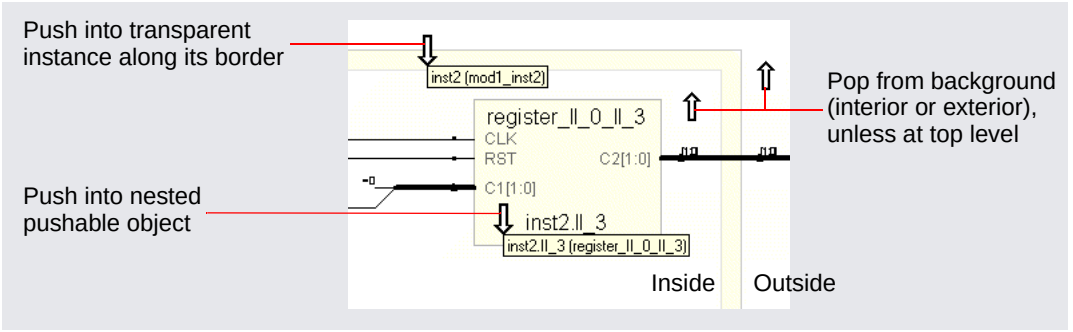
How to Push and Pop Hierarchical Levels

You push/pop design levels with the HDL Analyst Push/Pop mode. To enable or disable this mode, toggle View->Push/Pop Hierarchy, use the icon, or use the appropriate mouse strokes.



Once Push/Pop mode is enabled, you push or pop as follows:

- To *pop*, place the pointer in an empty area of the schematic background, then click or use the appropriate mouse stroke. The background area inside a transparent instance acts just like the background area outside the instance.
- To *push* into an object, place the mouse pointer over the object and click or use the appropriate mouse stroke. To push into a transparent instance, place the pointer over its pale yellow border, not its hollow (white) interior. Pushing into an object nested inside a transparent hierarchical instance descends to a lower level than pushing into the enclosing transparent instance. In the following figure, pushing into transparent instance inst2 descends one level; pushing into nested instance inst2.ll_3 descends two levels.



The following arrow mouse pointers indicate status in Push/Pop mode. For other indicators, see Pushable Schematic Objects, on page 103.

© 2022 Synopsys, Inc.

A down arrow 	Indicates that you can push (descend) into the object under the pointer and view its details at the next lower level.
An up arrow 	Indicates that there is a hierarchical level above the current sheet.
A crossed-out double arrow 	Indicates that there is no accessible hierarchy above or below the current pointer position. If the pointer is over the schematic background it indicates that the current level is the top and you cannot pop higher. If the pointer is over an object, the object is an object you cannot push into: a non-hierarchical instance, a hidden hierarchical instance, or a black box.

See also:

- Hidden Hierarchical Instances, on page 90
- Transparent and Opaque Display of Hierarchical Instances, on page 89
- Using Mouse Strokes, on page 50
- Navigating With a Hierarchy Browser, on page 106

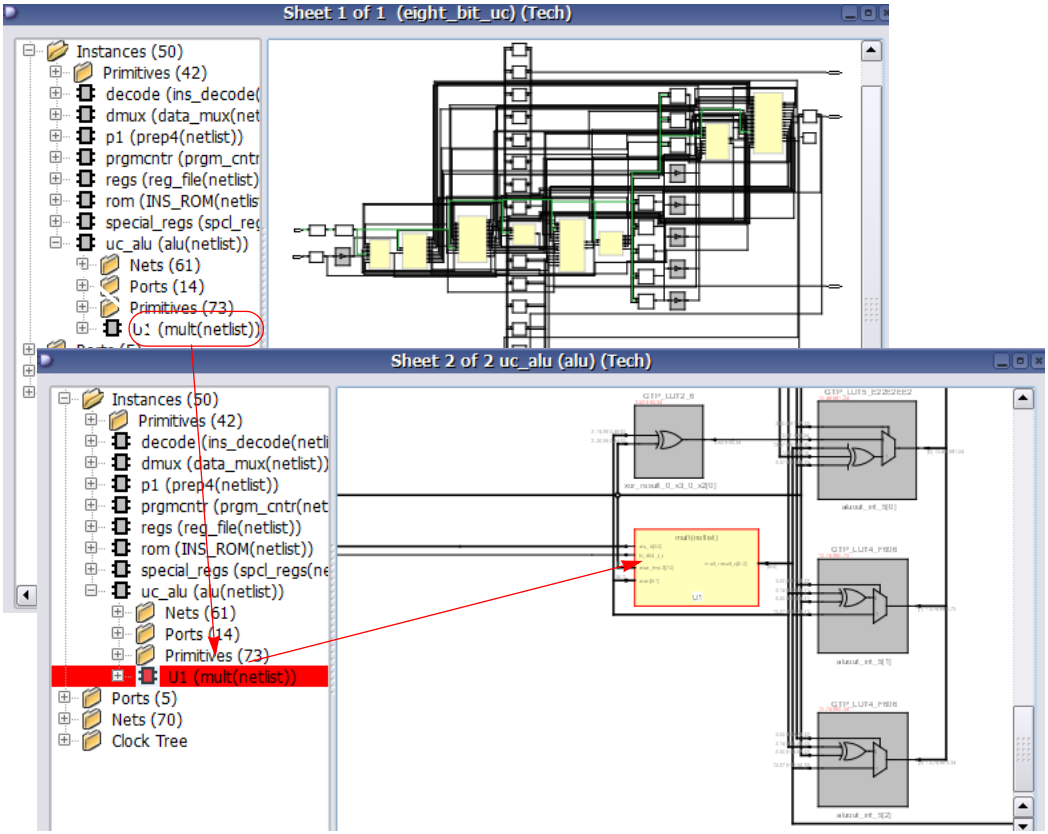
Navigating With a Hierarchy Browser

Hierarchy Browsers are designed for locating objects by browsing your design. To move between design levels of a particular object, use Push/Pop mode (see Pushing and Popping Hierarchical Levels, on page 103 and Looking Inside Hierarchical Instances, on page 108 for other ways of viewing design hierarchy).

The browser in the RTL view displays the hierarchy specified in the RTL design description. The browser in the Technology view displays the hierarchy of your design after technology mapping.

Selecting an object in the browser displays it in the schematic, because the two are linked. Use the Hierarchy Browser to traverse your hierarchy and select ports, nets, components, and submodules. The browser categorizes the objects, and accompanies each with a symbol that indicates the object type. The following figure shows crossprobing between a schematic and the hierarchy browser.

© 2022 Synopsys, Inc.



Selecting an object in the browser causes the schematic to display it also.

Explore the browser hierarchy by expanding or collapsing the categories in the browser. You can also use the arrow keys (left, right, up, down) to move up and down the hierarchy and select objects. To select more than one object, press Ctrl and select the objects in the browser. To select a range of schematic objects, click an object at one end of the range, then hold the Shift key while clicking the name of an object at the other end of the range.

© 2022 Synopsys, Inc.

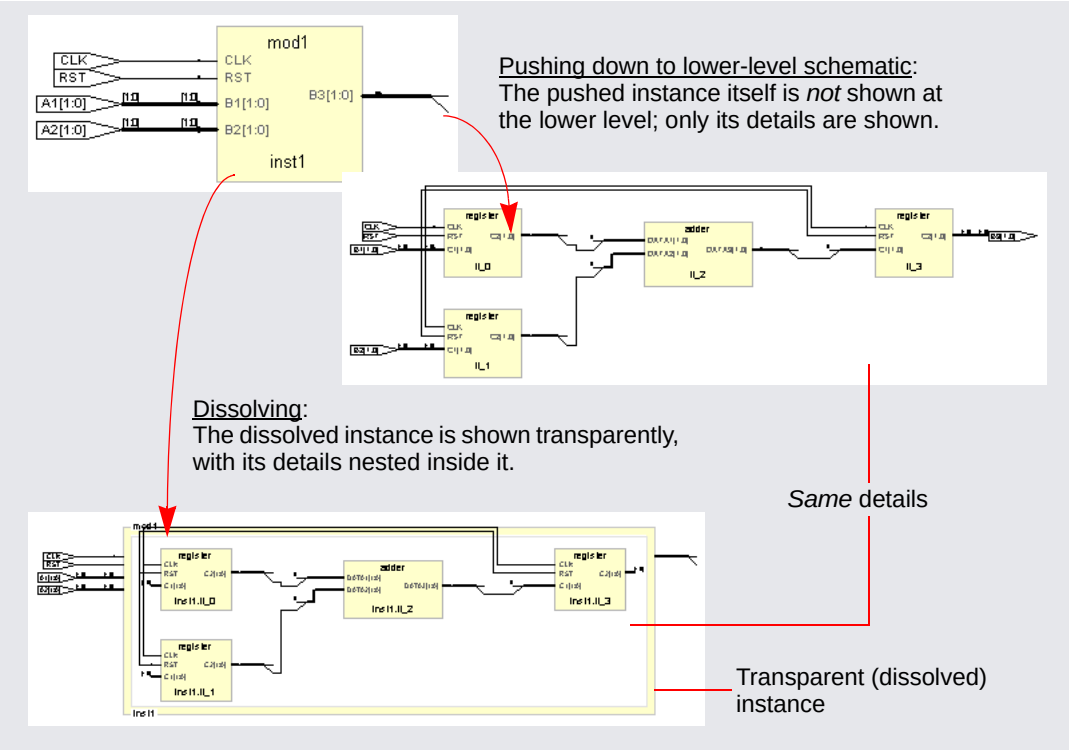
See also:

- Crossprobing Objects, on page 97
- Pushing and Popping Hierarchical Levels, on page 103
- Hierarchy Browser Popup Menu Commands, on page 438

Looking Inside Hierarchical Instances

An alternative method of viewing design hierarchy is to examine transparent hierarchical instances (see Navigating With a Hierarchy Browser, on page 106 and Navigating With a Hierarchy Browser, on page 106 for other ways of viewing design hierarchy). A transparent instance appears as a hollow box with a pale yellow border. Inside this border are transparent and opaque objects from lower design levels.

Transparent instances provide design context. They show the lower-level logic nested within the transparent instance at the current design level, while pushing shows the same logic a level down. The following figure compares the same lower-level logic viewed in a transparent instance and a push operation:



© 2022 Synopsys, Inc.

You cannot control the display of transparent instances directly. However, you can perform the following operations, which result in the display of transparent instances:

- Hierarchically expand an object (using the expansion commands in the HDL Analyst menu).
- Dissolve selected hierarchical instances in a *filtered* schematic (HDL Analyst -> Dissolve Instances).
- Filter a schematic, after selecting multiple objects at more than one level. See Commands That Result in Filtered Schematics, on page 111 for additional information.

These operations only make *non-hidden hierarchical* instances transparent. You cannot dissolve hidden or primitive instances (including technology-specific primitives). However, you can do the following:

- Unhide hidden instances, then dissolve them.
- Push down into technology-specific primitives to see their lower-level details, and you can show the interiors of all technology-specific primitives.

See also:

- Pushing and Popping Hierarchical Levels, on page 103
- Navigating With a Hierarchy Browser, on page 106
- HDL Analyst Command, on page 383
- Transparent and Opaque Display of Hierarchical Instances, on page 89
- Hidden Hierarchical Instances, on page 90

© 2022 Synopsys, Inc.

Filtering and Flattening Schematics

This section describes the HDL Analyst commands that result in filtered and flattened schematics. It describes:

- Commands That Result in Filtered Schematics, on page 111
- Combined Filtering Operations, on page 112
- Returning to The Unfiltered Schematic, on page 113
- Commands That Flatten Schematics, on page 113
- Selective Flattening, on page 114
- Filtering Compared to Flattening, on page 115

Commands That Result in Filtered Schematics

A filtered schematic shows a subset of your design. Any command that *results in a filtered schematic* is a filtering command. Some commands, like the Expand commands, increase the amount of logic displayed, but they are still considered filtering commands because they result in a filtered view of the design. Other commands like Filter Schematic and Isolate Paths remove objects from the current display.

Filtering commands include the following:

- Filter Schematic, Isolate Paths – reduce the displayed logic.
- Dissolve Instances (in a filtered schematic) – makes selected instances transparent.
- Expand, Expand to Register/Port, Expand Paths, Expand Inwards, Select Net Driver, Select Net Instances – display logic connected to the current selection.
- Show Critical Path, Flattened Critical Path, Hierarchical Critical Path – show critical paths.

All the filtering commands, except those that display critical paths, operate on the currently selected schematic object(s). The critical path commands operate on your entire design, regardless of what is currently selected.

All the filtering commands except Isolate Paths are accessible from the HDL Analyst menu; Isolate Paths is in the RTL view and Technology view popup menus (along with most of the other commands above).

© 2022 Synopsys, Inc.

For information about filtering procedures, see *Filtering Schematics, on page 308* in the *User Guide*.

See also:

- Filtered and Unfiltered Schematic Views, on page 84
- HDL Analyst Menu, on page 382 and RTL and Technology Views Popup Menus, on page 438

Combined Filtering Operations

Filtering operations are designed to be used in combination, successively. You can perform a sequence of operations like the following:

1. Use Filter Schematic to filter your design to examine a particular instance. See HDL Analyst Menu: Filtering and Flattening Commands, on page 385 for a description of the command.
2. Select Expand to expand from one of the output pins of the instance to add its immediate successor cells to the display. See HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383 for a description of the command.
3. Use Select Net Driver to add the net driver of a net connected to one of the successors. See HDL Analyst Menu: Hierarchical and Current Level Submenus, on page 383 for a description of the command.
4. Use Isolate Paths to isolate the net driver instance, along with any of its connecting paths that were already displayed. See HDL Analyst Menu: Analysis Commands, on page 389 for a description of the command.

Filtering operations add their resulting filtered schematics to the history of schematic displays, so you can use the View menu Forward and Back commands to switch between the filtered views. You can also combine filtering with the search operation. See Finding Schematic Objects, on page 95 for more information.

© 2022 Synopsys, Inc.

Returning to The Unfiltered Schematic

A filtered schematic often loses the design context, as it is removed from the display by filtering. After a series of multiple or complex filtering operations, you might want to view the context of a selected object. You can do this by:

- Selecting a higher level object in the Hierarchy Browser; doing so always crossprobes to the corresponding object in the original schematic.
- Using Show Context to take you directly from a selected instance to the corresponding context in the original, unfiltered schematic.
- Using Goto Net Driver to go from a selected net to the corresponding context in the original, unfiltered schematic.

There is no Unfilter command. Use Show Context to see the unfiltered schematic containing a given instance. Use View->Back to return to the previous, unfiltered display after filtering an unfiltered schematic. You can go back and forth between the original, unfiltered design and the filtered schematics, using the commands View->Back and Forward.

See also:

- RTL and Technology Views Popup Menus, on page 438
- View Menu: RTL and Technology Views Commands, on page 305

Commands That Flatten Schematics

A flattened schematic contains no hierarchical objects. Any command that results in a flattened schematic is a flattening command. This includes the following.

Command	Unfiltered Schematic	Filtered Schematic
Dissolve Instances	Flattens selected instances	--
Flatten Current Schematic (Flatten Schematic)	Flattens at the current level and all lower levels. RTL view: flattens to generic logic level Technology view: flattens to technology-cell level	Flattens only non-hidden transparent hierarchical instances; opaque and hidden hierarchical instances are not flattened.
RTL->Flattened View	Creates a new, unfiltered RTL schematic of the entire design, flattened to the level of generic logic cells.	

© 2022 Synopsys, Inc.

Command	Unfiltered Schematic Filtered Schematic
Technology-> Flattened View	Creates a new, unfiltered Technology schematic of the entire design, flattened to the level of technology cells.
Technology-> Flattened to Gates View	Creates a new, unfiltered Technology schematic of the entire design, flattened to the level of Boolean logic gates.
Technology-> Flattened Critical Path	Creates a filtered, flattened Technology view schematic that shows only the instances with the worst slack times and their path.
Unflatten Schematic	Undoes any flattening done by Dissolve Instances and Flatten Current Schematic at the current schematic level. Returns to the original schematic, as it was before flattening (and any filtering).

All the commands are on the HDL Analyst menu except Unflatten Schematic, which is available in a schematic popup menu.

The most versatile commands, are Dissolve Instances and Flatten Current Schematic, which you can also use for selective flattening (Selective Flattening, on page 114).

See also:

- Filtering Compared to Flattening, on page 115
- Selective Flattening, on page 114

Selective Flattening

By default, flattening operations are not very selective. However, you can selectively flatten particular instances with these command (see RTL and Technology Views Popup Menus, on page 438 for descriptions):

- Use Hide Instances to hide instances that you do *not* want to flatten, then flatten the others (flattening operations do not recognize hidden instances). After flattening, you can Unhide Instances that are hidden.

© 2022 Synopsys, Inc.

- Flatten selected hierarchical instances using one of these commands:
  - If the current schematic is unfiltered, use Dissolve Instances.
  - If the schematic is filtered, use Dissolve Instances, followed by Flatten Current Schematic. In a filtered schematic, Dissolve Instances makes the selected instances transparent and Flatten Current Schematic flattens only transparent instances.

The Dissolve Instances and Flatten Current Schematic (or Flatten Schematic) commands behave differently in filtered and unfiltered schematics as outlined in the following table:

Command	Unfiltered Schematic	Filtered Schematic
Dissolve Instances	Flattens selected instances	Provides virtual flattening: makes selected instances transparent, displaying their lower-level details.
Flatten Current Schematic Flatten Schematic	Flattens *everything* at the current level and below	Flattens only the non-hidden, *transparent* hierarchical instances: does not flatten opaque or hidden instances. See below for details of the process.

In a filtered schematic, flattening with Flatten Current Schematic is actually a two-step process:

1. The transparent instances of the schematic are flattened in the context of the entire design. The result of this step is the entire hierarchical design, with the transparent instances of the filtered schematic replaced by their internal logic.
2. The original filtering is then restored: the design is refiltered to show only the logic that was displayed before flattening.

Although the result displayed is that of Step 2, you can view the intermediate result of Step 1 with View->Back. This is because the display history is erased before flattening (Step 1), and the result of Step 1 is added to the history as if you had viewed it.

Filtering Compared to Flattening

As a general rule, use filtering to examine your design, and flatten it only if you really need it. Here are some reasons to use filtering instead of flattening:

© 2022 Synopsys, Inc.

- Filtering before flattening is a more efficient use of computer time and memory. Creating a new view where everything is flattened can take considerable time and memory for a large design. You then filter anyway to remove the flattened logic you do not need.
- Filtering is selective. On the other hand, the default flattening operations are global, the entire design is flattened from the current level down. Similarly, the inverse operation (UnFlatten Schematic) unflattens everything on the current schematic level.
- Flattening operations eliminate the *history* for the current view, you can not use View->Back after flattening. You can, however, use UnFlatten Schematic to regenerate the unflattened schematic.

See also:

- RTL and Technology Views Popup Menus, on page 438
- Selective Flattening, on page 114

© 2022 Synopsys, Inc.

Timing Information and Critical Paths

The HDL Analyst tool provides several ways of examining critical paths and timing information, to help you analyze problem areas. The different ways are described in the following sections.

- Timing Reports, on page 117
- Critical Paths and the Slack Margin Parameter, on page 118
- Examining Critical Path Schematics, on page 119

See the following for more information about timing and result analysis:

- Watch Window, on page 34
- Log File, on page 145
- Chapter 12, *Optimizing Processes for Productivity* in the *User Guide*

Timing Reports

When you synthesize a design, a default timing report is automatically written to the log file, which you can view using View->View Log File. This report provides a clock summary, I/O timing summary, and detailed timing information for your design.

For certain device technologies, you can use the Analysis->Timing Analyst command to generate a custom timing report. Use this command to specify start and end points of paths whose timing interests you, and set a limit for the number of paths to analyze between these points. By default, the sequential instances, input ports, and output ports that are currently selected in the Technology views of the design are the candidates for choosing start and end points. In addition, the start and end points of the previous Timing Analyst run become the default start and end points for the next run. When analyzing timing, any latches in the path are treated as level-sensitive registers.

The custom timing report is stored in a text file named *resultsfile*.ta, where *resultsfile* is the name of the results file (see Implementation Results Panel, on page 329). In addition, a corresponding output netlist file is generated, named *resultsfile*_ta.srm. Both files are in the implementation results directory.

© 2022 Synopsys, Inc.

The Timing Analyst dialog box provides check boxes for viewing the text report (Open Report) in the Text Editor and the corresponding netlist (Open Schematic) in a Technology view. This Technology view of the timing path, labeled Timing View in the title bar, is special in two ways:

- The Timing View shows only the paths you specify in the Timing Analyst dialog box. It corresponds to a special design netlist that contains critical timing data.
- The Timing Analyst and Show Critical Path commands (and equivalent icons and shortcuts) are unavailable whenever the Timing View is active.

See also:

- Analysis Menu, on page 370
- Timing Reports, on page 150
- Log File, on page 145

Critical Paths and the Slack Margin Parameter

The HDL Analyst tool can isolate critical paths in your design, so that you can analyze problem areas, add timing constraints where appropriate, and resynthesize for better results.

After you successfully run synthesis, you can display just the critical paths of your design using any of the following commands from the HDL Analyst menu:

- Hierarchical Critical Path
- Flattened Critical Path
- Show Critical Path

The first two commands create a new Technology view, hierarchical or flattened, respectively. The Show Critical Path command reuses the current Technology view. Neither the current selection nor the current sheet display have any effect on the result. The result is flat if the entire design was already flat; otherwise it is hierarchical. Use Show Critical Path if you want to maintain the existing display history.

All these commands filter your design to show only the instances (and their paths) with the worst slack times. They also enable HDL Analyst -> Show Timing Information, displaying timing information.

© 2022 Synopsys, Inc.

Negative slack times indicate that your design has not met its timing requirements. The worst (most negative) slack time indicates the amount by which delays in the critical path cause the timing of the design to fail. You can also obtain a *range* of worst slack times by setting the *slack margin* parameter to control the sensitivity of the critical-path display. Instances are displayed only if their slack times are within the slack margin of the (absolutely) worst slack time of the design.

The slack margin is the criterion for distinguishing worst slack times. The larger the margin, the more relaxed the measure of worst, so the greater the number of critical-path instances displayed. If the slack margin is zero (the default value), then only instances with the worst slack time of the design are shown. You use HDL Analyst->Set Slack Margin to change the slack margin.

The critical-path commands do not calculate a single critical path. They filter out instances whose slack times are not too bad (as determined by the slack margin), then display the remaining, worst-slack instances, together with their connecting paths.

For example, if the worst slack time of your design is -10 ns and you set a slack margin of 4 ns, then the critical path commands display all instances with slack times between -6 ns and -10 ns.

See also:

- HDL Analyst Menu, on page 382
- HDL Analyst Command, on page 383
- Handling Negative Slack, on page 334 of the *User Guide*
- Analyzing Timing in Schematic Views, on page 328 of the *User Guide*

Examining Critical Path Schematics

Use successive filtering operations to examine different aspects of the critical path. After filtering, use View -> Back to return to the previous point, then filter differently. For example, you could use the command Isolate Paths to examine the cone of logic from a particular pin, then use the Back command to return to the previous display, then use Isolate Paths on a different pin to examine a different logic cone, and so on.

© 2022 Synopsys, Inc.

Also, the Show Context and Goto Net Driver commands are particularly useful after you have done some filtering. They let you get back to the original, unfiltered design, putting selected objects in context.

See also:

- Returning to The Unfiltered Schematic, on page 113
- Filtering and Flattening Schematics, on page 111

© 2022 Synopsys, Inc.

SYNOPSYS®
Silicon to Software™

CHAPTER 4

Constraint Guidelines

Constraints are used in the FPGA synthesis environment to achieve optimal design results. Timing constraints set performance goals, non-timing constraints (design constraints) guide the tool through optimizations that further enhance performance and physical constraints define regions and locations for placement-aware synthesis.

This chapter provides an overview of how constraints are handled in the FPGA synthesis environment.

- Constraint Types, on page 122
- Constraint Files, on page 123
- FDC Constraints, on page 125
- Methods for Creating Constraints, on page 125
- Constraint Checking, on page 127
- Database Object Search, on page 129
- Forward Annotation, on page 129
- Auto Constraints, on page 130

© 2022 Synopsys, Inc.

Constraint Types

One way to ensure the FPGA synthesis tools achieve the best quality of results for your design is to define proper constraints. In the FPGA environment, constraints can be categorized by the following types:

Type	Description
Timing	Performance constraints that guide the synthesis tools to achieve optimal results. Examples: clocks (create_clock), clock groups (set_clock_groups), and timing exceptions like multicycle and false paths (set_multicycle_path...) See FDC Constraints, on page 125 for information on defining these constraints.
Design	Additional design goals that enhance or guide tool optimizations. Examples: Attributes and directives (define_attribute, define_global_attribute), I/O standards (define_io_standard), and compile points (define_compile_point).

The easiest way to specify constraints is through the SCOPE interface. The tool saves timing and design constraints to an FDC file that you add to your project.

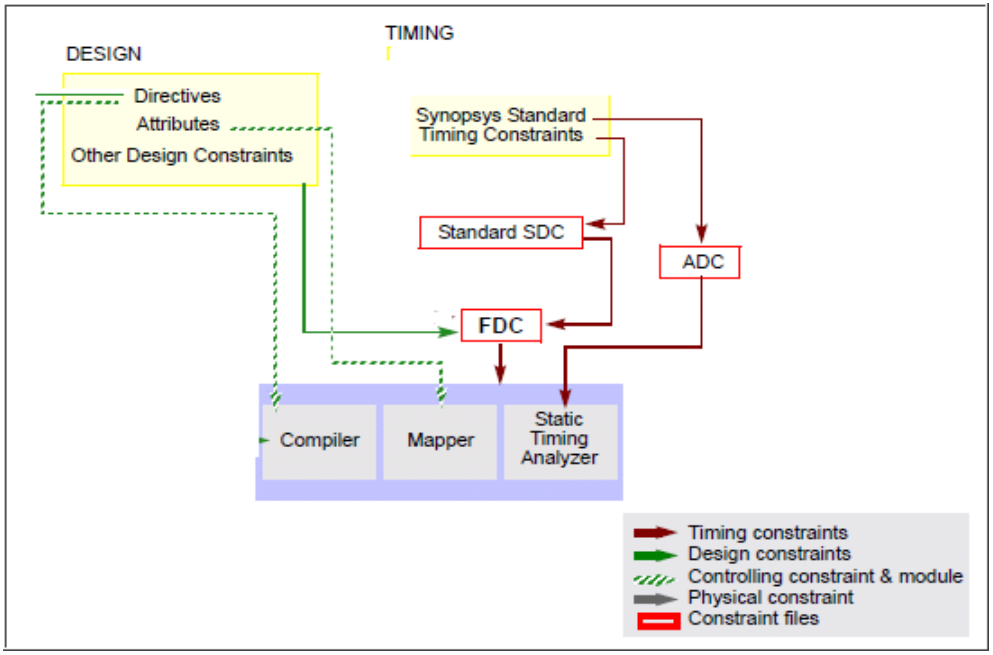
See Also

Constraint Files, on page 123	Overview of constraint files
FDC Constraints, on page 125	Overview of timing constraint definitions and FDC file generation.
SCOPE Constraints Editor, on page 197	Information about automatic generation of timing and design constraints.
Timing Constraints, on page 240	Timing constraint syntax
Design Constraints, on page 282	Design constraint syntax

© 2022 Synopsys, Inc.

Constraint Files

The figure below shows the files used for specifying various types of constraints. The FDC file is the most important one and is the primary file for both timing and non-timing design constraints. The other constraint files are used for specific features or as input files to generate the FDC file, as described in FDC Constraints, on page 125. The figure also indicates the specific processes controlled by attributes and directives.



© 2022 Synopsys, Inc.

The table is a summary of the various kinds of constraint files.

File	Type	Common Commands	Comments
FDC	Timing constraints Design constraints	create_clock, set_multicycle_delay … define_attribute, define_io_standard …	Used for synthesis. Includes timing constraints that follow the Synopsys standard format as well as design constraints.
ADC	Timing constraints for timing analysis	create_clock, set_multicycle_delay …	Used with the stand-alone timing analyzer.

© 2022 Synopsys, Inc.

FDC Constraints

The FPGA design constraints (FDC) file contains constraints that the tool uses during synthesis. This FDC file includes both timing constraints and non-timing constraints in a single file.

- Timing constraints define performance targets to achieve optimal results. The constraints follow the Synopsys standard format, such as create_clock, set_input_delay, and set_false_path.
- Non-timing (or design constraints) define additional goals that help the tool optimize results. These constraints are unique to the FPGA synthesis tools and include constraints such as define_attribute, define_io_standard, and define_compile_point.

The recommended method to define constraints is to enter them in the SCOPE editor, and the tool automatically generates the appropriate syntax. If you define constraints manually, use the appropriate syntax for each type of constraint (timing or non-timing), as described above. See Methods for Creating Constraints, on page 125 for details on generating constraint files.

Methods for Creating Constraints

Constraints are passed to the synthesis environment in FDC files using Tcl command syntax.

New Designs

For new designs, you can specify constraints using any of the following methods:

Definition Method	Description
SCOPE Editor (fdc file)– Recommended	Use this method to specify constraints wherever possible. The SCOPE editor automatically generates fdc constraints with the right syntax. You can use it for most constraints. See SCOPE Constraints Editor, on page 197, for information how to use SCOPE to automatically generate constraint syntax. Access: File->New->FPGA Design Constraints …

© 2022 Synopsys, Inc.

Definition Method	Description
Manually-Entered Text Editor (fdc file, all other constraint files)	You can manually enter constraints in a text file. Make sure to use the correct syntax for the timing and design commands. The SCOPE GUI includes a TCL View with an advanced text editor, where you can manually generate the constraint syntax. For a description of this view, see TCL View, on page 220. You can also open any constraint file in a text editor to modify it.
Source Code Attributes/Directives (HDL files)	Directives must be entered in the source code because they affect the compiler. Do not include any other constraints in the source code, as this makes the source code less portable. In addition, you must recompile the design for the constraints to take effect. Attributes can be entered through the SCOPE interface, as they affect the mapper, not the compiler.
Automatic— First Pass	Enable the Auto Constrain button in the Project view to have the tool automatically generate constraints based on inferred clocks. See Using Auto Constraints, on page 345 in the *User Guide* for details. Use this method as a quick first pass to get an idea of what constraints can be set.

If there are multiple timing exception constraints on the same object, the software uses the guidelines described in Conflict Resolution for Timing Exceptions, on page 236 to determine the constraint that takes precedence.

See Also

To specify the correct syntax for the timing and design commands, see:

- Chapter 4, *Constraints Commands*
- *Attribute Reference Manual*

To use the SCOPE editor, see:

- SCOPE Constraints Editor
- Chapter 5, *Specifying Constraints*

© 2022 Synopsys, Inc.

Constraint Checking

The synthesis tools include several features to help you debug and analyze design constraints. Use the constraint checker to check the syntax and applicability of the timing constraints in the project. The synthesis log file includes a timing report as well as detailed reports on the compiler, mapper, and resource usage information for the design. A stand-alone timing analyzer (STA) generates a customized timing report when you need more details about specific paths or want to modify constraints and analyze, without resynthesizing the design. The following sections provide more information about these features.

Constraint Checker

Check syntax and other pertinent information on your constraint files using Run->Constraint Check or the Check Constraints button in the SCOPE editor. This command generates a report that checks the syntax and applicability of the timing constraints that includes the following information:

- Constraints that are not applied
- Constraints that are valid and applicable to the design
- Wildcard expansion on the constraints
- Constraints on objects that do not exist

Note: Using collections with Tcl control constructs (such as if, for, foreach, and while) can produce unexpected synthesis results. Avoid defining constraints for collections with control constructs, especially since the constraint checker does not recognize these built-in Tcl commands.

See Constraint Checking Report, on page 157 for details.

Timing Constraint Report Files

The results of running constraint checking, synthesis, and stand-alone timing analysis are provided in reports that help you analyze constraints.

© 2022 Synopsys, Inc.

Use these files for additional timing constraint analysis:

File	Description
_cck.rpt	Lists the results of running the constraint checker.See Constraint Checking Report, on page 157).
_cck_fdc_rpt	Lists the wildcard expansion results of running the constraint checker for collections with the get_* and all_* object query commands using the check_fdc_query Tcl command. See check_fdc_query, on page 27 for more information.
_scck.rpt	Lists the results of running the constraint checker for collections with the get_* and all_* object query commands.
.ta	Reports timing analysis results. See Generating Custom Timing Reports with STA, on page 335).
.srr or .htm	Reports post-synthesis timing results as part of the text or HTML log file (see Timing Reports, on page 150 and Log File, on page 145).

Database Object Search

To apply constraints, you have to search the database to find the appropriate objects. Sometimes you might want to search for and apply the same constraint to multiple objects. The FPGA tools provide some Tcl commands to facilitate the search for database objects:

Commands	Common Commands	Description
Find	Tcl Find, open_design...	Lets you search for design objects to form collections that can apply constraints to the group. See Using Collections, on page 142 and find, on page 127.
Collections	define_collection, c_union...	Create, copy, evaluate, traverse, and filter collections. See Using Collections, on page 142 and Collection Commands, on page 144 for more information.

Forward Annotation

The tool can automatically generate vendor-specific constraint files for forward annotation to the place-and-route tools when you enable the Write Vendor Constraints switch (on the Implementation Results tab) or use the -write_apr_constraint option of the set_option command.

Vendor	File Extension
Pango	SCF

For information about how forward annotation is handled for your target technology, refer to the appropriate vendor chapter of the *FPGA Synthesis Reference Manual*.

© 2022 Synopsys, Inc.

Auto Constraints

Auto constraints are automatically generated by the synthesis tool, however, these do not replace regular timing constraints in the normal synthesis flow. Auto constraints are intended as a quick first pass to evaluate the kind of timing constraints you need to set in your design.

To enable this feature and automatically generate register-to-register constraints, use the Auto Constrain option on the left panel of the Project view. For details, see Using Auto Constraints, on page 345 in the *User Guide*.

© 2022 Synopsys, Inc.

SYNOPSYS®
Silicon to Software™

CHAPTER 5

Input and Result Files

This chapter describes the input and output files used by the synthesis tool.

- Input Files, on page 132
- Libraries, on page 136
- Output Files, on page 140
- Log File, on page 145
- Timing Reports, on page 150
- Constraint Checking Report, on page 157

© 2022 Synopsys, Inc.

Input Files

The following table describes the input files used by the synthesis tool.

Extension	File	Description
.adc	Analysis Design Constraint	Contains timing constraints to use for stand-alone timing analysis. Constraints in this file are used only for timing analysis and do not change the result files from synthesis. Constraints in the adc file are applied in addition to fdc constraints used during synthesis. Therefore, adc constraints affect timing results only if there are no conflicts with fdc constraints. You can forward annotate adc constraints to your vendor constraint file without rerunning synthesis. See Using Analysis Design Constraints, on page 338 of the *User Guide* for details.
.fdc	Synopsys FPGA Design Constraint	Create FPGA timing and design constraints with SCOPE.
.ini	Configuration and Initialization	Governs the behavior of the synthesis tool. You normally do *not* need to edit this file. For example, use the HDL Analyst Options dialog box, instead, to customize behavior. See HDL Analyst Options Command, on page 408. On the Windows 7 platforms, the ini file is in the C:\Users*userName*\AppData\Roaming\Synplicity directory. On Linux workstations, the ini file is in the following directory: (~/.Synplicity, where ~ is your home directory, which can be set with the environment variable $HOME).
.prj	Project	Contains all the information required to complete a design. It is in Tcl format, and contains references to source files, compilation, mapping, and optimization switches, specifications for target technology and other runtime options.
.sdc	Constraint	Contains the timing constraints (clock parameters, I/O delays, and timing exceptions) in Tcl format. You can either create this file manually or generate it by entering constraints in the SCOPE window.

© 2022 Synopsys, Inc.

Extension	File	Description
.sv	Source files (Verilog)	Design source files in SystemVerilog format. The sv source file is added to the Verilog directory in the Project view. For more information about the Verilog and SystemVerilog languages, and the synthesis commands and attributes you can include, see Verilog, on page 134, Chapter 1, *Verilog Language Support*, and Chapter 2, *SystemVerilog Language Support*. For information about using VHDL and Verilog files together in a design, see Using Mixed Language Source Files, on page 45 of the *User Guide*.
.vhd	Source files (VHDL)	Design source files in VHDL format. See VHDL, on page 134 and Chapter 3, *VHDL Language Support* for details. For information about using VHDL and Verilog files together in a design, see Using Mixed Language Source Files, on page 45 of the *User Guide*.
.v	Source files (Verilog)	Design source files in Verilog format. For more information about the Verilog language, and the synthesis commands and attributes you can include, see Verilog, on page 134, Chapter 1, *Verilog Language Support*, and Chapter 2, *SystemVerilog Language Support*. For information about using VHDL and Verilog files together in a design, see Using Mixed Language Source Files, on page 45 of the *User Guide*.

HDL Source Files

The HDL source files for a project can be in either VHDL (vhd), Verilog (v), or SystemVerilog (sv) format.

The synthesis tool contains built-in macro libraries for vendor macros like gates, counters, flip-flops, and I/Os. If you use the built-in macro libraries, you can easily instantiate vendor macros directly into the VHDL designs, and forward-annotate them to the output netlist. Refer to the appropriate vendor support documentation for more information.

© 2022 Synopsys, Inc.

VHDL

The Synopsys FPGA synthesis tool supports a synthesizable subset of VHDL93 (IEEE 1076), and the following IEEE library packages:

- numeric_bit
- numeric_std
- std_logic_1164

The synthesis tool also supports the following industry standards in the IEEE libraries:

- std_logic_arith
- std_logic_signed
- std_logic_unsigned

The synthesis tool library contains an attributes package (*installDirectory*/lib/vhd/synattr.vhd) of built-in attributes and timing constraints that you can use with VHDL designs. The package includes declarations for timing constraints (including black-box timing constraints), vendor-specific attributes, and synthesis attributes. To access these built-in attributes, add the following two lines to the beginning of each of the VHDL design units that uses them:

```
library synplify;
use synplify.attributes.all;
```

For more information about the VHDL language, and the synthesis commands and attributes you can include, see Chapter 3, *VHDL Language Support*.

Verilog

The synthesis tool supports a synthesizable subset of Verilog 2001 and Verilog 95 (IEEE 1364) and SystemVerilog extensions. For more information about the Verilog language, and the synthesis commands and attributes you can include, see Chapter 1, *Verilog Language Support* and Chapter 2, *SystemVerilog Language Support*.

© 2022 Synopsys, Inc.

The synthesis tool contains built-in macro libraries for vendor macros like gates, counters, flip-flops, and I/Os. If you use the built-in macro libraries, you can instantiate vendor macros directly into Verilog designs and forward-annotate them to the output netlist. Refer to the *User Guide* for more information.

© 2022 Synopsys, Inc.

Libraries

You can instantiate components from a library, which can be either in Verilog or VHDL. For example, you might have technology-specific or custom IP components in a library, or you might have generic library components. The *installDirectory*/lib directory included with the software contains some component libraries you can use for instantiation.

There are several kinds of libraries you can use:

- Technology-specific libraries that contain I/O pad, macro, or other component descriptions. The lib directory lists these kinds of libraries under vendor sub-directories. The libraries are named for the technology family, and in some cases also include a version number for the version of the place-and-route tool with which they are intended to be used.
- The open verification library is automatically included in the FPGA product installation. When using your own open verification library, follow the recommendation described in Open Verification Library (Verilog), on page 136.
- Technology-independent libraries that contain common components. You can have your own library or use the one Synopsys provides. This library is a Verilog library of common logic elements, much like the Synopsys® GTECH component library. See The Generic Technology Library, on page 137 for a description of this library.
- An ASIC Library Data Format file (.lib) is the technology library file that contains information about the functionality of each standard cell, its input capacitance, fanout, and timing information. For the synthesis flow to understand the instantiated or mapped ASIC primitives in the RTL, you would need to translate the functionality of the standard cell to equivalent synthesizable Verilog/VHDL definitions. To do this, you can use the lib2syn executable. For details, see ASIC Library Files, on page 138.

Open Verification Library (Verilog)

The open verification library is automatically included in the FPGA product installation. If you use your own version of the open verification library, then it is recommended that you disable loading the default synovl library to avoid any conflicts between the two libraries. To do this, set the -disable_synovl environment variable to 1. For example:

```
#in bash
export disable_synovl=1

#in csh
setenv disable_synovl 1
```

When the default synovl library is disabled, the following message is generated in the log file: `@N::Open Verification Library which is part of tool installation, is being disabled by option "disable_synovl"`..

The Generic Technology Library

The synthesis software includes this Verilog library for generic components under the *installDirectory*/lib/generic_technology directory. Currently, the library is only available in Verilog format. The library consists of technology-independent common logic elements, which help the designer to develop technology-independent parts. The library models extract the functionality of the component, but not its implementation. During synthesis, the mappers implement these generic components in implementations that are appropriate to the technology being used.

To use components from this directory, add the library to the project by doing either of the following:

- Add add_file -verilog "$LIB/generic_technology/gtech.v to your `prj` file or type it in the Tcl window.
- In the tool window, click the Add file button, navigate to the *installDirectory*/lib/generic_technology directory and select the gtech.v file.

When you synthesize the design, the tool uses components from this library.

You cannot use the generic technology library together with other generic libraries, as this could result in a conflict. If you have your own GTECH library that you intend to use, do not use the generic technology library.

© 2022 Synopsys, Inc.

ASIC Library Files

An ASIC Library Data Format file (.lib) is the technology library file that contains information about the functionality of each standard cell, its input capacitance, fanout, and timing information.

For the synthesis flow to understand the instantiated or mapped ASIC primitives in the RTL, you would need to manually translate the functionality of the standard cell to equivalent synthesizable Verilog/VHDL definitions. This .lib file conversion is not automated in the synthesis flow. This means that the tool will not automatically translate .lib files into corresponding and equivalent synthesizable Verilog/VHDL definitions.

However, you can use the lib2syn executable to facilitate this conversion process. The lib2syn.exe executable generates equivalent synthesizable Verilog/VHDL definitions for the cells defined in the input .lib file. You can find this executable at these locations:

- Windows: *installDirectory* / bin / lib2syn.exe
- Linux: *installDirectory* / bin / lib2syn

The executable can be run as shown in these examples:

- For Verilog output: lib2syn.exe test.lib -ovm a.vm -logfile test_lib2syn.log
- For VHDL output: lib2syn.exe test.lib -ovhm a.vhm -logfile test_lib2syn.log

The tool supports the Synopsys GTECH library flow by default, so you do not need the .lib file equivalent synthesizable Verilog/VHDL definitions for a NETLIST mapped to a GTECH library.

Note that for the synthesis flow, the lib2syn executable does not translate cells with state table definitions.

The synthesis tools do not read Synopsis Liberty format (.syn) files directly. However, there are workarounds.

- If your design has instantiated ASIC cells, do the following:
  - Get the Verilog functional files for the instantiated components.
  - Add the functional files to your project as libraries.
- If you have an ASIC library in the Liberty (.lib) or .sel format, do the following:

- Convert the ASIC library into a Verilog functional file with the lib2syn utility. The lib2syn command syntax is shown below:

  installDirectory /bin/lib2syn.exe *library*.lib - ovm *VerilogFunctionalFile*

  or

  installDirectory /bin/lib2syn.exe *library*.sel -ovm *VerilogFunctionalFile*

- Add the functional file to your project as a library.

© 2022 Synopsys, Inc.

Output Files

The synthesis tool generates reports about the synthesis run and files that you can use for simulation or placement and routing. The following table describes the output files, categorizing them as either synthesis result and report files, or output files generated as input for other tools.

Extension	File	Description
_cck.rpt	Constraint Checker Report	Checks the syntax and applicability of the timing constraints in the fdc file for your project and generates a report (*projectName*_cck.rpt). See Constraint Checking Report, on page 157 for more information.
_compiler.linkerlog	Compiler log file for HDL source file linking	Provides details of why the VHDL and/or Verilog components in the source files were not properly linked. This file is located in the synwork directory for the implementation.
.fse	FSM information file	Design-dependent. Contains information about encoding types and transition states for all state machines in the design.
.info	Design component files	Design-dependent. Contains detailed information about design components like state machines or ROMs.
.linkerlog	Mixed language ports/generics differences	Provides details of why the VHDL and/or Verilog components in the source files were not properly linked. This file is located in the synwork directory for the implementation. The same information is also reported in the log file.
.pfl	Message Filter criteria	Output file created after filtering messages in the Messages window. See Updating the *projectName.pfl file, on page 184* in the *User Guide*.

© 2022 Synopsys, Inc.

Extension	File	Description
Results file: `.vm`	Vendor-specific results file	Results file that contains the synthesized netlist, written out in a format appropriate to the technology and the place-and-route tool you are using. Specify this file on the Implementation Results panel of the Implementation Options dialog box (Implementation Results Panel, on page 329).
`run_options.txt`	Project settings for implementations	This file is created when a design is synthesized and contains the project settings and options used with the implementations. These settings and options are also processed for displaying the Project Status view after synthesis is run. For details, see Project Status Tab, on page 23.
`.sap`	Synplify Annotated Properties	This file is generated after the Annotated Properties for Analyst option is selected in the Device panel of the Implementation Options dialog box. After the compile stage, the tool annotates the design with properties like clock pins. You can find objects based on these annotated properties using Tcl Find. For more information, see find, on page 127Using the Tcl Find Command to Define Collections, on page 136 in the *User Guide*.
`.sar`	Archive file	Output of the Synopsys FPGA Archive utility in which design project files are stored into a single archive file. Archive files use Synopsys Proprietary Format. See Archive Project Command, on page 316 for details on archiving, unarchiving and copying projects.

© 2022 Synopsys, Inc.

Extension	File	Description
_scck.rpt	Constraint Checker Report (Syntax Only)	Generates a report that contains an overview of the design information, such as, the top-level view, name of the constraints file, if there were any constraint syntax issues, and a summary of clock specifications.
.srd	Intermediate mapping files	Used to save mapping information between synthesis runs. You do not need to use these files.
.srm	Mapping output files	Output file after mapping. It contains the actual technology-specific mapped design. This is the representation that appears graphically in a Technology view.
.srr	Synthesis log file	Provides information on the synthesis run, as well as area and timing reports. See Log File, on page 145, for more information.
.srs	Compiler output file	Output file after the compiler stage of the synthesis process. It contains an RTL-level representation of a design. This is the representation that appears graphically in an RTL view.
synlog folder	Intermediate technology mapping files	This folder contains intermediate netlists and log files after technology mapping has been run. Timestamp information is contained in these netlist files to manage jobs with up-to-date checks. For more information, see Using Up-to-date Checking for Job Management, on page 154.

© 2022 Synopsys, Inc.

Extension	File	Description
`synwork folder`	Intermediate pre-mapping files	This folder contains intermediate netlists and log files after pre-mapping has been run. Timestamp information is contained in these netlist files to manage jobs with up-to-date checks. For more information, see Using Up-to-date Checking for Job Management, on page 154.
`.ta`	Customized Timing Report	Contains the custom timing information that you specify through Analysis->Timing Analyst. See Analysis Menu, on page 370, for more information.
`_ta.srm`	Customized mapping output file	Creates a customized output netlist when you generate a custom timing report with HDL Analyst->Timing Analyst. It contains the representation that appears graphically in a Technology view. See Analysis Menu, on page 370 for more information.
`.tap`	Timing Annotated Properties	This file is generated after the Annotated Properties for Analyst option is selected in the Device panel of the Implementation Options dialog box. After the compile stage, the tool annotates the design with timing properties and the information can be analyzed in the RTL view and Design Planner. You can also find objects based on these annotated properties using Tcl Find. For more information, see Using the Tcl Find Command to Define Collections, on page 136 in the *User Guide*.
`.tlg`	Log file	This log file contains a list of all the modules compiled in the design.

© 2022 Synopsys, Inc.

Extension	File	Description
vendor constraint file	Constraints file for forward annotation	Contains synthesis constraints to be forward-annotated to the place-and-route tool. The constraint file type varies with the vendor and the technology. Refer to the vendor chapters for specific information about the constraints you can forward-annotate. Check the Implementation Results dialog (Implementation Options) for supported files. See Implementation Results Panel, on page 329.
.vm .vhm	Mapped Verilog or VHDL netlist	Optional post-synthesis netlist file in Verilog (.vm) or VHDL (.vhm) format. This is a structural netlist of the synthesized design, and differs from the original RTL used as input for synthesis. Specify these files in the Implementation Results dialog box (Implementation Options). See Implementation Results Panel, on page 329. Typically, you use this netlist for gate-level simulation, to verify your synthesis results. Some designers prefer to simulate before and after synthesis, and also after place-and-route. This approach helps them to isolate the stage of the design process where a problem occurred. The Verilog and VHDL output files are for functional simulation only. When you input stimulus into a simulator for functional simulation, use a cycle time for the stimulus of 1000 time ticks.

© 2022 Synopsys, Inc.

Log File

The log file report, located in the implementation directory, is written out in two file formats: text (*projectName*.srr), and HTML with an interactive table of contents (*projectName*.htm and *projectName*_srr.htm) where *projectName* is the name of your project. Select View Log File in HTML in the Options->Project View Options dialog box to enable viewing the log file in HTML. Select the View Log button in the Project view (Buttons and Options, on page 69) to see the log file report.

The log file is written each time you compile or synthesize (compile and map) the design. When you compile a design without mapping it, the log file contains only compiler information. As a precaution, a backup copy of the log file (srr) is written to the backup sub-directory in the Implementation Results directory. Only one backup log file is updated for subsequent synthesis runs.

The log file contains detailed reports on the compiler, mapper, timing, and resource usage information for your design. Errors, notes, warnings, and messages appear in both the log file and on the Messages tab in the Tcl window.

For further details about different sections of the log file, see the following:

For information about ...	See ...
Compiled files, messages (warnings, errors, and notes), user options set for synthesis, state machine extraction information, including a list of reachable states.	Compiler Report, on page 146
Buffers added to clocks in certain supported technologies.	Clock Buffering Report, on page 147
Buffers added to nets.	Net Buffering Report, on page 147
Compile point remapping	Compile Point Information, on page 147
Timing results. This section of the log file begins with “START TIMING REPORT” section. If you use the Timing Analyst to generate a custom timing report, its format is the same as the timing report in the log file, but the customized timing report is in a ta file.	Timing Reports, on page 150

© 2022 Synopsys, Inc.

For information about ...	See ...
Resources used by synthesis mapping	Resource Usage Report, on page 148
Design changes made as a result of retiming	Retiming Report, on page 148
Design changes made as a result of gated clock conversion	Gated Clock Conversion Report, on page 164

Compiler Report

This report starts with the compiler version and date, and includes the following:

- Project information - the top-level module.
- Design information - HDL syntax and synthesis checks, black box instantiations, FSM extractions and inferred RAMs/ROMs.

Premap Report

This report begins with the pre-mapper version and date, and reports the following:

- File loading times and memory usage
- Clock summary

Mapper Report

This report begins with the mapper version and date, and reports the following:

- Project information - the names of the constraint files, target technology, and attributes set in the design.
- Design information such as flattened instances, extraction of counters, FSM implementations, clock nets, buffered nets, replicated logic, RTL optimizations, and informational or warning messages.

© 2022 Synopsys, Inc.

Clock Buffering Report

This section of the log file reports any clocks that were buffered. For example:

```
Clock Buffers:
Inserting Clock buffer for port clock0,TNM=clock0
```

Net Buffering Report

Net buffering reports are generated for almost all of the supported FPGAs and CPLDs. This information is written in the log file, and includes the following information:

- The nets that were buffered or had their source replicated
- The number of segments created for that net
- The total number of buffers added during buffering
- The number of registers and look-up tables (or other cells) added during replication

Example: Net Buffering Report

```
Net buffering Report:
Badd_c[2] - loads: 24, segments 2, buffering source
Badd_c[1] - loads: 32, segments 2, buffering source
Badd_c[0] - loads: 48, segments 3, buffering source
Aadd_c[0] - loads: 32, segments 3, buffering source
Added 10 Buffers
Added 0 Registers via replication
Added 0 LUTs via replication
```

Compile Point Information

The Summary of Compile Points section of the log file (*projectName*.srr) lists each compile point, together with an indication of whether it was remapped, and, if so, why. Also, a timing report is generated for each compile point located in its respective results directories in the Implementation Directory. The compile point is the top-level design for this report file.

For more information on compile points and the compile-point synthesis flow, see Synthesizing Compile Points, on page 428 of the *User Guide*.

© 2022 Synopsys, Inc.

Timing Section

A default timing report is written to the log file (*projectName*.srr) in the "START OF TIMING REPORT" section. See Timing Reports, on page 150, for details.

You can use the Timing Analyst to generate additional timing reports for point-to-point analysis (see Analysis Menu, on page 370). Their format is the same as the timing report.

Resource Usage Report

A resource usage report is added to the log file each time you compile or synthesize. The format of the report varies, depending on the architecture you are using. The report provides the following information:

- The total number of cells, and the number of combinational and sequential cells in the design
- The number of clock buffers and I/O cells
- Details of how many of each type of cell in the design

See Checking Resource Usage, on page 172 in the *User Guide* for a brief procedure on using the report to check for overutilization.

Retiming Report

Whenever retiming is enabled, a retiming report is added to the log file (*projectName*.srr). It includes information about the design changes made as a result of retiming, such as the following:

- The number of flip-flops added, removed, or modified because of retiming. Flip-flops modified by retiming have a _ret suffix added to their names.
- Names of the flip-flops that were *moved* by retiming and no longer exist in the Technology view.

© 2022 Synopsys, Inc.

- Names of the flip-flops *created* as result of the retiming moves, that did not exist in the RTL view.
- Names of the flip-flops *modified* by retiming; for example, flip-flops that are in the RTL and Technology views, but have different fanouts because of retiming.

© 2022 Synopsys, Inc.

Timing Reports

Timing results can be written to one or more of the following files:

`.srr` or `.htm`	Log file that contains a default timing report. To find this information, after synthesis completes, open the log file (View -> Log File), and search for START OF TIMING REPORT.
`.ta`	Timing analysis file that contains timing information based on the parameters you specify in the stand-alone Timing Analyst (Analysis->Timing Analyst).
*designName*_async_clk `.rpt.scv`	Asynchronous clock report file that is generated when you enable the related option in the stand-alone Timing Analyzer (Analysis->Timing Analyst). This report can be displayed in a spreadsheet tool and contains information for paths that cross between multiple clock groups. See Asynchronous Clock Report, on page 155 for details on this report.

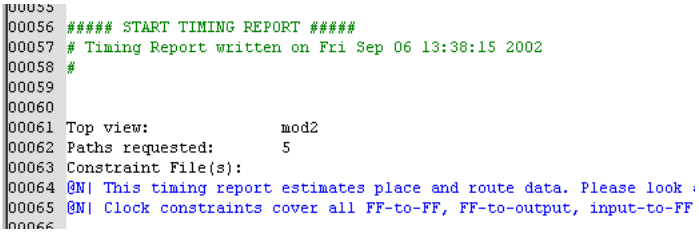
The timing reports in the `srr`/`htm` and `ta` files have the following sections:

- Timing Report Header, on page 151
- Performance Summary, on page 151
- Clock Relationships, on page 153
- Interface Information, on page 154
- Asynchronous Clock Report, on page 155

© 2022 Synopsys, Inc.

Timing Report Header

The timing report header lists the date and time, the name of the top-level module, the number of paths requested for the timing report, and the constraint files used.



You can control the size of the timing report by choosing Project -> Implementation Options, clicking the Timing Report tab of the panel, and specifying the number of start/end points and the number of critical paths to report. See Timing Report Panel, on page 331, for details.

Performance Summary

The Performance Summary section of the timing report reports estimated and requested frequencies for the clocks, with the clocks sorted by negative slack. The timing report has a different section for detailed clock information. The Performance Summary lists the following information for each clock in the design:

Performance Summary Column	Description
Starting Clock	Clock at the start point of the path. If the clock name is system, the clock is a collection of clocks with an undefined clock event. Rising and falling edge clocks are reported as one clock domain.
Requested/Estimated Frequency	Target frequency goal /estimated value after synthesis. See Cross-Clock Path Timing Analysis, on page 153 for information on how cross-clock path slack is reported.
Requested/Estimated Period	Target clock period/estimated value after synthesis.

© 2022 Synopsys, Inc.

Performance Summary Column	Description
Slack	Difference between estimated and requested period. See Cross-Clock Path Timing Analysis, on page 153 for information on how cross-clock path slack is reported.
Clock Type	The type of clock: inferred, declared, derived or system. For more information, see Clock Types, on page 152.
Clock Group	Name of the clock group that a clock belongs.

The synthesis tool does not report inferred clocks that have an unreasonable slack time. Also, a real clock might have a negative period. For example, suppose you have a clock going to a single flip-flop, which has a single path going to an output. If you specify an output delay of –1000 on this output, then the synthesis tool cannot calculate the clock frequency. It reports a negative period and no clock.

Clock Types

The synthesis timing reports include the following types of clocks:

- Declared Clocks

  User-defined clocks specified in the constraint file.

- System Clock

  The system clock is the delay for the combinational path. Additionally, a system clock can be reported if there are sequential elements in the design for a clock network that cannot be traced back to a clock. Also, the system clock can occur for unconstrained I/O ports. You must investigate these conditions.

Paths to/from black boxes are timed by the system clock. Add the black-box timing constraints. See syn_black_box, on page 28 for the black box source code directives.

© 2022 Synopsys, Inc.

Clock Relationships

For each pair of clocks in the design, the Clock Relationships section of the timing report lists both the required time (constraint) and the worst slack time for each of the intervals rise to rise, fall to fall, rise to fall, and fall to rise. See Cross-Clock Path Timing Analysis, on page 153 for details about cross-clock paths.

This information is provided for the paths between related clocks (that is, clocks in the same clock group). If there is no path at all between two clocks, then that pair is not reported. If there is no path for a given pair of edges between two clocks, then an entry of No paths appears.

For information about how these relationships are calculated, see Clock Groups, on page 200.

```
Clock Relationships
*******************

Clocks            |    rise  to  rise    |    fall  to  fall    |    rise  to  fall    |    fall  to  rise
-----------------------------------------------------------------------------------------------------------
Starting  Ending  |  constraint  slack   |  constraint  slack   |  constraint  slack   |  constraint  slack
-----------------------------------------------------------------------------------------------------------
clk1      clk1    |  25.000      15.943  |  25.000      17.764  |  No paths    -       |  No paths    -
clk1      clk2    |  1.000       -9.430  |  No paths    -       |  No paths    -       |  1.000       -1.531
clk2      clk1    |  No paths    -       |  1.000       -0.811  |  1.000       -1.531  |  No paths    -
clk2      clk2    |  8.000       0.764   |  8.000       -1.057  |  No paths    -       |  6.000       2.814
clk3      clk3    |  No paths    -       |  10.000      0.943   |  No paths    -       |  No paths    -
===========================================================================================================
```

Cross-Clock Path Timing Analysis

The following describe how the timing analyst calculates cross-clock path frequency and slack.

Cross-Clock Path Frequency

For each data path, the tool estimates the highest frequency that can be set for the clock(s) without a setup violation. It finds the largest scaling factor that can be applied to the clock(s) without causing a setup violation. If the start clock is not the same as the end clock, it scales both by the same factor.

scale = (*minimum time period* -(-*current slack*))/*minimum time period*

It assumes all other delays in the setup calculation (e.g., uncertainty) are fixed.

It applies relevant multicycle constraints to the setup calculation.

© 2022 Synopsys, Inc.

The estimated frequency for a clock is the minimum frequency over all paths that start or end on that clock, with the following exceptions:

- The tool does not consider paths between the system clock and another clock to estimate frequency.
- It considers paths with a path delay constraint to be asynchronous, and does not use them to estimate frequency.
- It considers paths between clocks in different domains to be asynchronous, and does not use them to estimate frequency.

Slack for Cross-Clock Paths

The slack reported for a cross-clock path is the worst slack for any path that starts on that clock. Note that this differs from the estimated frequency calculation, which is based on the worst slack for any path starting or ending on that clock.

Interface Information

The interface section of the timing report contains information on arrival times, required times, and slack for the top-level ports. It is divided into two subsections, one each for Input Ports and Output Ports. Bidirectional ports are listed under both. For each port, the interface report contains the following information.

Port parameter	**Description**
Port Name	Port name.
Starting Reference Clock	The reference clock.
User Constraint	The input/output delay. If a port has multiple delay records, the report contains the values for the record with the worst slack. The reference clock corresponds to the worst slack delay record.

Port parameter	**Description**
Arrival Time	Input ports: define_input_delay, or default value of 0. Output ports: path delay (including clock-to-out delay of source register). For purely combinational paths, the propagation delay is calculated from the driving input port.
Required Time	Input ports: clock period – (path delay + setup time of receiving register + define_reg_input_delay value). Output ports: clock period – define_output_delay. Default value of define_output_delay is 0.
Slack	Required Time – Arrival Time

Asynchronous Clock Report

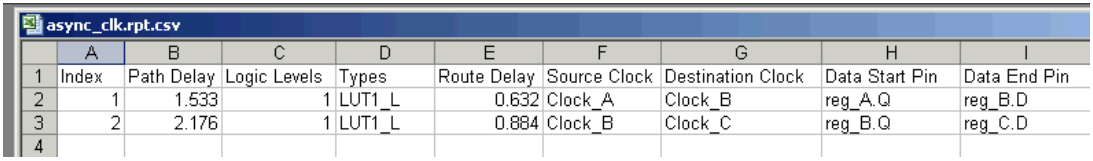
You can generate a report for paths that cross between clock groups using the stand-alone Timing Analyst (Analysis->Timing Analyst, Generate Asynchronous Clock Report check box). Generally, paths in different clock groups are automatically handled as false paths. This option provides a file that contains information on each of the paths and can be viewed in a spreadsheet tool. To display the CSV-format report:

1. Locate the file in your results directory *projectName*_async_clk.rpt.csv.
2. Open the file in your spreadsheet tool.

Column	**Description**
Index	Path number.
Path Delay	Delay value as reported in standard timing (ta) file.
Logic Levels	Number of logic levels in the path (such as LUTs, cells, and so on) that are between the start and end points.
Types	Cell types, such as LUT, logic cell, and so on.
Route Delay	As reported for each path in ta.
Source Clock	Start clock.
Destination Clock	End clock.

© 2022 Synopsys, Inc.

Column	Description
Data Start Pin	Sequential device output pin at start of path.
Data End Pin	Setup check pin at destination.



async_clk.rpt.csv

	A	B	C	D	E	F	G	H	I
1	Index	Path Delay	Logic Levels	Types	Route Delay	Source Clock	Destination Clock	Data Start Pin	Data End Pin
2	1	1.533	1	LUT1_L	0.632	Clock_A	Clock_B	reg_A.Q	reg_B.D
3	2	2.176	1	LUT1_L	0.884	Clock_B	Clock_C	reg_B.Q	reg_C.D
4									

Constraint Checking Report

Use the Run->Constraint Check command to generate a report on the constraint files in your project. The *projectName*_cck.rpt file provides information such as invalid constraint syntax, constraint applicability, and any warnings or errors. For details about running Constraint Check, see Tcl Syntax Guidelines for Constraint Files, on page 51 in the *User Guide*.

This section describes the following topics:

- Reporting Details, on page 157
- Inapplicable Constraints, on page 158
- Applicable Constraints With Warnings, on page 159
- Sample Constraint Check Report, on page 160

Reporting Details

This constraint checking file reports the following:

- Constraints that are not applied
- Constraints that are valid and applicable to the design
- Wildcard expansion on the constraints
- Constraints on objects that do not exist

It contains the following sections:

Summary	Statement which summarizes the total number of issues defined as an error or warning (x) out of the total number of constraints with issues (y) for the total number of constraints (z) in the fdc file. `Found <x> issues in <y> out of <z> constraints`
Clock Relationship	Standard timing report clock table, without slack.
Unconstrained Start/End Points	Lists I/O ports that are missing input/output delays.

© 2022 Synopsys, Inc.

Unapplied constraints	Constraints that cannot be applied because objects do not exist or the object type check is not valid. See Inapplicable Constraints, on page 158 for more information.
Applicable constraints with issues	Constraints will be applied either fully or partially, but there might be issues that generate warnings which should be investigated, such as some objects/collections not existing. Also, whenever at least one object in a list of objects is not specified with a valid object type, a warning is displayed. See Applicable Constraints With Warnings, on page 159 for more information.
Constraints with matching wildcard expressions	Lists constraints or collections using wildcard expressions up to the first 1000, respectively.

Inapplicable Constraints

Refer to the following table for constraints that were not applied because objects do not exist or the object type check was not valid:

For these constraints ...	Objects must be ...
Attributes	Valid definitions
create_clock	• Ports • Nets • Pins • Registers • Instantiated buffers
create_generated_clock	Clocks
define_compile_point	• Region • View
define_current_design	v:*view*

For these constraints ...	Objects must be ...
set_false_path set_multicycle_path set_max_delay	For -to or -from objects: • i:sequential instances • p:ports • i:black boxes For -through objects • n:nets • t:hierarchical ports • t:pins
set_multicycle_path	Specified as a positive integer
set_input_delay	• Input ports • bidir ports
set_output_delay	• Output ports • Bidir ports
set_reg_input_delay set_reg_output_delay	Sequential instances

Applicable Constraints With Warnings

The following table lists reasons for warnings in the report file:

For these constraints ...	Objects must be ...
create_clock	• Ports • Nets • Pins • Registers • Instantiated buffers
set_clock_uncertainty	A single object. Multiple objects are not supported.
define_compile_point	A single object. Multiple objects are not supported.
define_current_design	v:*view*

© 2022 Synopsys, Inc.

For these constraints ...	Objects must be ...
set_false_path set_multicycle_path set_path_delay	For -to or -from objects: • i:sequential instances • p:ports • i:black boxes For -through objects: • n:nets • t:hierarchical ports • t:pins
set_input_delay	A single object. Multiple objects are not supported.
set_output_delay	A single object. Multiple objects are not supported.
set_reg_input_delay set_reg_output_delay	A single object. Multiple objects are not supported.

Sample Constraint Check Report

The following is a sample report generated by constraint checking:

```
# Synopsys Constraint Checker, version maprc, Build 1138R, built Jun 7 2016
# Copyright (C) 1994-2016, Synopsys, Inc.

# Written on Fri Jun 7 09:42:22 2016
##### DESIGN INFO ############################################################

Top View:                "decode_top"
Constraint File(s):      "C:\timing_88\FPGA_decode_top.fdc"
##### SUMMARY ################################################################

Found 3 issues in 2 out of 27 constraints
```

```
##### DETAILS ################################################################

Clock Relationships
*******************

Starting  Ending  |  rise to rise  |  fall to fall  |  rise to fall  |  fall to rise
--------------------------------------------------------------------------------
clk2x     clk2x   |  24.000        |  24.000        |  12.000        |  12.000
--------------------------------------------------------------------------------
clk2x     clk     |  24.000        |  No paths      |  No paths      |  12.000
clk       clk2x   |  24.000        |  No paths      |  12.000        |  No paths
clk       clk     |  48.000        |  No paths      |  No paths      |  No paths
================================================================================
Note:
'No paths' indicates there are no paths in the design for that pair of clock edges.
'Diff grp' indicates that paths exist but the starting clock and ending clock are in
different clock groups

Unconstrained Start/End Points
******************************
p:test_mode

Inapplicable constraints
************************

set_false_path -from p:next_synd -through i:core.tab1.ram_loader
@E:|object "i:core.tab1.ram_loader" does not exist
@E:|object "i:core.tab1.ram_loader" is incorrect type; "-through" objects must be of
type net (n:), or pin (t:)

Applicable constraints with issues
**********************************

set_false_path -from {core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.omega_inst.omega_tmp_d_lch[7:0]}
@W:|object "core.decoder.root_mult*.root_prod_pre[*]" is missing qualifier which may
result in undesired results; "-from" objects must be of type clock (c:), inst (i:), port
(p:), or pin (t:)

Constraints with matching wildcard expressions
**********************************************

set_false_path -from {core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.omega_inst.omega_tmp_d_lch[7:0]}
@N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]
```


© 2022 Synopsys, Inc.

```
set_false_path -from {i:core.decoder.*.root_prod_pre[*]} -to {i:core.decoder.t_*_[*]}
@N:|expression "core.decoder.*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]
@N:|expression "core.decoder.t_*_[*]" applies to objects:
core.decoder.t_20_[7:0]
core.decoder.t_19_[7:0]
core.decoder.t_18_[7:0]
core.decoder.t_17_[7:0]
core.decoder.t_16_[7:0]
core.decoder.t_15_[7:0]
core.decoder.t_14_[7:0]
core.decoder.t_13_[7:0]
core.decoder.t_12_[7:0]
core.decoder.t_11_[7:0]
core.decoder.t_10_[7:0]
core.decoder.t_9_[7:0]
core.decoder.t_8_[7:0]
core.decoder.t_7_[7:0]
core.decoder.t_6_[7:0]
core.decoder.t_5_[7:0]
core.decoder.t_4_[7:0]
core.decoder.t_3_[7:0]
core.decoder.t_2_[7:0]
core.decoder.t_1_[7:0]
core.decoder.t_0_[7:0]

set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.err[7:0]}
N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.omega_inst.deg_omega[4:0]}
@N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.omega_inst.omega_tmp[0:7]}
@N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.root[7:0]}
@N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]
```

© 2022 Synopsys, Inc.

```
set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.root_inst.count[3:0]}
N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.root_inst.q_reg[7:0]}
@N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult*.root_prod_pre[*]} -to
{i:core.decoder.root_inst.q_reg_d_lch[7:0]}
@N:|expression "core.decoder.root_mult*.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult.root_prod_pre[*]} -to
{i:core.decoder.error_inst.den[7:0]}
@N:|expression "core.decoder.root_mult.root_prod_pre[*]" applies to objects:
core.decoder.root_mult.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.root_mult1.root_prod_pre[*]} -to
{i:core.decoder.error_inst.num1[7:0]}
@N:|expression "core.decoder.root_mult1.root_prod_pre[*]" applies to objects:
core.decoder.root_mult1.root_prod_pre[14:0]

set_false_path -from {i:core.decoder.synd_reg_*_[7:0]} -to {i:core.decoder.b_*_[7:0]}
@N:|expression "core.decoder.synd_reg_*_[7:0]" applies to objects:
core.decoder.un1_synd_reg_0_[7:0]
core.decoder.synd_reg_20_[7:0]
core.decoder.synd_reg_19_[7:0]
core.decoder.synd_reg_18_[7:0]
core.decoder.synd_reg_17_[7:0]
core.decoder.synd_reg_16_[7:0]
core.decoder.synd_reg_15_[7:0]
core.decoder.synd_reg_14_[7:0]
core.decoder.synd_reg_13_[7:0]
core.decoder.synd_reg_12_[7:0]
core.decoder.synd_reg_11_[7:0]
core.decoder.synd_reg_10_[7:0]
core.decoder.synd_reg_9_[7:0]
core.decoder.synd_reg_8_[7:0]
core.decoder.synd_reg_7_[7:0]
core.decoder.synd_reg_6_[7:0]
core.decoder.synd_reg_5_[7:0]
core.decoder.synd_reg_4_[7:0]
core.decoder.synd_reg_3_[7:0]
core.decoder.synd_reg_2_[7:0]
core.decoder.synd_reg_1_[7:0]
```


© 2022 Synopsys, Inc.

```
@N:|expression "core.decoder.b_*_[7:0]" applies to objects:
core.decoder.un1_b_0_[7:0]
core.decoder.b_calc.un1_lambda_0_[7:0]
core.decoder.b_20_[7:0]
core.decoder.b_19_[7:0]
core.decoder.b_18_[7:0]
core.decoder.b_17_[7:0]
core.decoder.b_16_[7:0]
core.decoder.b_15_[7:0]
core.decoder.b_14_[7:0]
core.decoder.b_13_[7:0]
core.decoder.b_12_[7:0]
core.decoder.b_11_[7:0]
core.decoder.b_10_[7:0]
core.decoder.b_9_[7:0]
core.decoder.b_8_[7:0]
core.decoder.b_7_[7:0]
core.decoder.b_6_[7:0]
core.decoder.b_5_[7:0]
core.decoder.b_4_[7:0]
core.decoder.b_3_[7:0]
core.decoder.b_2_[7:0]
core.decoder.b_1_[7:0]
core.decoder.b_0_[7:0

Library Report
**************

# End of Constraint Checker Report
```

Gated Clock Conversion Report

When using the Clock Conversion option, both the sequential element that could not be converted and the reason why the conversion did not occur are reported.

For more information about using gated clocks, see Working with Gated Clocks, on page 472.

© 2022 Synopsys, Inc.

SYNOPSYS®
Silicon to Software™

CHAPTER 6

RAM and ROM Inference

This chapter provides guidelines and Verilog or VHDL examples for coding RAMs for synthesis. It covers the following topics:

- Guidelines and Support for RAM Inference, on page 166
- Block RAM Examples, on page 167
- Initial Values for RAMs, on page 179
- RAM Instantiation with SYNCORE, on page 186
- ROM Inference, on page 187

© 2022 Synopsys, Inc.

Guidelines and Support for RAM Inference

There are two methods to handle RAMs—instantiation and inference. Many FPGA families provide technology-specific RAMs that you can instantiate in your HDL source code. The software supports instantiation, but you can also set up your source code so that it infers the RAMs. The following table sums up the pros and cons of the two approaches.

Inference in Synthesis	Instantiation
Advantages	**Advantages**
Portable coding style Automatic timing-driven synthesis No additional tool dependencies	Most efficient use of the RAM primitives of a specific technology Supports all kinds of RAMs
Limitations	**Limitations**
Glue logic to implement the RAM might result in a sub-optimal implementation Can only infer synchronous RAMs No support for address wrapping Pin name limitations means some pins are always active or inactive	Source code is not portable because it is technology-dependent Limited or no access to timing and area data if the RAM is a black box Inter-tool access issues, if the RAM is a black box created with another tool

You must structure your source code correctly for the type of RAM you want to infer. The following table lists the supported technology-specific RAMs that can be generated by the synthesis tool.

RAM Type	Pango
Single Port	x
Dual Port	x
True Dual Port	x

© 2022 Synopsys, Inc.

Block RAM Examples

The examples below show you how to define RAM in the RTL code so that the synthesis tool can infer block RAM. See the following for details:

- Block RAM Mode Examples, on page 167
- Single-Port Block RAM Examples, on page 171
- Dual-Port Block RAM Examples, on page 173
- True Dual-Port RAM Examples, on page 176

For details about inferring block RAM, see Initializing RAMs, on page 365 in the *User Guide*.

Block RAM Mode Examples

The coding style supports the enable and reset pins of the block RAM primitive. The tool supports different write mode operations for single-port and dual-port RAM. This section contains examples of how to specify the supported block RAM output modes:

- WRITE_FIRST Mode Example, on page 167
- READ_FIRST Mode Example, on page 169
- NO_CHANGE Mode Example, on page 170

WRITE_FIRST Mode Example

This example shows the WRITE_FIRST mode operation with active enable.

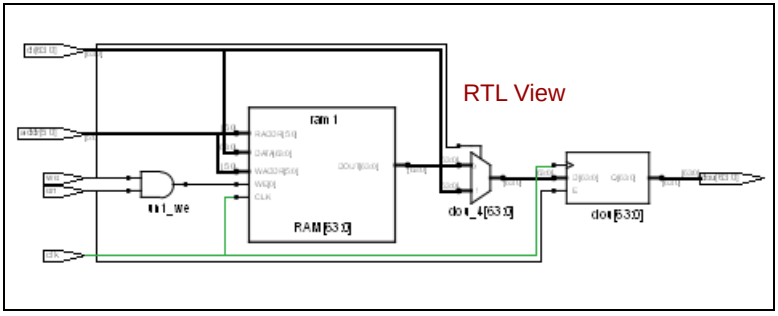
```
module v_rams_02a (clk, we, en, addr, di, dou);
input clk;
input we;
input en;
input [5:0] addr;
input [63:0] di;
output [63:0] dou;
reg [63:0] RAM [63:0];
reg [63:0] dou;
```


© 2022 Synopsys, Inc.

```
always @(posedge clk)
begin
if (en)
   begin
   if (we)
      begin
         RAM[addr] <= di;
         dou <= di;
      end
   else
      dou <= RAM[addr];
   end
end
endmodule

always @(posedge clk)
if (en & we) mem[addr] <= data_in;
endmodule
```

The following figure shows the RTL view of a WRITE_FIRST mode RAM with output registered. The Technology view shows that the RAM is mapped to a block RAM.



READ_FIRST Mode Example

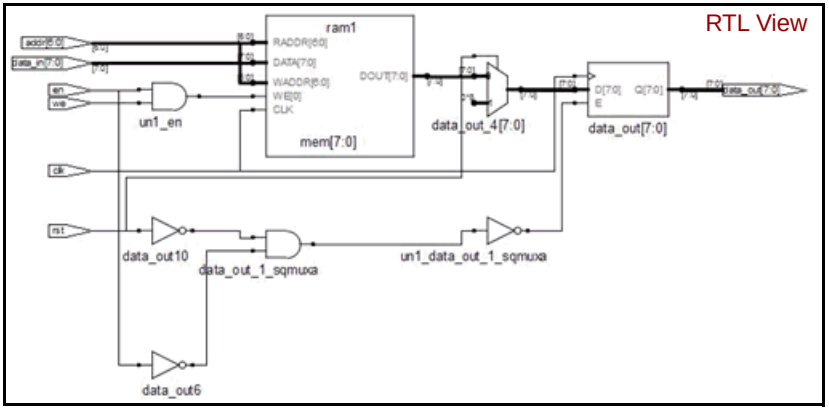
The following piece of code is an example of READ_FIRST mode with both enable and reset, with reset taking precedence:

```
module ram_test(data_out, data_in, addr, clk, rst, en, we);
output [7:0]data_out;
input [7:0]data_in;
input [6:0]addr;
input clk, en, rst, we;
reg [7:0] mem [127:0] /* synthesis syn_ramstyle = "block_ram" */;
reg [7:0] data_out;

always@(posedge clk)
if(rst == 1)
   data_out <= 0;
else begin
   if(en) begin
      data_out <= mem[addr];
   end
end

always @(posedge clk)
if (en & we) mem[addr] <= data_in;
endmodule
```

The following figure shows the RTL view of a READ_FIRST RAM with inferred enable and reset, with reset taking precedence. Select the Technology view to see that the inferred RAM is mapped to a block RAM.



© 2022 Synopsys, Inc.

NO_CHANGE Mode Example

This NO_CHANGE mode example has neither enable nor reset. If you register the read address and the output address, the software infers block RAM.

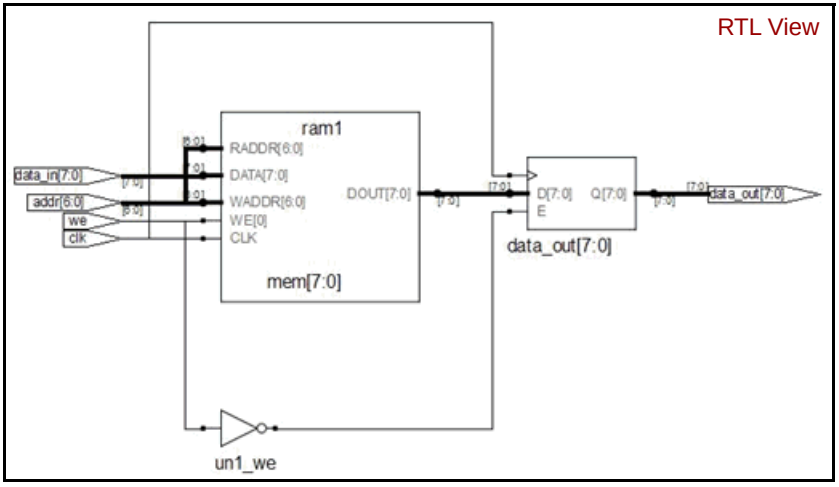
```
module ram_test(data_out, data_in, addr, clk, we);
output [7:0]data_out;
input [7:0]data_in;
input [6:0]addr;
input clk,we;
reg [7:0] mem [127:0] /* synthesis syn_ramstyle = "block_ram" */;
reg [7:0] data_out;

always@(posedge clk)
if(we == 1)
   data_out <= data_out;
else
   data_out <= mem[addr];

always @(posedge clk)
if (we) mem[addr] <= data_in;

endmodule
```

The next figure shows the RTL view of a NO_CHANGE RAM. Select the Technology view to see that the RAM is mapped to block RAM.



Single-Port Block RAM Examples

This section describes the coding style required to infer single-port block RAMs. For single-port RAM, the same address is used to index the write-to and read-from RAM. See the following examples:

- Single-Port Block RAM Examples, on page 171
- Single-Port RAM with RAM Output Registered Examples, on page 172
- Dual-Port Block RAM Examples, on page 173

Single-Port RAM with Read Address Registered Example

In these examples, the read address is registered, but the write address (which is the same as the read address) is not registered. There is one clock for the read address and the RAM.

Verilog Example: Read Address Registered

```
module ram_test(q, a, d, we, clk);
output [7:0] q;
input [7:0] d;
input [6:0] a;
input clk, we;
reg [6:0] read_add;
/* The array of an array register ("mem") from which the RAM is
inferred*/
reg [7:0] mem [127:0] ;
assign q = mem[read_add];

always @(posedge clk) begin
read_add <= a;
if(we)
   /* Register RAM Data */
   mem[a] <= d;
end

endmodule
```

VHDL Example: READ Address Registered

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
```


© 2022 Synopsys, Inc.

```
entity ram_test is
   port (d : in std_logic_vector(7 downto 0);
         a : in std_logic_vector(6 downto 0);
         we : in std_logic;
         clk : in std_logic;
         q : out std_logic_vector(7 downto 0) );
end ram_test;

architecture rtl of ram_test is
type mem_type is array (127 downto 0) of
   std_logic_vector (7 downto 0);
signal mem: mem_type;
signal read_add : std_logic_vector(6 downto 0);
begin
   process (clk)
   begin
      if rising_edge(clk) then
         if (we = '1') then
            mem(conv_integer(a)) <= d;
         end if;
         read_add <= a;
      end if;
   end process;

q <= mem(conv_integer(read_add));
end rtl ;
```

Single-Port RAM with RAM Output Registered Examples

In this example, the RAM output is registered, but the read and write addresses are unregistered. The write address is the same as the read address. There is one clock for the RAM and the output.

Verilog Example: Data Output Registered

```
module ram_test(q, a, d, we, clk);
output [7:0] q;
input [7:0] d;
input [6:0] a;
input clk, we;
/* The array of an array register ("mem") from which the RAM is
inferred */
reg [7:0] mem [127:0] ;
reg [7:0] q;
```

```
always @(posedge clk) begin
q = mem[a];
if(we)
   /* Register RAM Data */
   mem[a] <= d;
end

endmodule
```

VHDL Example: Data Output Registered

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity ram_test is
   port (d: in std_logic_vector(7 downto 0);
         a: in integer range 127 downto 0;
         we: in std_logic;
         clk: in std_logic;
      q: out std_logic_vector(7 downto 0) );
end ram_test;

architecture rtl of ram_test is
type mem_type is array (127 downto 0) of
   std_logic_vector (7 downto 0);
signal mem: mem_type;
begin
   process(clk)
   begin
      if (clk'event and clk='1') then
         q <= mem(a);
            if (we='1') then
               mem(a) <= d;
            end if;
      end if;
   end process;
end rtl;
```

Dual-Port Block RAM Examples

The following example or RTL code results in simple dual-port block RAMs being implemented in supported technologies.

© 2022 Synopsys, Inc.

Verilog Example: Dual-Port RAM

This Verilog example has two read addresses, both of which are registered, and one address for write (same as a read address), which is unregistered. It has two outputs for the RAM, which are unregistered. There is one clock for the RAM and the addresses.

```
module dualportram ( q1,q2,a1,a2,d,we,clk1) ;
output [7:0]q1,q2;
input [7:0] d;
input [6:0]a1,a2;
input clk1,we;
wire [7:0] q1;
reg [6:0] read_addr1,read_addr2;
reg[7:0] mem [127:0] /* synthesis syn_ramstyle = "no_rw_check" */;
assign q1 = mem [read_addr1];
assign q2 = mem[read_addr2];

always @ ( posedge clk1) begin
read_addr1 <= a1;
read_addr2 <= a2;
if (we)
   mem[a2] <= d;
end

endmodule
```

VHDL Example: Dual-Port RAM

The following VHDL example is of READ_FIRST mode for a dual-port RAM:

```
Library IEEE ;
use IEEE.std_logic_1164.all ;
use IEEE.std_logic_arith.all ;
use IEEE.std_logic_unsigned.all ;

entity Dual_Port_ReadFirst is
   generic (data_width: integer :=4;
   address_width: integer :=10);
```

© 2022 Synopsys, Inc.

```
port (write_enable: in std_logic;
   write_clk, read_clk: in std_logic;
   data_in: in std_logic_vector (data_width-1 downto 0);
   data_out: out std_logic_vector (data_width-1 downto 0);
   write_address: in std_logic_vector (address_width-1 downto 0);
   read_address: in std_logic_vector (address_width-1 downto 0)
   );
end Dual_Port_ReadFirst;

architecture behavioral of Dual_Port_ReadFirst is
type memory is array (2**(address_width-1) downto 0) of
   std_logic_vector (data_width-1 downto 0);
signal mem : memory;

signal reg_write_address : std_logic_vector (address_width-1 downto 0);
signal reg_write_enable: std_logic;

attribute syn_ramstyle : string;
attribute syn_ramstyle of mem : signal is "block_ram";

begin
register_enable_and_write_address:
   process (write_clk,write_enable,write_address,data_in)
   begin
      if (rising_edge(write_clk)) then
         reg_write_address <= write_address;
         reg_write_enable <= write_enable;
      end if;
   end process;

write:
   process (read_clk,write_enable,write_address,data_in)
   begin
      if (rising_edge(write_clk)) then
         if (write_enable='1') then
            mem(conv_integer(write_address))<=data_in;
         end if;
      end if;
   end process;

read:
   process (read_clk,write_enable,read_address,write_address)
   begin
      if (rising_edge(read_clk)) then
         if (reg_write_enable='1') and (read_address =
            reg_write_address) then data_out <= "XXXX";
```


© 2022 Synopsys, Inc.

```
            else
               data_out<=mem(conv_integer(read_address));
            end if;
         end if;
      end process;

   end behavioral;
```

True Dual-Port RAM Examples

You must use a registered read address when you code the RAM or have writes to one process. If you have writes to multiple processes, you must use the syn_ramstyle attribute to infer the RAM.

There are two situations which can result in this error message:

```
"@E:MF216: ram.v(29)|Found NRAM mem_1[7:0] with multiple
processes"
```

- An nram with two clocks and two write addresses has syn_ramstyle set to a value of registers. The software cannot implement this, because there is a physical FPGA limitation that does not allow registers with multiple writes.
- You have a registered output for an nram with two clocks and two write addresses.

Verilog Example: True Dual-Port RAM

The following RTL example shows the recommended coding style for true dual-port block RAM. It is a Verilog example where the tool infers true dual-port RAM from a design with multiple writes:

```
module ram(data0, data1, waddr0, waddr1, we0,we1,
   clk0, clk1, q0, q1);
parameter d_width = 8;
parameter addr_width = 8;
parameter mem_depth = 256;
input [d_width-1:0] data0, data1;
input [addr_width-1:0] waddr0, waddr1;
input we0, we1, clk0, clk1;
output [d_width-1:0] q0, q1;
```

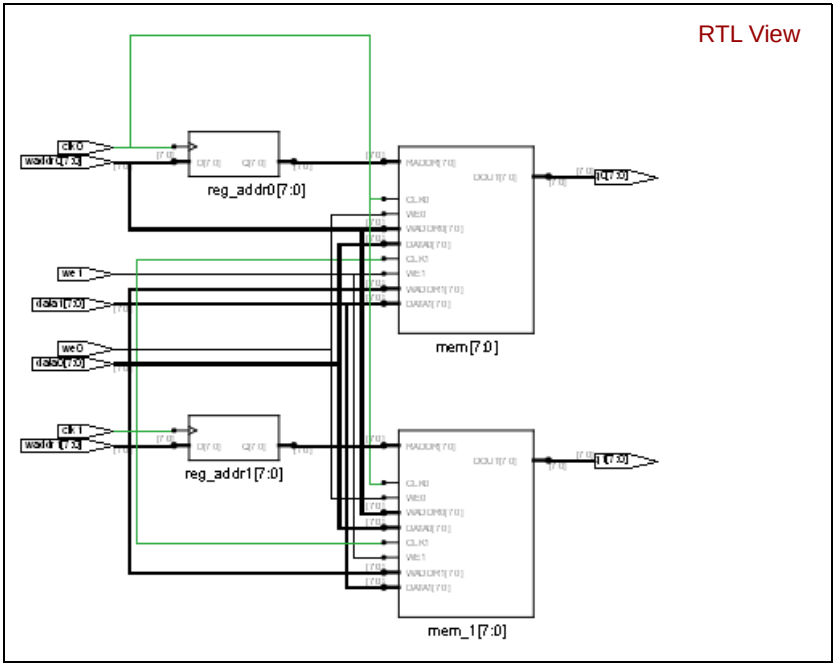
© 2022 Synopsys, Inc.

```
reg [addr_width-1:0] reg_addr0, reg_addr1;
reg [d_width-1:0] mem [mem_depth-1:0] /* synthesis
syn_ramstyle="no_rw_check" */;
assign q0 = mem[reg_addr0];
assign q1 = mem[reg_addr1];

always @(posedge clk0)
begin
   reg_addr0 <= waddr0;
   if (we0)
      mem[waddr0] <= data0;
end

always @(posedge clk1)
begin
   reg_addr1 <= waddr1;
   if (we1)
      mem[waddr1] <= data1;
end

endmodule
```



© 2022 Synopsys, Inc.

VHDL Example: True Dual-Port RAM

The following RTL example shows the recommended coding style for true dual-port block RAM. It is a VHDL example where the tool infers true dual-port RAM from a design with multiple writes:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity one is
generic (data_width : integer := 4;
address_width :integer := 5 );
   port (data_a:in std_logic_vector(data_width-1 downto 0);
         data_b:in std_logic_vector(data_width-1 downto 0);
         addr_a:in std_logic_vector(address_width-1 downto 0);
         addr_b:in std_logic_vector(address_width-1 downto 0);
         wren_a:in std_logic;
         wren_b:in std_logic;
         clk:in std_logic;
         q_a:out std_logic_vector(data_width-1 downto 0);
         q_b:out std_logic_vector(data_width-1 downto 0) );
end one;

architecture rtl of one is
type mem_array is array(0 to 2**(address_width) -1) of
std_logic_vector(data_width-1 downto 0);
signal mem : mem_array;
attribute syn_ramstyle : string;
attribute syn_ramstyle of mem : signal is "no_rw_check" ;
signal addr_a_reg : std_logic_vector(address_width-1 downto 0);
signal addr_b_reg : std_logic_vector(address_width-1 downto 0);
begin
   WRITE_RAM : process (clk)
   begin
      if rising_edge(clk) then
         if (wren_a = '1') then
            mem(to_integer(unsigned(addr_a))) <= data_a;
         end if;
         if (wren_b='1') then
            mem(to_integer(unsigned(addr_b))) <= data_b;
         end if;
         addr_a_reg <= addr_a;
         addr_b_reg <= addr_b;
```

```
      end if;
   end process WRITE_RAM;
q_a <= mem(to_integer(unsigned(addr_a_reg)));
q_b <= mem(to_integer(unsigned(addr_b_reg)));
end rtl;
```

Limitations to RAM Inference

RAM inference is only supported for synchronous RAMs.

Initial Values for RAMs

You can specify initial values for a RAM in a data file and then include the appropriate task enable statement, $readmemb or $readmemh, in the initial statement of the RTL code for the module. The inferred logic can be different due to the initial statement. The syntax for these two statements is as follows:

$readmemh ("*fileName***", *memoryName* [, *startAddress* [, *stopAddress*]]);**

$readmemb ("*fileName***", *memoryName* [, *startAddress* [, *stopAddress*]]);**

$readmemb	Use this with a binary data file.
$readmemh	Use this with a hexadecimal data file.
fileName	Name of the data file that contains initial values. See Initialization Data File, on page 183 for format examples.
memoryName	The name of the memory.
startAddress	Optional starting address for RAM initialization; if omitted, defaults to first available memory location.
stopAddress	Optional stopping address for RAM initialization; *startAddress* must be specified

© 2022 Synopsys, Inc.

This section contains the following topics:

- Example 1: RAM Initialization, on page 180
- Example 2: Cross-Module Referencing for RAM Initialization, on page 181
- Initialization Data File, on page 183
- Forward Annotation of Initial Values, on page 186

Example 1: RAM Initialization

This example shows a single-port RAM that is initialized using the $readmemb binary task enable statement which reads the values specified in the binary mem.ini file. See Initialization Data File, on page 183 for details of the binary and hexadecimal file formats.

```
module ram_inference (data, clk, addr, we, data_out);
input [27:0] data;
input clk, we;
input [10:0] addr;
output [27:0] data_out;
reg [27:0] mem [0:2000] /* synthesis syn_ramstyle = "no_rw_check" */;
reg [10:0] addr_reg;

initial
begin
   $readmemb ("mem.ini", mem, 2, 1900) /* Initialize RAM with contents */
      /* from locations 2 thru 1900*/;
end

always @(posedge clk)
begin
   addr_reg <= addr;
end

always @(posedge clk)
begin
   if(we)
   begin
      mem[addr] <= data;
   end
end

assign data_out = mem[addr_reg];
endmodule
```

Example 2: Cross-Module Referencing for RAM Initialization

The following example shows how a RAM using cross-module referencing (XMR) can be accessed hierarchically and initialized with the $readmemb / $readmemh statement which reads the values specified in the mem.txt file from the top-level design.

Example 2A: XMR for RAM Initialization

```
   (Top-Level Module)

//Top

module top (input[27:0] data, input clk, we, input[10:0] addr,

   output[27:0] data_out);

ram_inference ram_inst (.*);

initial

begin

   $readmemb (“mem.txt”, top.ram_inst.mem, 0, 10);

end

endmodule
```

This code example implements cross-module referencing of the RAM block and is initialized with the $readmemb statement in the top-level module.

Example 2B: XMR for RAM Initialization (RAM)

```
//RAM

module ram_inference (input[27:0] data, input clk, input[10:0]

   addr, output[27:0] data_out);

reg[27:0] mem[0:2000] /*synthesis syn_ramstyle = “no_rw_check”*/;

reg [10:0] addr_reg;

always @(posedge clk)

begin

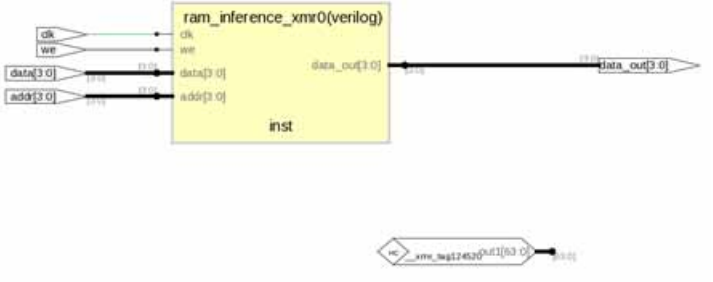
   addr_reg <= addr;
```


© 2022 Synopsys, Inc.

```
end
always @(posedge clk)
begin
   if(we)
   begin
      mem[addr] <= data;
   end
end
assign data_out = mem[addr_reg];
endmodule
```

Here is the code example of the RAM block to be implemented for cross-module referencing and initialized.

The following shows the HDL Analyst view of a RAM module that must be accessed hierarchically to be initialized.



RAM Initialization Limitations with XMR

XMR for RAM initialization requires that the following conditions be met:

- Variables must be recognized as inferred memories.
- Cross-module referencing of memory variables cannot occur between HDL languages.
- Cross-module referencing paths must be static and cannot include an index with a dynamic value.

Initialization Data File

The initialization data file, read by the $readmemb and $readmemh system tasks, contains the initial values to be loaded into the memory array. This initialization file can reside in the project directory or can be referenced by an include path relative to the project directory. The system $readmemb or $readmemh task first looks in the project directory for the named file and, if not found, searches for the file in the list of directories on the Verilog tab in include-path order.

If the initialization data file does not contain initial values for every memory address, the unaddressed memory locations are initialized to 0. Also, if a width mismatch exists between an initialization value and the memory width, loading of the memory array is terminated; any values initialized before the mismatch is encountered are retained.

Unless an internal address is specified (see Internal Address Format, on page 185), each value encountered is assigned to a successive word element of the memory. If no addressing information is specified either with the $readmem task statement or within the initialization file itself, the default starting address is the lowest available address in the memory. Consecutive words are loaded until either the highest address in the memory is reached or the data file is completely read.

If a start address is specified without a finish address, loading starts at the specified start address and continues upward toward the highest address in the memory. In either case, loading continues upward. If both a start address and a finish address are specified, loading begins at the start address and continues until the finish address is reached (or until all initialization data is read).

© 2022 Synopsys, Inc.

For example:

```
initial
begin
//$readmemh ("mem.ini", ram_bank1)
   /* Initialize RAM with contents from locations 0 thru 31*/;

//$readmemh ("mem.ini", ram_bank1,0)
   /* Initialize RAM with contents from locations 0 thru 31*/;

$readmemh ("mem.ini", ram_bank1, 0, 31)
   /* Initialize RAM with contents from locations 0 thru 31*/;

$readmemh ("mem.ini", ram_bank2, 31, 0)
   /* Initialize RAM with contents from locations 31 thru 0*/;
```

The data initialization file can contain the following:

- White space (spaces, new lines, tabs, and form-feeds)
- Comments (both comment formats are allowed)
- Binary values for the $readmemb task, or hexadecimal values for the $readmemh tasks

In addition, the data initialization file can include any number of hexadecimal addresses (see Internal Address Format, on page 185).

Binary File Format

The binary data file mem.ini that corresponds to the example in Example 1: RAM Initialization, on page 180 looks like this:

```
11111111111111111111100110111   /* data for address 0 */
11111111111111111111101100111   /* data for address 1 */
11111111111111111111111000010
11111111111111111111100100001
11111111111111111111101110000
11111111111111111111011100110
... /* continues until Address 1999 */
```

© 2022 Synopsys, Inc.

Hex File Format

If you use $readmemh instead of $readmemb, the hexadecimal data file for the example in Example 1: RAM Initialization, on page 180 looks like this:

```
FFFFF37   /* data for address 0 */
FFFFF63   /* data for address 1 */
FFFFFC2
FFFFF21
.../* continues until Address 1999 */
```

Internal Address Format

In addition to the binary and hex formats described above, the initialization file can include embedded hexadecimal addresses. These hexadecimal addresses must be prefaced with an at sign (@) as shown in the example below.

```
FFFFF37  /* data for address 0 */
FFFFF63  /* data for address 1 */
@0EA     /* memory address 234
FFFFFC2  /* data for address 234*/
FFFFF21  /* data for address 235*/
...
@0A7     /* memory address 137
FFFFF77  /* data for address 137*/
FFFFF7A  /* data for address 138*/
...
```

Either uppercase or lowercase characters can be used in the address. No white space is allowed between the @ and the hex address. Any number of address specifications can be included in the file, and in any order. When the $readmemb or $readmemh system task encounters an embedded address specification, it begins loading subsequent data at that memory location.

When addressing information is specified both in the system task and in the data file, the addresses in the data file must be within the address range specified by the system task arguments; otherwise, an error message is issued, and the load operation is terminated.

© 2022 Synopsys, Inc.

Forward Annotation of Initial Values

Initial values for RAMs and sequential shift components are forward annotated to the netlist. The compiler currently generates netlist (srs) files with seqshift, ram1, ram2, and nram components. If initial values are specified in the HDL code, the synthesis tool attaches an attribute to the component in the srs file.

RAM Instantiation with SYNCORE

The SYNCORE Memory Compiler in the IP Wizard helps you generate HDL code for your specific RAM implementation requirements. For information on using the SYNCORE Memory Compiler, see Specifying RAMs with SYNCore, on page 223 in the *User Guide*.

© 2022 Synopsys, Inc.

ROM Inference

As part of BEST (Behavioral Extraction Synthesis Technology) feature, the synthesis tool infers ROMs (read-only memories) from your HDL source code, and generates block components for them in the RTL view.

The data contents of the ROMs are stored in a text file named rom.info. To quickly view rom.info in read-only mode, synthesize your HDL source code, open an RTL view, then push down into the ROM component.

Generally, the synthesis tool infers ROMs from HDL source code that uses case statements, or equivalent if statements, to make 16 or more signal assignments using constant values (words). The constants must all be the same width.

Another requirement for ROM inference is that values must be specified for at least half of the address space. For example, if the ROM has 5 address bits, then the address space is 32 and at least 16 of the different addresses must be specified.

Verilog Example

```
module rom(z,a);
output [3:0] z;
input [4:0] a;
reg [3:0] z;

always @(a) begin
   case (a)
      5'b00000 : z = 4'b0001;
      5'b00001 : z = 4'b0010;
      5'b00010 : z = 4'b0110;
      5'b00011 : z = 4'b1010;
      5'b00100 : z = 4'b1000;
      5'b00101 : z = 4'b1001;
      5'b00110 : z = 4'b0000;
      5'b00111 : z = 4'b1110;
      5'b01000 : z = 4'b1111;
      5'b01001 : z = 4'b1110;
      5'b01010 : z = 4'b0001;
      5'b01011 : z = 4'b1000;
      5'b01100 : z = 4'b1110;
      5'b01101 : z = 4'b0011;
      5'b01110 : z = 4'b1111;
```


© 2022 Synopsys, Inc.

```
      5'b01111 : z = 4'b1100;
      5'b10000 : z = 4'b1000;
      5'b10001 : z = 4'b0000;
      5'b10010 : z = 4'b0011;
      default : z = 4'b0111;
   endcase
end
endmodule
```

VHDL Example

```
library ieee;
use ieee.std_logic_1164.all;

entity rom4 is
   port (a : in std_logic_vector(4 downto 0);
         z : out std_logic_vector(3 downto 0) );
end rom4;

architecture behave of rom4 is
begin
   process(a)
   begin
      if a = "00000" then
         z <= "0001";
      elsif a = "00001" then
         z <= "0010";
      elsif a = "00010" then
         z <= "0110";
      elsif a = "00011" then
         z <= "1010";
      elsif a = "00100" then
         z <= "1000";
      elsif a = "00101" then
         z <= "1001";
      elsif a = "00110" then
         z <= "0000";
      elsif a = "00111" then
         z <= "1110";
      elsif a = "01000" then
         z <= "1111";
      elsif a = "01001" then
         z <= "1110";
      elsif a = "01010" then
         z <= "0001";
      elsif a = "01011" then
```

```
            z <= "1000";
         elsif a = "01100" then
            z <= "1110";
         elsif a = "01101" then
            z <= "0011";
         elsif a = "01110" then
            z <= "1111";
         elsif a = "01111" then
            z <= "1100";
         elsif a = "10000" then
            z <= "1000";
         elsif a = "10001" then
            z <= "0000";
         elsif a = "10010" then
            z <= "0011";
         else
            z <= "0111";
         end if;
      end process;
   end behave;
```

ROM Table Data (rom.info File)

Note: This data is for viewing only.

```
ROM work.rom4(behave)-z_1[3:0]
address width: 5
data width: 4
inputs:
0: a[0]
1: a[1]
2: a[2]
3: a[3]
4: a[4]
outputs:
0: z_1[0]
1: z_1[1]
2: z_1[2]
3: z_1[3]

data:
00000 -> 0001
00001 -> 0010
00010 -> 0110
00011 -> 1010
00100 -> 1000
```


© 2022 Synopsys, Inc.

```
00101 -> 1001
00110 -> 0000
00111 -> 1110
01000 -> 1111
01001 -> 1110
01010 -> 0001
01011 -> 1000
01100 -> 1110
01101 -> 0011
01110 -> 0010
01111 -> 0010
10000 -> 0010
10001 -> 0010
10010 -> 0010
default -> 0111
```

ROM Initialization with Generate Block

The software supports conditional ROM initialization with the generate block, as shown in the following example:

```
generate
   if (INIT) begin
      initial
      begin
         $readmemb("init.hex",mem);
      end
   end
endgenerate
```

CHAPTER 7

SynCore IP Tool

This chapter describes the SYNCore IP functionality that is bundled with the synthesis tool.

- SYNCore FIFO Compiler, on page 192
- SYNCore RAM Compiler, on page 223
- SYNCore Byte-Enable RAM Compiler, on page 245
- SYNCore ROM Compiler, on page 260
- SYNCore Adder/Subtractor Compiler, on page 274
- SYNCore Counter Compiler, on page 297

© 2022 Synopsys, Inc.

SYNCore FIFO Compiler

The SYNCore synchronous FIFO compiler offers an IP wizard that generates Verilog code for your FIFO implementation. This section describes the following:

- Synchronous FIFO Overview, on page 192
- Specifying FIFOs with SYNCore, on page 193
- SYNCore FIFO Wizard, on page 197
- FIFO Read and Write Operations, on page 207
- FIFO Ports, on page 208
- FIFO Parameters, on page 211
- FIFO Status Flags, on page 213
- FIFO Programmable Flags, on page 216

Synchronous FIFO Overview

A FIFO is a First-In-First-Out memory queue. Different control logic manages the read and write operations. A FIFO also has various handshake signals for interfacing with external user modules.

The SYNCore FIFO compiler generates synchronous FIFOs with symmetric ports and one clock controlling both the read and write operations. The FIFO is symmetric because the read and write ports have the same width.

When the Write_enable signal is active and the FIFO has empty locations, data is written into FIFO memory on the rising edge of the clock. A Full status flag indicates that the FIFO is full and that no more write operations can be performed. See FIFO Write Operation, on page 207 for details.

When the FIFO has valid data and Read_enable is active, data is read from the FIFO memory and presented at the outputs. The FIFO Empty status flag indicates that the FIFO is empty and that no more read operations can be performed. See FIFO Read Operation, on page 208 for details.

The FIFO is not corrupted by an invalid request, for example, if a read request is made while the FIFO is empty or a write request is received when the FIFO is full. Invalid requests do not corrupt the data, but they cause the corre-

© 2022 Synopsys, Inc.

sponding read or write request to be ignored and the Overflow or Underflow flags to be asserted. You can monitor these status flags for invalid requests. These and other flags are described in FIFO Status Flags, on page 213 and FIFO Programmable Flags, on page 216.

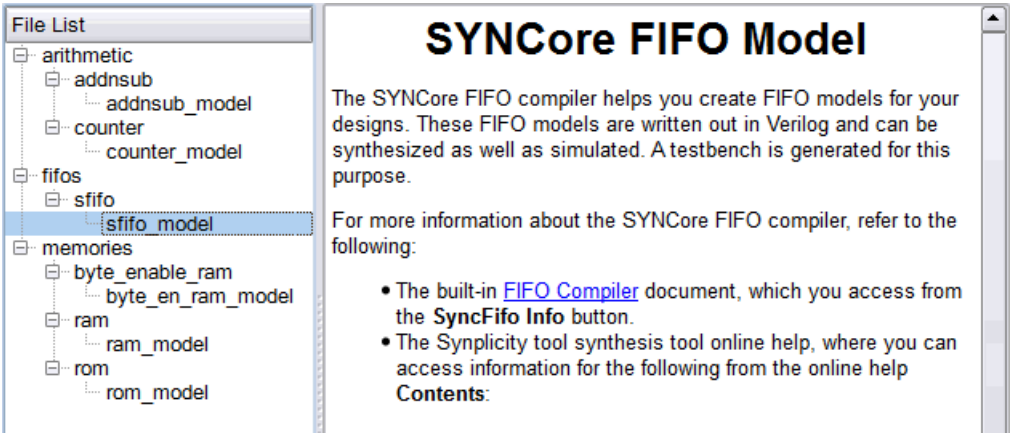
At any point in time, Data count reflects the available data inside the FIFO. In addition, you can use the Programmable Full and Programmable Empty status flags for user-defined thresholds.

Specifying FIFOs with SYNCore

The SYNCore IP Wizard helps you generate Verilog code for your FIFO implementations. The following procedure shows you how to generate Verilog code for a FIFO using the SYNCore IP wizard.

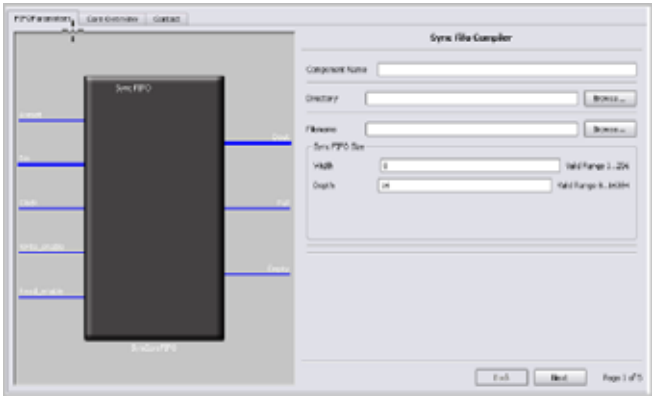
Note: The SYNCore FIFO model uses Verilog 2001. When adding a FIFO model to a Verilog-95 design, be sure to enable the Verilog 2001 check box on the Verilog tab of the Implementation Options dialog box or include a set_option -vlog_std v2001 statement in your project file to prevent a syntax error.

1. Start the wizard.
   - From the synthesis tool GUI, select Run->Launch SYNCore or click the Launch SYNCore icon to start the SYNCore IP wizard.



© 2022 Synopsys, Inc.

– In the window that opens, select sfifo_model and click Ok. This opens the first screen of the wizard.



2. Specify the parameters you need in the five pages of the wizard. For details, refer to Specifying SYNCore FIFO Parameters, on page 195.

   The FIFO symbol on the left reflects the parameters you set.

3. After you have specified all the parameters you need, click the Generate button (lower left).

   The tool displays a confirmation message (TCL execution successful!) and writes the required files to the directory you specified in the parameters. The HDL code is in Verilog.

   The FIFO generated is a synchronous FIFO with symmetric ports and with the same clock controlling both the read and write operations. Data is written or read on the rising edge of the clock. All resets are synchronous with the clock. All edges (clock, enable, and reset) are considered positive.

   SYNCore also generates a testbench for the FIFO that you can use for simulation. The testbench covers a limited set of vectors for testing.

   You can now close the SYNCore wizard.

4. Add the FIFO you generated to your design.

   – Use the Add File command to add the Verilog design file that was generated and the syncore_sfifo.v file to your project. These files are in

the directory for output files that you specified on page 1 of the wizard.

- Use a text editor to open the instantiation_file.vin template file, which is located in the same directory. Copy the lines that define the memory, and paste them into your top-level module. The following shows a template file (in red text) inserted into a top-level module.

```
module top (
input Clk,
input [15:0] DataIn,
input WrEn,
input RdEn,
output Full,
output Empty,
output [15:0] DataOut
);

fifo_a32 <instanceName>(
.Clock(Clock)
,.Din(Din)
,.Write_enable(Write_enable)
,.Read_enable(Read_enable)
,.Dout(Dout)
,.Full(Full)
,.Empty(Empty)
)

endmodule
```

template

5. Edit the template port connections so that they agree with the port definitions in your top-level module as shown in the example below (the updated connection names are shown in red). You can also assign a unique name to each instantiation.

   Note that currently, the FIFO models will not be implemented with the dedicated FIFO blocks available in certain technologies.

Specifying SYNCore FIFO Parameters

The following elaborates on the parameter settings for SYNCore FIFOs. The status, handshaking, and programmable flags are optional. For descriptions of the parameters, see *SYNCore FIFO Wizard, on page 197*.

© 2022 Synopsys, Inc.

1. Start the SYNCore wizard, as described in Specifying FIFOs with SYNCore, on page 193.
2. Do the following on page 1 of the FIFO wizard:
   - In Component Name, specify a name for the FIFO. Do not use spaces.
   - In Directory, specify a directory where you want the output files to be written. Do not use spaces.
   - In Filename, specify a name for the Verilog output file with the FIFO specifications. Do not use spaces.
   - Click Next. The wizard opens another page where you can set parameters.
3. For a FIFO with no status, handshaking, or programmable flags, use the default settings. You can generate the FIFO, as described in Specifying FIFOs with SYNCore, on page 193.
4. To set an almost full status flag, do the following on page 2 of the FIFO wizard:
   - Enable Almost Full.
   - Set associated handshaking flags for the signal as desired, with the Overflow Flag and Write Acknowledge options.
   - Click Next when you are done.
5. To set an almost empty status flag, do the following on page 3:
   - Enable Almost Empty.
   - Set associated handshaking flags for the signal as desired, with the Underflow Flag and Read Acknowledge options.
   - Click Next when you are done.
6. To set a programmable full flag, do the following:
   - Make sure you have enabled Full on page 2 of the wizard and set any handshaking flags you require.
   - Go to page 4 and enable Programmable Full.
   - Select one of the four mutually exclusive configurations for Programmable Full on page 4. See Programmable Full, on page 217 for details.
   - Click Next when you are done.

© 2022 Synopsys, Inc.

7. To set a programmable empty flag, do the following:
   - Make sure you have enabled Empty on page 3 of the wizard and set any handshaking flags you require.
   - Go to page 5 and enable Programmable Empty.
   - Select one of the four mutually exclusive configurations for Programmable Empty on page 5. See Programmable Empty, on page 219 for details.

   You can now generate the FIFO and add it to the design, as described in Specifying FIFOs with SYNCore, on page 193.

SYNCore FIFO Wizard

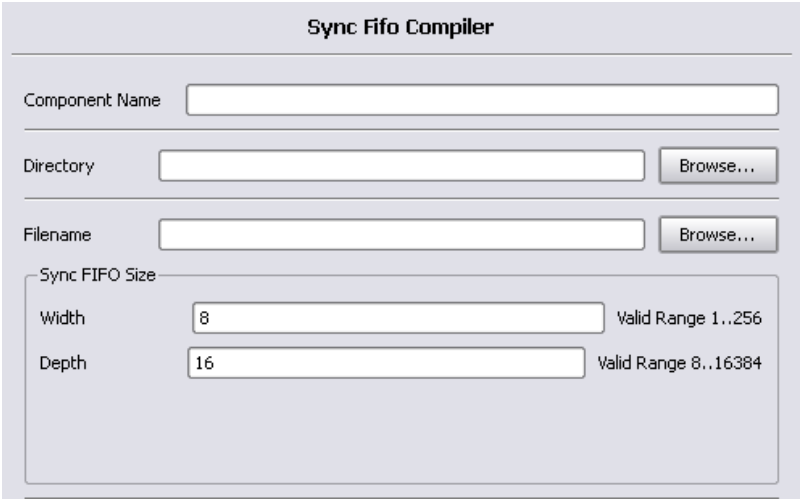
The following describe the parameters you can set in the FIFO wizard, which opens when you select sfifo_model:

- SYNCore FIFO Parameters Page 1, on page 197
- SYNCore FIFO Parameters Page 2, on page 199
- SYNCore FIFO Parameters Page 3, on page 201
- SYNCore FIFO Parameters Page 4, on page 203
- SYNCore FIFO Parameters Page 5, on page 205

SYNCore FIFO Parameters Page 1

The page 1 parameters define the FIFO. Data is written/read on the rising edge of the clock.

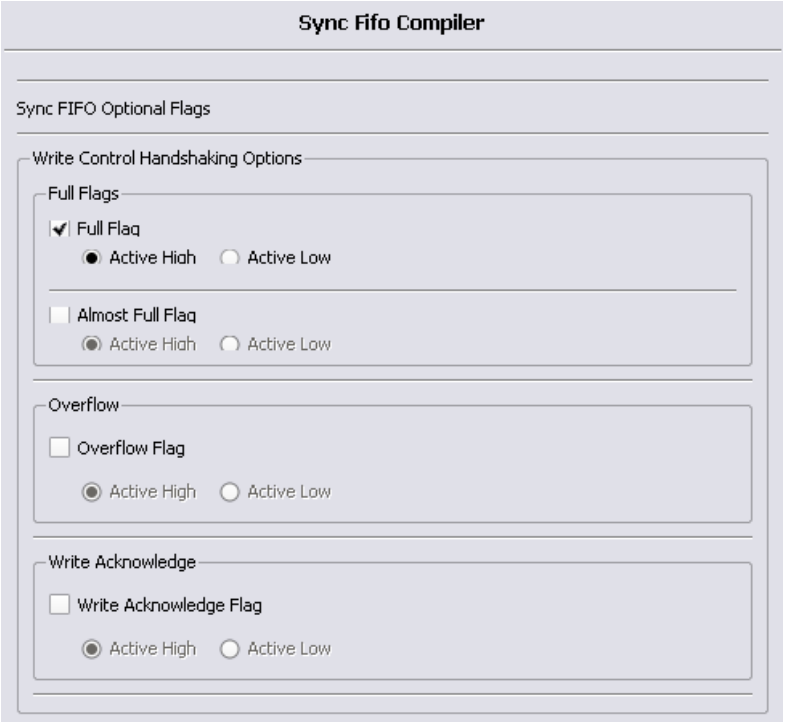
© 2022 Synopsys, Inc.



Parameter	Function
Component Name	Specifies a name for the FIFO. This is the name that you instantiate in your design file to create an instance of the SYNCore FIFO in your design. Do not use spaces.
Directory	Indicates the directory where the generated files will be stored. Do not use spaces.
Filename	Specifies the name of the generated file containing the HDL description of the generated FIFO. Do not use spaces.
Width	Specifies the width of the FIFO data input and output. It must be within the valid range.
Depth	Specifies the depth of the FIFO. It must be within the valid range.

© 2022 Synopsys, Inc.

SYNCore FIFO Parameters Page 2



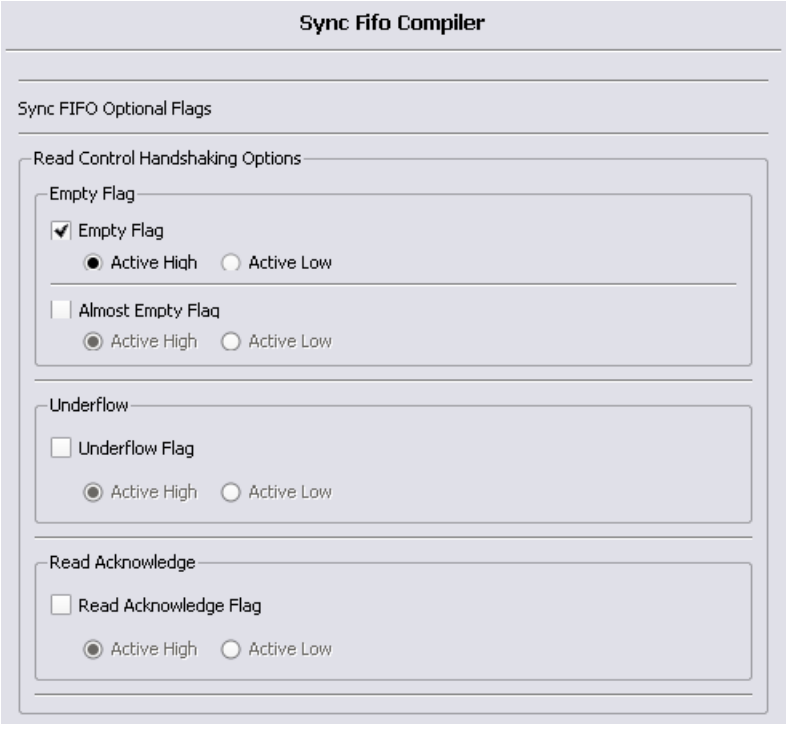
© 2022 Synopsys, Inc.

The page 2 parameters let you specify optional handshaking flags for FIFO write operations. When you specify a flag, the symbol on the left reflects your choice. Data is written/read on the rising edge of the clock.

Parameter	Function
Full Flag	Specifies a Full signal, which is asserted when the FIFO memory queue is full and no more writes can be performed until data is read. Enabling this option makes the Active High and Active Low options (FULL_FLAG_SENSE parameter) available for the signal. See Full/Almost Full Flags, on page 213 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Almost Full Flag	Specifies an Almost_full signal, which is asserted to indicate that there is one location left and the FIFO will be full after one more write operation. Enabling this option makes the Active High and Active Low options available for the signal (AFULL_FLAG_SENSE parameter). See Full/Almost Full Flags, on page 213 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Overflow Flag	Specifies an Overflow signal, which is asserted to indicate that the write operation was unsuccessful because the FIFO was full. Enabling this option makes the Active High and Active Low options available for the signal (OVERFLOW_FLAG_SENSE parameter). See Handshaking Flags, on page 214 f and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Write Acknowledge Flag	Specifies a Write_ack signal, which is asserted at the completion of a successful write operation. Enabling this option makes the Active High and Active Low options (WACK_FLAG_SENSE parameter) available for the signal. See Handshaking Flags, on page 214 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Active High	Sets the specified signal to active high (1).
Active Low	Sets the specified signal to active low (0).

SYNCore FIFO Parameters Page 3

The page 3 parameters let you specify optional handshaking flags for FIFO read operations. Data is written/read on the rising edge of the clock.

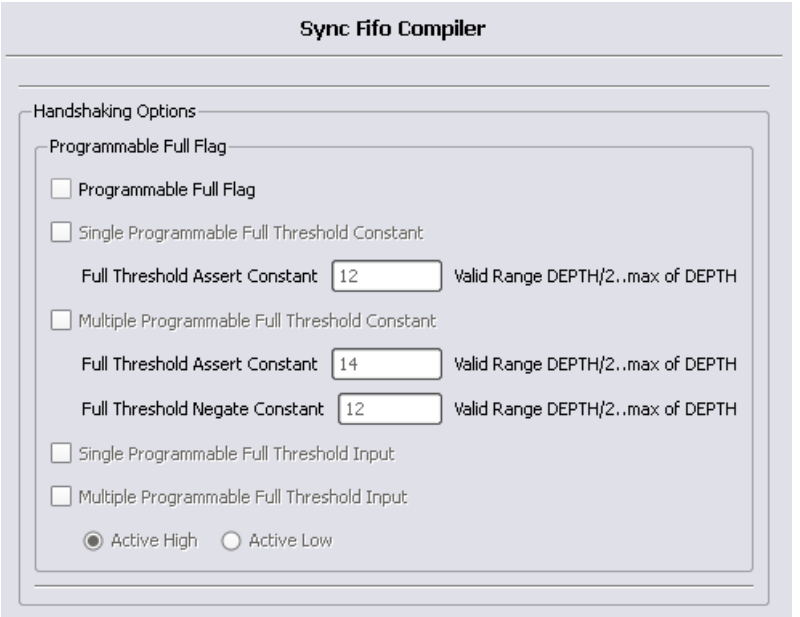


Parameter	Function
Empty Flag	Specifies an Empty signal, which is asserted when the memory queue for the FIFO is empty and no more reads can be performed until data is written. Enabling this option makes the Active High and Active Low options (EMPTY_FLAG_SENSE parameter) available for the signal. See Empty/Almost Empty Flags, on page 214 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.

© 2022 Synopsys, Inc.

Parameter	Function
Almost Empty Flag	Specifies an Almost_empty signal, which is asserted when there is only one location left to be read. The FIFO will be empty after one more read operation. Enabling this option makes the Active High and Active Low options (AEMPTY_FLAG_SENSE parameter) available for the signal. See Empty/Almost Empty Flags, on page 214 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Underflow Flag	Specifies an Underflow signal, which is asserted to indicate that the read operation was unsuccessful because the FIFO was empty. Enabling this option makes the Active High and Active Low options (UNDRFLW_FLAG_SENSE parameter) available for the signal. See Handshaking Flags, on page 214 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Read Acknowledge Flag	Specifies a Read_ack signal, which is asserted at the completion of a successful read operation. Enabling this option makes the Active High and Active Low options (RACK_FLAG_SENSE parameter) available for the signal. See Handshaking Flags, on page 214 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.
Active High	Sets the specified signal to active high (1).
Active Low	Sets the specified signal to active low (0).

SYNCore FIFO Parameters Page 4



The page 4 parameters let you specify optional handshaking flags for FIFO programmable full operations. To use these options, you must have a Full signal specified. See FIFO Programmable Flags, on page 216 for details and FIFO Parameters, on page 211 for a list of the FIFO parameters. Data is written/read on the rising edge of the clock.

Parameter	Function
Programmable Full Flag	Specifies a Prog_full signal, which indicates that the FIFO has reached a user-defined full threshold. You can only enable this option if you set Full Flag on page 2. When it is enabled, you can specify other options for the Prog_Full signal (PFULL_FLAG_SENSE parameter). See Programmable Full, on page 217 and FIFO Parameters, on page 211 for descriptions of the flag and parameter.

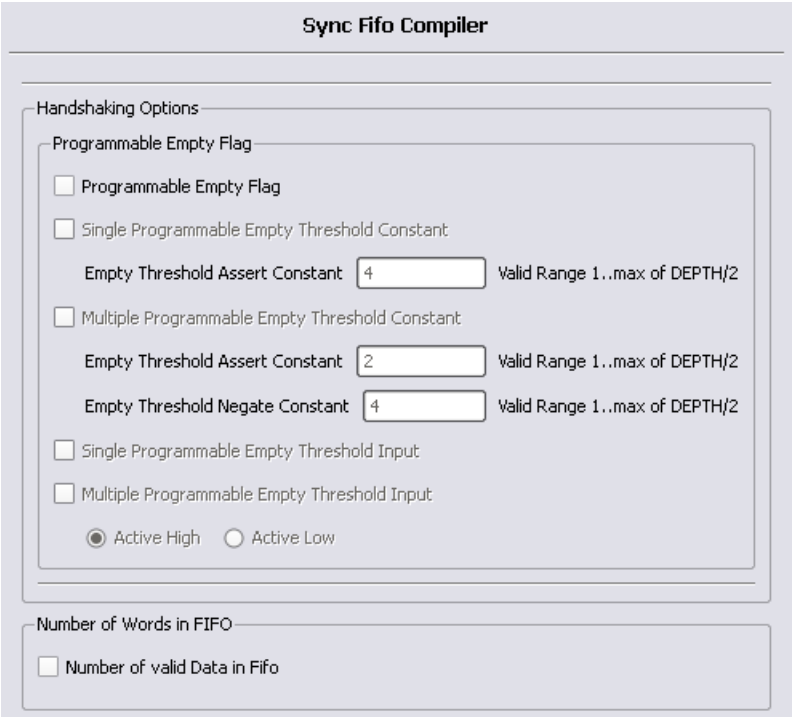
© 2022 Synopsys, Inc.

Parameter	Function
Single Programmable Full Threshold Constant	Specifies a Prog_full signal with a single constant defining the assertion threshold (PGM_FULL_TYPE=1 parameter). See Programmable Full with Single Threshold Constant, on page 217 for details. Enabling this option makes Full Threshold Assert Constant available.
Multiple Programmable Full Threshold Constant	Specifies a Prog_full signal (PGM_FULL_TYPE=2 parameter), with multiple constants defining the assertion and de-assertion thresholds. See Programmable Full with Multiple Threshold Constants, on page 218 for details. Enabling this option makes Full Threshold Assert Constant and Full Threshold Negate Constant available.
Full Threshold Assert Constant	Specifies a constant that is used as a threshold value for asserting the Prog_full signal It sets the PGM_FULL_THRESH parameter for PGM_FULL_TYPE=1 and the PGM_FULL_ATHRESH parameter for PGM_FULL_TYPE=2.
Full Threshold Negate Constant	Specifies a constant that is used as a threshold value for de-asserting the Prog_full signal (PGM_FULL_NTHRESH parameter).
Single Programmable Full Threshold Input	Specifies a Prog_full signal (PGM_FULL_TYPE=3 parameter), with a threshold value specified dynamically through a Prog_full_thresh input port during the reset state. See Programmable Full with Single Threshold Input, on page 218 for details. Enabling this option adds the Prog_full_thresh input port to the FIFO.
Multiple Programmable Full Threshold Input	Specifies a Prog_full signal (PGM_FULL_TYPE=4 parameter), with threshold assertion and deassertion values specified dynamically through input ports during the reset state. See Programmable Full with Multiple Threshold Inputs, on page 219 for details. Enabling this option adds the Prog_full_thresh_assert and Prog_full_thresh_negate input ports to the FIFO.
Active High	Sets the specified signal to active high (1).
Active Low	Sets the specified signal to active low (0).

© 2022 Synopsys, Inc.

SYNCore FIFO Parameters Page 5

These options specify optional handshaking flags for FIFO programmable empty operations. To use these options, you first specify an Empty signal on page 3. See FIFO Programmable Flags, on page 216 for details and FIFO Parameters, on page 211 for a list of the FIFO parameters. Data is written/read on the rising edge of the clock.



Parameter	Function
Programmable Empty Flag	Specifies a Prog_empty signal (PEMPTY_FLAG_SENSE parameter), which indicates that the FIFO has reached a user-defined empty threshold. See Programmable Empty, on page 219 and FIFO Parameters, on page 211 for descriptions of the flag and parameter. Enabling this option makes the other options available to specify the threshold value, either as a constant or through input ports. You can also specify single or multiple thresholds for each of these options.

© 2022 Synopsys, Inc.

Parameter	Function
Single Programmable Empty Threshold Constant	Specifies a Prog_empty signal (PGM_EMPTY_TYPE=1 parameter), with a single constant defining the assertion threshold. See Programmable Empty with Single Threshold Input, on page 221 for details. Enabling this option makes Empty Threshold Assert Constant available.
Multiple Programmable Empty Threshold Constant	Specifies a Prog_empty signal (PGM_EMPTY_TYPE=2 parameter), with multiple constants defining the assertion and de-assertion thresholds. See Programmable Empty with Multiple Threshold Constants, on page 220 for details. Enabling this option makes Empty Threshold Assert Constant and Empty Threshold Negate Constant available.
Empty Threshold Assert Constant	Specifies a constant that is used as a threshold value for asserting the Prog_empty signal. It sets the PGM_EMPTY_THRESH parameter for PGM_EMPTY_TYPE=1 and the PGM_EMPTY_ATHRESH parameter for PGM_EMPTY_TYPE=2.
Empty Threshold Negate Constant	Specifies a constant that is used as a threshold value for de-asserting the Prog_empty signal (PGM_EMPTY_NTHRESH parameter).
Single Programmable Empty Threshold Input	Specifies a Prog_empty signal (PGM_EMPTY_TYPE=3 parameter), with a threshold value specified dynamically through a Prog_empty_thresh input port during the reset state. See Programmable Empty with Single Threshold Input, on page 221 for details. Enabling this option adds the Prog_full_thresh input port to the FIFO.
Multiple Programmable Empty Threshold Input	Specifies a Prog_empty signal (PGM_EMPTY_TYPE=4 parameter), with threshold assertion and deassertion values specified dynamically through Prog_empty_thresh_assert and Prog_empty_thresh_negate input ports during the reset state. See Programmable Empty with Multiple Threshold Inputs, on page 221 for details. Enabling this option adds the input ports to the FIFO.
Active High	Sets the specified signal to active high (1).
Active Low	Sets the specified signal to active low (0).

© 2022 Synopsys, Inc.

Parameter	Function
Number of Valid Data in FIFO	Specifies the Data_cnt signal for the FIFO output. This signal contains the number of words in the FIFO in the read domain.

FIFO Read and Write Operations

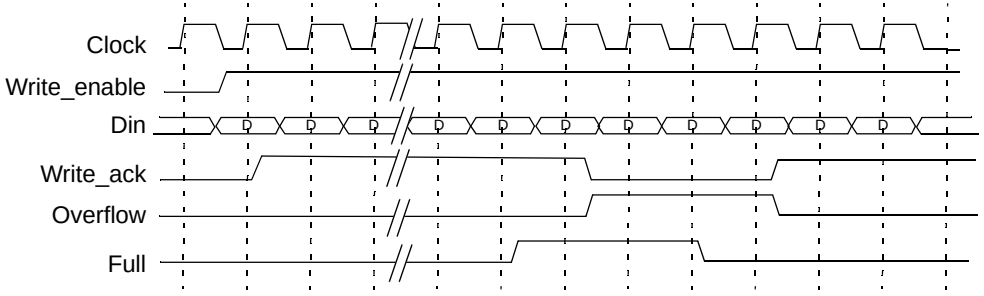
This section describes FIFO behavior with read and write operations.

FIFO Write Operation

When write enable is asserted and the FIFO is not full, data is added to the FIFO from the input bus (Din) and write acknowledge (Write_ack) is asserted. If the FIFO is continuously written without being read, it will fill with data. The status outputs are asserted when the number of entries in the FIFO is greater than or equal to the corresponding threshold, and should be monitored to avoid overflowing the FIFO.

When the FIFO is full, any attempted write operation fails and the overflow flag is asserted.

The following figure illustrates the write operation. Write acknowledge (Write_ack) is asserted on the next rising clock edge after a valid write operation. When Full is asserted, there can be no more legal write operations. This example shows that asserting Write_enable when Full is high causes the assertion of Overflow.



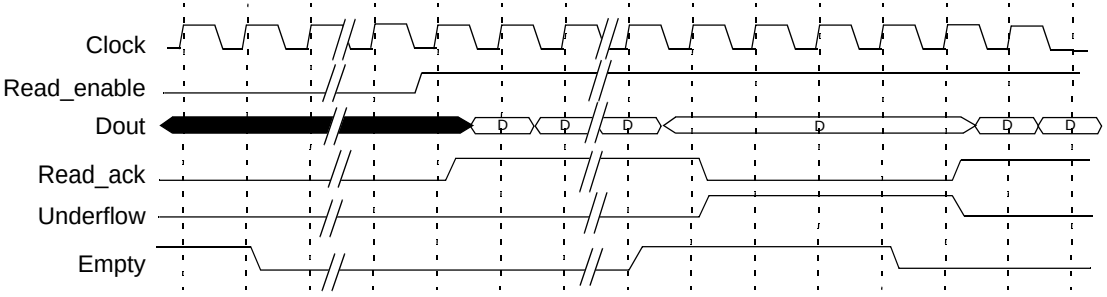
© 2022 Synopsys, Inc.

FIFO Read Operation

When read enable is asserted and the FIFO is not empty, the next data word in the FIFO is driven on the output bus (Dout) and a read valid is asserted. If the FIFO is continuously read without being written, the FIFO will empty. The status outputs are asserted when the number of entries in the FIFO are less than or equal to the corresponding threshold, and should be monitored to avoid underflow of the FIFO. When the FIFO is empty, all read operations fail and the underflow flag is asserted.

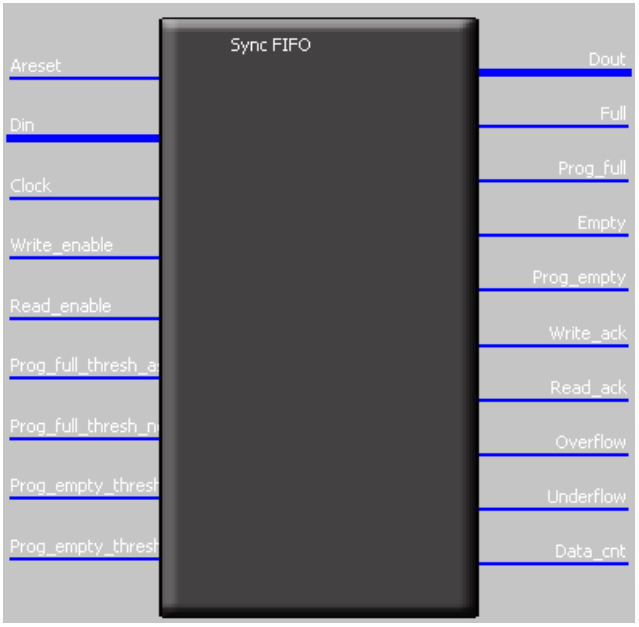
If read and write operation occur simultaneously during the empty state, the write operation will be valid and empty, and is de-asserted at the next rising clock edge. There cannot be a legal read operation from an empty FIFO, so the underflow flag is asserted.

The following figure illustrates a typical read operation. If the FIFO is not empty, Read_ack is asserted at the rising clock edge after Read_enable is asserted and the data on Dout is valid. When Empty is asserted, no more read operations can be performed. In this case, initiating a read causes the assertion of Underflow on the next rising clock edge, as shown in this figure.



FIFO Ports

The following figure shows the FIFO ports.



Port Name	Description
Almost_empty	Almost empty flag output (active high). Asserted when the FIFO is almost empty and only one more read can be performed. Can be active high or active low.
Almost_full	Almost full flag output (active high). Asserted when only one more write can be performed into the FIFO. Can be active high or active low.
AReset	Asynchronous reset input. Resets all internal counters and FIFO flag outputs.
Clock	Clock input for write and read. Data is written/read on the rising edge.
Data_cnt	Data word count output. Indicates the number of words in the FIFO in the read clock domain.
Din [width:0]	Data input word to the FIFO.
Dout [width:0]	Data output word from the FIFO.

© 2022 Synopsys, Inc.

Port Name	Description
Empty	FIFO empty output (active high). Asserted when the FIFO is empty and no additional reads can be performed. Can be active high or active low.
Full	FIFO full output (active high). Asserted when the FIFO is full and no additional writes can be performed. Can be active high or active low.
Overflow	FIFO overflow output flag (active high). Asserted when the FIFO is full and the previous write was rejected. Can be active high or active low.
Prog_empty	Programmable empty output flag (active high). Asserted when the words in the FIFO exceed or equal the programmable empty assert threshold. De-asserted when the number of words is more than the programmable full negate threshold. Can be active high or active low.
Prog_empty_ thresh	Programmable FIFO empty threshold input. User-programmable threshold value for the assertion of the Prog_empty flag. Set during reset.
Prog_empty_ thresh_assert	Programmable FIFO empty threshold assert input. User-programmable threshold value for the assertion of the Prog_empty flag. Set during reset.
Prog_empty_ thresh_negate	Programmable FIFO empty threshold negate input. User programmable threshold value for the de-assertion of the Prog_full flag. Set during reset.
Prog_full	Programmable full output flag (active high). Asserted when the words in the FIFO exceed or equal the programmable full assert threshold. De-asserted when the number of words is less than the programmable full negate threshold. Can be active high or active low.
Prog_full_thresh	Programmable FIFO full threshold input. User-programmable threshold value for the assertion of the Prog_full flag. Set during reset.
Prog_full_thresh_ assert	Programmable FIFO full threshold assert input. User-programmable threshold value for the assertion of the Prog_full flag. Set during reset.
Prog_full_thresh_ negate	Programmable FIFO full threshold negate input. User-programmable threshold value for the de-assertion of the Prog_full flag. Set during reset.

© 2022 Synopsys, Inc.

Port Name	Description
Read_ack	Read acknowledge output (active high). Asserted when valid data is read from the FIFO. Can be active high or active low.
Read_enable	Read enable output (active high). If the FIFO is not empty, data is read from the FIFO on the next rising edge of the read clock.
Underflow	FIFO underflow output flag (active high). Asserted when the FIFO is empty and the previous read was rejected.
Write_ack	Write Acknowledge output (active high). Asserted when there is a valid write into the FIFO. Can be active high or active low.
Write_enable	Write enable input (active high). If the FIFO is not full, data is written into the FIFO on the next rising edge.

FIFO Parameters

Parameter	Description
AEMPTY_FLAG_SENSE	FIFO almost empty flag sense 0 Active Low 1 Active High
AFULL_FLAG_SENSE	FIFO almost full flag sense 0 Active Low 1 Active High
DEPTH	FIFO depth
EMPTY_FLAG_SENSE	FIFO empty flag sense 0 Active Low 1 Active High
FULL_FLAG_SENSE	FIFO full flag sense 0 Active LowOVERFLOW_ 1 Active High
OVERFLOW_FLAG_ SENSE	FIFO overflow flag sense 0 Active Low 1 Active High

© 2022 Synopsys, Inc.

Parameter	Description
PEMPTY_FLAG_SENSE	FIFO programmable empty flag sense 0 Active Low 1 Active High
PFULL_FLAG_SENSE	FIFO programmable full flag sense 0 Active Low 1 Active High
PGM_EMPTY_ATHRESH	Programmable empty assert threshold for PGM_EMPTY_TYPE=2
PGM_EMPTY_NTHRESH	Programmable empty negate threshold for PGM_EMPTY_TYPE=2
PGM_EMPTY_THRESH	Programmable empty threshold for PGM_EMPTY_TYPE=1
PGM_EMPTY_TYPE	Programmable empty type. See Programmable Empty, on page 219 for details. 1 Programmable empty with single threshold constant. 2 Programmable empty with multiple threshold constant 3 Programmable empty with single threshold input 4 Programmable empty with multiple threshold input
PGM_FULL_ATHRESH	Programmable full assert threshold for PGM_FULL_TYPE=2
PGM_FULL_NTHRESH	Programmable full negate threshold for PGM_FULL_TYPE=2
PGM_FULL_THRESH	Programmable full threshold for PGM_FULL_TYPE=1
PGM_FULL_TYPE	Programmable full type. See Programmable Full, on page 217 for details. 1 Programmable full with single threshold constant 2 Programmable full with multiple threshold constant 3 Programmable full with single threshold input 4 Programmable full with multiple threshold input
RACK_FLAG_SENSE	FIFO read acknowledge flag sense 0 Active Low 1 Active High

© 2022 Synopsys, Inc.

Parameter	Description
UNDERFLOW_FLAG_SENSE	FIFO underflow flag sense 0 Active Low 1 Active High
WACK_FLAG_SENSE	FIFO write acknowledge flag sense 0 Active Low 1 Active High
WIDTH	FIFO data input and data output width

FIFO Status Flags

You can set the following status flags for FIFO read and write operations.

- Full/Almost Full Flags, on page 213
- Empty/Almost Empty Flags, on page 214
- Handshaking Flags, on page 214
- Programmable full and empty flags, which are described in Programmable Full, on page 217 and Programmable Empty, on page 219.

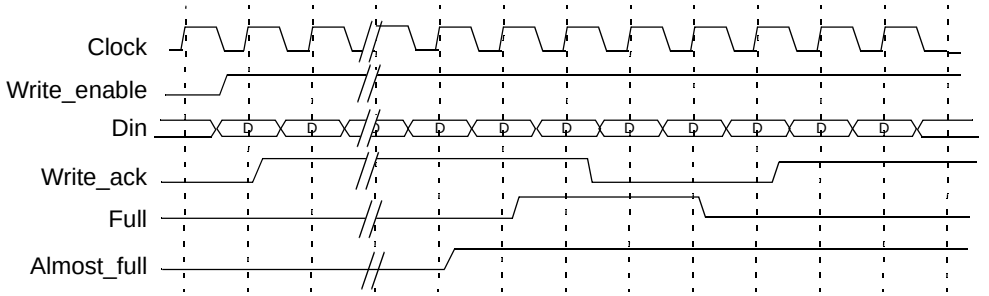
Full/Almost Full Flags

These flags indicate the status of the FIFO memory queue for write operations:

Full	Indicates that the FIFO memory queue is full and no more writes can be performed until data is read. Full is synchronous with the clock (Clock). If a write is initiated when Full is asserted, the write does not succeed and the overflow flag is asserted.
Almost_full	The almost full flag (Almost_full) indicates that there is one location left and the FIFO will be full after one more write operation. Almost full is synchronous to Clock. This flag is guaranteed to be asserted when the FIFO has one remaining location for a write operation.

The following figure displays the behavior of these flags. In this example, asserting Wriite_enable when Almost_full is high causes the assertion of Full on the next rising clock edge.

© 2022 Synopsys, Inc.

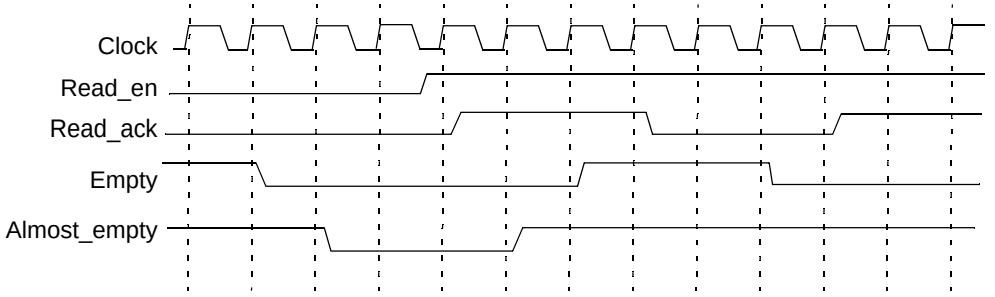


Empty/Almost Empty Flags

These flags indicate the status of the FIFO memory queue for read operations:

Empty	Indicates that the memory queue for the FIFO is empty and no more reads can be performed until data is written. The output is active high and is synchronous to the clock. If a read is initiated when the empty flag is true, the underflow flag is asserted.
Almost_ empty	Indicates that the FIFO will be empty after one more read operation. Almost_empty is active high and is synchronous to the clock. The flag is guaranteed to be asserted when the FIFO has one remaining location for a read operation.

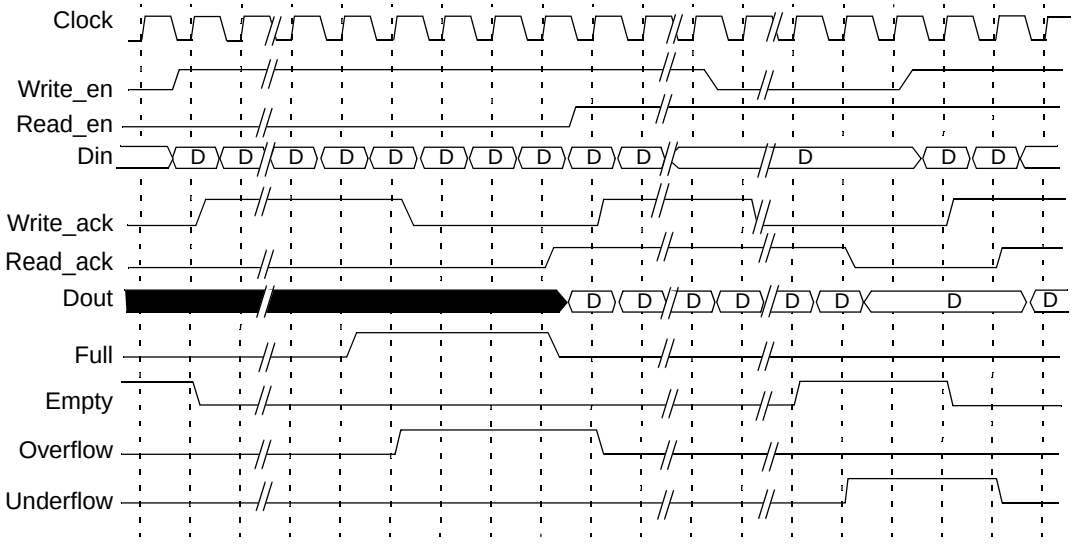
The following figure illustrates the behavior of the FIFO with one word remaining.



Handshaking Flags

You can specify optional Read_ack, Write_ack, Overflow, and Underflow handshaking flags for the FIFO.

© 2022 Synopsys, Inc.



Read_ack Asserted at the completion of each successful read operation. It indicates that the data on the Dout bus is valid. It is an optional port that is synchronous with Clock and can be configured as active high or active low.

Read_ack is deasserted when the FIFO is underflowing, which indicates that the data on the Dout bus is invalid. Read_ack is asserted at the next rising clock edge after read enable. Read_enable is asserted when the FIFO is not empty.

© 2022 Synopsys, Inc.

Write_ack	Asserted at the completion of each successful write operation. It indicates that the data on the Din port has been stored in the FIFO. It is synchronous with the clock, and can be configured as active high or active low. Write_ack is deasserted for a write to a full FIFO, as illustrated in the figure. Write_ack is deasserted one clock cycle after Full is asserted to indicate that the last write operation was valid and no other write operations can be performed.
Overflow	Indicates that a write operation was unsuccessful because the FIFO was full. In the figure, Full is asserted to indicate that no more writes can be performed. Because the write enable is still asserted and the FIFO is full, the next cycle causes Overflow to be asserted. Note that Write_ack is not asserted when FIFO is overflowing. When the write enable is deasserted, Overflow deasserts on the next clock cycle.
Underflow	Indicates that a read operation was unsuccessful, because the read was attempted on an empty FIFO. In the figure, Empty is asserted to indicate that no more reads can be performed. As the read enable is still asserted and the FIFO is empty, the next cycle causes Underflow to be asserted. Note that Read_ack is not asserted when FIFO is underflowing. When the read enable is deasserted, the Underflow flag deasserts on the next clock cycle.

FIFO Programmable Flags

The FIFO supports completely programmable full and empty flags to indicate when the FIFO reaches a predetermined user-defined fill level. See the following:

Prog_full	Indicates that the FIFO has reached a user-defined full threshold. See Programmable Full, on page 217 for more information.
Prog_empty	Indicates that the FIFO has reached a user-defined empty threshold. See Programmable Empty, on page 219 for more information.

Both flags support various implementation options. You can do the following:

- Set a constant value.
- Set dedicated input ports so that the thresholds can change dynamically in the circuit.
- Use hysteresis, so that each flag has different assert and negative values.

© 2022 Synopsys, Inc.

Programmable Full

The Prog_full flag (programmable full) is asserted when the number of entries in the FIFO is greater than or equal to a user-defined assert threshold. If the number of words in the FIFO is less than the negate threshold, the flag is de-asserted. The following is the valid range of threshold values:

Assert threshold value	Depth/2 to Max of Depth For multiple threshold types, the assert value should always be larger than the negate value in multiple threshold types.
Negate threshold value	Depth/2 to Max of Depth

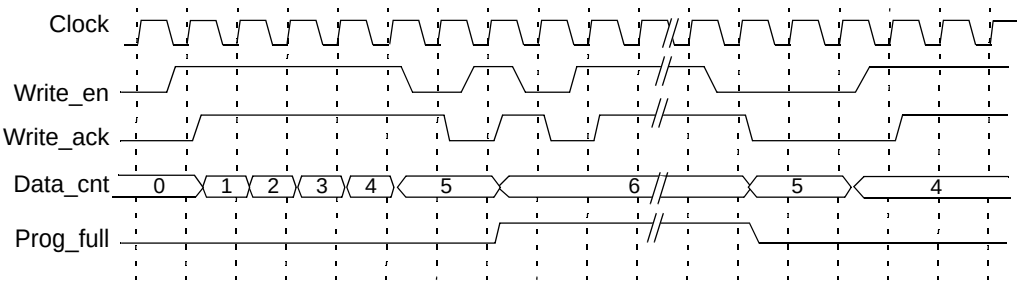
Prog_full has four threshold types:

- Programmable Full with Single Threshold Constant, on page 217
- Programmable Full with Multiple Threshold Constants, on page 218
- Programmable Full with Single Threshold Input, on page 218
- Programmable Full with Multiple Threshold Inputs, on page 219

Programmable Full with Single Threshold Constant

PGM_FULL_TYPE = 1

This option lets you set a single constant value for the threshold. It requires significantly fewer resources when the FIFO is generated. This figure illustrates the behavior of Prog_full when configured as a single threshold constant with a value of 6.

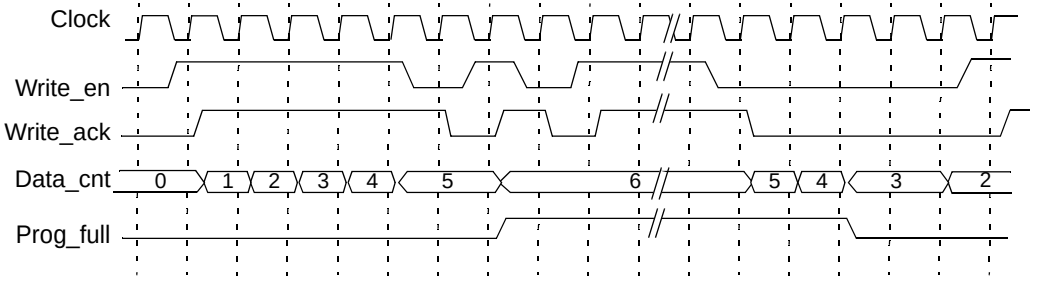


© 2022 Synopsys, Inc.

Programmable Full with Multiple Threshold Constants

PGM_FULL_TYPE = 2

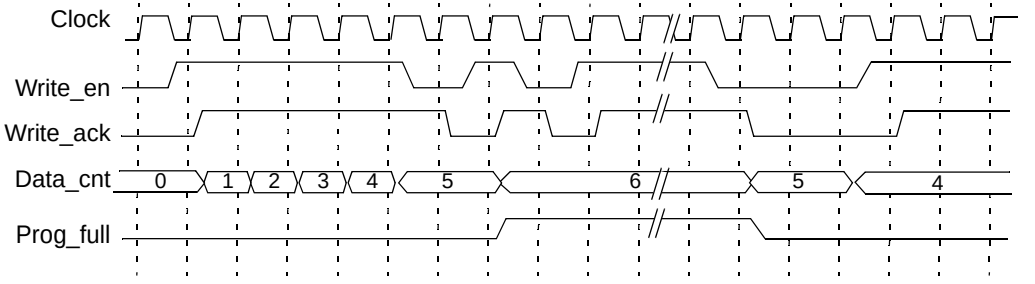
The programmable full flag is asserted when the number of words in the FIFO is greater than or equal to the full threshold assert value. If the number of FIFO words drops to less than the full threshold negate value, the programmable full flag is de-asserted. Note that the negate value must be set to a value less than the assert value. The following figure illustrates the behavior of Prog_full configured as multiple threshold constants with an assert value of 6 and a negate value of 4.



Programmable Full with Single Threshold Input

PGM_FULL_TYPE = 3

This option lets you specify the threshold value through an input port (Prog_full_thresh) during the reset state, instead of using constants. The following figure illustrates the behavior of Prog_full configured as a single threshold input with a value of 6.

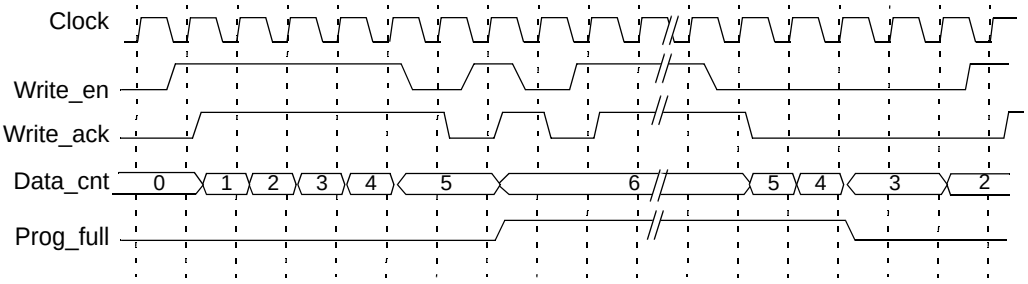


Programmable Full with Multiple Threshold Inputs

PGM_FULL_TYPE = 4

This option lets you specify the assert and negate threshold values dynamically during the reset stage using the Prog_full_thresh_assert and Prog_full_thresh_negate input ports. You must set the negate value to a value less than the assert value.

The programmable full flag is asserted when the number of words in the FIFO is greater than or equal to the Prog_full_thresh_assert value. If the number of FIFO words goes below Prog_full_thresh_negate value, the programmable full flag is deasserted. The following figure illustrates the behavior of Prog_full configured as multiple threshold inputs with an assert value of 6 and a negate value of 4.



Programmable Empty

The programmable empty flag (Prog_empty) is asserted when the number of entries in the FIFO is less than or equal to a user-defined assert threshold. If the number of words in the FIFO is greater than the negate threshold, the flag is deasserted. The following is the valid range of threshold values:

Assert threshold value	1 to Max of Depth/2 For multiple threshold types, the assert value should always be lower than the negate value in multiple threshold types.
Negate threshold value	1 to Max of Depth/2

There are four threshold types you can specify:

- Programmable Empty with Single Threshold Constant, on page 220

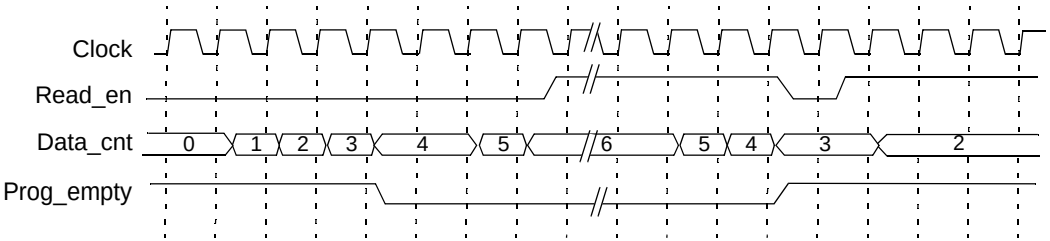
© 2022 Synopsys, Inc.

- Programmable Empty with Multiple Threshold Constants, on page 220
- Programmable Empty with Single Threshold Input, on page 221
- Programmable Empty with Multiple Threshold Inputs, on page 221

Programmable Empty with Single Threshold Constant

PGM_EMPTY_TYPE = 1

This option lets you specify an empty threshold value with a single constant. This approach requires significantly fewer resources when the FIFO is generated. The following figure illustrates the behavior of Prog_empty configured as a single threshold constant with a value of 3.



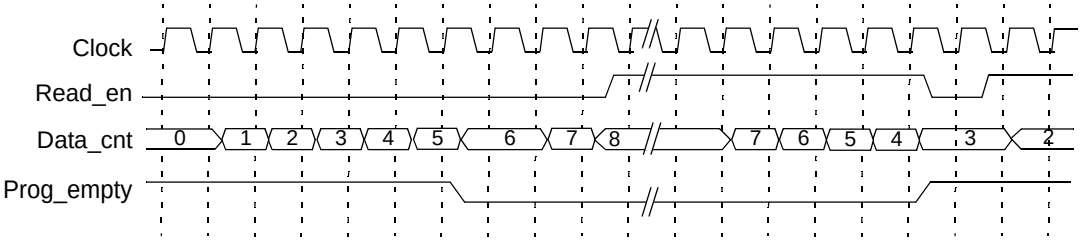
Programmable Empty with Multiple Threshold Constants

PGM_EMPTY_TYPE = 2

This option lets you specify constants for the empty threshold assert value and empty threshold negate value. The programmable empty flag asserts and deasserts in the range set by the assert and negate values. The assert value must be set to a value less than the negate value. When the number of words in the FIFO is less than or equal to the empty threshold assert value, the Prog_empty flag is asserted. When the number of words in FIFO is greater than the empty threshold negate value, Prog_empty is deasserted.

The following figure illustrates the behavior of Prog_empty when configured as multiple threshold constants with an assert value of 3 and a negate value of 5.

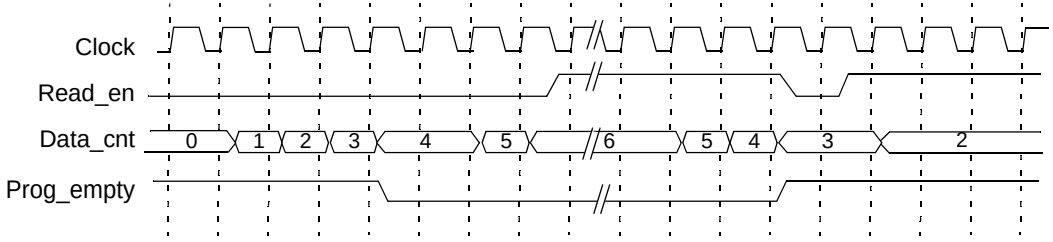
© 2022 Synopsys, Inc.



Programmable Empty with Single Threshold Input

PGM_EMPTY_TYPE = 3

This option lets you specify the threshold value dynamically during the reset state with the Prog_empty_thresh input port, instead of with a constant. The Prog_empty flag asserts when the number of FIFO words is equal to or less than the Prog_empty_thresh value and deasserts when the number of FIFO words is more than the Prog_empty_thresh value. The following figure illustrates the behavior of Prog_empty when configured as a single threshold input with a value of 3.



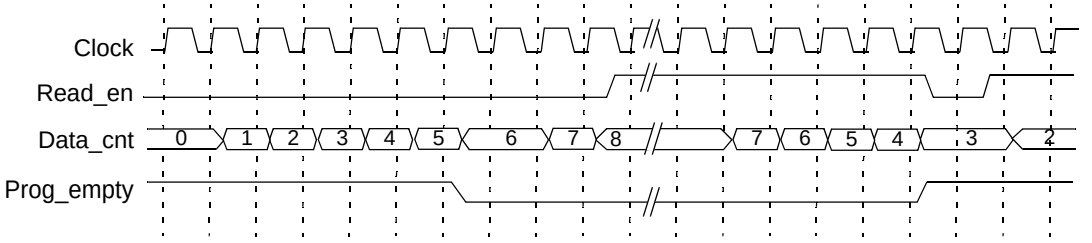
Programmable Empty with Multiple Threshold Inputs

PGM_EMPTY_TYPE = 4

This option lets you specify the assert and negate threshold values dynamically during the reset stage using the Prog_empty_thresh_assert and Prog_empty_thresh_negate input ports instead of constants. The programmable empty flag asserts and deasserts according to the range set by the assert and negate values. The assert value must be set to a value less than the negate value.

© 2022 Synopsys, Inc.

When the number of FIFO words is less than or equal to the empty threshold assert value, Prog_empty is asserted. If the number of FIFO words is greater than the empty threshold negate value, the flag is deasserted. The following figure illustrates the behavior of Prog_empty configured as multiple threshold inputs, with an assert value of 3 and a negate value of 5.



SYNCore RAM Compiler

The SYNCore RAM Compiler generates Verilog code for your RAM implementation. This section describes the following:

- Specifying RAMs with SYNCore, on page 223
- SYNCore RAM Wizard, on page 231
- Single-Port Memories, on page 235
- Dual-Port Memories, on page 237
- Read/Write Timing Sequences, on page 242

Specifying RAMs with SYNCore

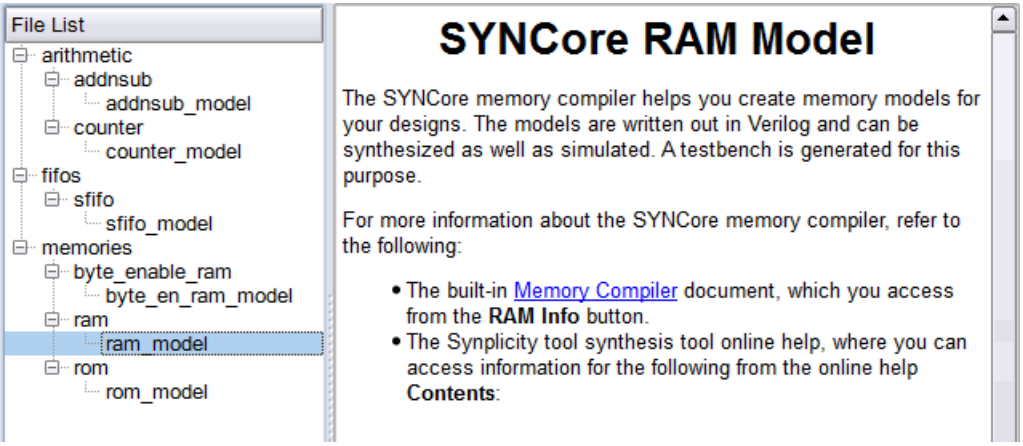
The SYNCore IP wizard helps you generate Verilog code for your RAM implementation requirements. The following procedure shows you how to generate Verilog code for a RAM using the SYNCore IP wizard.

Note: The SYNCore RAM model uses Verilog 2001. When adding a RAM model to a Verilog-95 design, be sure to enable the Verilog 2001 check box on the Verilog tab of the Implementation Options dialog box or include a set_option -vlog_std v2001 statement in your project file to prevent a syntax error.

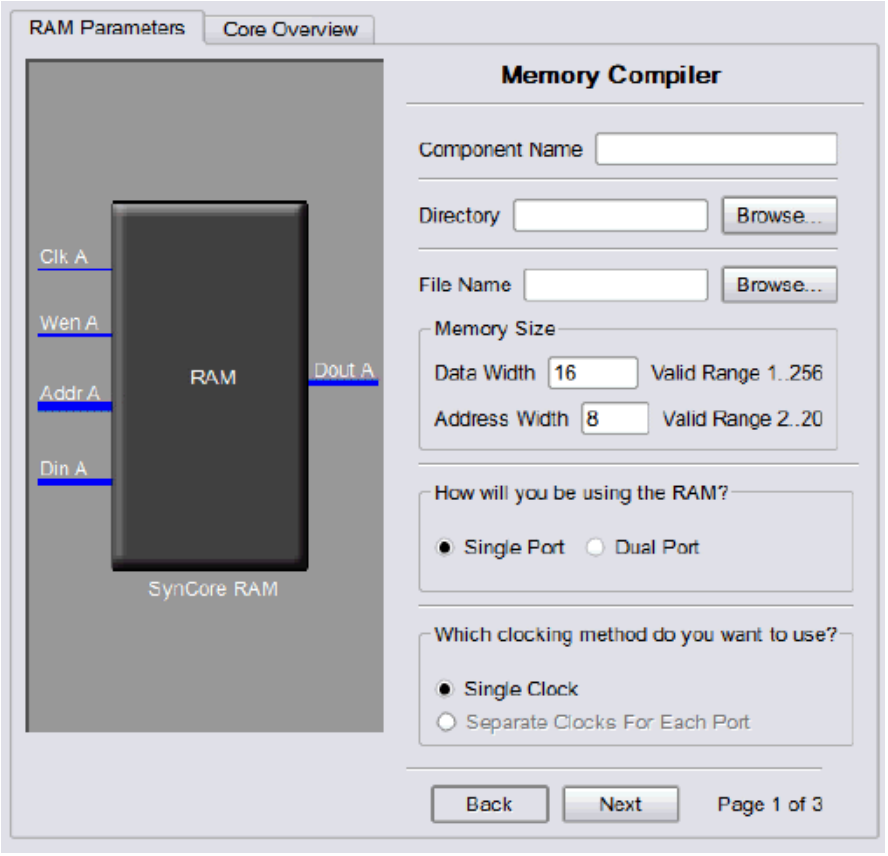
---

1. Start the wizard.
   - From the synthesis tool GUI, select Run->Launch SYNCore or click the Launch SYNCore icon  to start the SYNCore IP wizard.

© 2022 Synopsys, Inc.



– In the window that opens, select ram_model and click Ok. This opens the first screen of the wizard.



2. Specify the parameters you need in the wizard.
   - For details about the parameters for a single-port RAM, see Specifying Parameters for Single-Port RAM, on page 228.
   - For details about the parameters for a dual-port RAM, see Specifying Parameters for Dual-Port RAM, on page 229. Note that dual-port implementations are only supported for some technologies.

   The RAM symbol on the left reflects the parameters you set.

   The default settings for the tool implement a block RAM with synchronous resets, and where all edges (clock, enable, and reset) are considered positive.

© 2022 Synopsys, Inc.

3. After you have specified all the parameters you need, click the Generate button in the lower left corner.

   The tool displays a confirmation message is displayed (TCL execution successful!) and writes the required files to the directory you specified in the parameters. The HDL code is in Verilog.

   SYNCore also generates a testbench for the RAM. The testbench covers a limited set of vectors.

   You can now close the SYNCore Memory Compiler.

4. Edit the RAM files if necessary.

   – The default RAM has a no_rw_check attribute enabled. If you do not want this, edit syncore_ram.v and comment out the `define SYN_MULTI_PORT_RAM statement, or use `undef SYN_MULTI_PORT_RAM.

   – If you want to use the synchronous RAMs available in the target technology, make sure to register either the read address or the outputs.

5. Add the RAM you generated to your design.

   – Use the Add File command to add the Verilog design file that was generated and the syncore_ram.v file to your project. These files are in the directory for output files that you specified on page 1 of the wizard.

   – Use a text editor to open the instantiation_file.vin template file, which is located in the same directory. Copy the lines that define the memory, and paste them into your top-level module. The following figure shows a template file (in red text) inserted into a top-level module.

© 2022 Synopsys, Inc.

```
module top (

input ClkA,
input [7:0] AddrA,
input [15:0] DataInA,
input WrEnA,

output [15:0] DataOutA

);

myram2 <InstanceName> (
.PortAClk(PortAClk)
, .PortAAddr(PortAAddr)
, .PortADataIn(PortADataIn)
, .PortAWriteEnable(PortAWriteEnable)
, .PortADataOut(PortADataOut)
);

endmodule
```

template

6. Edit the template port connections so that they agree with the port definitions in your top-level module as shown in the example below (the updated connection names are shown in red). You can also assign a unique name to each instantiation.

© 2022 Synopsys, Inc.

```
module top (

input ClkA,
input [7:0] AddrA,
input [15:0] DataInA,
input WrEnA,

output [15:0] DataOutA

);


myram2 decoderram(
.PortAClk(ClkA)
, .PortAAddr(AddrA)
, .PortADataIn(DataInA)
, .PortAWriteEnable(WrEnA)
, .PortADataOut(DataOutA)
);

endmodule
```

Specifying Parameters for Single-Port RAM

To create a single-port RAM with the SYNCore Memory Compiler, you need to specify a single read/write address (single port) and a single clock. You only need to configure Port A. The following procedure lists what you need to specify. For descriptions of each parameter, refer to *SYNCore RAM Wizard, on page 231*.

1. Start the SYNCore RAM wizard, as described in Specifying RAMs with SYNCore, on page 223.
2. Do the following on page 1 of the RAM wizard:
   - In Component Name, specify a name for the memory. Do not use spaces.
   - In Directory, specify a directory where you want the output files to be written. Do not use spaces.
   - In Filename, specify a name for the Verilog file that will be generated with the RAM specifications. Do not use spaces.

© 2022 Synopsys, Inc.

- Enter data and address widths.
- Enable Single Port, to specify that you want to generate a single-port RAM. This automatically enables Single Clock.
- Click Next. The wizard opens another page where you can set parameters for Port A.

The RAM symbol dynamically updates to reflect the parameters you set.

3. Do the following on page 2 of the RAM wizard:
   - Set Use Write Enable to the setting you want.
   - Set Register Read Address to the setting you want.
   - Set Synchronous Reset to the setting you want. Register Outputs is always enabled
   - Specify the read access you require for the RAM.

   You can now generate the RAM by clicking Generate, as described in Specifying RAMs with SYNCore, on page 223. You do not need to specify any parameters on page 3, as this is a single-port RAM and you do not need to specify Port B. All output files are in the directory you specified on the first page of the wizard.

   For details about setting dual-port RAM parameters, see Specifying Parameters for Dual-Port RAM, on page 229. For read/write timing diagrams, see Read/Write Timing Sequences, on page 242.

Specifying Parameters for Dual-Port RAM

The following procedure shows you how to set parameters for dual-port memory in the SYNCore wizard. Dual-port RAMs are only supported for some technologies. For information about generating single-port RAMs, see Specifying Parameters for Single-Port RAM, on page 228. It shows you how to generate these common RAM configurations:

- One read access and one write access
- Two read accesses and one write access
- Two read accesses and two write accesses

For the corresponding read/write timing diagrams, see Read/Write Timing Sequences, on page 242.

© 2022 Synopsys, Inc.

1. Start the SYNCore RAM wizard, as described in Specifying RAMs with SYNCore, on page 223.
2. Do the following on page 1 of the RAM wizard:
   - In Component Name, specify a name for the memory. Do not use spaces.
   - In Directory, specify a directory where you want the output files to be written. Do not use spaces.
   - In Filename, specify a name for the Verilog file that will be generated with the RAM specifications. Do not use spaces.
   - Enter data and address widths.
   - Enable Dual Port, to specify that you want to generate a dual-port RAM.
   - Specify the clocks.

For a single clock ...	Enable Single Clock.
For separate clocks for each of the ports ...	Enable Separate Clocks For Each Port.

   - Click Next. The wizard opens another page where you can set parameters for Port A.
3. Do the following on page 2 of the RAM wizard to specify settings for Port A:
   - Set parameters according to the kind of memory you want to generate:

One read & one write	Enable Read Only Access.
Two reads & one write	Enable Read and Write Access. Specify a setting for Use Write Enable.
Two reads & two writes	Enable Read and Write Access. Specify a setting for Use Write Enable. Specify a read access option for Port A.

   - Specify a setting for Register Read Address.
   - Set Synchronous Reset to the setting you want. Register Outputs is always enabled.

– Click Next. The wizard opens another page where you can set parameters for Port B. The page and the parameters are identical to the previous page, except that the settings are for Port B instead of Port A.

4. Specify the settings for Port B on page 3 of the wizard according to the kind of memory you want to generate:

One read & one write	Enable Write Only Access. Set Use Write Enable to the setting you want.
Two reads & one write	Enable Read Only Access. Specify a setting for Register Read Address.
Two reads & two writes	Enable Read and Write Access. Specify a setting for Use Write Enable. Specify a setting for Register Read Address. Set Synchronous Reset to the setting you want. Note that Register Outputs is always enabled. Select a read access option for Port B.

The RAM symbol on the left reflects the parameters you set. All output files are written to the directory you specified on the first page of the wizard.

You can now generate the RAM by clicking Generate, as described in Specifying RAMs with SYNCore, on page 223, and add it to your design.

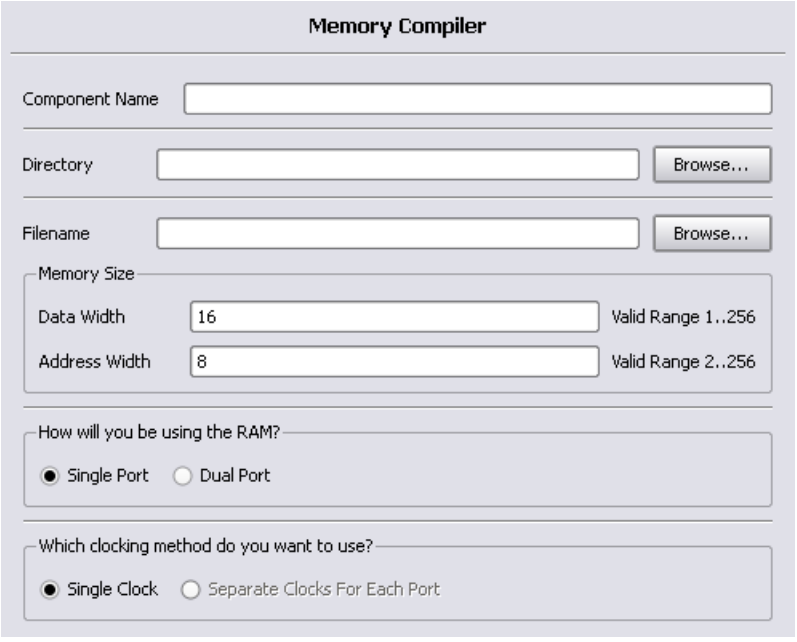
SYNCore RAM Wizard

The following describe the parameters you can set in the RAM wizard, which opens when you select ram_model:

- SYNCore RAM Parameters Page 1, on page 232
- SYNCore RAM Parameters Pages 2 and 3, on page 234

© 2022 Synopsys, Inc.

SYNCore RAM Parameters Page 1



Component Name	Specifies the name of the component. This is the name that you instantiate in your design file to create an instance of the SYNCore RAM in your design. Do not use spaces. For example:
	`ram101 <ComponentName> (`
	`.PortAClk(PortAClk)`
	`, .PortAAddr(PortAAddr)`
	`, .PortADataIn(PortADataIn)`
	`, .PortAWriteEnable(PortAWriteEnable)`
	`, .PortBDataIn(PortBDataIn)`
	`, .PortBAddr(PortBAddr)`
	`, .PortBWriteEnable(PortBWriteEnable)`
	`, .PortADataOut(PortADataOut)`
	`, .PortBDataOut(PortBDataOut)`
	`);`

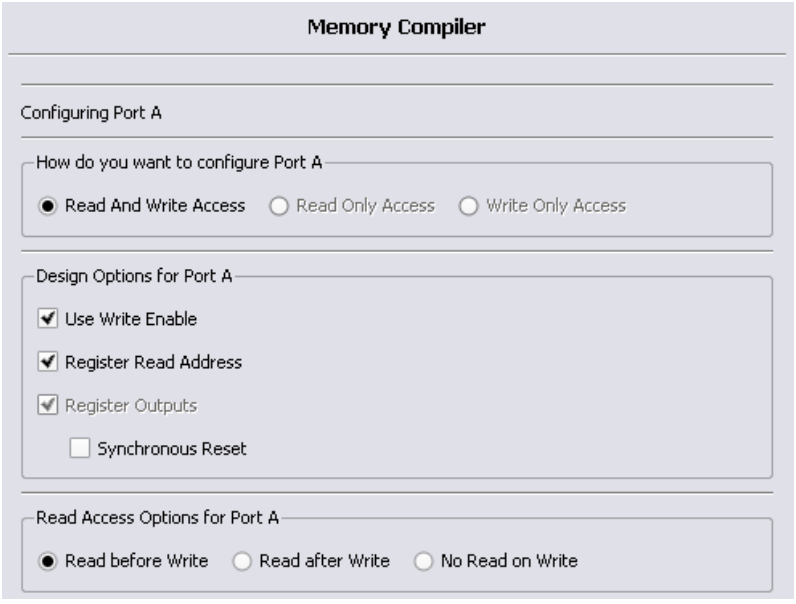
Directory	Specifies the directory where the generated files are stored. Do not use spaces. The following files are created: • filelist.txt – lists files written out by SYNCore • options.txt – lists the options selected in SYNCore • readme.txt – contains a brief description and known issues • syncore_ram.v – Verilog library file required to generate RAM model • testbench.v – Verilog testbench file for testing the RAM model • instantiation_file.vin – describes how to instantiate the wrapper file • *component*.v – RAM model wrapper file generated by SYNCore Note that running the Memory Compiler wizard in the same directory overwrites the existing files.
Filename	Specifies the name of the generated file containing the HDL description of the compiled RAM. Do not use spaces.
Data Width	Is the width of the data you need for the memory. The unit used is the number of bits.
Address Width	Is the address depth you need for the memory. The unit used is the number of bits.
Single Port	When enabled, generates a single-port RAM.

© 2022 Synopsys, Inc.

Dual Port	When enabled, generates a dual-port RAM.
Single Clock	When enabled, generates a RAM with a single clock for dual-port configurations.
Separate Clocks for Each Port	When enabled, generates separate clocks for each port in dual-port RAM configurations.

SYNCore RAM Parameters Pages 2 and 3

The port implementation parameters on pages 2 and 3 are identical, but page 2 applies to Port A (single- and dual-port configurations), and page 3 applies to Port B (dual-port configurations only). The following figure shows the parameters on page 2 for Port A.



Read and Write Access	Specifies that the port can be accessed by both read and write operations.
Read Only Access	Specifies that the port can only be accessed by read operations.
Write Only Access	Specifies that the port can only be accessed by write operations.
Use Write Enable	Includes write-enable control. The RAM symbol on the left reflects the selections you make.

© 2022 Synopsys, Inc.

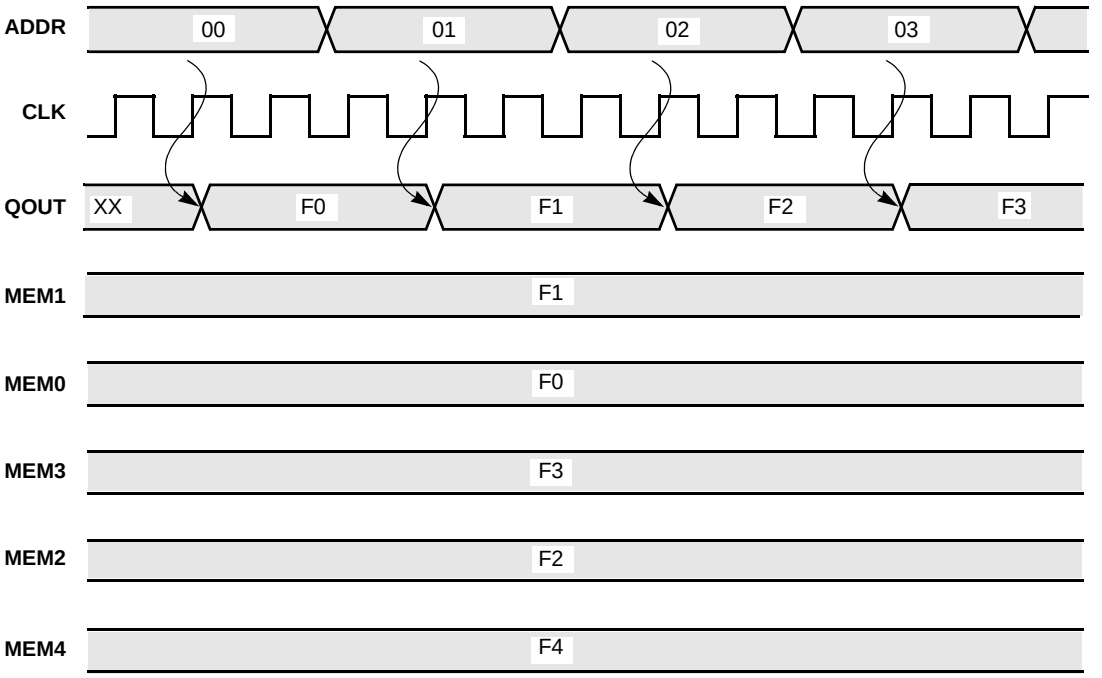
Register Read Address	Adds registers to the read address lines. The RAM symbol on the left reflects the selections you make.
Register Outputs	Adds registers to the write address lines when you specify separate read/write addressing. The register outputs are always enabled. The RAM symbol on the left reflects the selections you make.
Synchronous Reset	Individually synchronizes the reset signal with the clock when you enable Register Outputs. The RAM symbol on the left reflects the selections you make.
Read before Write	Specifies that the read operation takes place before the write operation for port configurations with both read and write access (Read And Write Access is enabled). For a timing diagram, see Read Before Write, on page 242.
Read after Write	Specifies that the read operation takes place after the write operation for port configurations with both read and write access (Read And Write Access is enabled). For a timing diagram, see Write Before Read, on page 243.
No Read on Write	Specifies that no read operation takes place when there is a write operation for port configurations with both read and write access (Read And Write Access is enabled). For a timing diagram, see No Read on Write, on page 244.

Single-Port Memories

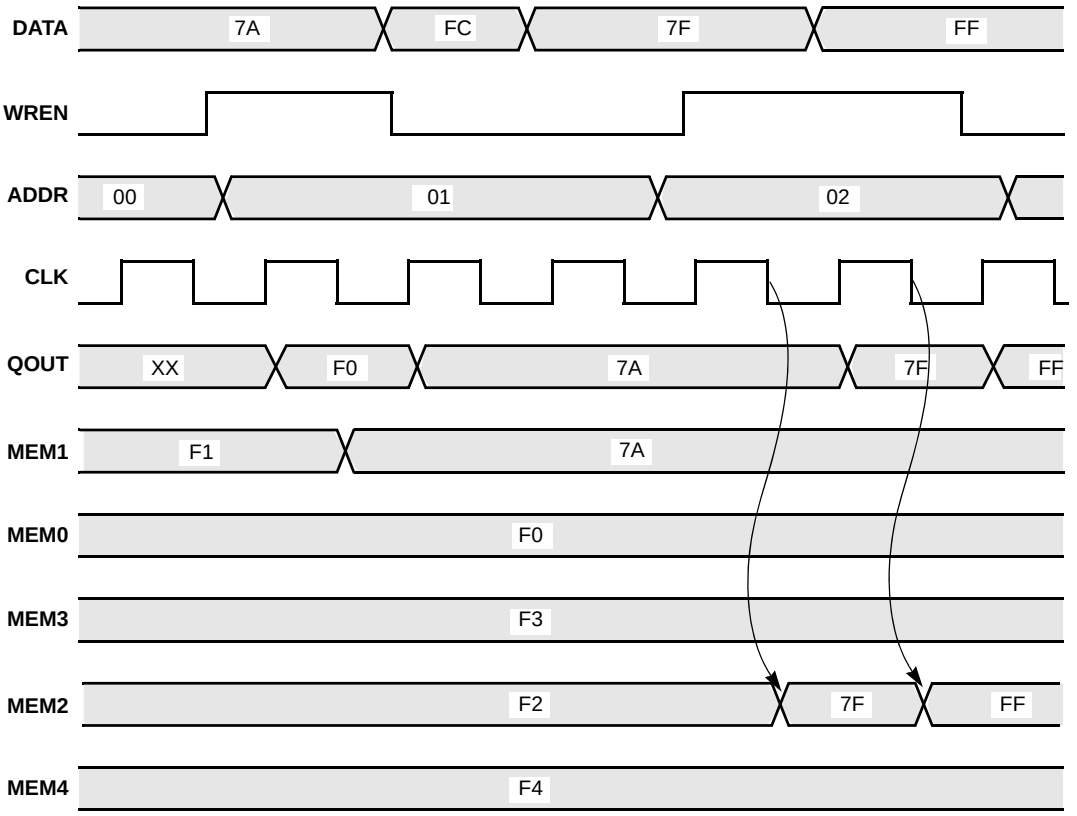
For single-port RAM, it is only necessary to configure Port A. The following diagrams show the read-write timing for single-port memories. See Specifying RAMs with SYNCore, on page 223 for a procedure.

© 2022 Synopsys, Inc.

Single-Port Read



Single-Port Write



Dual-Port Memories

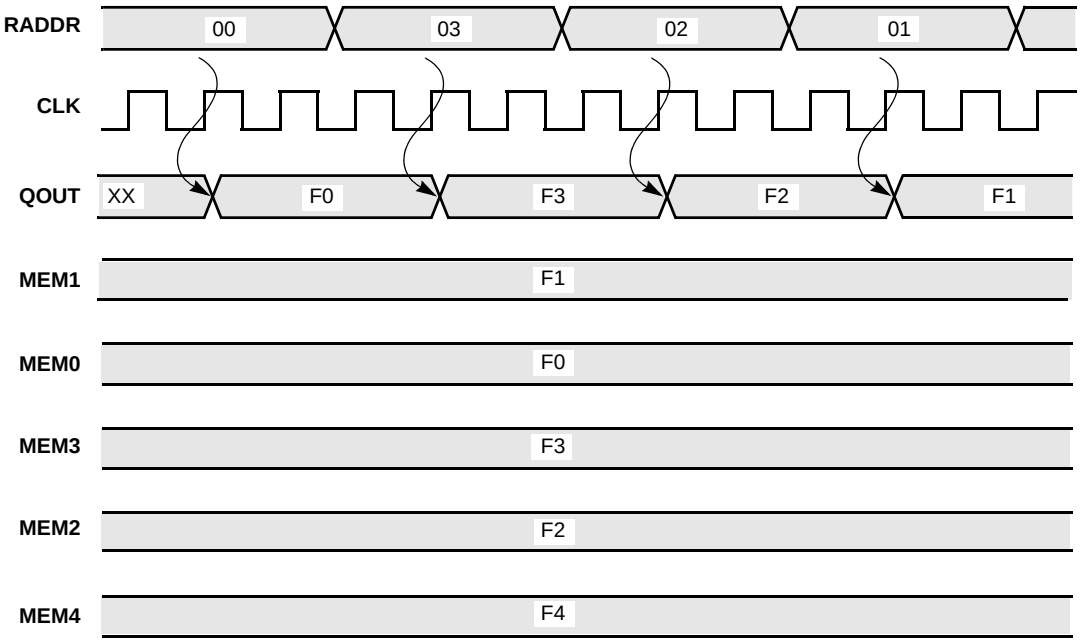
SYNCore dual-port memory includes the following common configurations:

- One read access and one write access
- Two read accesses and one write access
- Two read accesses and two write accesses

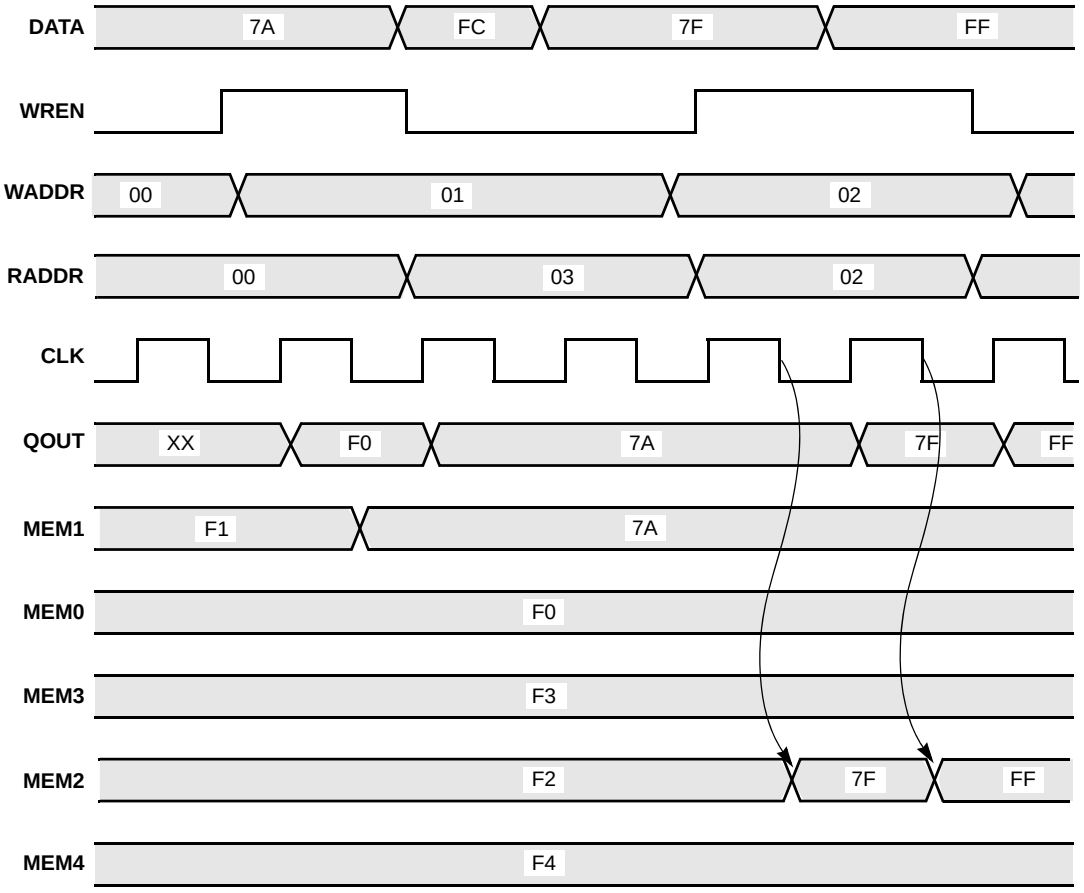
The following diagrams show the read-write timing for dual-port memories. See Specifying RAMs with SYNCore, on page 223 for a procedure to specify a dual-port RAM with SYNCore.

© 2022 Synopsys, Inc.

Dual-Port Single Read

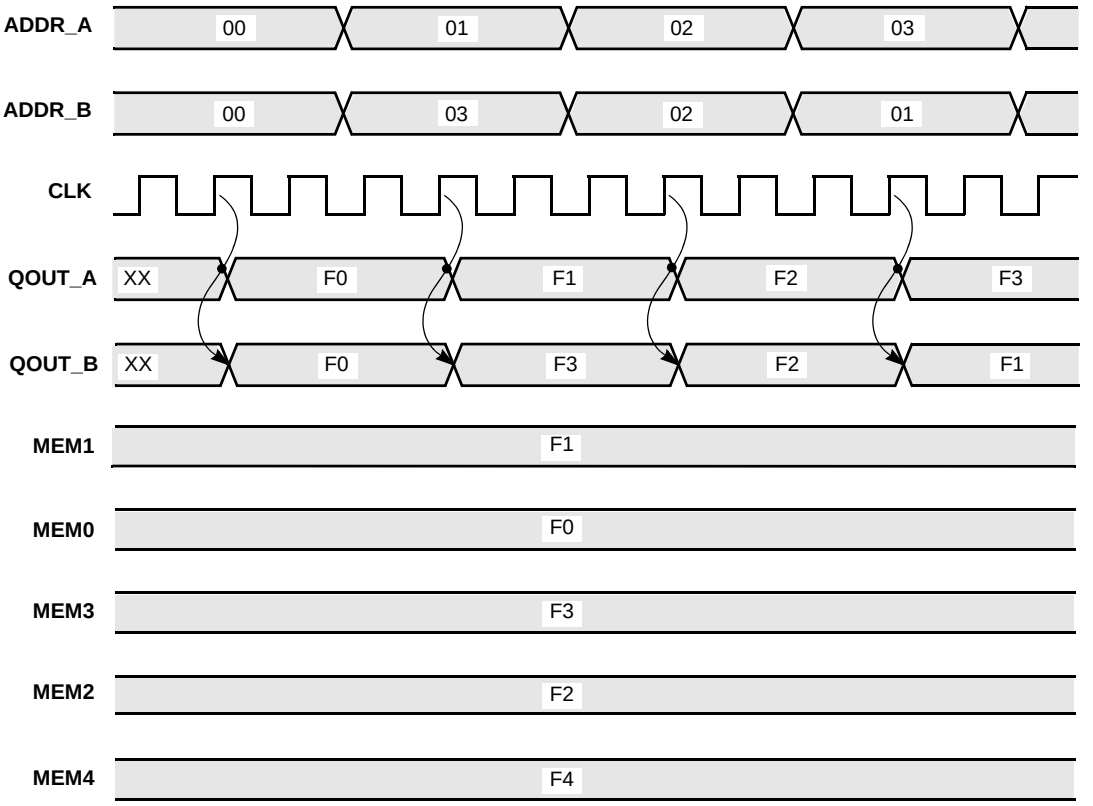


Dual-Port Single Write

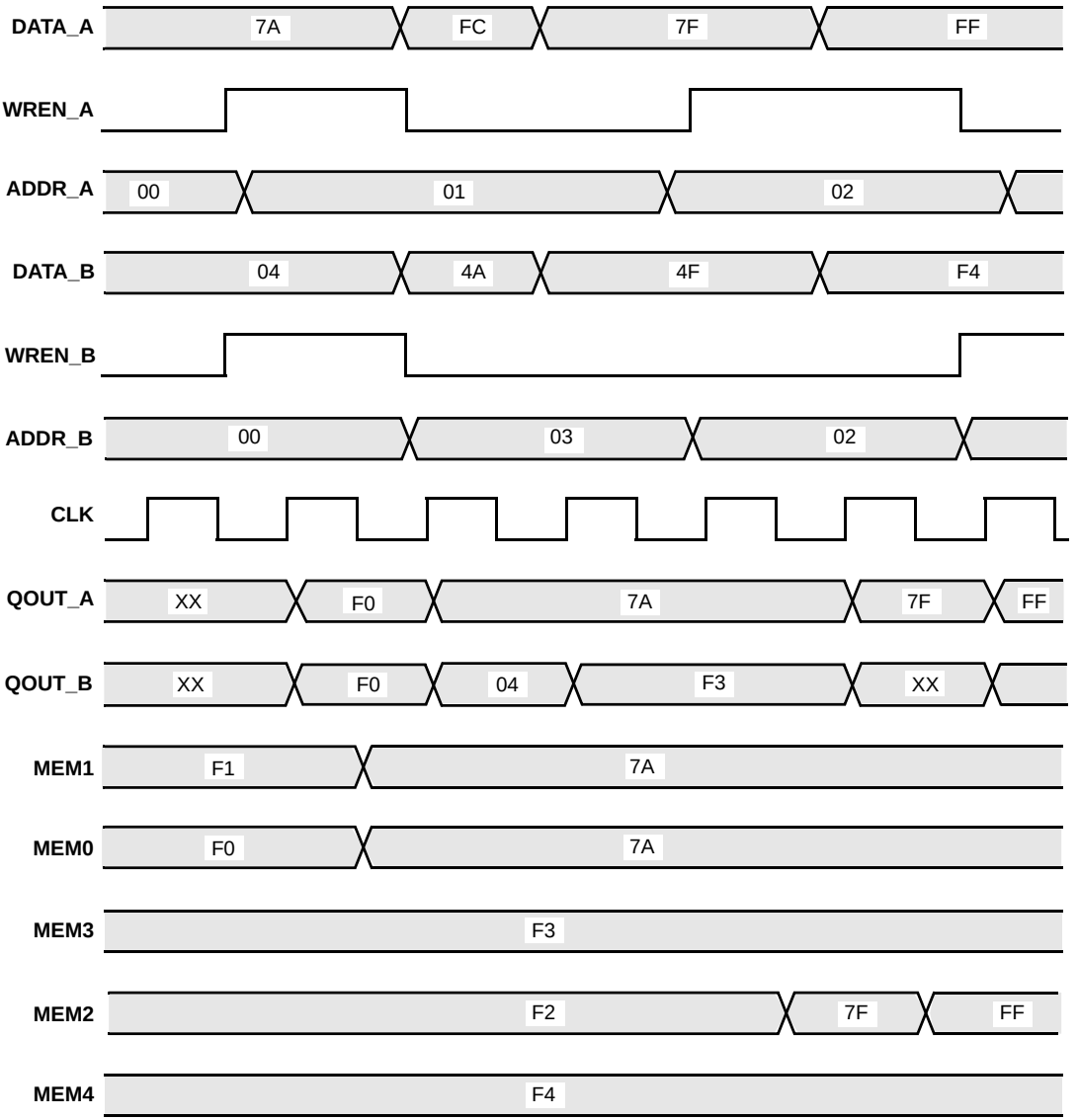


© 2022 Synopsys, Inc.

Dual-Port Read



Dual-Port Write



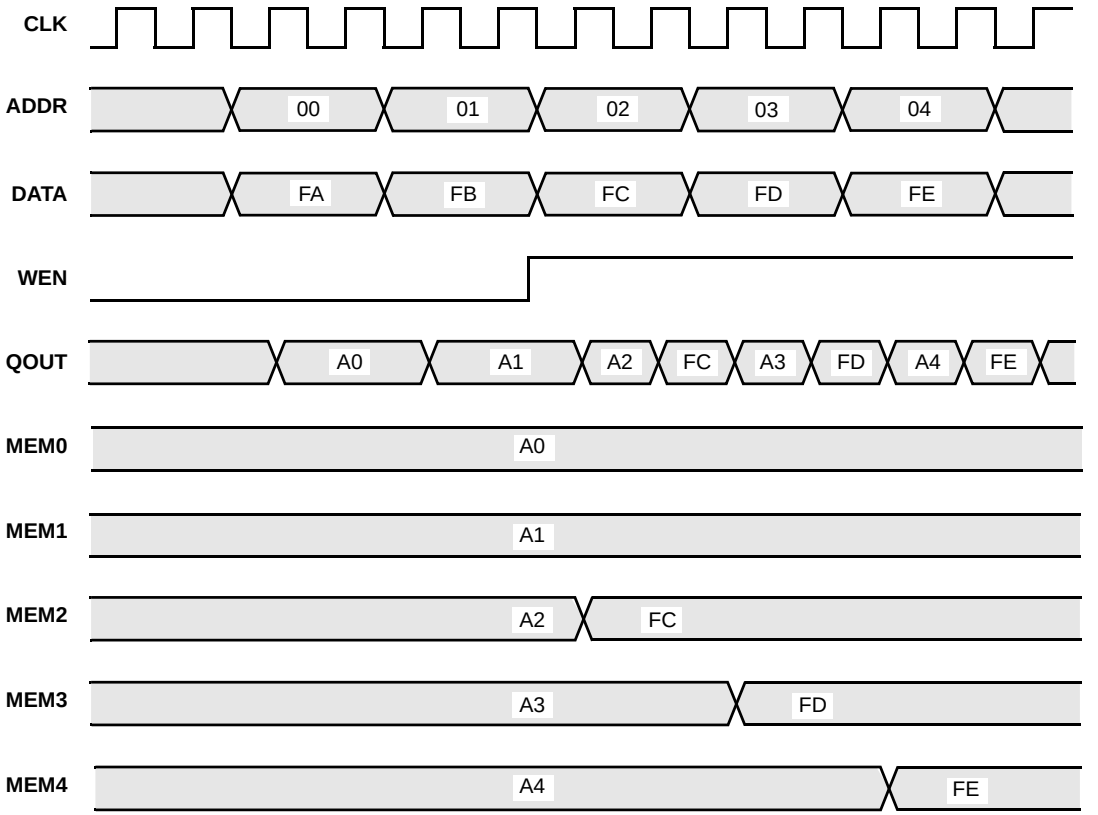
© 2022 Synopsys, Inc.

Read/Write Timing Sequences

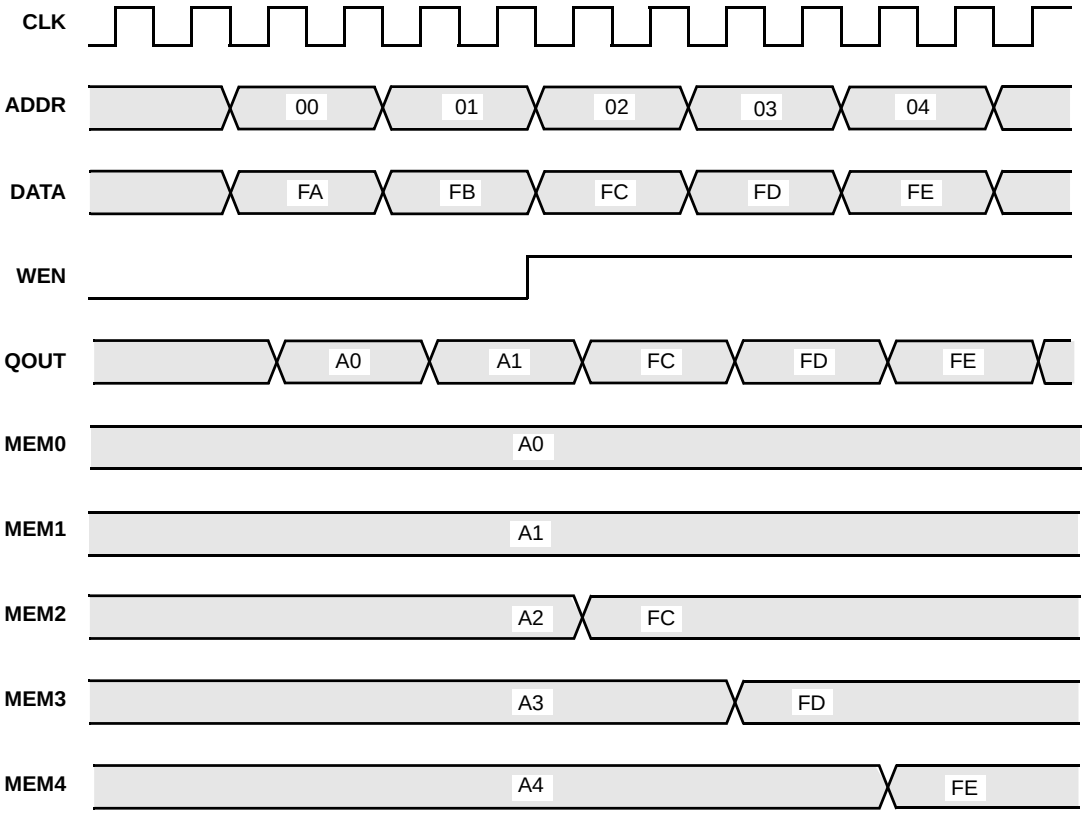
The waveforms in this section describe the behavior of the RAM when both read and write are enabled and the address is the same operation. The waveforms show the behavior when each of the read-write sequences is enabled. The waveforms are merged with the simple waveforms shown in the previous sections. See the following:

- Read Before Write, on page 242
- Write Before Read, on page 243
- No Read on Write, on page 244

Read Before Write

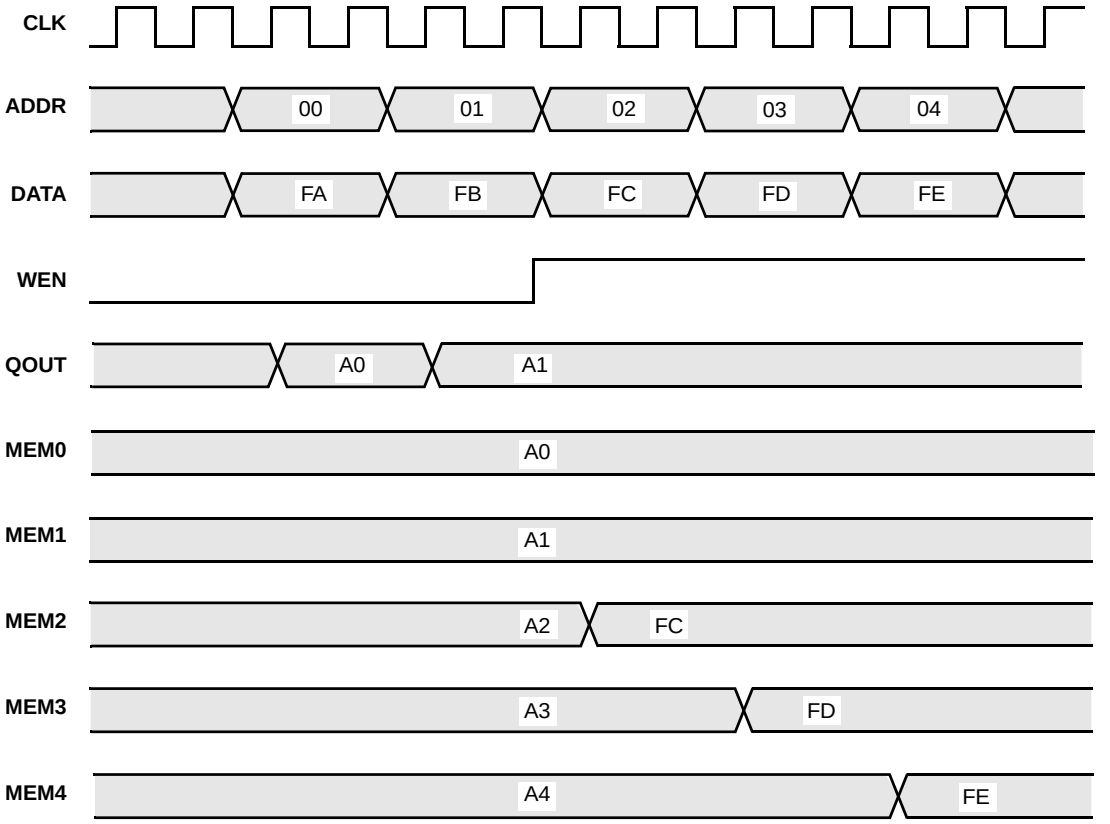


Write Before Read



© 2022 Synopsys, Inc.

No Read on Write



© 2022 Synopsys, Inc.

SYNCore Byte-Enable RAM Compiler

The SYNCore byte-enable RAM compiler generates SystemVerilog code describing byte-enabled RAMs. The data width of each byte is calculated by dividing the total data width by the write enable width. The byte-enable RAM compiler supports both single- and dual-port configurations.

This section describes the following:

- Functional Overview, on page 297
- Specifying Byte-Enable RAMs with SYNCore, on page 246
- SYNCore Byte-Enable RAM Wizard, on page 252
- Read/Write Timing Sequences, on page 255
- Parameter List, on page 258

Functional Overview

The SYNCore byte-enable RAM component supports bit/byte-enable RAM implementations using block RAM and distributed memory. For each configuration, design optimizations are made for optimum use of core resources. The timing diagram that follows, illustrates the supported signals for byte-enable RAM configurations.

Byte-enable RAM can be configured in both single- and dual-port configurations. In the dual-port configuration, each port is controlled by different clock, enable, and control signals. User configuration controls include selecting the enable level, reset type, and register type for the read data outputs and address inputs.

Reset applies only to the output read data registers; default value of read data on reset can be changed by you while generating the core file. Reset option is inactive when output read data is not registered.

© 2022 Synopsys, Inc.

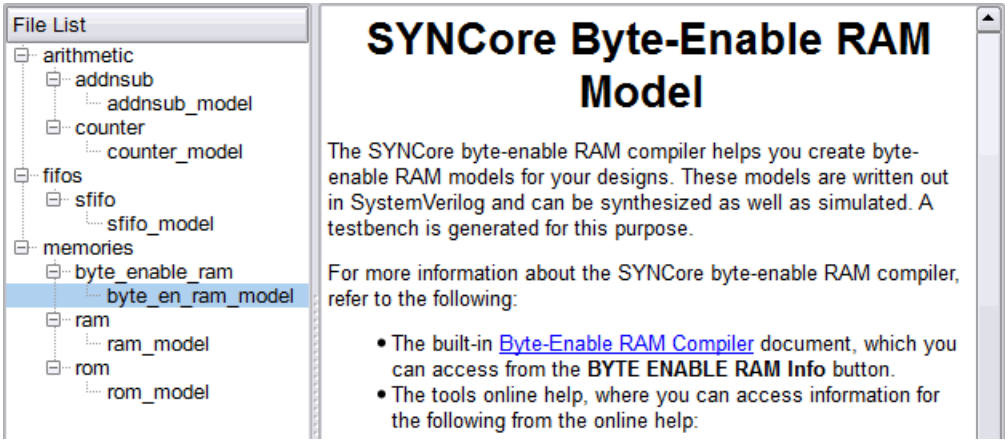
Specifying Byte-Enable RAMs with SYNCore

The SYNCore IP wizard helps you generate SystemVerilog code for your byte-enable RAM implementation requirements. The following procedure shows you how to generate SystemVerilog code for a byte-enable RAM using the SYNCore IP wizard.

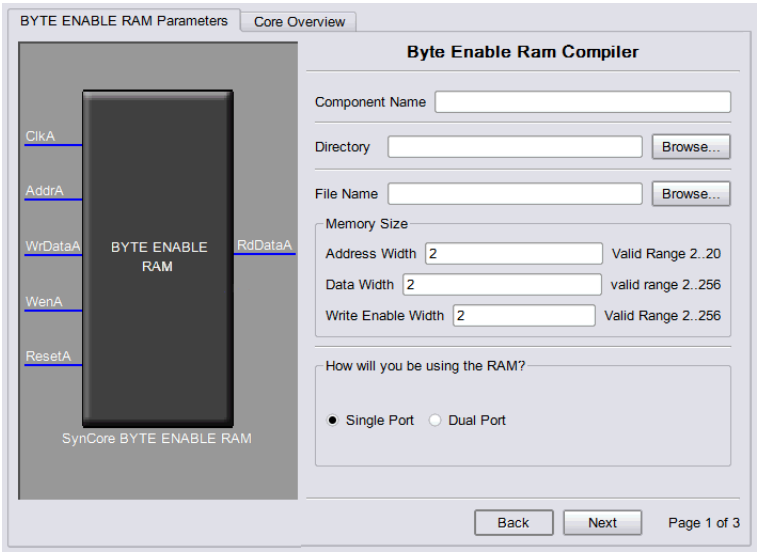
Note: The SYNCore byte-enable RAM model uses SystemVerilog. When adding a byte-enable RAM to your design, be sure to enable the System Verilog check box on the Verilog tab of the Implementation Options dialog box or include a set_option -vlog_std sysv statement in your project file to prevent a syntax error.

---

1. Start the wizard.
   - From the FPGA synthesis tool GUI, select Run->Launch SYNCore or click the Launch SYNCore icon  to start the SYNCore IP wizard.



   - In the window that opens, select byte_en_ram_model and click Ok to open the first page (page1) of the wizard.



2. Specify the parameters you need in the wizard. For details about the parameters, see Specifying Byte-Enable RAM Parameters, on page 250. The BYTE ENABLE RAM symbol on the left reflects any parameters you set.

3. After you have specified all the parameters you need, click the Generate button in the lower left corner. The tool displays a confirmation message (TCL execution successful!) and writes the required files to the directory you specified on page 1 of the wizard. The HDL code is in SystemVerilog.

   SYNCore also generates a test bench for the byte-enable RAM component. The test bench covers a limited set of vectors. You can now close the SYNCore byte-enable RAM compiler.

4. Edit the generated files for the byte-enable RAM component if necessary.

5. Add the byte-enable RAM that you generated to your design.
   - On the Verilog tab of the Implementation Options dialog box, make sure that SystemVerilog is enabled.
   - Use the Add File command to add the Verilog design file that was generated (the filename entered on page 1 of the wizard) and the

© 2022 Synopsys, Inc.

syncore_*.v file to your project. These files are in the directory for output files that you specified on page 1 of the wizard.

- Use a text editor to open the instantiation_file.vin template file. This file is located in the same output files directory. Copy the lines that define the byte-enable RAM and paste them into your top-level module.
- Edit the template port connections so that they agree with the port definitions in the top-level module; also change the instantiation name to agree with the component name entered on page 1. The following figure shows a template file inserted into a top-level module with the updated component name and port connections in red.

```
module top
   (input ClockA,
    input [3:0] AddA
    input [31:0] DataIn
    input WrEnA,
    input Reset
    output [31:0] DataOut
   )

INST_TAG

SP_RAM #
   (.ADD_WIDTH(4),
    .WE_WIDTH(2),
    .RADDR_LTNCY_A(1), // 0 - No Latency, 1 - 1 Cycle Latency
    .RDATA_LTNCY_A(1), // 0 - No Latency, 1 - 1 Cycle Latency
    .RST_TYPE_A(1), // 0 - No Reset, 1 synchronous
    .RST_RDATA_A({32{1'b1}}),
    .DATA_WIDTH(32)
   )

4x32spram
   (// Output Ports
    .RdDataA(DataIn),
    // Input Ports
    .WrDataA(DataOut),
    .WenA(WeEnA),
    .AddrA(AddA),
    .ResetA(Reset),
    .ClkA(ClockA)
    ;
```

© 2022 Synopsys, Inc.

Port List

Port A interface signals are applicable for both single-port and dual-port configurations; Port B signals are applicable for dual-port configuration only.

Name	Type	Description
ClkA	Input	Clock input for Port A
WenA	Input	Write enable for Port A; present when Port A is in write mode
AddrA	Input	Memory access address for Port A
ResetA	Input	Reset for memory and all registers in core; present with registered read data when Reset is enabled; active low (cannot be changed)
WrDataA	Input	Write data to memory for Port A; present when Port A is in write mode
RdDataA	Output	Read data output for Port A; present when Port A is in read or read/write mode
ClkB	Input	Clock input for Port B; present in dual-port mode
WenB	Input	Write enable for Port B; present in dual-port mode when Port B is in write mode
AddrB	Input	Memory access address for Port B; present in dual-port mode
ResetB	Input	Reset for memory and all registers in core; present in dual-port mode when read data is registered and Reset is enabled; active low (cannot be changed)
WrDataB	Input	Write data to memory for Port B; present in dual-port mode when Port B is in write mode
RdDataB	Output	Read data output for Port B; present in dual-port mode when Port B is in read or read/write mode

© 2022 Synopsys, Inc.

Specifying Byte-Enable RAM Parameters

When creating a single-port, byte-enable RAM with the SYNCore IP wizard, you must specify a single read address and a single clock; you only need to configure the Port A parameters on page 2 of the wizard.

When creating a dual-port, byte-enable RAM, you must additionally configure the Port B parameters on page 3 of the wizard.

The following procedure lists the parameters you need to specify. For descriptions of each parameter, refer to Parameter List, on page 258.

1. Start the SYNCore byte-enable RAM wizard as described in Specifying Byte-Enable RAMs with SYNCore, on page 246.
2. Do the following on page 1 of the byte-enable RAM wizard:
   - Specify a name for the memory in the Component Name field; do not use spaces.
   - Specify a directory name in the Directory field where you want the output files to be written; do not use spaces.
   - Specify a name in the File Name field for the SystemVerilog file to be generated with the byte-enable RAM specifications; do not use spaces.
   - Enter a value for the address width of the byte-enable RAM; the maximum depth of memory is limited to 2^256.
   - Enter a value for the data width for the byte-enable RAM; data width values range from 2 to 256.
   - Enter a value for the write enable width; write-enable width values range from 1 to 4.
   - Select Single Port to generate a single-port, byte-enable RAM or select Dual Port to generate a dual-port, byte-enable RAM.
   - Click Next to open page 2 of the wizard.

   The Byte Enable RAM symbol dynamically updates to reflect the parameters that you set.
3. Do the following on page 2 (configuring Port A) of the wizard:
   - Select the Port A configuration. Only Read and Write Access mode is valid for single-port configurations; this mode is selected by default.

© 2022 Synopsys, Inc.

- Set Pipelining Address Bus and Output Data according to your application. By default, read data is registered; you can register both the address and data registers.
- Set the Configure Reset Options. Enabling the checkbox enables the synchronous reset for read data. This option is enabled only when the read data is registered. Reset is active low and cannot be changed.
- Configure output reset data value options under Specify output data on reset; reset data can be set to default value of all '1' s or to a user-defined decimal value. Reset data value options are disabled when the reset is not enabled for Port A.
- Set Write Enable for Port A value; default for the write-enable level is active high.

4. If you are generating a dual-port, byte-enable RAM, set the Port B parameters on page 3 (note that the Port B parameters are only enabled when Dual Port is selected on page 1).

   The Port B parameters are identical to the Port A parameters on page 2. When using the dual-port configuration, when one port is configured for read access, the other port can only be configured for read/write access or write access.

5. Generate the byte-enable RAM by clicking Generate. Add the file to your project and edit the template file as described in Specifying Byte-Enable RAMs with SYNCore, on page 246. For read/write timing diagrams, see Read/Write Timing Sequences, on page 255.

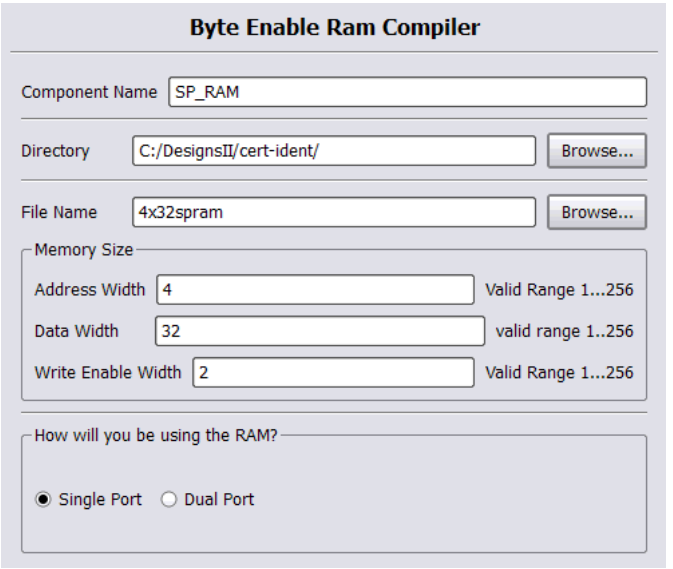
© 2022 Synopsys, Inc.

SYNCore Byte-Enable RAM Wizard

The following describes the parameters you can set in the byte-enable RAM wizard, which opens when you select byte_en_ram.

- SYNCore Byte-Enable RAM Parameters Page 1, on page 252
- SYNCore Byte-Enable RAM Parameters Pages 2 and 3, on page 253

SYNCore Byte-Enable RAM Parameters Page 1

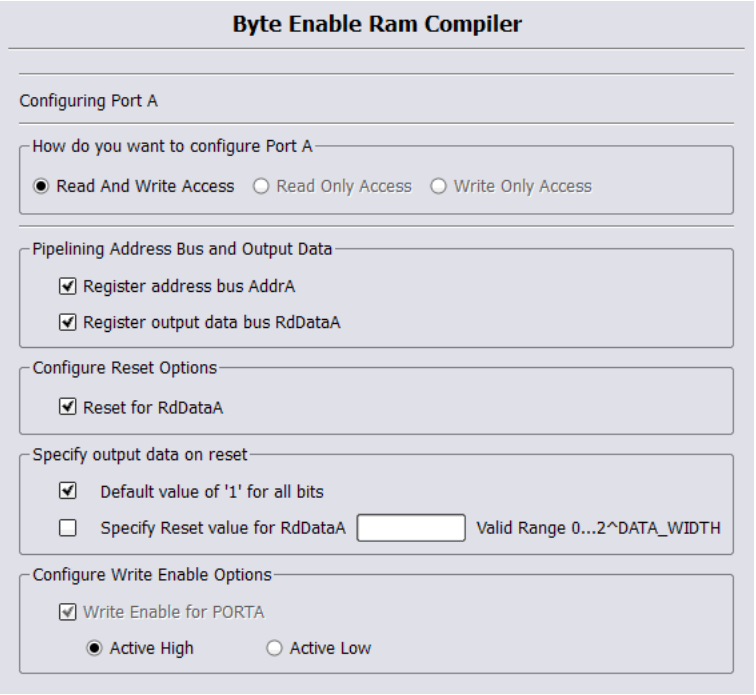


Component Name	Specifies the name of the component. This is the name that you instantiate in your design file to create an instance of the SYNCore byte-enable RAM in your design. Do not use spaces.
Directory	Specifies the directory where the generated files are stored. Do not use spaces. The following files are created: • filelist.txt – lists files written out by SYNCore • options.txt – lists the options selected in SYNCore • readme.txt – contains a brief description and known issues • syncore_be_ram_sdp.v – SystemVerilog library file required to generate single or simple dual-port, byte-enable RAM model • syncore_be_ram_tdp.v – SystemVerilog library file required to generate true dual-port byte-enable RAM model • testbench.v – Verilog testbench file for testing the byte-enable RAM model • instantiation_file.vin – describes how to instantiate the wrapper file • *component*.v – Byte-enable RAM model wrapper file generated by SYNCore Note that running the byte-enable RAM wizard in the same directory overwrites the existing files.
Filename	Specifies the name of the generated file containing the HDL description of the compiled byte-enable RAM. Do not use spaces.
Address Width	Specifies the address depth for Ports A and B. The unit used is the number of bits; the default is 2.
Data Width	Specifies the width of the data for Ports A and B. The unit used is the number of bits; the default is 2.
Write Enable Width	Specifies the write enable width for Ports A and B. The unit used is the number of byte enables; the default is 2, the maximum is 4.
Single Port	When enabled, generates a single-port, byte-enable RAM (automatically enables single clock).
Dual Port	When enabled, generates a dual-port, byte-enable RAM (automatically enables separate clocks for each port).

SYNCore Byte-Enable RAM Parameters Pages 2 and 3

The port implementation parameters on pages 2 and 3 are identical, but page 2 applies to Port A (single- and dual-port configurations), and page 3 applies to Port B (dual-port configurations only). The following figure shows the parameters on page 2 for Port A.

© 2022 Synopsys, Inc.



Read and Write Access	Specifies that the port can be accessed by both read and write operations (only mode allowed for single-port configurations).
Read Only Access	Specifies that the port can only be accessed by read operations (dual-port mode only).
Write Only Access	Specifies that the port can only be accessed by write operations (dual-port mode only).
Register address bus AddrA/B	Adds registers to the read address lines.
Register output data bus RdDataA/B	Adds registers to the read data lines. By default, the read data register is enabled.

© 2022 Synopsys, Inc.

Reset for RdDataA/B	Specifies the reset type for registered read data: • Reset type is synchronous when Reset for RdDataA/B is enabled • Reset type is no reset when Reset for RdDataA/B is disabled
Specify output data on reset	Specifies reset value for registered read data (applies only when RdDataA/B is enabled): • Default value of '1' for all bits – sets read data to all 1's on reset • Specify Reset value for RdDataA/B – specifies reset value for read data; when enabled, value is entered in adjacent field
Write Enable for Port A/B	Specifies the write enable level for Port A/B. Default is Active High.

Read/Write Timing Sequences

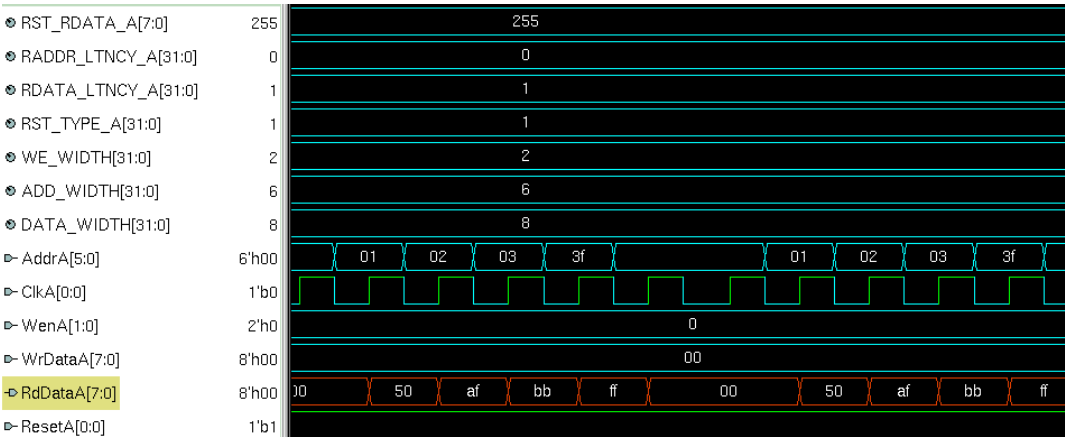
The waveforms in this section describe the behavior of the byte-enable RAM for both read and write operations.

Read Operation

On each active edge of the clock when there is a change in address, data is valid on the same clock or next clock (depending on latency parameter values for read address and read data ports). Active reset ignores any change in input address, and data and output data are initialized to user-defined values set by parameters RST_RDATA_A and RST_RDATA_B for port A and port B, respectively.

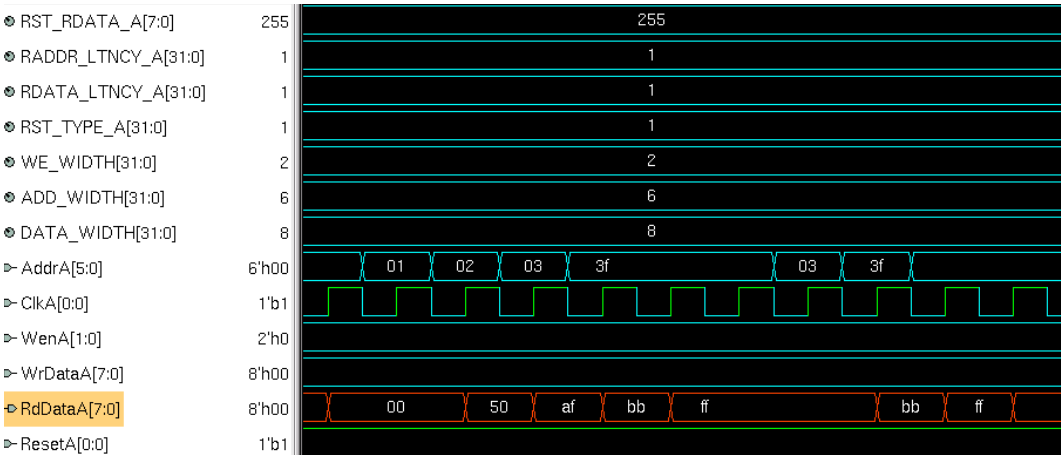
The following waveform shows the read sequence of the byte-enable RAM component with read data registered in single-port mode.

© 2022 Synopsys, Inc.



As shown in the above waveform, output read data changes on the same clock following the input address changed. When the address changes from 'h00 to 'h01, read data changes to 50 on the same clock, and data will be valid on the next clock edge.

The following waveform shows the read sequence with both the read data and address registered in single-port mode.



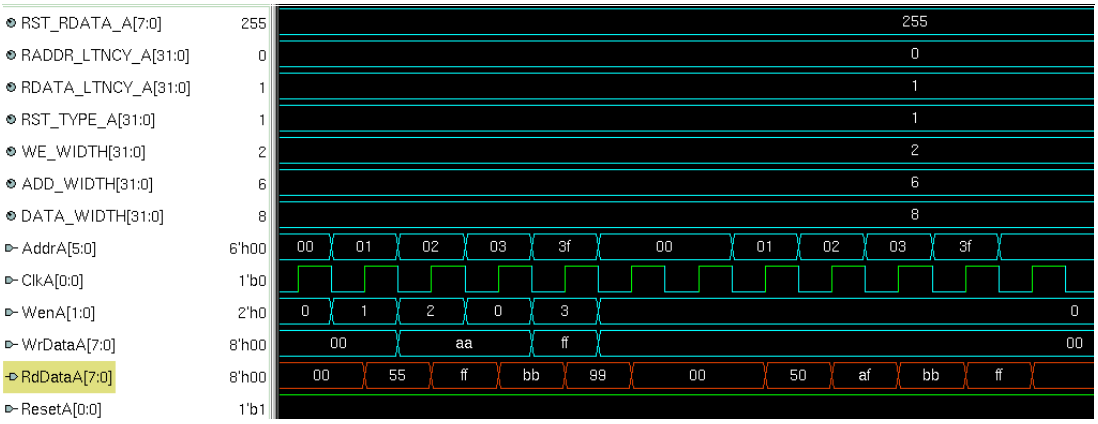
As shown in the above waveform, output read data changes on the next clock edge after the input address changes. When the address changes from 'h00 to 'h01, read data changes to 50 on the next clock, and data is valid on the next clock edge.

© 2022 Synopsys, Inc.

Note: The read sequence for dual-port mode is the same as single port, read/write conflicts occurring due to accessing the same location from both ports are the user's responsibility.

Write Operation

The following waveform shows a write sequence with read-after write in single-port mode.



On each active edge of the clock when there is a change in address with an active enable, data is written into memory on the same clock. When enable is not active, any change in address or data is ignored. Active reset ignores any change in input address and data.

The width of the write enable is controlled by the WE_WIDTH parameter. Input data is symmetrically divided and controlled by each write enable. For example, with a data width of 32 and a write enable width of 4, each bit of the write enable controls 8 bits of data (32/4=8). The byte-enable RAM compiler will error for wrong combination data width and write enable values.

The above waveform shows a write sequence with all possible values for write enable followed by a read:

- Value for parameter WE_WIDTH is 2 and DATA_WIDTH is 8 so each write enable controls 4 bits of input data.
- WenA value changes from 1 to 2, 2 to 0, and 0 to 3 which toggles all possible combinations of write enable.

© 2022 Synopsys, Inc.

The first sequence of address, write enable changes to perform a write sequence and the data patterns written to memory are 00, aa, ff. The read data pattern reflects the current content of memory before the write.

The second address sequence is a read (WenA is always zero). As shown in the read pattern, only the respective bits of data are written according to the write enable value.

Note: The write sequence for dual-port mode is the same as single port; conflicts occurring due to writing to the same location from both ports are the user's responsibility.

Parameter List

The following table lists the file entries corresponding to the byte-enable RAM wizard parameters.

Name	Description	Default Value	Range
ADDR_WIDTH	Bit/byte enable RAM address width	2	multiples of 2
DATA_WIDTH	Data width for input and output data, common to both Port A and Port B	8	2 to 256
WE_WIDTH	Write enable width, common to both Port A and Port B	2	
CONFIG_PORT	Selects single/dual port configuration	1 (single port)	0 = dual-port 1 = single-port
RST_TYPE_A/B	Port A/B reset type selection	1 (synchronous)	0 = no reset 1 = synchronous
RST_RDATA_A/B	Default data value for Port A/B on active reset	All 1's	decimal value

© 2022 Synopsys, Inc.

WEN_SENSE_A/B	Port A/B write enable sense	1 (active high)	0 = active low 1 = active high
RADDR_LTNCY_A/B	Optional read address register select Port A/B	1	0 = no latency 1 = one cycle latency
RDATA_LTNCY_A/B	Optional read data register select Port A/B	1	0 = no latency 1 = one cycle latency

© 2022 Synopsys, Inc.

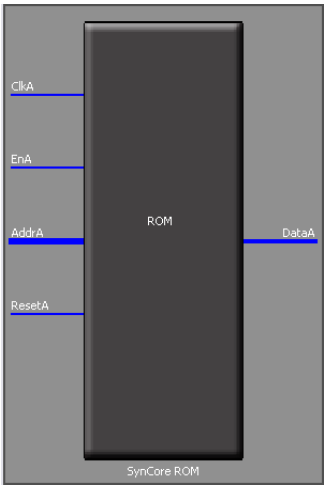
SYNCore ROM Compiler

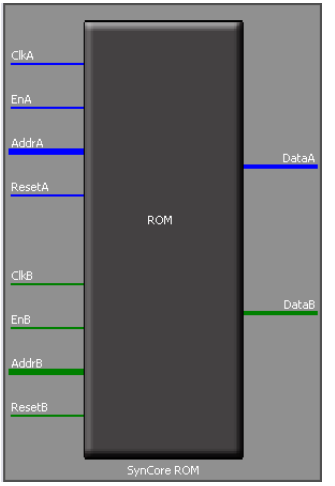
The SYNCore ROM Compiler generates Verilog code for your ROM implementation. This section describes the following:

- Functional Overview, on page 260
- Specifying ROMs with SYNCore, on page 262
- SYNCore ROM Wizard, on page 266
- Single-Port Read Operation, on page 270
- Dual-Port Read Operation, on page 271
- Parameter List, on page 271

Functional Overview

The SYNCore ROM component supports ROM implementations using block ROM or logic memory. For each configuration, design optimizations are made for optimum usage of core resources. Both single- and dual-port memory configurations are supported. Single-port ROM allows read access to memory through a single port, and dual-port ROM allows read access to memory through two ports. The following figure illustrates the supported signals for both configurations.





In the single-port (Port A) configuration, signals are synchronized to ClkA; ResetA can be synchronous or asynchronous depending on parameter selection. The read address (AddrA) and/or data output (DataA) can be registered to increase memory performance and improve timing. In the dual-port configuration, all Port A signals are synchronized to ClkA, and all PortB signals are synchronized to ClkB. ResetA and ResetB can be synchronous or asynchronous depending on parameter selection, and both data outputs can be registered and are subject to the same clock latencies. Registering the data output is recommended.

Note: When the data output is unregistered, the data is immediately set to its predefined reset value concurrent with an active reset signal.

© 2022 Synopsys, Inc.

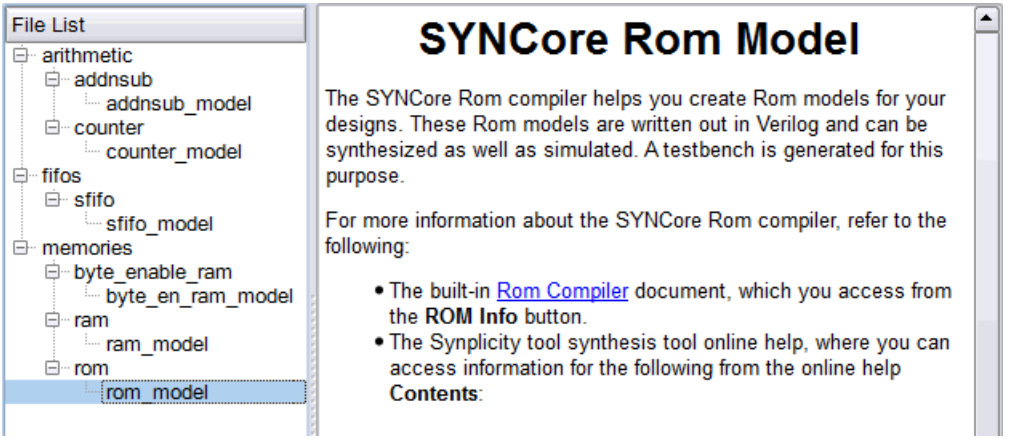
Specifying ROMs with SYNCore

The SYNCore IP wizard helps you generate Verilog code for your ROM implementation requirements. The following procedure shows you how to generate Verilog code for a ROM using the SYNCore IP wizard.

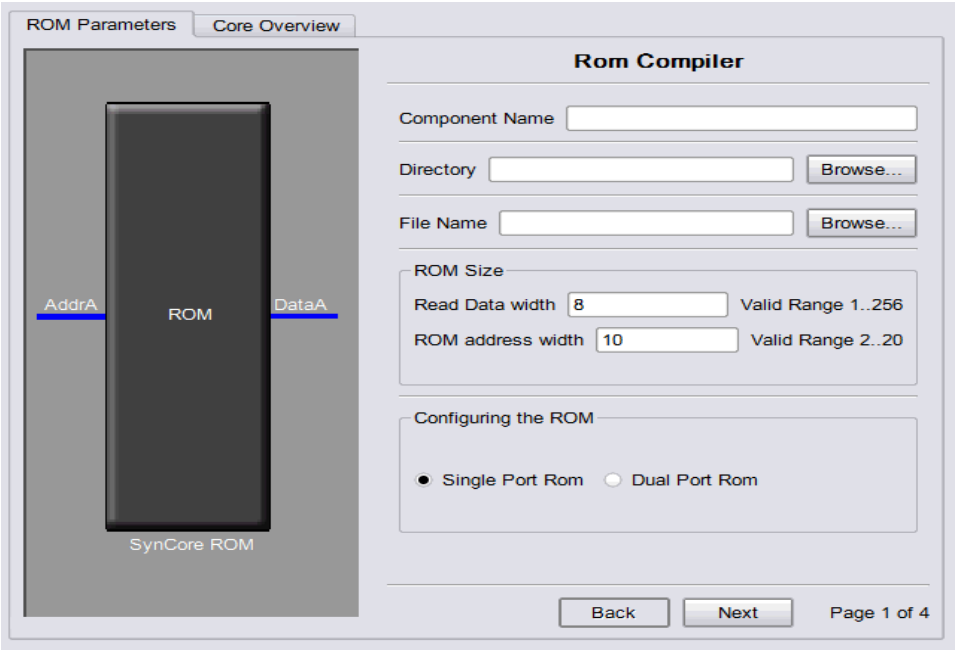
Note: The SYNCore ROM model uses Verilog 2001. When adding a ROM model to a Verilog-95 design, be sure to enable the Verilog 2001 check box on the Verilog tab of the Implementation Options dialog box or include a set_option -vlog_std v2001 statement in your project file to prevent a syntax error.

---

1. Start the wizard.
   - From the FPGA synthesis tool GUI, select Run->Launch SYNCore or click the Launch SYNCore icon  to start the SYNCore IP wizard.



   - In the window that opens, select rom_model and click Ok to open page 1 of the wizard.



2. Specify the parameters you need in the wizard. For details about the parameters, see Specifying ROM Parameters, on page 265. The ROM symbol on the left reflects any parameters you set.

3. After you have specified all the parameters you need, click the Generate button in the lower left corner. The tool displays a confirmation message (TCL execution successful!) and writes the required files to the directory you specified on page 1 of the wizard. The HDL code is in Verilog.

   SYNCore also generates a testbench for the ROM. The testbench covers a limited set of vectors.

   You can now close the SYNCore ROM Compiler.

4. Edit the ROM files if necessary. If you want to use the synchronous ROMs available in the target technology, make sure to register either the read address or the outputs.

5. Add the ROM you generated to your design.
   - Use the Add File command to add the Verilog design file that was generated and the syncore_rom.v file to your project. These files are in

© 2022 Synopsys, Inc.

the directory for output files that you specified on page 1 of the wizard.

- Use a text editor to open the instantiation_file.vin template file. This file is located in the same output files directory. Copy the lines that define the ROM, and paste them into your top-level module. The following figure shows a template file (in red text) inserted into a top-level module.

```
module test_rom_style(z,a,clk,en,rst);
input clk,en,rst;
output reg [3:0] z;
input [6:0] a;

my1stROM   <InstanceName> (
   // Output Ports
   .DataA(DataA),

   // Input  Ports
   .ClkA(ClkA),
   .EnA(EnA),
   .ResetA(ResetA),
   .AddrA(AddrA)
);
```

template

6. Edit the template port connections so that they agree with the port definitions in your top-level module as shown in the example below (the updated connection names are shown in red). You can also assign a unique name to each instantiation.

Port List

PortA interface signals are applicable for both single-port and dual-port configurations; PortB signals are applicable for dual-port configuration only.

Name	Type	Description
ClkA	Input	Clock input for Port A
EnA	Input	Enable input for Port A

© 2022 Synopsys, Inc.

AddrA	Input	Read address for Port A
ResetA	Input	Reset or interface disable pin for Port A
DataA	Output	Read data output for Port A
ClkB	Input	Clock input for Port B
EnB	Input	Enable input for Port B
AddrB	Input	Read address for Port B
ResetB	Input	Reset or interface disable pin for Port B
DataB	Output	Read data output for Port B

Specifying ROM Parameters

If you are creating a single-port ROM with the SYNCore IP wizard, you need to specify a single read address and a single clock, and you only need to configure the Port A parameters on page 2. If you are creating a dual-port ROM, you must additionally configure the Port B parameters on page 3. The following procedure lists what you need to specify. For descriptions of each parameter, refer to *SYNCore RAM Wizard, on page 231*.

1. Start the SYNCore ROM wizard, as described in Specifying ROMs with SYNCore, on page 262.
2. Do the following on page 1 of the ROM wizard:
   - In Component Name, specify a name for the memory. Do not use spaces.
   - In Directory, specify a directory where you want the output files to be written. Do not use spaces.
   - In Filename, specify a name for the Verilog file that will be generated with the ROM specifications. Do not use spaces.
   - Enter values for Read Data width and ROM address width (minimum depth value is 2; maximum depth of the memory is limited to 2^256).
   - Select Single Port Rom to indicate that you want to generate a single-port ROM or select Dual Port Rom to generate a dual-port ROM.
   - Click Next. The wizard opens page 2 where you set parameters for Port A.

   The ROM symbol dynamically updates to reflect any parameters you set.

© 2022 Synopsys, Inc.

3. Do the following on page 2 (Configuring Port A) of the RAM wizard:
   - For synchronous ROMs, select Register address bus AddrA and/or Register output data bus DataA to register the read address and/or the outputs. Selecting either checkbox enables the Enable for Port A checkbox which is used to select the Enable level.
   - Set the Configure Reset Options. Enabling the checkbox enables the type of reset (asynchronous or synchronous) and allows an output data pattern (all 1's or a specified pattern) to be defined on page 4.
4. If you are generating a dual-port ROM, set the port B parameters on page 3 (the page 3 parameters are only enabled when Dual Port Rom is selected on page 1).
5. On page 4, specify the location of the ROM initialization file and the data format (Hexadecimal or Binary). ROM initialization is supported using memory-coefficient files. The data format is either binary or hexadecimal with each data entry on a new line in the memory-coefficient file (specified by parameter INIT_FILE). Supported file types are `txt`, `mem`, `dat`, and `init` (recommended).
6. Generate the ROM by clicking Generate, as described in Specifying ROMs with SYNCore, on page 262 and add it to your design. All output files are in the directory you specified on page 1 of the wizard.

   For read/write timing diagrams, see Read/Write Timing Sequences, on page 242.

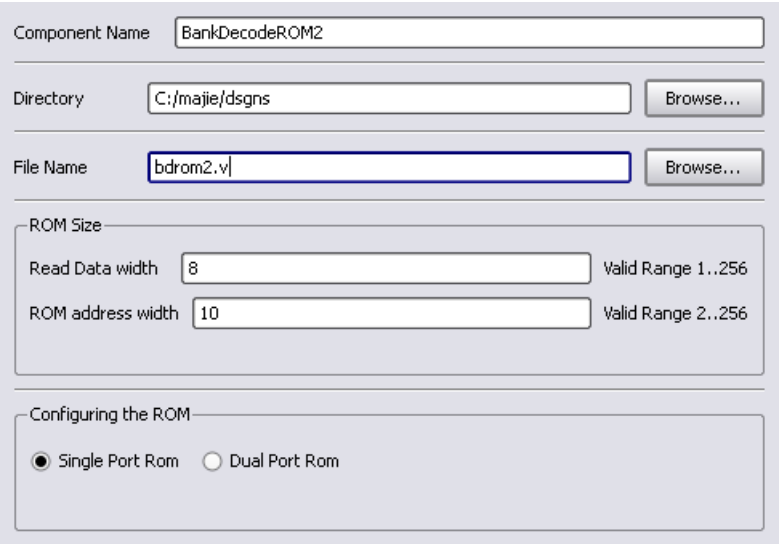
SYNCore ROM Wizard

The following describe the parameters you can set in the ROM wizard, which opens when you select rom_model:

- SYNCore ROM Parameters Page 1, on page 267
- SYNCore ROM Parameters Pages 2 and 3, on page 268
- SYNCore ROM Parameters Page 4, on page 270

© 2022 Synopsys, Inc.

SYNCore ROM Parameters Page 1



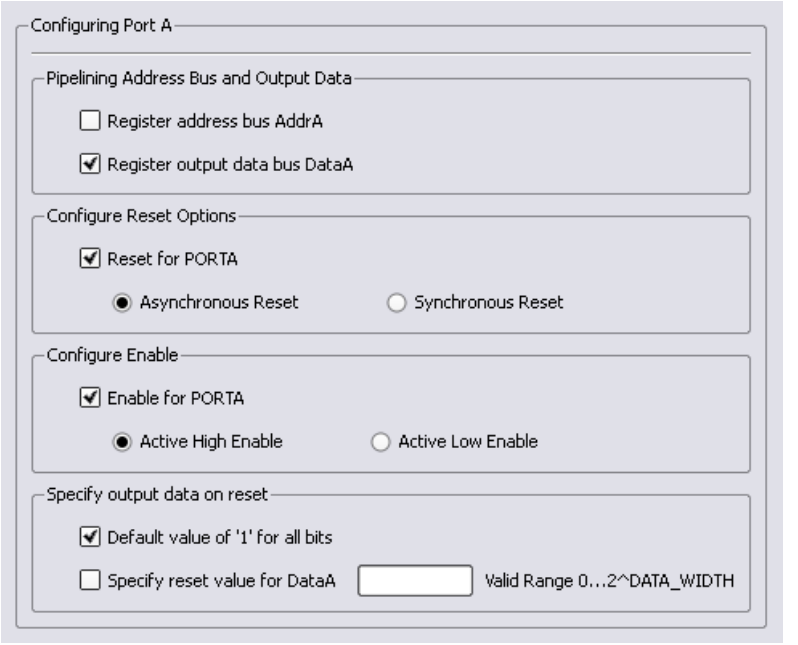
Component Name	Specifies the name of the component. This is the name that you instantiate in your design file to create an instance of the SYNCore ROM in your design. Do not use spaces.
Directory	Specifies the directory where the generated files are stored. Do not use spaces. The following files are created: filelist.txt – lists files written out by SYNCore options.txt – lists the options selected in SYNCore readme.txt – contains a brief description and known issues syncore_rom.v – Verilog library file required to generate ROM model testbench.v – Verilog testbench file for testing the ROM model instantiation_file.vin – describes how to instantiate the wrapper file *component*.v – ROM model wrapper file generated by SYNCore Note that running the ROM wizard in the same directory overwrites the existing files.
File Name	Specifies the name of the generated file containing the HDL description of the compiled ROM. Do not use spaces.

© 2022 Synopsys, Inc.

Read Data Width	Specifies the read data width of the ROM. The unit used is the number of bits and ranges from 2 to 256. Default value is 8. The read data width is common to both Port A and Port B. The corresponding file parameter is DATA_WIDTH=n.
ROM address width	Specifies the address depth for the memory. The unit used is the number of bits. Default value is 10. The corresponding file parameter is ADD_WIDTH=n.
Single Port Rom	When enabled, generates a single-port ROM. The corresponding file parameter is CONFIG_PORT="single".
Dual Port Rom	When enabled, generates a dual-port ROM. The corresponding file parameter is CONFIG_PORT="dual".

SYNCore ROM Parameters Pages 2 and 3

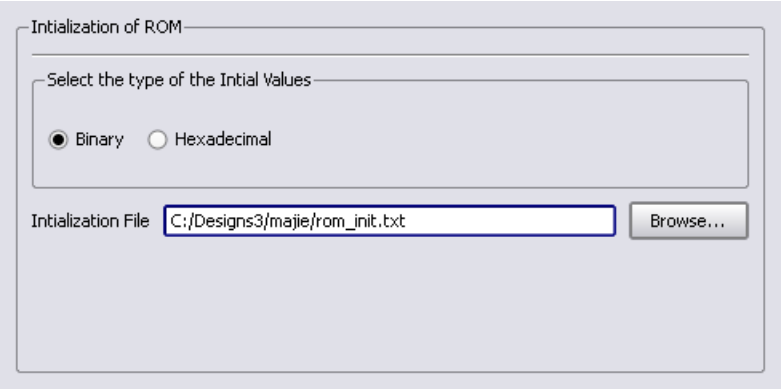
The port implementation parameters on pages 2 and 3 are the same; page 2 applies to Port A (single- and dual-port configurations), and page 3 applies to Port B (dual-port configurations only).



Register address bus AddrA	Used with synchronous ROM configurations to register the read address. When checked, also allows chip enable to be configured.
Register output data bus DataA	Used with synchronous ROM configurations to register the data outputs. When checked, also allows chip enable to be configured.
Asynchronous Reset	Sets the type of reset to asynchronous (Configure Reset Options must be checked). Configuring reset also allows the output data pattern on reset to be defined. The corresponding file parameter is RST_TYPE_A=1/RST_TYPE_B=1.
Synchronous Reset	Sets the type of reset to synchronous (Configure Reset Options must be checked). Configuring reset also allows the output data pattern on reset to be defined. The corresponding file parameter is RST_TYPE_A=0/RST_TYPE_B=0.
Active High Enable	Sets the level of the chip enable to high for synchronous ROM configurations. The corresponding file parameter is EN_SENSE_A=1/EN_SENSE_B=1.
Active Low Enable	Sets the level of the chip enable to low for synchronous ROM configurations. The corresponding file parameter is EN_SENSE_A=0/EN_SENSE_B=0.
Default value of '1' for all bits	Specifies an output data pattern of all 1's on reset. The corresponding file parameter is RST_DATA_A={n{1'b1} }/RST_DATA_B={n{1'b1} }.
Specify reset value for DataA/DataB	Specifies a user-defined output data pattern on reset. The pattern is defined in the adjacent field. The corresponding file parameter is RST_TYPE_A=pattern/RST_TYPE_B=pattern.

© 2022 Synopsys, Inc.

SYNCore ROM Parameters Page 4

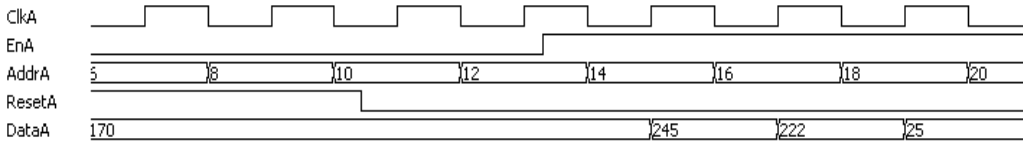


Binary	Specifies binary-formatted initialization file.
Hexadecimal	Specifies hexadecimal-formatted initial file.
Initialization File	Specifies path and filename of initialization file. The corresponding file parameter is INIT_FILE="*filename*".

Single-Port Read Operation

For single-port ROM, it is only necessary to configure Port A (see Specifying ROMs with SYNCore, on page 262). The following diagram shows the read timing for a single-port ROM.

On every active edge of the clock when there is a change in address with an active enable, data will be valid on the same clock or the next clock (depending on latency parameter values). When enable is inactive, any address change is ignored, and the data port maintains the last active read value. An active reset ignores any change in input address and forces the output data to its predefined initialization value. The following waveform shows the functional behavior of control signals in single-port mode.



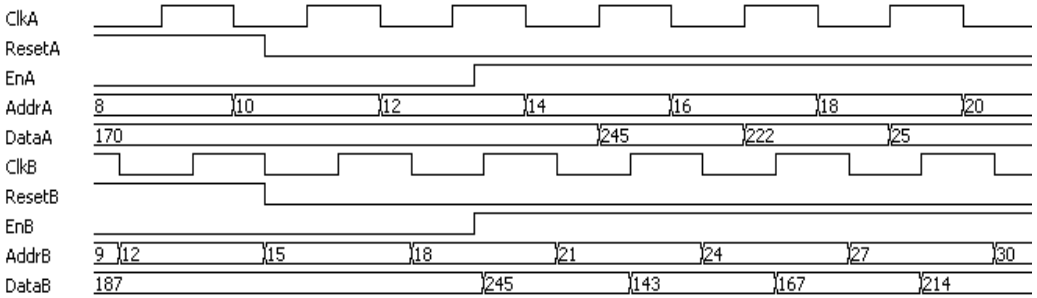
© 2022 Synopsys, Inc.

When reset is active, the output data holds the initialization value (i.e., 255). When reset goes inactive (and enable is active), data is read from the addressed location of ROM. Reset has priority over enable and always sets the output to the predefined initialization value. When both enable and reset are inactive, the output holds its previous read value.

Note: In the above timing diagram, reset is synchronous.

Dual-Port Read Operation

Dual-port ROMs allow read access to memory through two ports. For dual-port ROM, both port A and port B must be configured (see Specifying ROMs with SYNCore, on page 262). The following diagram shows the read timing for a dual-port ROM.



When either reset is active, the corresponding output data holds the initialization value (i.e., 255). When a reset goes inactive (and its enable is active), data is read form the addressed location of ROM. Reset has priority over enable and always sets the output to the predefined initialization value. When both enable and reset are inactive, the output holds its previous read value.

Note: In the above timing diagram, reset is synchronous.

Parameter List

The following table lists the file entries corresponding to the ROM wizard parameters.

© 2022 Synopsys, Inc.

Name	Description	Default Value	Range
ADD_WIDTH	ROM address width value. Default value is 10	10	--
DATA_WIDTH	Read data width, common to both Port A and Port B	8	2 to 256
CONFIG_PORT	Parameter to select single/dual configuration	dual (Dual Port)	dual (Dual), single (Single).
RST_TYPE_A	Port A reset type selection (synchronous, asynchronous)	1 - asynchronous	1(asyn), 0 (sync)
RST_TYPE_B	Port B reset type selection (synchronous, asynchronous)	1 - asynchronous	1 (asyn), 0 (sync)
RST_DATA_A	Default data value for Port A on active Reset	'1' for all data bits	0 – 2^DATA_WIDTH - 1
RST_DATA_B	Default data value for Port A on active Reset	'1' for all data bits	0 – 2^DATA_WIDTH - 1
EN_SENSE_A	Port A enable sense	1 – active high	0 - active low, 1- active high
EN_SENSE_B	Port B enable sense	1 – active high	0 - active low, 1- active high
ADDR_LTNCY_A	Optional address register select Port A	1- address registered	1(reg), 0(no reg)
ADDR_LTNCY_B	Optional address register select Port B	1- address registered	1(reg), 0(no reg)

© 2022 Synopsys, Inc.

DATA_LTNCY_A	Optional data register select Port A	1- data registered	1(reg), 0(no reg)
DATA_LTNCY_B	Optional data register select Port B	1- data registered	1(reg), 0(no reg)
INIT_FILE	Initial values file name	init.txt	--

© 2022 Synopsys, Inc.

SYNCore Adder/Subtractor Compiler

The SYNCore adder/subtractor compiler generates Verilog code for a parametrizable, pipelined adder/subtractor. This section describes the functionality of this block in detail.

- Functional Description, on page 274
- Specifying Adder/Subtractors with SYNCore, on page 275
- SYNCore Adder/Subtractor Wizard, on page 282
- Adder, on page 286
- Subtractor, on page 289
- Dynamic Adder/Subtractor, on page 292

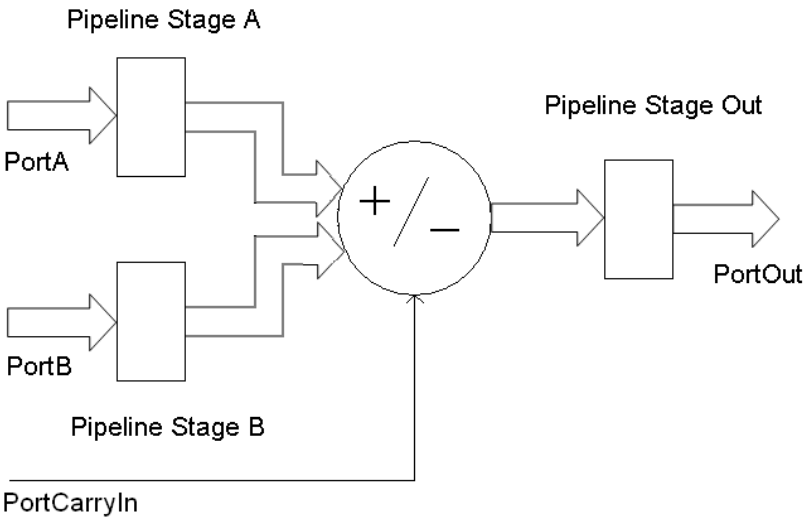
Functional Description

The adder/subtractor has a single clock that controls the entire pipeline stages (if used) of the adder/subtractor.

As its name implies, this block just adds/subtracts the inputs and provides the output result. One of the inputs can be configured as a constant. The data inputs and outputs of the adder/subtractor can be pipelined; the pipeline stages can be 0 or 1, and can be configured individually. The individual pipeline stage registers include their own reset and enable ports.

The reset to all of the pipeline registers can be configured either as synchronous or asynchronous using the RESET_TYPE parameter. The reset type of the pipeline registers cannot be configured individually.

© 2022 Synopsys, Inc.



SYNCore adder/subtractor has ADD_N_SUB parameter, which can take three values ADD, SUB, or DYNAMIC. Based on this parameter value, the adder/subtractor can be configured as follows.

- Adder
- Subtractor
- Dynamic Adder and Subtractor

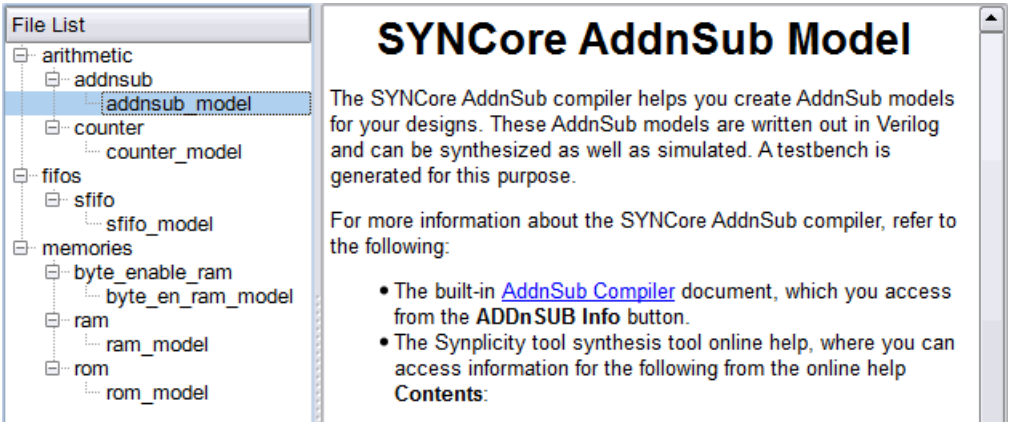
Specifying Adder/Subtractors with SYNCore

The SYNCore IP wizard helps you generate Verilog code for your adder/subtractor implementation requirements. The following procedure shows you how to generate Verilog code for an adder/subtractor using the SYNCore IP wizard.

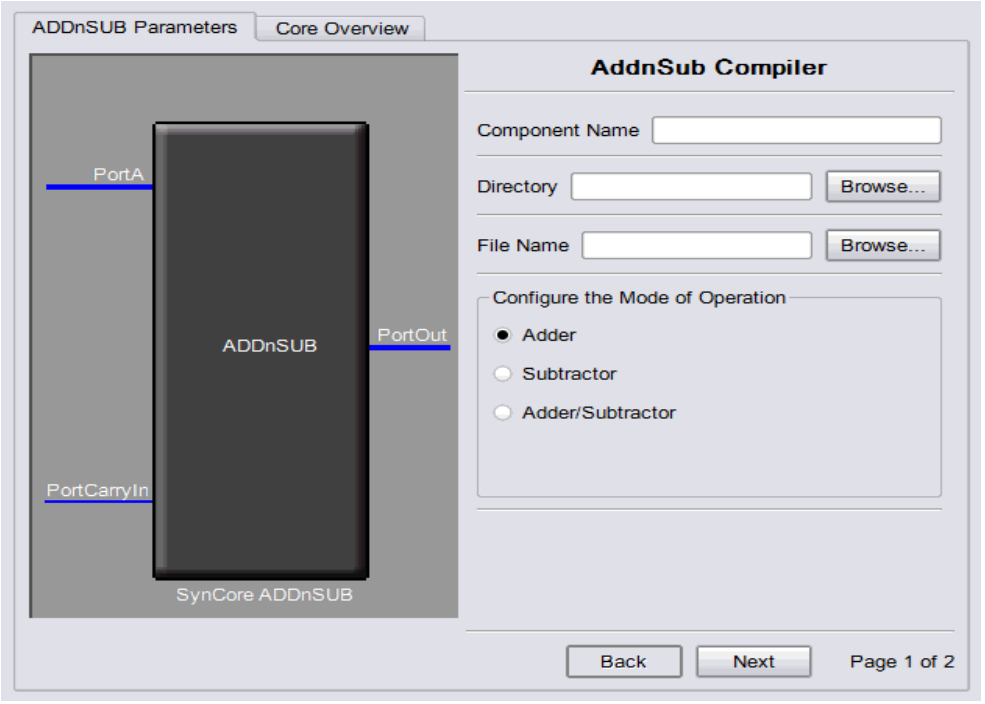
Note: The SYNCore adder/subtractor models use Verilog 2001. When adding an adder/subtractor model to a Verilog-95 design, be sure to enable the Verilog 2001 check box on the Verilog tab of the Implementation Options dialog box or include a set_option -vlog_std v2001 statement in your project file to prevent a syntax error.

© 2022 Synopsys, Inc.

1. Start the wizard.
   - From the FPGA synthesis tool GUI, select Run->Launch SYNCore or click the Launch SYNCore icon to start the SYNCore IP wizard.



   - In the window that opens, select addnsub_model and click Ok to open page 1 of the wizard.



2. Specify the parameters you need in the wizard. For details about the parameters, see Specifying Adder/Subtractor Parameters, on page 281. The ADDnSUB symbol on the left reflects any parameters you set.

3. After you have specified all the parameters you need, click the Generate button in the lower left corner.

   The tool displays a confirmation message (TCL execution successful!) and writes the required files to the directory you specified on page 1 of the wizard. The HDL code is in Verilog.

   The SYNCore wizard also generates a testbench for your adder/subtractor. The testbench covers a limited set of vectors. You can now close the wizard.

4. Add the adder/subtractor you generated to your design.
   - Edit the adder/subtractor files if necessary.

© 2022 Synopsys, Inc.

- Use the Add File command to add the Verilog design file that was generated and the syncore_addnsub.v file to your project. These files are in the directory for output files that you specified on page 1 of the wizard.
- Use a text editor to open the instantiation_file.v template file. This file is located in the same output files directory. Copy the lines that define the adder/subtractor and paste them into your top-level module. The following figure shows a template file (in red text) inserted into a top-level module.

```
module top (
    output [15 : 0] Out,
    input Clk,
    input [15 : 0] A,
    input CEA,
    input RSTA,
    input [15 : 0] B,
    input CEB,
    input RSTB,
    input CEOut,
    input RSTOut,
    input ADDnSUB,
    input CarryIn);

My_ADDnSUB ADDnSUB_inst(
// Output Ports
    .PortOut(Out),
// Input  Ports
    .PortClk(Clk),
    .PortA(A),
    .PortCEA(CEA),
    .PortRSTA(RSTA),
    .PortB(B),
    .PortCEB(CEB),
    .PortRSTB(RSTB),
    .PortCEOut(CEOut),
    .PortRSTOut(RSTOut),
    .PortADDnSUB(ADDnSUB),
    .PortCarryIn(CarryIn));

endmodule
```

template

5. Edit the template port connections so that they agree with the port definitions in your top-level module as shown in the example below (the updated connection names are shown in red). You can also assign a unique name to each instantiation.

```
module top (
   output [15 : 0] Out,
   input Clk,
   input [15 : 0] A,
   input CEA,
   input RSTA,
   input [15 : 0] B,
   input CEB,
   input RSTB,
   input CEOut,
   input RSTOut,
   input ADDnSUB,
   input CarryIn);

My_ADDnSUB ADDnSUB_inst(
// Output Ports
   .PortOut(Out),
// Input  Ports
   .PortClk(Clk),
   .PortA(A),
   .PortCEA(CEA),
   .PortRSTA(RSTA),
   .PortB(B),
   .PortCEB(CEB),
   .PortRSTB(RSTB),
   .PortCEOut(CEOut),
   .PortRSTOut(RSTOut),
   .PortADDnSUB(ADDnSUB),
   .PortCarryIn(CarryIn));
endmodule
```

Port List

The following table lists the port assignments for all possible configurations; the third column specifies the conditions under which the port is available.

© 2022 Synopsys, Inc.

Port Name	Description	Required/Optional
PortA	Data input for adder/subtractor Parameterized width and pipeline stages	Always present
PortB	Data input for adder/subtractor Parameterized width and pipeline stages	Not present if adder/subtractor is configured as a constant adder/subtractor
PortClk	Primary clock input; clocks all registers in the unit	Always present
PortRstA	Reset input for port A pipeline registers (active high)	Not present if pipeline stage for port A is 0
PortRstB	Reset input for port B pipeline registers (active high)	Not present if pipeline stage for port B is 0 or for constant adder/subtractor
PortADDnSUB	Selection port for dynamic operation	Not present if adder/subtractor configured as standalone adder or subtractor
PortRstOut	Reset input for output register (active high)	Not present if output pipeline stage is 0
PortCEA	Clock enable for port A pipeline registers (active high)	Not present if pipeline stage for port A is 0
PortCEB	Clock enable for port B pipeline registers (active high)	Not present if pipeline stage for port B is 0 or for constant adder/subtractor
PortCarryIn	Carry input for adder/subtractor	Always present
PortCEOut	Clock enable for output register (active high)	Not present if output pipeline stage is 0
PortOut	Data output	Always present

© 2022 Synopsys, Inc.

Specifying Adder/Subtractor Parameters

The SYNCore adder/subtractor can be configured as any of the following:

- Adder
- Subtractor
- Dynamic Adder/Subtractor

If you are creating a constant input adder, subtractor, or a dynamic adder/subtractor with the SYNCore IP wizard, you must select Constant Value Input and specify a value for port B in the Constant Value/Port B Width field on page 2 of the parameters. The following procedure lists the parameters you need to define when generating an adder/subtractor. For descriptions of each parameter, see *SYNCore Adder/Subtractor Wizard, on page 282*.

1. Start the SYNCore adder/subtractor wizard as described in *Specifying Adder/Subtractors with SYNCore, on page 275*.
2. Enter the following on page 1 of the wizard:
   - In the Component Name field, specify a name for your adder/subtractor. Do not use spaces.
   - In the Directory field, specify a directory where you want the output files to be written. Do not use spaces.
   - In the Filename field, specify a name for the Verilog file that will be generated with the adder/subtractor definitions. Do not use spaces.
   - Select the appropriate configuration in Configure the Mode of Operation.
3. Click Next. The wizard opens page 2 where you set parameters for port A and port B.
4. Configure Port A and B.
   - In the Configure Port A section, enter a value in the Port A Width field.
   - If you are defining a synchronous adder/subtractor, check Register Input A and then check Clock Enable for Register A and/or Reset for Register A.
   - To configure port B as a constant port, go to the Configure Port B section and check Constant Value Input. Enter the constant value in the Constant Value/Port B Width field.

© 2022 Synopsys, Inc.

- To configure port B as a dynamic port, go to the Configure Port B section and check Enable Port B and enter the port width in the Constant Value/Port B Width field.
- To define a synchronous adder/subtractor, check Register Input B and then check Clock Enable for Register B and/or Reset for Register B.

5. In the Configure Output Port section:
   - Enter a value in the Output port Width field.
   - If you are registering the output port, check Register output Port.
   - If you are defining a synchronous adder/subtractor check Clock Enable for Register PortOut and/or Reset for Register PortOut.
6. In the Configure Reset type for all Reset Signal section, click Synchronous Reset or Asynchronous Reset as appropriate.

   As you enter the page 2 parameters, the ADDnSUB symbol dynamically updates to reflect the parameters you set.
7. Generate the adder/subtractor by clicking the Generate button as described in Specifying Adder/Subtractors with SYNCore, on page 275 and add it to your design. All output files are in the directory you specified on page 1 of the wizard.

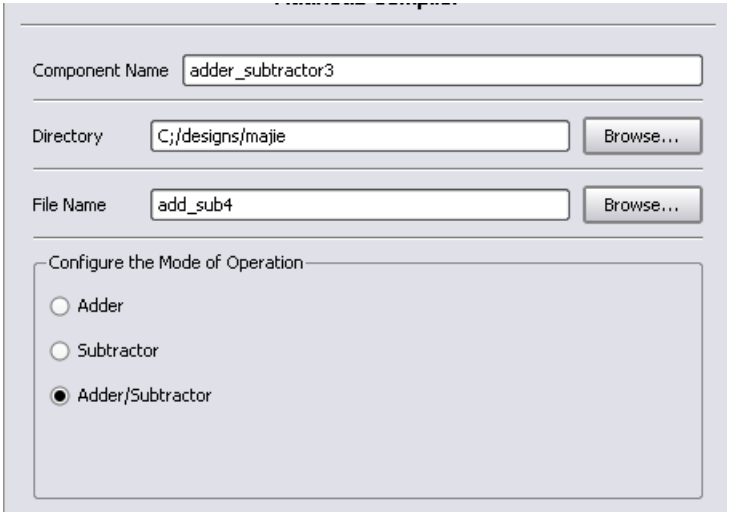
SYNCore Adder/Subtractor Wizard

The following describe the parameters you can set in the adder/subtractor wizard, which opens when you select addnsub_model:

- SYNCore Adder/Subtractor Parameters Page 1, on page 283
- SYNCore Adder/Subtractor Parameters Page 2, on page 284

© 2022 Synopsys, Inc.

SYNCore Adder/Subtractor Parameters Page 1

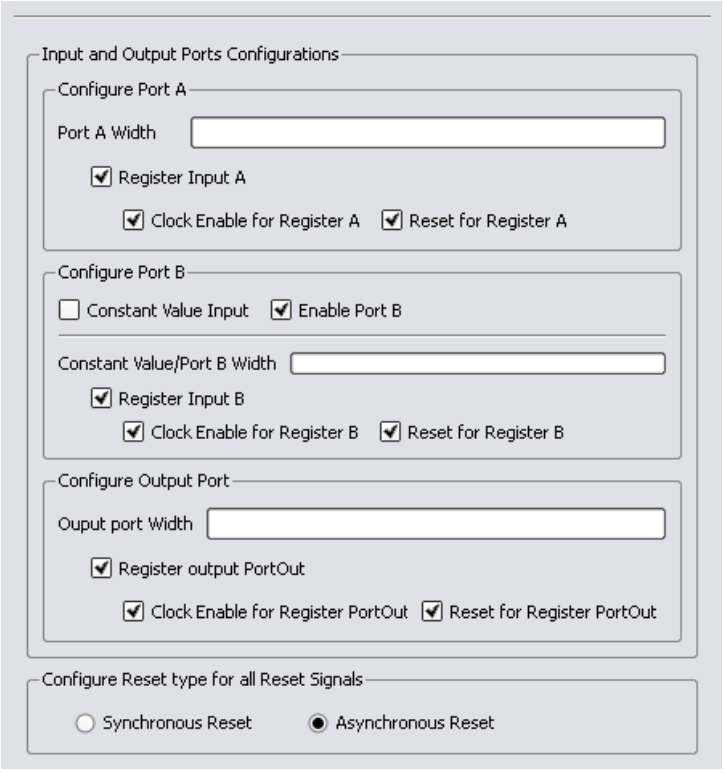


Component Name	Specifies a name for the adder/subtractor. This is the name that you instantiate in your design file to create an instance of the SYNCore adder/subtractor in your design. Do not use spaces.
Directory	Indicates the directory where the generated files will be stored. Do not use spaces. The following files are created: • filelist.txt – lists files written out by SYNCore • options.txt – lists the options selected in SYNCore • readme.txt – contains a brief description and known issues • syncore_ADDnSUB.v – Verilog library file required to generate adder/subtractor model • testbench.v – Verilog testbench file for testing the adder/subtractor model • instantiation_file.vin – describes how to instantiate the wrapper file • *component*.v – adder/subtractor model wrapper file generated by SYNCore Note that running the wizard in the same directory overwrites any existing files.
Filename	Specifies the name of the generated file containing the HDL description of the generated adder/subtractor. Do not use spaces.

© 2022 Synopsys, Inc.

Adder	When enabled, generates an adder (the corresponding file parameter is ADD_N_SUB ="ADD").
Subtractor	When enabled, generates a subtractor (the corresponding file parameter is ADD_N_SUB ="SUB").
Adder/Subtractor	When enabled, generates a dynamic adder/subtractor (the corresponding file parameter is ADD_N_SUB ="DYNAMIC").

SYNCore Adder/Subtractor Parameters Page 2



Port A Width	Specifies the width of port A (the corresponding file parameter is PORT_A_WIDTH=n).
Register Input A	Used with synchronous adder/subtractor configurations to register port A. When checked, also allows clock enable and reset to be configured (the corresponding file parameter is PORTA_PIPELINE_STAGE='0' or '1').
Clock Enable for Register A	Specifies the enable for port A register.
Reset for Register A	Specifies the reset for port A register.
Constant Value Input	Specifies port B as a constant input when checked and allows you to enter a constant value in the Constant Value/Port B Width field (the corresponding file parameter is CONSTANT_PORT ='0').
Enable Port B	Specifies port B as an input when checked and allows you to enter a port B width in the Constant Value/Port B Width field (the corresponding file parameter is CONSTANT_PORT ='1').
Constant Value/Port B Width	Specifies either a constant value or port B width depending on Constant Value Input and Enable Port B selection (the corresponding file parameters are CONSTANT_VALUE= n or PORT_B_WIDTH=n).
Register Input B	Used with synchronous adder/subtractor configurations to register port B. When checked, also allows clock enable and reset to be configured (the corresponding file parameter is PORTB_PIPELINE_STAGE='0' or '1').
Clock Enable for Register B	Specifies the enable for the port B register.
Reset for Register B	Specifies the reset for the port B register.
Output port Width	Specifies the width of the output port (the corresponding file parameter is PORT_OUT_WIDTH=n).
Register output PortOut	Used with synchronous adder/subtractor configurations to register the output port. When checked, also allows clock enable and reset to be configured (the corresponding file parameter is PORTOUT_PIPELINE_STAGE='0' or '1'.
Clock Enable for Register PortOut	Specifies the enable for the output port register.

© 2022 Synopsys, Inc.

Reset for Register PortOut	Specifies the reset for the output port register.
Synchronous Reset	Sets the type of reset to synchronous (the corresponding file parameter is RESET_TYPE='0').
Asynchronous Reset	Sets the type of reset to asynchronous (the corresponding file parameter is RESET_TYPE='1').

Adder

Based on the parameter CONSTANT_PORT, the adder can be configured in two ways.

- CONSTANT_PORT='0' – adder with two input ports (port A and port B)
- CONSTANT_PORT='1' – adder with one constant port

Adder with Two Input Ports (Port A and Port B)

In this mode, port A and port B values are added. Optional pipeline stages can also be inserted at port A, port B or at both port A and port B. Optionally, pipeline stages can also be added at the output port. Depending on pipeline stages, a number of the adder configurations are given below.

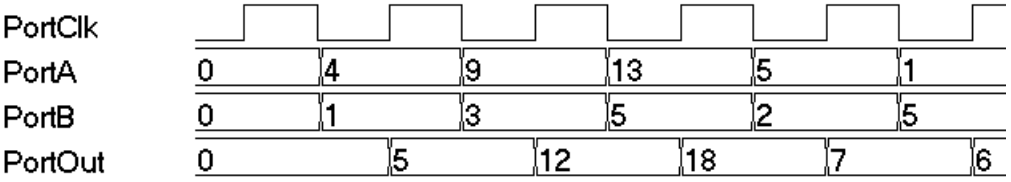
Adder with No Pipeline Stages – In this mode, the port A and port B inputs are added. The adder is purely combinational, and the output changes immediately with respect to the inputs.

Parameters: PORTA_PIPELINE_STAGE= '0'

PortA	0	4	9	13	5	1
PortB	0	1	3	5	2	5
PortOut	0	5	12	18	7	6

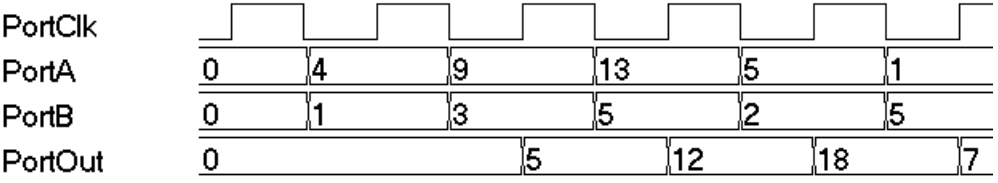
Adder with Pipeline Stages at Input Only – In this mode, the port A and port B inputs are pipelined and added. Because there is no pipeline stage at the output, the result is valid at each rising edge of the clock.

Parameters: PORTA_PIPELINE_STAGE= '1'
PORTB_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '0'



Adder with Pipeline Stages at Input and Output – In this mode, the port A and port B inputs are pipelined and added, and the result is pipelined. The result is valid only on the second rising edge of the clock.

Parameters: PORTA_PIPELINE_STAGE= '1'
PORTB_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '1'



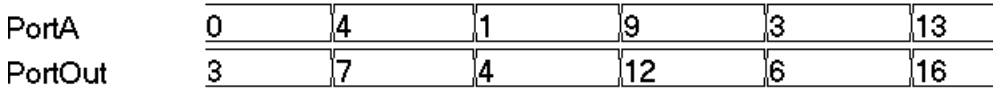
Adder with a Port Constant

In this mode, port A is added with a constant value (the constant value can be passed though the parameter CONSTANT_VALUE). Optional pipeline stages can also be inserted at port A. Optionally, pipeline stages can also be added at the output port. Depending on the pipeline stages, a number of the adder configurations are given below (here CONSTANT_VALUE='3')

© 2022 Synopsys, Inc.

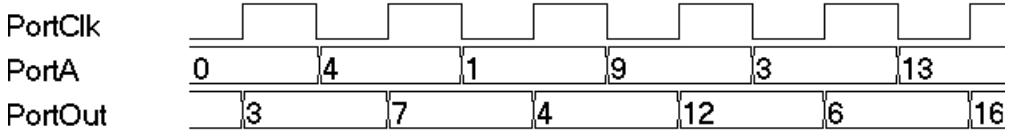
Adder with No Pipeline Stages – In this mode, input port A is added with a constant value. The adder is purely combinational, and the output changes immediately with respect to the input.

Parameters: PORTA_PIPELINE_STAGE= '0'
PORTOUT_PIPELINE_STAGE= '0'



Adder with Pipeline Stage at Input Only – In this mode, input port A is pipelined and added with a constant value. Because there is no pipeline stage at the output, the result is valid at each rising edge of the clock.

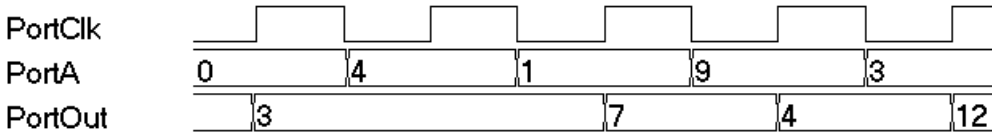
Parameters: PORTA_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '0'



Adder with Pipeline Stages at Input and Output – In this mode, input port A is pipelined and added with a constant value, and the result is pipelined. The result is valid only on the second rising edge of the clock.

Parameters: PORTA_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '1'

© 2022 Synopsys, Inc.



Subtractor

Based on the parameter CONSTANT_PORT, the subtractor can be configured in two ways.

CONSTANT_PORT='0' – subtractor with two input ports (port A and port B)

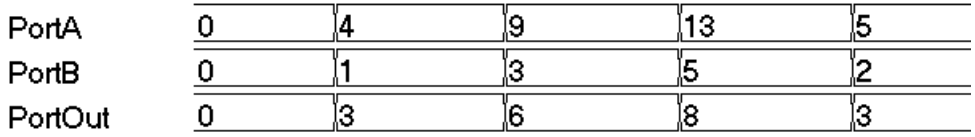
CONSTANT_PORT='1' – subtractor with one constant port

Subtractor with Two Input Ports (Port A and Port B)

In this mode, port B is subtracted from port A. Optional pipeline stages can also be inserted at port A, port B, or both ports. Optionally, pipeline stages can also be added at the output port. Depending on the pipeline stages, a number of the subtractor configurations are given below.

Subtractor with No Pipeline Stages – In this mode, input port B is subtracted from port A, and the subtractor is purely combinational. The output changes immediately with respect to the inputs.

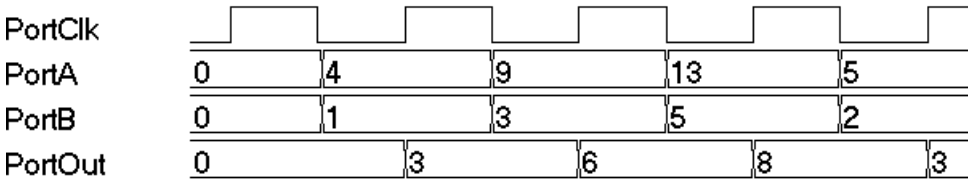
Parameters: PORTA_PIPELINE_STAGE= '0'
PORTB_PIPELINE_STAGE= '0'
PORTOUT_PIPELINE_STAGE= '0'



© 2022 Synopsys, Inc.

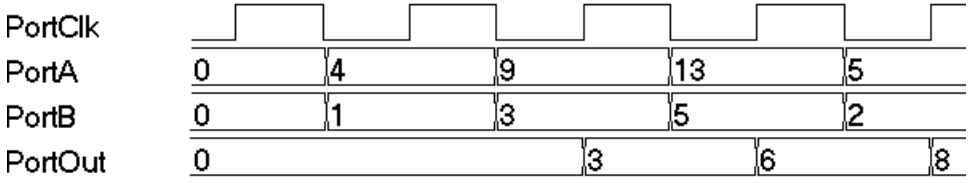
Subtractor with Pipeline Stages at Input Only – In this mode, input port B and input PortA are pipelined and then subtracted. Because there is no pipeline stage at the output, the result is valid at each rising edge of the clock.

Parameters: PORTA_PIPELINE_STAGE= '1'
PORTB_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '0'



Subtractor with Pipeline Stages at Input and Output – In this mode, input PortA and PortB are pipelined and then subtracted, and the result is pipelined. The result is valid only at the second rising edge of the clock.

Parameters: PORTA_PIPELINE_STAGE= '1'
PORTB_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '1'

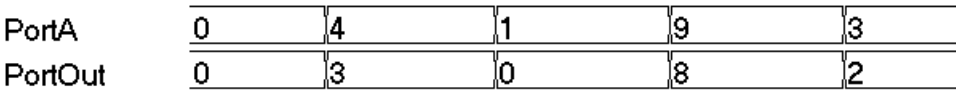


Subtractor with a Port Constant

In this mode, a constant value is subtracted from port A (the constant value can be passed though the parameter CONSTANT_VALUE). Optional pipeline stages can also be inserted at port A, Optionally, pipeline stages can also be added at the output port. Depending on pipeline stages, a number of the subtractor configurations are given below (here `CONSTANT_VALUE='1'`).

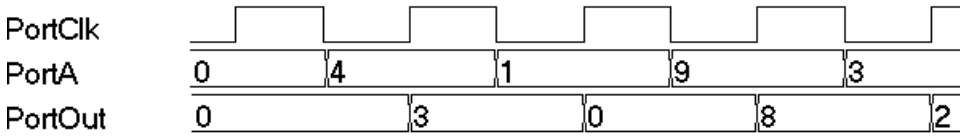
Subtractor with No Pipeline Stages – In this mode, a constant value is subtracted from port A. The subtractor is purely combinational, and the output changes immediately with respect to the input.

Parameters: PORTA_PIPELINE_STAGE= '0'
PORTOUT_PIPELINE_STAGE= '0'



Subtractor with Pipeline Stages at Input Only – In this mode, a constant value is subtracted from pipelined input port A. Because there is no pipeline stage at the output, the output is valid at each rising edge of the clock.

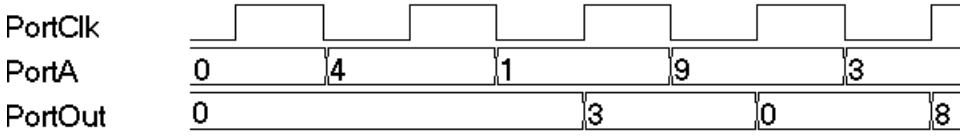
Parameters: PORTA_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '0'



Subtractor with Pipeline Stages at Input and Output – In this mode, a constant value is subtracted from pipelined port A, and the output is pipelined. The result is valid only at the second rising edge of the clock.

© 2022 Synopsys, Inc.

Parameters: PORTA_PIPELINE_STAGE= ‘1’
PORTOUT_PIPELINE_STAGE= ‘1’



Dynamic Adder/Subtractor

In dynamic adder/subtractor mode, port PortADDnSUB controls adder/subtractor operation.

PortADDnSUB='0' – adder operation

PortADDnSUB='1' – subtractor operation

Based on the parameter CONSTANT_PORT, the dynamic adder/subtractor can be configured in one of two ways:

CONSTANT_PORT='0' – dynamic adder/subtractor with two input ports

CONSTANT_PORT='1' – dynamic adder/subtractor with one constant port

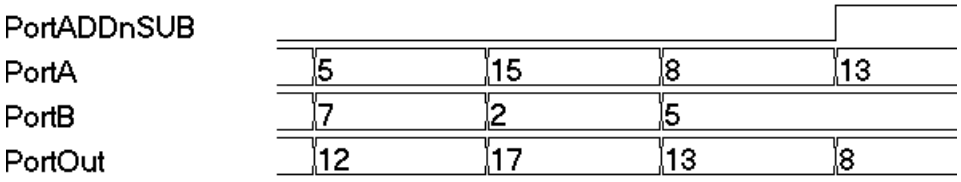
Dynamic Adder/Subtractor with Two Input Ports (Port A and Port B)

In this mode, the addition and subtraction is dynamic based on the value of input port PortADDnSUB. Optional pipeline stages can also be inserted at Port A, Port B, or both Port A and Port B. Optionally, pipeline stages can also be added at the output port. Depending on pipeline stages, some of the dynamic adder/subtractor configurations are given below.

Dynamic Adder/Subtractor with No Pipeline Registers – In this mode, the dynamic adder/subtractor is a purely combinational, and output changes immediately with respect to the inputs.

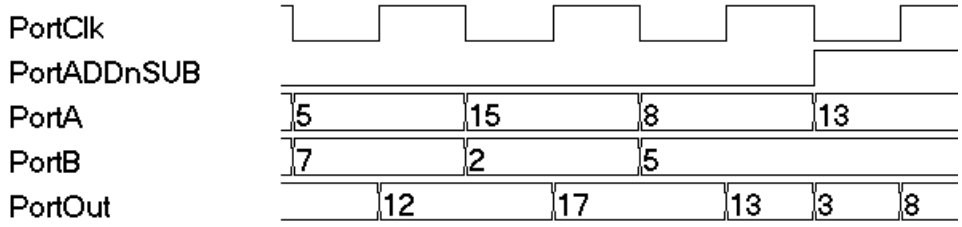
© 2022 Synopsys, Inc.

Parameters: PORTA_PIPELINE_STAGE= '0'
PORTB_PIPELINE_STAGE= '0'
PORTOUT_PIPELINE_STAGE= '0'



Dynamic Adder/Subtractor with Pipeline Stages at Input Only – In this mode, input port A and port B are pipelined and then added/subtracted based on the value of port PortADDnSUB. Because there is no pipeline stage at the output port, the result immediately changes with respect to the PortADDnSUB signal.

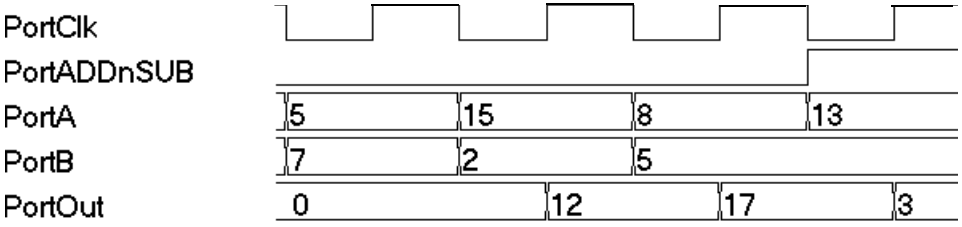
Parameters: PORTA_PIPELINE_STAGE= '1'
PORTB_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '0'



Dynamic Adder/Subtractor with Pipeline Stages at Input and Output – In this mode, input port A and port B are pipelined and then added/subtracted based on the value of port PortADDnSUB. Because the output port is pipelined, the result is valid only on the second rising edge of the clock.

© 2022 Synopsys, Inc.

Parameters: PORTA_PIPELINE_STAGE= ‘1’
PORTB_PIPELINE_STAGE= ‘1’
PORTOUT_PIPELINE_STAGE= ‘1’

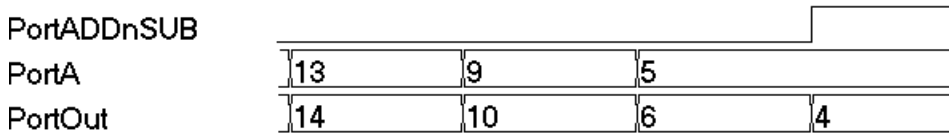


Dynamic Adder/Subtractor with a Port Constant

In this mode, a constant value is either added or subtracted from port A based on input port value PortADDnSUB (the constant value can be passed though the parameter CONSTANT_VALUE). Optional pipeline stages can also be inserted at port A, Optionally, pipeline stages can also be added at the output port. Depending on the pipeline stages, a number of the dynamic adder/subtractor configurations are given below (here `CONSTANT_VALUE='1'`).

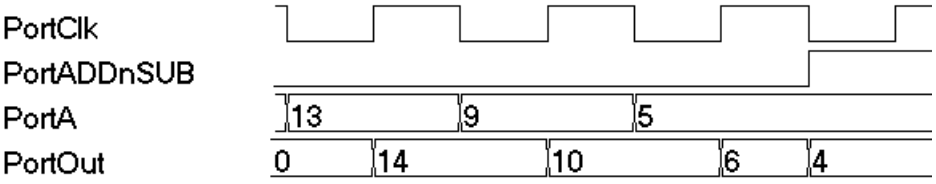
Dynamic Adder/Subtractor with No Pipeline Registers – In this mode, dynamic adder/subtractor is purely combinational, and the output changes immediately with respect to the input.

Parameters: PORTA_PIPELINE_STAGE= ‘0’
PORTOUT_PIPELINE_STAGE= ‘0’



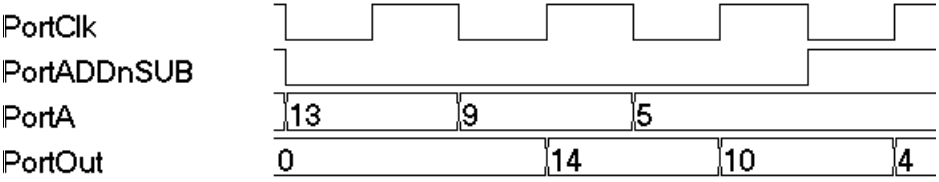
Dynamic Adder/Subtractor with Pipeline Stages at Input Only – In this mode, a constant value is either added or subtracted from the pipelined version of port A based on the value of port PortADDnSUB. Because there is no pipeline stage on the output port, the result changes immediately with respect to the PortADDnSUB signal.

Parameters: PORTA_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '0'



Dynamic Adder/Subtractor with Pipeline Stages at Input and Output – In this mode, a constant value is either added or subtracted from the pipelined version of port A based on the value of port PortADDnSUB. Because the output port is pipelined, the result is valid only on the second rising edge of the clock.

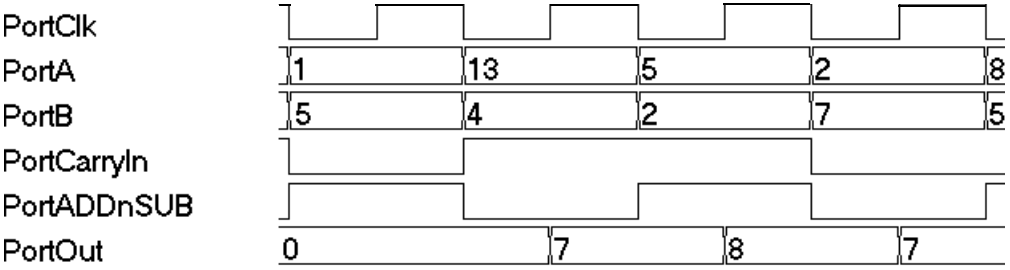
Parameters: PORTA_PIPELINE_STAGE= '1'
PORTOUT_PIPELINE_STAGE= '1'



© 2022 Synopsys, Inc.

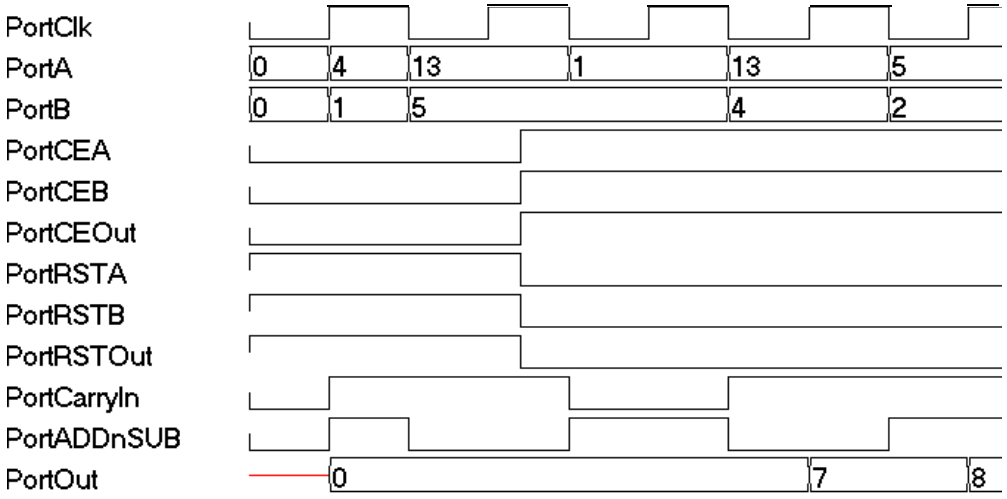
Dynamic Adder/Subtractor with Carry Input

The following waveform shows the behavior of the dynamic adder/subtractor with a carry input (the carry input is assumed to be 0).



Dynamic Adder/Subtractor with Complete Control Signals

The following waveform shows the complete signal set for the dynamic adder/subtractor. The enable and reset signals are always present in all of the previous cases.



SYNCore Counter Compiler

The SYNCore counter compiler generates Verilog code for your up, down, and dynamic (up/down) counter implementation. This section describes the following:

- Functional Overview, on page 297
- Specifying Counters with SYNCore, on page 298
- SYNCore Counter Wizard, on page 304
- UP Counter Operation, on page 308
- Down Counter Operation, on page 308
- Dynamic Counter Operation, on page 309

Functional Overview

The SYNCore counter component supports up, down, and dynamic (up/down) counter implementations using DSP blocks or logic elements. For each configuration, design optimizations are made for optimum use of core resources.

As its name implies, the COUNTER block counts up (increments) or down (decrements) by a step value and provides an output result. You can load a constant or a variable as an intermediate value or base for the counter. Reset to the counter on the PortRST input is active high and can be configured either as synchronous or asynchronous using the RESET_TYPE parameter. Count enable on the PortCE input must be value high to enable the counter to increment or decrement.

© 2022 Synopsys, Inc.

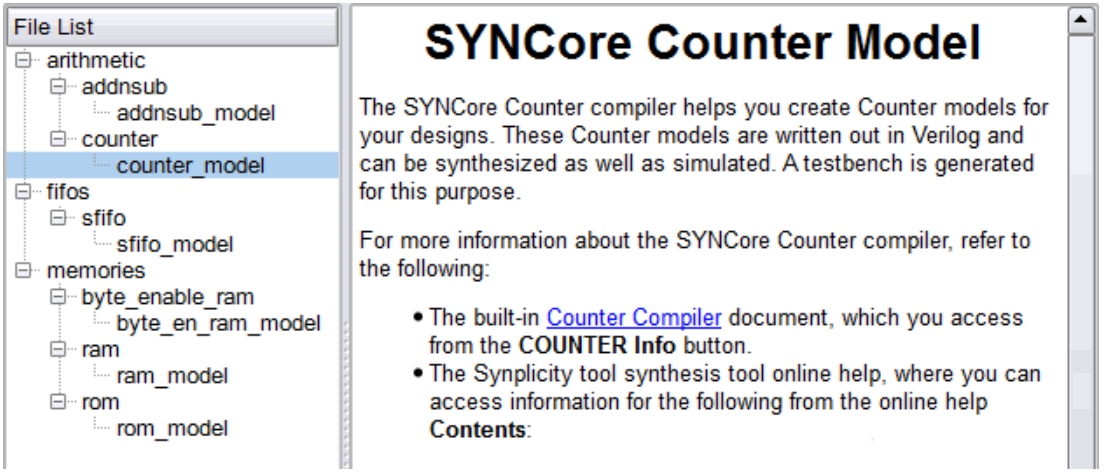
Specifying Counters with SYNCore

The SYNCore IP wizard helps you generate Verilog code for your counter implementation requirements. The following procedure shows you how to generate Verilog code for a counter using the SYNCore IP wizard.

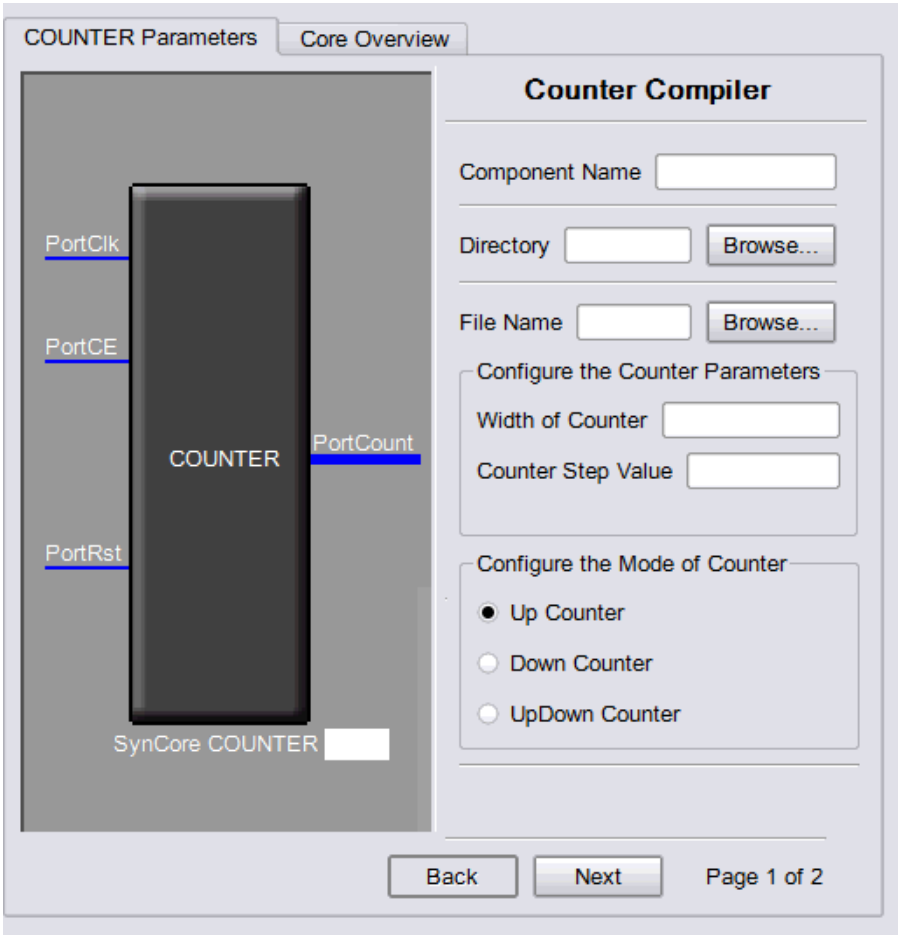
Note: The SYNCore counter model uses Verilog 2001. When adding a counter model to a Verilog-95 design, be sure to enable the Verilog 2001 check box on the Verilog tab of the Implementation Options dialog box or include a set_option -vlog_std v2001 statement in your project file to prevent a syntax error.

---

1. Start the wizard.
   - From the FPGA synthesis tool GUI, select Run->Launch SYNCore or click the Launch SYNCore icon  to start the SYNCore IP wizard.



   - In the window that opens, select counter_model and click Ok to open page1 of the wizard.



2. Specify the parameters you need in the wizard. For details about the parameters, see Specifying Counter Parameters, on page 303. The COUNTER symbol on the left reflects any parameters you set.

3. After you have specified all the parameters you need, click the Generate button in the lower left corner.

   The tool displays a confirmation message (TCL execution successful!) and writes the required files to the directory you specified on page 1 of the wizard. The HDL code is in Verilog.

© 2022 Synopsys, Inc.

The SYNCore wizard also generates a testbench for your counter. The testbench covers a limited set of vectors. You can now close the wizard.

4. Add the counter you generated to your design.
   - Edit the counter files if necessary.
   - Use the Add File command to add the Verilog design file that was generated and the syncore_addnsub.v file to your project. These files are in the directory for output files that you specified on page 1 of the wizard.
   - Use a text editor to open the instantiation_file.v template file. This file is located in the same output files directory. Copy the lines that define the counter and paste them into your top-level module. The following figure shows a template file (in red text) inserted into a top-level module.

```
module counter #(
    parameter COUNT_WIDTH = 5,
    parameter STEP = 2,
    parameter RESET_TYPE = 0,
    parameter LOAD = 2,
    parameter MODE = "Dynamic")

(
// Output Ports
       output wire [WIDTH-1:0] Count,
// Input Ports
       input wire Clock,
       input wire Reset,
       input wire Up_Down,
       input wire Load,
       input wire [WIDTH-1:0] LoadValue,
       input wire Enable );

SynCoreCounter #(
    .COUNT_WIDTH(COUNT_WIDTH),
    .STEP(STEP),
    .RESET_TYPE(RESET_TYPE),
    .LOAD(LOAD),
    .MODE(MODE))

SynCoreCounter_ins1 (
    .PortCount(PortCount),
    .PortClk(PortClock),
    .PortRST(PortRST),
    .PortUp_nDown(PortUp_nDown),
    .PortLoad(PortLoad),
    .PortLoadValue(PortLoadLoadValue)
    .PortCE(PortCE) );

endmodule
```

template

5. Edit the template port connections so that they agree with the port definitions in your top-level module as shown in the example below (the updated connection names are shown in red). You can also assign a unique name to each instantiation. You can also assign a unique name to each instantiation.

© 2022 Synopsys, Inc.

```
module counter #(
   parameter COUNT_WIDTH = 5,
   parameter STEP = 2,
   parameter RESET_TYPE = 0,
   parameter LOAD = 2,
   parameter MODE = "Dynamic")

(
// Output Ports
      output wire [WIDTH-1:0] Count,
// Input Ports
      input wire Clock,
      input wire Reset,
      input wire Up_Down,
      input wire Load,
      input wire [WIDTH-1:0] LoadValue,
      input wire Enable);

SynCoreCounter #(
   .COUNT_WIDTH(COUNT_WIDTH),
   .STEP(STEP),
   .RESET_TYPE(RESET_TYPE),
   .LOAD(LOAD),
   .MODE(MODE))

SynCoreCounter_ins1 (
   .PortCount(Count),
   .PortClk(Clock),
   .PortRST(Reset),
   .PortUp_nDown(Up_Down),
   .PortLoad(Load),
   .PortLoadValue(LoadValue),
   .PortCE(Enable));

endmodule
```

Port List

The following table lists the port assignments for all possible configurations; the third column specifies the conditions under which the port is available.

Port Name	Description	Required/Optional
PortCE	Count Enable input pin with size one (active high)	Always present
PortClk	Primary clock input	Always present
PortLoad	Load Enable input which loads the counter (active high).	Not present for parameter LOAD=0
PortLoadValue	Load value primary input (active high)	Not present for parameter LOAD=0 and LOAD=1
PortRST	Reset input which resets the counter (active high)	Always present
PortUp_nDown	Primary input which determines the counter mode. 0 = Up counter 1 = Down counter	Present only for MODE="Dynamic"
PortCount	Counter primary output	Always present

Specifying Counter Parameters

The SYNCore counter can be configured for any of the following functions:

- Up Counter
- Down Counter
- Dynamic Up/Down Counter

The counter core can have a constant or variable input load or no load value. If you are creating a constant-load counter, you will need to select Enable Load and Load Constant Value on page 2 of the wizard. If you are creating a variable-load counter, you will need to select Enable Load and Use Variable Port Load on page 2. The following procedure lists the parameters you need to define when generating a counter. For descriptions of each parameter, see SYNCore Counter Wizard, on page 304.

1. Start the SYNCore counter wizard, as described in *Specifying Counters with SYNCore, on page 298*.

© 2022 Synopsys, Inc.

2. Enter the following on page 1 of the wizard:
   - In the Component Name field, specify a name for your counter. Do not use spaces.
   - In the Directory field, specify a directory where you want the output files to be written. Do not use spaces.
   - In the Filename field, specify a name for the Verilog file that will be generated with the counter definitions. Do not use spaces.
   - Enter the width and depth of the counter in the Configure the Counter Parameters section.
   - Select the appropriate configuration in the Configure the Mode of Counter section.
3. Click Next. The wizard opens page 2 where you set parameters for PortLoad and PortLoadValue.
   - Select Enable Load option and the required load option in Configure Load Value section.
   - Select the required reset type in the Configure Reset type section.

   The COUNTER symbol dynamically updates to reflect the parameters you set.
4. Generate the counter core by clicking Generate button. All output files are written to the directory you specified on page1 of the wizard.

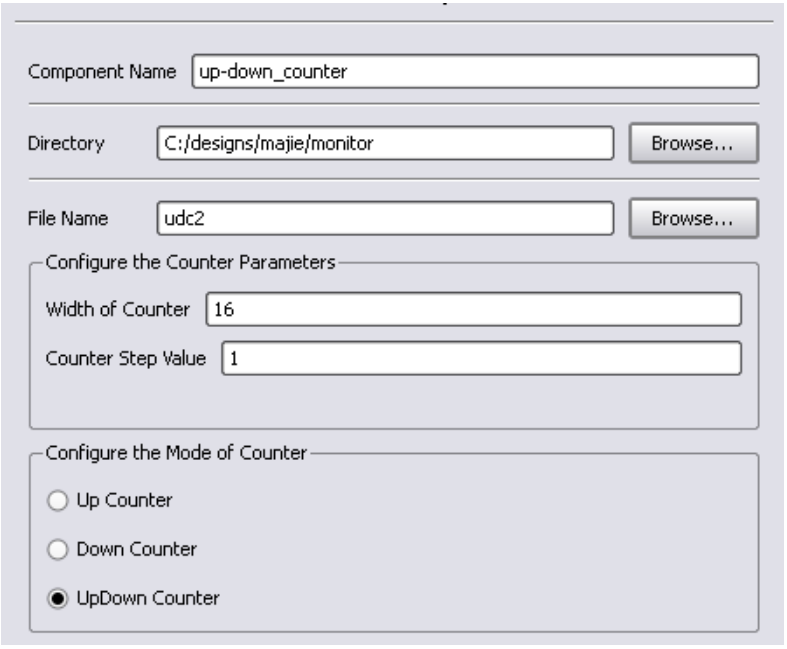
SYNCore Counter Wizard

The following describe the parameters you can set in the ROM wizard, which opens when you select counter_model:

- SYNCore Counter Parameters Page 1, on page 305
- SYNCore Counter Parameters Page 2, on page 307

© 2022 Synopsys, Inc.

SYNCore Counter Parameters Page 1

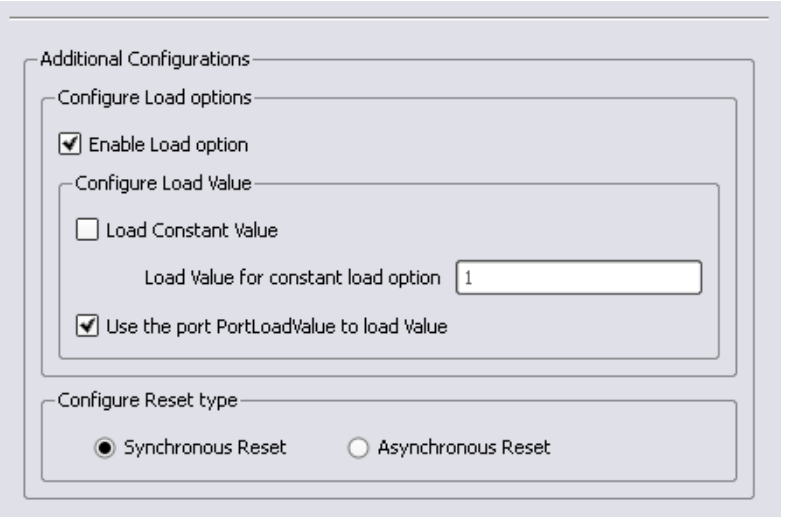


© 2022 Synopsys, Inc.

Component Name	Specifies a name for the counter. This is the name that you instantiate in your design file to create an instance of the SYNCore counter in your design. Do not use spaces.
Directory	Indicates the directory where the generated files will be stored. Do not use spaces. The following files are created: • filelist.txt - lists files written out by SYNCore • options.txt - lists the options selected in SYNCore • readme.txt - contains a brief description and known issues • syncore_counter.v - Verilog library file required to generate counter model • testbench.v - Verilog testbench file for testing the counter model • instantiation_file.vin - describes how to instantiate the wrapper file • *component*.v - counter model wrapper file generated by SYNCore Note that running the wizard in the same directory overwrites any existing files.
Filename	Specifies the name of the generated file containing the HDL description of the generated counter. Do not use spaces.
Width of Counter	Determines the counter width (the corresponding file parameter is COUNT_WIDTH=n).
Counter Step Value	Determines the counter step value (the corresponding file parameter is STEP=n).
Up Counter	Specifies an up counter (the default) configuration (the corresponding file parameter is MODE=Up).
Down Counter	Specifies an down counter configuration (the corresponding file parameter is MODE=Down).
UpDown Counter	Specifies a dynamic up/down counter configuration (the corresponding file parameter is MODE=Dynamic).

© 2022 Synopsys, Inc.

SYNCore Counter Parameters Page 2



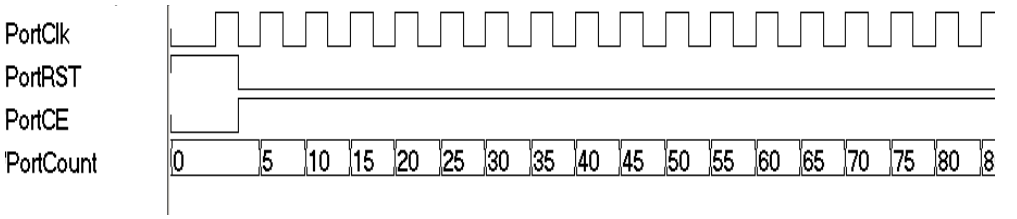
Enable Load option	Enables the load options
Load Constant Value	Load the constant value specified in the Load Value for constant load option field; (the corresponding file parameter is LOAD=1).
Load Value for constant load option	The constant value to be loaded.
Use the port PortLoadValue to load Value	Loads variable value from PortLoadValue (the corresponding file parameter is LOAD=2).
Synchronous Reset	Specifies a synchronous (the default) reset input (the corresponding file parameter is MODE=0).
Asynchronous Reset	Specifies an asynchronous reset input (the corresponding file parameter is MODE=1).

© 2022 Synopsys, Inc.

UP Counter Operation

In this mode, the counter is incremented by the step value defined by the STEP parameter. When reset is asserted (when PostRST is active high), the counter output is reset to 0. After the assertion of PortCE, the counter starts counting upwards coincident with the rising edge of the clock. The following waveform is with a constant STEP value of 5 and no load value.

Parameters: MODE= 'Up'
LOAD= '0'



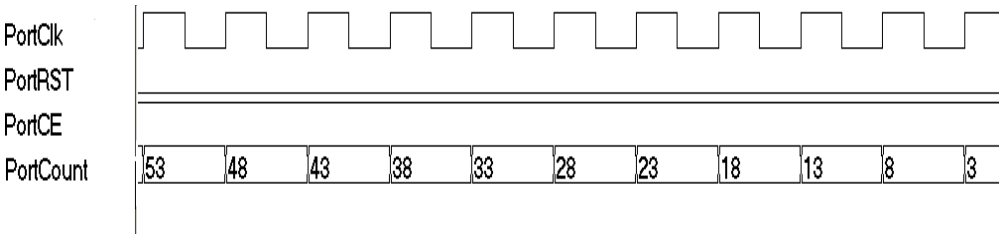
Note: Counter core can be configured to use a constant or dynamic load value in Up Counter mode (for the counter to load the PortLoadValue, PortCE must be active). This functionality is explained in Dynamic Counter Operation, on page 309.

Down Counter Operation

In this mode, the counter is decremented by the step value defined by the STEP parameter. When reset is asserted (when PostRST is active high), the counter output is reset to 0. After the assertion of PortCE, the counter starts counting downwards coincident with the rising edge of the clock. The following waveform is with a constant STEP value of 5 and no load value.

Parameters: MODE= 'Down'
LOAD= '0'

© 2022 Synopsys, Inc.



Note: Counter core can be configured to use a constant or dynamic load value in Down Counter mode (for the counter to load the PortLoadValue, PortCE must be active). This functionality is explained in Dynamic Counter Operation, on page 309.

Dynamic Counter Operation

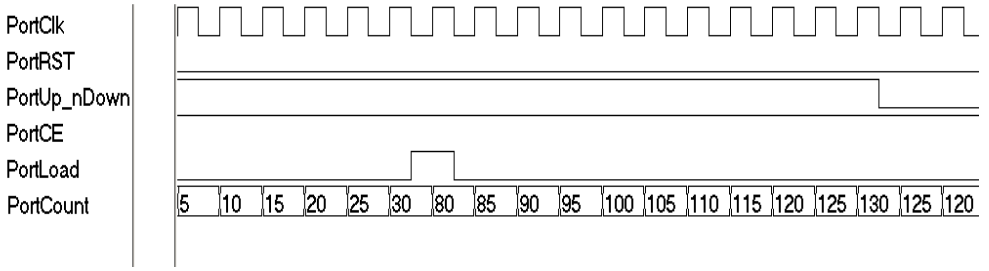
In this mode, the counter is incremented or decremented by the step value defined by the STEP parameter; the count direction (up or down) is controlled by the PortUp_nDown input (1 = up, 0 = down).

Dynamic Up/Down Counters with Constant Load Value*

On de-assertion of PortRST, the counter starts counting up or down based on the PortUp_nDown input value. The following waveform is with STEP value of 5 and a LOAD_VALUE of 80. When PortLoad is asserted, the counter loads the constant load value on the next active edge of clock and resumes counting in the specified direction.

Parameters: MODE= ‘Dynamic’
LOAD= ‘1’

© 2022 Synopsys, Inc.



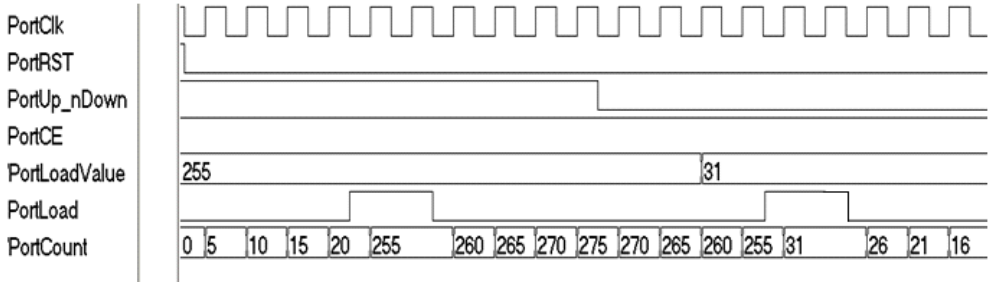
Note: *For counter to load the PortLoadValue, PortCE must be active.

Dynamic Up/Down Counters with Dynamic Load Value*

On de-assertion of PortRST, the counter starts counting up or down based on the PortUp_nDown input value. The following waveform is with STEP value of 5 and a LOAD_VALUE of 80. When PortLoad is asserted, the counter loads the constant load value on the next active edge of clock and resumes counting in the specified direction.

In this mode, the counter counts up or down based on the PortUp_nDown input value. On the assertion of PortLoad, the counter loads a new PortLoadValue and resumes up/down counting on the next active clock edge. In this example, a variable PortLoadValue of 8 is used with a counter STEP value of 5.

Parameters: MODE= 'Dynamic'
LOAD= '2'



Note: * For counter to load the PortLoadValue, PortCE should be active.

© 2022 Synopsys, Inc.

© 2022 Synopsys, Inc.

APPENDIX A

Designing with Pango

This chapter discusses the following topics for synthesizing Pango designs:

- Pango Technology Support, on page 314
- Pango Features, on page 315
- Pango Design Options, on page 319
- Pango Device Support, on page 322
- Pango Attribute and Directive Summary, on page 325

© 2022 Synopsys, Inc.

Pango Technology Support

This section describes general guidelines for using the synthesis tool and Pango place-and-route (P&R) tool with Pango devices. Refer to:

- Supported Device Families, on page 314
- Netlist Format, on page 314

Supported Device Families

The synthesis tool creates technology-specific netlists for Pango Titan, Logos and Compact technologies.

New devices are added on an ongoing basis. For the most current list of supported devices, select Project->Implementation Options and check the list of technologies on the Device tab.

Netlist Format

The synthesis tool outputs Verilog netlist files for use with the Pango P&R application. This file has a vm extension.

© 2022 Synopsys, Inc.

Pango Features

This section describes supported Pango features:

- Register and Latch Support, on page 315
- Combinational Logic Support, on page 315
- RAM Support, on page 316
- Block RAM Support, on page 316
- Distributed RAM Support, on page 317
- Distributed ROM Support, on page 317
- DSP Support, on page 318
- Multiplier Support, on page 318
- I/O Primitives, on page 318

Register and Latch Support

The synthesis tool supports instantiation and inference of registers and latches with different control signals, such as an enable or synchronous/asynchronous set/reset. Initial values on registers and latches are also supported.

The tool supports initial value on DFF registers and on DLATCH. The assigned initial value is forward annotated in the netlist through the parameter INIT. This feature is supported for the Logos 2,Titan 2 and Titan 3 technologies.

Combinational Logic Support

The synthesis tool supports the instantiation and inference of GTP_INV, GTP_LUT2, GTP_LUT3, GTP_LUT4, and GTP_LUT5. GTP_LUT5CARRY, GTP_LUTMUX4 and level MUX primitives, such as GTP_MUX2LUT6, GTP_MUX-2LUT7, and GTP_MUX2LUT8 are also supported for inferencing.

© 2022 Synopsys, Inc.

RAM Support

RAM mapping using GTP_DRM18K_E1 and GTP_DMR9K is supported for the Logos technology.

Inferred RAM and ROM are mapped using GTP_DRM18K_E3 and GTP_DRM36K_E3 on Titan 3. All RAM features such as memory initialization, control set packing and syn_ramstyle/syn_romstyle attributes to control RAM inference are supported.

Mapping of sequential shift registers using block RAM, distributed RAM or registers is supported on the Logos 2 and Titan 2 devices. Mapping of sequential shift registers using GTP_DRM18K_E3 and GTP_DRM36K_E3 is supported on Titan 3. Attribute syn_srlstyle controls the mapping of sequential shift registers. See syn_srlstyle, on page 157 in the *Attribute Reference*.

Attribute syn_allowed_resources is applied on the top-level module for specifying the upper limit of Block RAMs or DSPs to be inferred in the design. See syn_allowed_resources, on page 24 in the *Attribute Reference*.

Block RAM Support

The synthesis tool supports instantiation and inference of the following block RAM primitives based on the configuration.

- GTP_RAM144K – Single-port/true dual-port RAM with two ports that can be configured individually as read and write. Support is provided for normal mode, write-through mode, and read-before-write mode. Initial values for the RAM are supported.
- GTP_RAM144KSDP – Dual-port RAM with two ports that can be configured individually as either read or write. This RAM does not support any modes. Initial values for the RAM are supported.

Block RAM is inferred if the RAM or synchronous ROM is greater than 256 bits. RAM or ROM with less than 64 bits are mapped to registers. You can use the syn_ramstyle attribute with the value of block_ram/registers to control the block RAM inferencing. See syn_ramstyle, on page 133 in the *Attribute Reference*.

© 2022 Synopsys, Inc.

Limitations

- Asynchronous reset extraction is not supported.
- Byte enables and mixed data width mode are not supported.

Distributed RAM Support

The synthesis tool supports instantiation and inference for the following Distributed RAM (DRAM) primitives based on the configuration.

- GTP_RAM16X4SP – Single-port distributed RAM, where inference is based on the data width for the different primitives. Initial values for the DRAM are supported.
- GTP_RAM16X4DP – Dual-port distributed RAM, where inference is based on the data width for the different primitives. Initial values for the DRAM are supported.

Distributed RAM is inferred if the RAM is greater than 64 bits and less than 256 bits. You can use the syn_ramstyle attribute with the value of block_ram / registers to control the distributed RAM inferencing. See syn_ramstyle, on page 133 in the *Attribute Reference*.

Distributed ROM Support

The synthesis tool supports instantiation and inference for the following distributed ROM primitives based on the configuration.

GTP_ROM32X1, GTP_ROM64X1, GTP_ROM128X1, GTP_ROM256X1—The software infers asynchronous ROMs (ROM with read only) using the primitives based on its size. These primitives are mapped if the RAM is greater than 64 bits and less than 256 bits. ROM with less than 64 bits is mapped to registers.

You can use syn_romstyle attribute with the value of lut_rom / registers to control the distributed ROM inferencing. See syn_romstyle, on page 146 in the *Attribute Reference*.

© 2022 Synopsys, Inc.

DSP Support

Mapping of DSP functions like multiplier, multiplier-adder, multiplier-subtractor, wide multiplier and multiplier-adder cascade chain is supported using GTP_APM_E1 for the Logos technology.

The following DSP functions are supported using GTP_APM_E2:

- Packing of signed 25x18 multiplier and unsigned 24x17 multiplier
- Packing of direct input multiplier - adder/subtractor
- Packing of accumulator logic along with multiplier
- Cascade chain inference for multiplier-adder/subtractor chains

GTP_APM_E4 for Titan 3 supports the following functions:

- Mapping of signed 27x18 and unsigned 26x17 multiplier
- Mapping of mult-add and mult-sub with direct input Z
- Mapping of mult-acc and mult-accsub
- Packing of register with synchronous and asynchronous reset
- syn_dspstyle attribute

Multiplier Support

The synthesis tool supports instantiation and inference of following multiplier primitives.

GTP_MULT9X9, GTP_MULT18X18, GTP_MULT27X27, GTP_MULT18X36 – The primitives inferred are based on the multiplier widths. Smaller multipliers (4 bits or less inputs) are mapped to combinational logic.

You can use the syn_dspstyle attribute to control whether the multiplier is inferred as a MULT primitive or logic. See syn_dspstyle, on page 47 in the *Attribute Reference*.

I/O Primitives

The synthesis tool supports instantiation and inference of the GTP_INBUF, GTP_OUTBUF, GTP_OUTBUFT and GTP_IOBUF primitives.

© 2022 Synopsys, Inc.

Pango Design Options

The following options are located on the Options tab of the Implementation Options dialog box.

- FSM Compiler, on page 319
- Resource Sharing, on page 319
- Pipelining, on page 320
- Retiming, on page 320
- Gated and Generated Clocks, on page 320

FSM Compiler

Turning on the FSM Compiler enables special FSM optimizations. See Optimizing State Machines, on page 398 in the *User Guide*.

Resource Sharing

When Resource Sharing is enabled, synthesis uses resource sharing techniques. A resource sharing report is available in the log file (see Resource Usage Report, on page 148). See Sharing Resources, on page 396 in the *User Guide*.

© 2022 Synopsys, Inc.

Pipelining

Pipelining multipliers and ROMs allows your design to operate at a faster frequency. The synthesis tool moves registers that follow a multiplier or ROM into the multiplier or ROM, provided that you have registers in your RTL code. You can pipeline multipliers and ROMs if the ROM is bigger than 512 words.

- To invoke pipelining and apply it globally, enable the Pipelining option in the Project view.
- To apply this attribute locally, add the syn_pipeline attribute to the registers driven by the multipliers or ROMs. See syn_pipeline, on page 119 for the syntax.

For detailed information about using pipelining, see Pipelining, on page 376 of the *User Guide*.

Retiming

Retiming is a powerful technique for improving the timing performance of sequential circuits. The retiming process moves storage devices (flip-flops) across computational elements with no memory (gates/LUTs) to improve the performance of the circuit.

Set global retiming either from the Project view check box or from the Device tab under the Implementation Options button. Use the syn_allow_retiming attribute to enable or disable retiming for individual flip-flops. See syn_allow_retiming, on page 21 for syntax details, or Retiming, on page 380 of the *User Guide*.

Pipelining is automatically enabled when retiming is enabled. Use the syn_pipeline attribute to control pipelining. See *syn_pipeline, on page 119* for syntax details.

Gated and Generated Clocks

The Clock Conversion option on the GCC tab, when enabled, attempts to remove the gated clocks from synchronous logic and to convert generated clocks to the original clock with an enable.

© 2022 Synopsys, Inc.

Gated Clocks

Gated clocks are converted by removing the enable logic from the clock path. Extracting the enable logic allows clock constraints to be applied globally and eliminates broken paths. Converting gated clocks also makes use of the dedicated clock networks to simplify ASIC prototyping. For information on using the Clock Conversion option with gated clocks, see Working with Gated Clocks, on page 472 in the *User Guide*.

Generated Clocks

Generated clocks are converted to a circuit with an enable to improve performance by reducing clock skew. For more information on using the Clock Conversion option with generated clocks, see Optimizing Generated Clocks, on page 508 in the *User Guide*.

© 2022 Synopsys, Inc.

Pango Device Support

This section provides guidelines for using the synthesis tool with devices for the Pango technology. The topics include:

- Pango Device Mapping Options, on page 322
- set_option Command for Pango Architectures, on page 324
- Pango Technologies, on page 324

Pango Device Mapping Options

Specifies the implementation options for a design. You can set options for synthesis such as target technology, device architecture, and synthesis styles. To specify these options, select Project->Implementation Options and set the options on the Device or Options tab.

The following table lists the options for the Pango technology.

Annotated Properties for Analyst	Annotates the design with properties after the design is compiled. You can view these properties in the schematic views, and use them to create collections using the Tcl Find command. See Annotating Timing Information in the Schematic Views, on page 329 for more information.
Automatic Compile Point	Enables the automatic compile point flow, which can analyze a design and identify modules that can automatically be defined as compile points and mapped in parallel using multiprocessing. See The Automatic Compile Point Flow, on page 428 in the *User Guide*.
Clock Conversion	Performs gated- and generated-clock optimizations when enabled. See Working with Gated Clocks, on page 472 and Optimizing Generated Clocks, on page 508 in the *User Guide* for details.
Disable I/O Insertion	When enabled, it prevents automatic I/O insertion during synthesis. By default, it is disabled.
Fanout Guide	Sets a guideline for the fanout limit. The default is 100. For tips on setting and using fanout limits, see Setting Fanout Limits, on page 392 and Controlling Buffering and Replication, on page 394 in the *User Guide*.

© 2022 Synopsys, Inc.

FSM Compiler	Enables or disables the FSM compiler. Controls the use of FSM synthesis for state machines. The default is false (FSM Compiler disabled). Value can be 1 or true, 0 or false. See FSM Compiler, on page 319.
Pipelining	Pipelines registers located outside of a multiplier module back into the module to create pipeline stages. Default is 1 for ON. See *Pipelining, on page 376* in the *User Guide* for a procedure.
Read Write Check on RAM	Lets the synthesis tool insert bypass logic around the RAM to prevent a simulation mismatch between the RTL and post-synthesis simulations. The synthesis software globally inserts bypass logic around the RAM that reads and writes to the same address simultaneously. Disable this option, when you cannot simultaneously read and write to the same RAM location and want to minimize overhead logic. For details about using this option in conjunction with the syn_ramstyle attribute, see syn_ramstyle, on page 133.
Resolve Mixed Drivers	When a net is driven by a VCC or GND and active drivers, enable this option to connect the net to the VCC or GND driver.
Resource Sharing	Enables or disables resource sharing. See Sharing Resources, on page 396 in the *User Guide* for information about using it.
Retiming	When enabled (1), registers may be moved into combinational logic to improve performance. The default value is 0 (disabled). Enabling -retiming also enables -pipe. See Retiming, on page 380 in the *User Guide* for a procedure.
Update Compile Point Timing Data	Determines whether changes to a locked compile point force its parents to be remapped and updated with the new timing model of the child. See Compile Point Basics, on page 408, for details.

© 2022 Synopsys, Inc.

set_option Command for Pango Architectures

set_option Device Options

This is the syntax for setting device options for Pango devices.

-technology *keyword*
-part *partName*
-speed_grade *value*
-package *packageName*

-disable_io_insertion 1 | 0
-fanout_guide *value -> default fanout limit is 10000*
-maxfan *value*
-resolve_multiple_driver 1 | 0
-run_prop_extract 1 | 0
-rw_check_on_ram 1 | 0
-update_models_cp 1 | 0

Pango Technologies

The following table lists the supported Pango technologies and corresponding parameter values for the set_option -technology Tcl command.

Family	Valid -technology Keywords
Titan	Titan
Logos	Logos
Compact	Compact

Pango Attribute and Directive Summary

The following table summarizes the synthesis and Pango-specific attributes and directives available with the Pango Technology. Complete descriptions and examples can be found in Attributes and Directives Summary, on page 9.

Attribute/Directive	Description
full_case	Specifies that a Verilog case statement has covered all possible cases.
loop_limit	Specifies a loop iteration limit for for loops.
parallel_case	Specifies a parallel multiplexed structure in a Verilog case statement, rather than a priority-encoded structure.
syn_allow_retiming	Determines whether registers may be moved across combinational logic to improve performance in devices that support retiming.
syn_allowed_resources	Specifies the maximum number of resources available for use in a design.
syn_black_box	Defines a black box for synthesis.
syn_direct_enable	Identifies which signal to use as the enable input to an enable flip-flop, when multiple candidates are possible.
syn_direct_reset	Controls the assignment of a net to the dedicated reset pin of a synchronous storage element (flip-flop). Using this attribute, you can direct the mapper to only use a particular net as the reset when the design has a conditional reset on multiple candidates.
syn_direct_set	Controls the assignment of a net to the dedicated set pin of a synchronous storage element (flip-flop). Using this attribute, you can direct the mapper to only use a particular net as the set when the design has a conditional set on multiple candidates.
syn_dspstyle	Determines how multiplier logic is implemented.
syn_encoding	Specifies the encoding style for state machines.

© 2022 Synopsys, Inc.

Attribute/Directive	Description
syn_gatedclk_clock_en	Specifies the name of the enable pin inside a black box to support the *fix gated clocks* feature.
syn_gatedclk_clock_en_polarity	Indicates the polarity of the clock enable port on a black box, so that the software can apply the algorithm to fix gated clocks.
syn_hier	Controls the handling of hierarchy boundaries of a module or component during optimization and mapping.
syn_insert_buffer	Inserts a clock buffer according to the vendor-specified value.
syn_isclock	Specifies an input port on a black box as a clock.
syn_keep	Prevents the internal signal from being removed during synthesis and optimization.
syn_looplimit	Specifies a loop iteration limit for while loops.
syn_maxfan	Sets a fanout limit for an individual input port or register output.
syn_netlist_hierarchy	Determines if the generated netlist is to be hierarchical or flat.
syn_no_compile_point	Apply if you do not want the software to create a compile point for a particular view or module.
syn_noclockbuf	Turns off automatic clock buffer usage.
syn_noprune	Controls the automatic removal of instances that have outputs that are not driven.
syn_pad_type	Specifies an I/O buffer standard.
syn_pipeline	Specifies that registers be moved into multipliers and ROMs in order to improve frequency.

© 2022 Synopsys, Inc.

Attribute/Directive	Description
syn_preserve	Prevents sequential optimizations across a flip-flop boundary during optimization, and preserves the signal.
syn_probe	Inserts probe points for testing and debugging the internal signals of a design.
syn_ramstyle	Determines the way in which RAMs are implemented.
syn_reduce_controlset_size	Controls the minimum size of the unique control-set on which control-set optimizations can occur.
syn_replicate	Controls replication of registers during optimization.
syn_romstyle	Determines the way in which ROMs are implemented.
syn_smhigheffort	Uses higher threshold effort when the tool extracts a state-machine on individual state registers.
syn_srlstyle	Controls the mapping of sequential shift registers.
syn_state_machine	Enables/disables state-machine optimization on individual state registers in the design.
syn_ternaryadder	Supports implementation of ternary adders using Ripple Carry adder implementation.
syn_useenables	Controls the use of clock-enable registers within a design.
syn_useioff	Determines the packing of flip-flops in I/O pad cells.
translate_off/translate_on	Synthesizes designs originally written for use with other synthesis tools without needing to modify source code.

© 2022 Synopsys, Inc.

© 2022 Synopsys, Inc.

Index

Symbols

_ta.srm file 143
.adc file 132
.fse file 140
.info file 140
.ini file 132
.prj file 132
.sap
  annotated properties for analyst 141
.sar file 141
.sdc file 132
.srd file 142
.srm file 142
.srr file 145
  watching selected information 34
.srs file 142
  initial values (Verilog) 186
.sv file 133
  SystemVerilog source file 133
.ta file
  See timing report file 143
.v file 133
.vhd file 133
.vhm file 144
.vm file 144

A

adc file (analysis design constraint) 132
adder
  SYNCore 274
adders
  SYNCore 275
Allow Docking command 35
Alt key, selecting columns in Text Editor 44
analysis design constraint file (.adc) 132
Analyst toolbar 56
annotated properties for analyst
  .sap 141
  .timing annotated properties (.tap) 143
archive file (.sar) 141
arrow keys, selecting objects in Hierarchy Browser 107
arrow pointers for push and pop 105
asynchronous clock report
  description 155
Attributes demo 47
auto constraints 130
  Maximize option 70

B

block RAM
  dual-port RAM examples 173
  NO_CHANGE mode example 170
  READ_FIRST mode example 169
  single-port RAM examples 171
  WRITE_FIRST mode example 167
buttons and options, Project view 69
byte-enable RAMs
  SYNCore 246

C

cck.rpt file (constraint checking report) 140
check boxes, Project view 69
clock buffering report, log file (.srr) 147
clock groups
  Clock Relationships (timing report) 153
clock pin drivers, selecting all 80
clock relationships, timing report 153
clock report
  asynchronous 150
Clock Tree, HDL Analyst tool 80
clocks
  asynchronous report 155
  declared clock 152
  defining 80
  inferred clock 152
  system clock 152

color coding
  Text Editor 44
commenting out code (Text Editor) 44
compiler report, log file (.srr) 146
Constraint Check command 157
constraint checking report 157
constraint files 123
  .sdc 132
  automatic. *See* auto constraints
constraint files (.sdc)
  creating 55
constraints
  auto constraints. *See* auto constraints
  report file 157
  types 122
context help editor 45
context of filtered schematic, displaying 113
context sensitive help
  using the F1 key 15
copying
  for pasting 62
counter compiler
  SYNCore 297
counters
  SYNCore 298
critical paths 117
  analyzing 118
  finding 118
cross-clock paths, timing analysis 153
cross-hair mouse pointer 52
crossprobing 97
  definition 97
Ctrl key
  avoiding docking 54
  multiple selection 51
  zooming using the mouse wheel 53
cutting (for pasting) 56

D

declared clock 152
deleting
  See removing
design size, schematic sheet
  setting 100
Dissolve Instances command 115
docking 35
  avoiding 54
docking GUI entities
  toolbar 54
dual-port RAM examples 173
dual-port RAMs
  SYNCore parameters 229

E

editor view
  context help 45
examples
  Interactive Attribute Examples 47

F

failures, timing (definition) 119
fdc constraints 125
fdc file
  relationship with other constraint files 123
FIFO compiler
  SYNCore 192
FIFO flags
  empty/almost empty 214
  full/almost full 213
  handshaking 214
  programmable 216
  programmable empty 219
  programmable full 217
FIFOs
  compiling with SYNCore 193
files
  .adc 132
  .fdc 132
  .fse 140
  .info 140
  .ini 132
  .prj 132
  .sar 141
  .sdc 132
  .srm 142, 143
  .srr 145
    watching selected information 34
  .srs 142
  .ta 143
  .v 133
  .vhd 133
  .vhm 144
  .vm 144
  compiler output (.srs) 142
  constraint (.adc) 132

constraint (.sdc) 132
creating 55
customized timing report (.ta) 143
design component info (.info) 140
initialization (.ini) 132
log (.srr) 145
watching selected information 34
mapper output (.srm) 142, 143
output
See output files
project (.prj) 132
RTL view (.srs) 142
srr 145
watching selected information 34
state machine encoding (.fse) 140
Synopsys archive file (.sar) 141
synthesis output 140
Technology view (.srm) 142, 143
Verilog (.v) 133
VHDL (.vhd) 133
files for synthesis 132
filtered schematic
compared with unfiltered 84
filtering 111
commands 111
compared with flattening 115
FSM states and transitions 83
paths from pins or ports 119
filtering critical paths 118
finding
critical paths 118
information on synthesis tool 16
GUI 15
finite state machines
See state machines
Flatten Current Schematic command 115
Flatten Schematic command 115
flattening
commands 113
compared with filtering 115
selected instances 114
Float command
Watch window popup menu 35
floating
toolbar 54
floating toolbar popup menu 54
forward annotation
initial values 186
Forward Annotation of Initial Values
Verilog 186
frequency
cross-clock paths 153
Frequency (Mhz) option, Project view 70
fse file 140
FSM Compiler option, Project view 70
FSM Compiler, enabling and disabling
globally
with GUI 70, 319
FSM encoding file (.fse) 140
FSM toolbar 59
FSM Viewer 81
FSMs (finite state machines)
See state machines

G

generic technology library 137
graphical user interface (GUI), overview 19
GTECH library. *See* generic technology library
gtech.v library 137
gui
synthesis software 14
GUI (graphical user interface), overview 19

H

HDL Analyst tool 73
accessing commands 85
analyzing critical paths 117
Clock Tree 80
crossprobing 97
filtering designs 111
finding objects 95
hierarchical instances. *See* hierarchical instances
object information 86
preferences 100
push/pop mode 103
ROM table viewer 187
schematic sheet size 100
schematics, filtering 111
schematics, multiple-sheet 100
status bar information 86
title bar information 100
HDL Analyst toolbar
See Analyst toolbar
HDL Analyst views 74

See also RTL view, Technology view
HDL files, creating 55
header, timing report 151
help
online
accessing 15
hidden hierarchical instances 90
are not flattened 115
Hide command
floating toolbar popup menu 54
Log Watch window popup menu 35
Tcl Window popup menu 38
hierarchical instances 89
compared with primitive 88
display in HDL Analyst 89
hidden 90
opaque 89
transparent 89
hierarchical schematic sheet, definition 100
hierarchy
flattening
compared with filtering 115
pushing and popping 103
schematic sheets 100
Hierarchy Browser 106
changing size in view 75
Clock Tree 80
finding schematic objects 95
moving between objects 80
RTL view 75
symbols (legend) 80
Technology view 77
trees of objects 79

I

Identify Instrumentor
launching 60
IEEE 1364 Verilog 95 standard 134
Implementation Directory 28
Implementation Results 28
indenting a block of text 44
indenting text (Text Editor) 44
inferred clock 152
info file (design component info) 140
ini file 132
initial value data file
Verilog 183
Initial Values
forward annotation 186
initial values
$readmemb 179
$readmemh 179
initial values (Verilog)
netlist file (.srs) 186
initialization file (.ini) 132
input files 132
.adc 132
.ini 132
.sdc 132
.sv 133
.v 133
.vhd 133
inserting
bookmarks (Text Editor) 44
instances
hierarchical
dissolving 108
making transparent 108
hierarchical. *See* hierarchical instances
primitive. *See* primitive instances
Interactive Attribute Examples 47
interface information, timing report 154
IPs
SYNCore byte-enable RAMs 246
SYNCore counters 298
SYNCore FIFOs 193
SYNCore RAMs 223
SYNCore ROMs 262
SYNCore subtractors 275
isolating paths from pins or ports 119

K

keyboard shortcuts 61
arrow keys (Hierarchy Browser) 107
keyword completion, Text Editor 44
keywords
completing in Text Editor 44

L

latches
in timing analysis 117
Launch Identify Instrumentor icon 60
lib2syn
using 138
libraries

general technology 136
macro, built-in 133
technology-independent 136
VHDL
attributes and constraints 134
linkerlog file 140
log file (.srr) 145
watching selected information 34
log file report 145
clock buffering 147
compiler 146
mapper 146
net buffering 147
resource usage 148
retiming 148
summary of compile points 147
timing 148
Log Watch Configuration dialog box 36
Log Watch window 34
Output Windows 42
positioning commands 35

M

macros
libraries 133
mapper output file (.srm) 142, 143
mapper report
log file (.srr) 146
margin, slack 118
message viewer
description 38
Messages Tab 38
mouse button operations 51
mouse operations 49
Mouse Stroke Tutor 50
mouse wheel operations 53
Move command
floating toolbar window 54
Log Watch window popup menu 35
Tcl window popup menu 38
moving between objects in the Hierarchy Browser 80
moving GUI entities
toolbar 54
multiple-sheet schematics 100
multisheet schematics
transparent hierarchical instances 102

N

navigating
among hierarchical levels
by pushing and popping 103
with the Hierarchy Browser 106
among the sheets of a schematic 100
nesting design details (display) 108
net buffering report, log file 147
netlist file 144
initial values (Verilog) 186

O

object information
status bar, HDL Analyst tool 86
viewing in HDL Analyst tool 86
objects
crossprobing 97
dissolving 108
making transparent 108
objects, schematic
See schematic objects
Online help
F1 key 15
online help
accessing 15
opaque hierarchical instances 89
are not flattened 115
options
Project view 69
Frequency (Mhz) 70
FSM Compiler 70
Pipelining 71
Resource Sharing 71
Retiming 71
output files 140
.info 140
.sar 141
.srm 142, 143
.srr 145
watching selected information 34
.srs 142
.ta 143
.vhm 144
.vm 144
netlist 144
See also files
Output Windows 42

Overview of the Synopsys FPGA Synthesis Tools 12

P

Pango
  attributes and directives 325
parameters
  SYNCore adder/subtractor 282
  SYNCore counter 304
partitioning of schematics into sheets 100
pasting 56
performance summary, timing report 151
pins
  displaying
    on transparent instances 93
  displaying on technology-specific primitives 93
  isolating paths from 119
pipelining
  option 71
Pipelining option, Project view 71
pointers, mouse
  cross-hairs 52
  push/pop arrows 105
popping up design hierarchy 103
popup menus
  floating toolbar 54
  Log Watch window 35, 36
  Log Watch window positioning 35
  Tcl window 38
preferences
  HDL Analyst tool 100
  Project view display 54
primitive instances 88
primitives
  pin names in Technology view 93
prj file 132
probes
  inserting 327
Process View 30
project files (.prj) 132
project results
  Implementation Directory 28
  Process View 30
  Project Status View 23
Project Results View 23
Project Status View 23
Project toolbar 55
Project view 20
  buttons and options 69
  options 69
  Synplify Pro 20
  Synplify tool 20
Project window 20
project_name_cck.rpt file 157
push/pop mode, HDL Analyst tool 103

R

RAMs
  compiling with SYNCore 223
  initial values (Verilog) 179
  SYNCore 223
  SYNCore, byte-enable 246
RAMs, inferring
  advantages 166
reference manual, role in document set 11
removing
  bookmark (Text Editor) 44
  window (view) 54
reports
  constraint checking (cck.rpt) 157
Resource Sharing option, Project view 71
resource usage report, log file 148
retiming
  report, log file 148
Retiming option, Project view 71
ROM compiler
  SYNCore 260
ROM inference examples 187
ROM initialization
  with rom.info file 189
  with Verilog generate block 190
rom.info file 187
ROMs
  SYNCore 262
RTL view 75
  displaying 57
  file (.srs) 142

S

schematic objects
  crossprobing 97
  definition 86
  dissolving 108
  finding 95

making transparent 108
status bar information 86
schematic sheets 100
hierarchical (definition) 100
navigating among 100
setting size 100
schematics
configuring amount of logic on a sheet 100
crossprobing 97
filtered 84
filtering commands 111
flattening compared with filtering 115
flattening selectively 114
hierarchical (definition) 100
multiple-sheet 100
multiple-sheet. *See also* schematic sheets
object information 86
partitioning into sheets 100
sheet connectors 87
sheets
navigating among 100
size, setting 100
size in view, changing 75
unfiltered 84
unfiltering 113
selecting
text column (Text Editor) 44
selecting multiple objects using the Ctrl key 51
sheet connectors 87
Shift key 54
shortcuts
keyboard
See keyboard shortcuts
single-port RAM examples 171
single-port RAMs
SYNCore parameters 228
slack
cross-clock paths 154
defined 152
margin
definition 119
setting 118
source files
See also files
creating 55
srd file 142
srm file 142, 143
srr file 145
watching selected information 34
srs file 142
initial values (Verilog) 186
standards, supported
Verilog 134
VHDL 134
state machines
encoding
displaying 83
encoding file (.fse) 140
filtering states and transitions 83
state encoding, displaying 83
status bar information, HDL Analyst tool 86
structural netlist file (.vhm) 144
structural netlist file (.vm) 144
subtractor
SYNCore 274
subtractors
SYNCore 275
summary of compile points report
log file (.srr) 147
supported standards
Verilog 134
VHDL 134
symbols
Hierarchy Browser (legend) 80
SYNCore
adder/subtractor 274
adder/subtractor parameters 282
adders 275
byte-enable RAM compiler
byte-enable RAM compiler
SYNCore 245
counter compiler 297
counter parameters 304
counters 298
FIFO compiler 192, 193
RAM compiler
RAM compiler
SYNCore 223
RAMs 223
RAMs, byte-enable 246
RAMs, dual-port parameters 229
RAMs, single-port parameters 228
ROM compiler 260
ROMs 262
ROMs, parameters 265

subtractors 275
SYNCore adder/subtractor
adders 286
dynamic adder/subtractor 292
functional description 274
subtractors 289
SYNCore FIFOs
definition 192
parameter definitions 211
port list 208
read operations 208
status flags 213
write operations 207
SYNCore ROMs
clock latency 274
dual-port read 271
parameter list 271
single-port read 270
Synopsys FPGA Synthesis Tools
overview 12
Synplify Pro tool
Project view 20
user interface 14
Synplify tool
Project view 20
user interface 14
synthesis
log file (.srr) 145
watching selected information 34
synthesis software
gui 14
system clock 152
SystemVerilog keywords
context help 45

T

ta file (customized timing report) 143
Tcl commands
constraint files 125
pasting 38
Tcl Script window
Output Windows 42
Tcl window
popup menu commands 38
popup menus 38
Technology view 77
displaying 57
file (.srm) 142, 143
Text Editor
features 44
indenting a block of text 44
opening 43
selecting text column 44
view 42
text editor
completing keywords 44
Text Editor view 42
timing analysis of critical paths (HDL Analyst tool) 117
timing analyst
cross-clock paths 153
timing annotated properties (.tap) 143
timing constraints
See constraints
timing failures, definition 119
timing report 150
clock relationships 153
customized (.ta file) 143
file (.ta) 143
header 151
interface information 154
performance summary 151
timing reports
asynchronous clocks 155
log file (.srr) 148
title bar information, HDL Analyst tool 100
toolbars 54
FSM 59
moving and docking 54
transparent hierarchical instances 90
lower-level logic on multiple sheets 102
operations resulting in 110
pins and pin names 93
trees of objects, Hierarchy Browser 79
trees, browser, collapsing and expanding 80

U

unfiltered schematic, compared with filtered 84
unfiltering schematic 113
user interface
Synplify Pro tool 14
Synplify tool 14
user interface, overview 19
using the mouse 49
utilities
lib2syn 138

V

v file 133
Verilog
  Forward Annotation of Initial Values 186
  generic technology library 137
  initial value data file 183
  initial values for RAMs 179
  netlist file 144
  ROM inference 187
  source files (.v) 133
  structural netlist file (.vm) 144
  supported standards 134
Verilog 2001 134
Verilog 95 134
Verilog source file (.v) 133
vhd file 133
vhd source file 133
VHDL
  libraries
    attributes, supplied with synthesis tool 134
  source files (.vhd) 133
  structural netlist file (.vhm) 144
  supported standards 134
VHDL source file (.vhd) 133
vhm file 144
views 33
  FSM 81
  Project 20
  removing 54
  RTL 75
  Technology 77
vm file 144

W

Watch Window. *See* Log Watch window
window
  Project 20
windows 33
  closing 63
  log watch 34
  removing 54

Z

zoom
  using the mouse wheel and Ctrl key 53